

Basics of Statistical Learning

Weijia Jia

2026-06-05

Table of contents

Preface	3
References	3
 I Foundations	 5
1 What is Statistical Learning?	6
1.1 Introduction	6
1.2 Types of Learning	7
1.2.1 Supervised Learning	7
1.2.2 Unsupervised Learning	7
1.2.3 Reinforcement Learning	8
1.3 Types of Data	8
1.3.1 Tabular Data	8
1.3.2 Other Types of Data	9
1.4 Generalization and Model Complexity	10
1.4.1 Training Error and Test Error	10
1.4.2 Bias–Variance Tradeoff	11
 2 Classification Decisions and Evaluation	 13
2.1 Decision quality	14
2.1.1 Binary Classification Setup	14
2.1.2 Accuracy	15
2.1.3 Recall (sensitivity)	15
2.1.4 Precision	16
2.1.5 F1 Score	17
2.1.6 Recall vs Precision	18
2.2 Ranking quality	22
2.2.1 ROC	22
2.2.2 ROC and Hypothesis Testing	23
2.2.3 AUC	23
2.2.4 Precision-Recall Curve	24
2.3 Probability Quality	24
2.3.1 Calibration Curve	25
2.4 Model Tuning versus Threshold Tuning	25

2.5	Python Example: Breast Cancer Classification	26
2.5.1	Load the dataset	26
2.5.2	Fit models	27
2.5.3	Confusion Matrix and Basic Metrics	27
2.5.4	Modifying Thresholds	28
2.5.5	Tuning Thresholds	29
2.5.6	Choosing a Threshold from Costs	31
2.5.7	ROC Curve and AUC	32
2.5.8	Precision-Recall Curve	34
2.5.9	Calibration Curves	35
II	Parametric Supervised Learning	38
3	Linear Regression	39
3.1	Quick Overview	39
3.2	The Ordinary Least Squares Estimator	39
3.3	Linear Regression in Python	40
3.3.1	Linear regression models	41
3.3.2	Measure model performance	41
3.3.3	Model Diagnostics	42
3.4	Model evaluation	51
3.4.1	Train-Test Split	51
3.4.2	Cross-Validation	52
3.4.3	Modern Hyperparameter Tuning	71
3.5	Feature Engineering and Preprocessing	72
3.5.1	Creating Features Manually	72
3.5.2	Polynomial Features	73
3.5.3	Categorical Variables	74
3.5.4	Scaling Variables	77
3.5.5	ColumnTransformer	79
3.6	Pipelines and transformations of y	81
3.6.1	TransformedTargetRegressor	84
3.6.2	Full Example	84
4	Regularization	88
4.1	Motivation	88
4.2	Ridge Regression	88
4.2.1	OLS estimator review	88
4.2.2	Ridge estimator	89
4.2.3	About intercept	91
4.2.4	Alternative Formulation	92
4.2.5	SVD and Tikhonov Regularization	94

4.2.6	Bias-Variance Tradeoff	96
4.3	Ridge Regression in Python	98
4.3.1	Ridge estimator	99
4.3.2	Contour map	101
4.3.3	Choosing λ	108
4.3.4	Validation curve and Coefficient paths	110
4.3.5	LOOCV	112
4.4	Lasso Regression	114
4.4.1	Definition	114
4.4.2	Geometry and sparsity	115
4.4.3	One-variable lasso and soft-thresholding	117
4.5	Lasso in Python	119
4.5.1	A simulated example	119
4.5.2	Initial model fitting	120
4.5.3	Coefficient paths	122
4.5.4	Tuning with cross-validation	124
4.5.5	Validation curves	127
4.5.6	Summary	129
5	Dimension reduction	131
5.1	Principal Component Regression (PCR)	131
5.1.1	Data matrix and its geometric views	132
5.1.2	Singular Value Decomposition (SVD)	134
5.1.3	Principal components (PC)	134
5.1.4	Covariance and Optimality	136
5.1.5	Summary	137
5.1.6	Choosing the number of components (m)	137
5.2	PCR in Python	138
5.2.1	PCA	138
5.2.2	PCR	141
5.2.3	Choosing number of components by cross-validation	141
5.3	Partial Least Squares (PLS)	143
5.3.1	Algorithm intuition	144
5.3.2	Key properties	144
5.3.3	Comparison with PCA	145
5.3.4	Geometric meaning (Krylov Subspace)	145
5.4	PLS in Python	147
5.4.1	Example	147
5.4.2	Another example	150
6	Logistic Regression	152
6.1	Binary Response	152
6.1.1	Probability	152

6.1.2	Odds	152
6.1.3	Logit and Log-Odds	153
6.2	The Logistic Regression Model	153
6.2.1	From Log-Odds Back to Probability	154
6.2.2	Logistic Regression as a Linear Classifier	155
6.2.3	Interpreting the Coefficients	156
6.3	Maximum Likelihood Estimation	157
6.3.1	Binary Cross-Entropy and a Single Neuron	158
6.4	Regularized Logistic Regression	159
6.5	Logistic Regression in <code>sklearn</code>	159
6.5.1	Regularization	160
6.6	Example 1: Binary classification	161
6.6.1	Tuning <code>C</code>	162
6.6.2	Tuning the threshold	162
6.7	Multiclass Extension	164
6.7.1	Cross-Entropy Loss	165
6.8	Example 2: Multinomial Logistic Regression	167
6.8.1	Probabilities and Softmax	167
6.8.2	Confusion Matrix	168
III	Nonparametric Supervised Learning	171
7	K-Nearest Neighbors	172
7.1	The KNN Model	172
7.1.1	KNN Regression	172
7.1.2	KNN Classification	173
7.1.3	Distance	174
7.1.4	The Role of K	175
7.2	KNN in <code>sklearn</code>	176
7.3	Example 1: One-Dimensional Regression	176
7.3.1	Comparing Different Values of K	178
7.3.2	Choosing K by Cross-Validation	179
7.4	Example: Iris Classification	181
7.4.1	Pipeline	183
7.4.2	Choosing K for Classification	184
7.4.3	Decision Boundary	185
8	Kernel Smoothing	187
8.1	From KNN to Kernel Weights	187
8.2	Kernel Density Estimation	189
8.2.1	Uniform Kernel and Histograms	190
8.2.2	Gaussian KDE	191

8.2.3	Expectation of the KDE	193
8.3	Kernel Regression	194
8.3.1	Comparing KNN and Kernel Regression	195
8.3.2	Local Averaging at One Target Point	197
8.4	Bandwidth Tuning	198
8.4.1	Effect of Bandwidth in KDE	199
8.4.2	Effect of Bandwidth in Kernel Regression	200
8.4.3	Rule-of-Thumb Bandwidth Selection	201
8.4.4	Bandwidth Selection by Cross-Validation	202
8.5	Extensions	204
8.5.1	Local Linear Regression	204
8.5.2	Local Polynomial Regression	206
8.5.3	Multiple Dimensions	208
8.6	Summary	209
9	Tree-Based Methods	210
9.1	CART: Classification and Regression Trees	212
9.1.1	Splitting a Dataset	212
9.1.2	Classification Trees: Gini Impurity	213
9.1.3	Regression Trees: RSS	215
9.1.4	Trees are local	216
9.1.5	Trees are nonlinear	216
9.1.6	Automatic interaction effects	216
9.2	Tuning hyperparameters	217
9.2.1	Pruning	219
9.2.2	Pre-pruning	219
9.3	Trees in Python	220
9.3.1	Example 1: Classify <code>iris</code> dataset	221
9.3.2	Example 2: Regression with the <code>diabetes</code> dataset	225
10	Bagging and Random Forests	229
10.1	Ensemble Averaging	229
10.2	Bagging	231
10.2.1	Out-of-Bag Error	232
10.3	Random Forests	233
10.3.1	<code>max_features</code>	234
10.3.2	Random Forest as Adaptive Local Averaging	234
10.3.3	Feature Importance	236
10.4	Different Types of Randomization in Tree Ensembles	237
10.4.1	(Almost) No randomness: Single Decision Tree	238
10.4.2	Bootstrap Randomness: Bagging	238
10.4.3	Feature Randomness: Random Forest	239
10.4.4	Threshold Randomness: Extra Trees	239

10.5	Bagging in Python	240
10.5.1	Basic Bagging classifier	240
10.5.2	Random Forest Classifier	241
10.5.3	Useful Attributes	242
10.6	Tuning Hyperparameters	254
10.6.1	n_jobs	257
10.7	Example: Diabetes dataset	259
10.7.1	Tuning the Random Forest	261
10.7.2	Find the Number of Trees	262
11	Boosting	279
11.1	AdaBoost	279
11.1.1	Weak Learners	280
11.1.2	Weights of observations	281
11.1.3	The AdaBoost Algorithm	282
11.1.4	Why Does AdaBoost Work?	284
11.2	AdaBoost in Python	285
11.2.1	The Margin Effect of AdaBoost	286
11.2.2	Why Margins Matter	286
11.2.3	Margin study in Python	287
11.3	Tuning AdaBoost Hyperparameters	304
11.3.1	Learning Rate and Number of Trees	305
11.3.2	Base Tree Depth	305
11.3.3	Grid Search for AdaBoost	305
11.3.4	Example: AdaBoost on a Nonlinear Classification Problem	306
11.4	XGBoost and Gradient Boosting	307
11.4.1	Additive model	308
11.4.2	Functional gradient viewpoint	308
11.4.3	Regression tree fitting	309
11.4.4	XGBoost: Initial idea	310
11.4.5	Intuition of the quadratic approximation	312
11.4.6	Tree structure in XGBoost	312
11.4.7	Regularization	313
11.4.8	Learning rate	315
11.4.9	GBDT and XGBoost for classification	316
11.5	XGBoost in Python	317
11.6	Tuning XGBoost	320
11.6.1	RandomSearchCV	320
11.6.2	Workflow	321
11.6.3	Early Stopping	321
11.7	Example: XGBoost on the Diabetes Data	322
11.7.1	Baseline model	323
11.7.2	RandomSearchCV	323

11.7.3	Early stopping	324
11.7.4	Refined grid search	325
11.7.5	Feature Importance	328
Appendices		329
A Python IDE Setup		329
A.1	VS Code	329
A.2	Python Virtual Environment installation	332
A.2.1	Install the first Python	333
A.2.2	Setup the environment for the course	333
A.3	Back to VS Code and start Jupyter notebook	335
A.3.1	Output to html and pdf	340
A.4	WSL	341
B Jupyter Notebook and Markdown		344
B.1	Text Formatting	346
B.2	Headings	346
B.3	Lists	347
B.4	Essential LaTeX	347
C Python Basics for sklearn		349
C.1	Running Python Code	349
C.1.1	Indentation	349
C.1.2	if and for	350
C.1.3	Functions and Objects	350
C.1.4	Importing Packages	352
C.2	Data structure	353
C.2.1	Basic Objects	353
C.2.2	Lists	353
C.2.3	Dictionaries	354
C.3	numpy.array	355
C.3.1	2D Arrays	355
C.3.2	Indexing	356
C.3.3	Common Operations	357
C.3.4	Axis-wise	359
C.3.5	Reshaping	360
C.3.6	Arrays versus Lists	361
C.4	Random Numbers	361
C.5	pandas	362
C.5.1	DataFrame and Series	362
C.5.2	Read and write files	364

C.5.3	Relative paths	366
C.6	matplotlib Basics	367
C.6.1	Scatter Plot	367
C.6.2	Line Plot	368
C.6.3	Scatter and Line Together	369
C.6.4	Multiple Plots	370
C.6.5	Histograms	371
C.7	Common Errors	372
C.7.1	One-Dimensional X	372
C.7.2	Class Name versus Model Object	372
C.7.3	Mixing Lists and Arrays	373
D	Some Python concepts	374
D.1	Language	374
D.2	Interpreters	374
D.3	REPL	375
D.4	IPython	375
D.5	Jupyter	376
D.5.1	Multi-kernels setup	377

Preface

These are notes for STAT 432 2026 Summer at UIUC. If you have any questions please contact me.

References

- [1] BERGSTRA, J. and BENGIO, Y. (2012). [Random search for hyper-parameter optimization](#). *Journal of Machine Learning Research* **13** 281–305.
- [2] TIKHONOV, A. N. and ARSENIN, V. Y. (1977). *Solutions of ill-posed problems*. Winston.
- [3] HASTIE, T., TIBSHIRANI, R. and FRIEDMAN, J. (2009). *The elements of statistical learning*. Springer.
- [4] ALLEN, D. M. (1974). [The relationship between variable selection and data augmentation and a method for prediction](#). *Technometrics* **16** 125–7.
- [5] VERHULST, P.-F. (1838). Notice sur la loi que la population poursuit dans son accroissement. *Correspondance Mathématique et Physique* **10** 113–21.
- [6] BERKSON, J. (1944). [Application of the logistic function to bio-assay](#). *Journal of the American Statistical Association* **39** 357–65.
- [7] BREIMAN, L. (2001). Random forests. *Machine Learning* **45** 5–32.
- [8] LIN, Y. and JEON, Y. (2006). [Random forests and adaptive nearest neighbors](#). *Journal of the American Statistical Association* **101** 578–90.
- [9] FREUND, Y. and SCHAPIRE, R. E. (1995). [A decision-theoretic generalization of on-line learning and an application to boosting](#). In *Computational learning theory* pp 23–37. Springer Berlin Heidelberg.

- [10] FREUND, Y. and SCHAPIRE, R. E. (1997). [A decision-theoretic generalization of on-line learning and an application to boosting](#). *Journal of Computer and System Sciences* **55** 119–39.
- [11] SCHAPIRE, R. E., FREUND, Y., BARTLETT, P. and LEE, W. S. (1998). [Boosting the margin: A new explanation for the effectiveness of voting methods](#). *The Annals of Statistics* **26** 1651–86.
- [12] FRIEDMAN, J., HASTIE, T. and TIBSHIRANI, R. (2000). [Additive logistic regression: A statistical view of boosting \(with discussion and a rejoinder by the authors\)](#). *The Annals of Statistics* **28** 337–407.
- [13] BREIMAN, L. (1999). [Prediction games and arcing algorithms](#). *Neural Computation* **11** 1493–517.

Part I

Foundations

1 What is Statistical Learning?

1.1 Introduction

Statistical learning studies how to use data to understand relationships and make predictions.

In supervised learning, a common abstract model is

$$Y = f(X) + \varepsilon,$$

where:

- X represents features;
- Y represents the response;
- f is the underlying relationship;
- ε represents random noise or unexplained variation.

The main objective is to estimate the unknown function f using observed data.

More broadly, modern statistical learning is fundamentally about learning patterns that generalize well to unseen data.

Depending on the objective, statistical learning problems can be viewed from several different perspectives.

Task	Main Question
Prediction	What will happen?
Inference	Which variables matter?
Causality	What happens if we intervene?

- Prediction mainly focuses on predictive performance and generalization. A common formulation is

$$\hat{f}(x) \approx \mathbb{E}[Y \mid X = x].$$

- Inference focuses on understanding relationships between variables and often relies on tools such as confidence intervals and hypothesis testing.
- Causality studies intervention effects and is often written as

$$\mathbb{E}[Y \mid \text{do}(X = x)].$$

Important distinctions

1. Good training performance does not imply good generalization.
2. Good prediction does not imply valid inference.
3. Association does not imply causation.

Tip

In this course, we will primarily focus on **prediction** and **generalization** rather than formal statistical inference.

1.2 Types of Learning

Statistical learning problems can be divided into several major categories depending on the available data and learning objective.

We will mainly focus on supervised learning in this course, with some topics from unsupervised learning.

1.2.1 Supervised Learning

In supervised learning, we observe both predictors X and response Y .

The goal is to predict or explain the response, such as house price prediction, spam detection, medical diagnosis, etc..

Supervised learning is usually divided into two major tasks:

- **regression**, where the response is quantitative;
- **classification**, where the response is categorical.

Example 1.1.

- predicting salary is a regression problem;
- predicting whether a tumor is benign or malignant is a classification problem.

1.2.2 Unsupervised Learning

In unsupervised learning, we observe predictors X but no response Y .

The goal is to discover hidden structure in the data, such as clustering, dimensionality reduction, embedding learning, etc..

1.2.3 Reinforcement Learning

In reinforcement learning, an agent interacts with an environment and learns through rewards and penalties, such as robotics, game playing, recommendation systems, etc..

1.3 Types of Data

Machine learning methods are strongly connected to data structure, and different data types often require different modeling strategies.

This course mainly focuses on **tabular data**, which is the classical setting in statistics and applied machine learning.

1.3.1 Tabular Data

Tabular data is one of the most common data formats in statistical learning. It is typically represented as

$$X \in \mathbb{R}^{n \times p},$$

where:

- n is the number of observations;
- p is the number of predictors.

Example:

Age	Income	Balance	Student	Default
23	50000	1200	Yes	No
45	90000	3000	No	Yes

In tabular data:

- rows represent observations;
- columns represent variables or features.

For supervised learning, datasets usually contain:

- a feature matrix X ;
- a target variable y .

For unsupervised learning, we typically observe only the feature matrix X .

Since this course mainly focuses on tabular data, we will usually treat:

$$X \in \mathbb{R}^{n \times p}, \quad y \in \mathbb{R}^n.$$

 columns correspond to variables

The key feature of tabular data is NOT merely the matrix representation. More importantly, each column corresponds to a variable with its own semantic meaning, such as age, income, blood pressure, or transaction count.

 Tip

Tabular data is particularly well suited for methods such as linear models and tree-based models. Therefore, these model families will be the main focus of this course.

1.3.2 Other Types of Data

Besides tabular data, many machine learning problems involve other data formats. We briefly mention several common examples that appear in modern learning tasks.

 Image Data

Image data contains spatial structure, meaning nearby pixels are highly related. Modern image models frequently use:

- convolutional neural networks,
- vision transformers.

A central goal is to extract meaningful visual patterns such as edges, textures, shapes, and objects.

 Text Data

Text data contains sequential and contextual structure. Modern NLP is largely based on transformers and large language models.

i Time Series Data

Time series data is ordered over time such as stock prices, weather data and sensor measurements.

i Graph Data

Graph data represents relationships between objects, such as social networks, citation networks, molecular structures.

Modern graph learning often uses graph neural networks.

1.4 Generalization and Model Complexity

1.4.1 Training Error and Test Error

Classical regression focuses heavily on checking model assumptions, such as independence of errors, constant variance, approximate normality. These assumptions are especially important for formal inference, including confidence intervals and hypothesis testing.

In statistical learning, however, the primary goal is often **predictive performance** rather than formal inference. As a result, diagnostic tools still remain useful, but they usually play a secondary role. Their main purpose is often to detect serious issues such as strong nonlinearity, influential observations, data quality problems, severe model misspecification.

The central question becomes: How well does the model predict new observations?

Definition 1.1 (Training Error and Test Error).

- The training error measures how well a model fits the training data.
- The test error measures prediction error on new observations not used to fit the model.

For example, suppose we evaluate a regression model using mean squared error (MSE).

The **training MSE** measures fit on the observed training sample, while the **test MSE** measures prediction performance on new data drawn from the same population.

A model may achieve extremely small training error while still performing poorly on unseen data. This happens because the model begins fitting random noise rather than the underlying signal.

Definition 1.2 (Overfitting). Overfitting occurs when a model fits the training data very closely but fails to generalize well to new data. In this case, the model captures random noise or idiosyncrasies in the training sample rather than the underlying relationship.

Model selection methods attempt to balance model complexity and prediction accuracy in order to avoid overfitting.

1.4.2 Bias–Variance Tradeoff

A central idea behind generalization is the **bias–variance tradeoff**.

Suppose

$$Y = f(X) + \varepsilon, \quad \text{Var}(\varepsilon) = \sigma^2.$$

For a fitted estimator $\hat{f}$, the test mean squared error at x_0 satisfies

$$\mathbb{E} \left[(Y_0 - \hat{f}(x_0))^2 \right] = \text{Bias}^2(\hat{f}(x_0)) + \text{Var}(\hat{f}(x_0)) + \sigma^2.$$

The three terms represent:

- **bias**: systematic modeling error: $\text{Bias}(\hat{f}(x)) = \mathbb{E}[\hat{f}(x)] - f(x)$. It measures the systematic difference between the average prediction and the true underlying function;
- **variance**: sensitivity to the training sample: $\text{Var}(\hat{f}(x))$. It measures how much the fitted model changes when the training data changes. Highly flexible models often have larger variance;
- **irreducible error**: σ^2 . This represents random noise or variability in the data generation process that cannot be explained or removed, even with the true model.

Roughly speaking, as model flexibility increases:

- training error usually decreases;
- test error often first decreases and then increases.

Model selection aims to balance these two effects in order to achieve good generalization performance.

i Main Point

Many modern methods can be viewed as ways to control the bias–variance tradeoff. Regularization, cross-validation, ensemble methods, trees, boosting, and neural networks can all be understood from this perspective.

i Information Criteria and Estimated Test Error

Classical regression often estimates expected test error indirectly using only the training data.

Some commonly used criteria are:

- AIC (Akaike Information Criterion),
- BIC (Bayesian Information Criterion),
- Mallows' C_p .

These criteria reward good fit while penalizing excessive model complexity.

Although they are useful for model comparison, they still rely on approximations derived from the training sample.

2 Classification Decisions and Evaluation

Unlike regression, where metrics such as Mean Squared Error (MSE) evaluate prediction error along a continuous scale, classification evaluation is fundamentally more nuanced.

A modern statistical learning classifier often produces a score or estimated probability rather than directly outputting a categorical decision. For example, a model may estimate

$$\hat{p}(x) = \Pr(Y = 1 \mid X = x),$$

and a separate decision rule then converts this soft prediction into a hard classification. As a result, classification performance can be evaluated from several different perspectives.

Broadly speaking, classification evaluation can be divided into three related but distinct goals.

Goal	Main question	Common tools
Decision quality	Are the final classification decisions good?	Accuracy, precision, recall, F1-score
Ranking quality	Are positive observations ranked above negative observations?	ROC curve, AUC, PR curve
Probability quality	Are predicted probabilities numerically trustworthy?	Calibration curve, Brier score, log loss

These goals are related but fundamentally different. Accurate probability estimates are the richest output: they can support ranking and threshold-based decisions. However, good decisions, good rankings, and calibrated probabilities are different evaluation goals and one does not generally imply the others.

If a model accurately estimates the true conditional probability distribution $\Pr(Y \mid X)$, then it naturally induces good rankings and supports effective decision-making across many possible thresholds or cost functions. However, the converse implications generally fail. Optimizing only a threshold-based metric such as accuracy provides no guarantee that the underlying probabilities are calibrated or that the ranking behavior is reliable.

In practice, however, decision quality is often easier to achieve and is frequently the primary objective in applied statistical learning. Many real-world applications ultimately require

concrete actions, such as approving a loan, detecting fraud, or diagnosing disease. Consequently, threshold-based metrics such as accuracy, precision, recall, and F1-score often play the central role in applied machine learning.

i Note

Classification evaluation can become quite sophisticated and may involve ranking performance, probability calibration, cost-sensitive learning, threshold optimization, and decision theory.

In this course, however, our primary goal is to understand the core ideas of statistical learning and predictive modeling rather than to develop a complete theory of statistical decision making. Therefore, we will mainly focus on accuracy and closely related threshold-based classification metrics.

2.1 Decision quality

Decision quality evaluates the final hard classifications after applying a threshold to predicted scores or probabilities.

This perspective focuses on whether the model makes useful decisions in practice. Metrics such as accuracy, precision, recall, and F1-score belong to this category.

2.1.1 Binary Classification Setup

Suppose the response is binary:

$$Y \in \{0, 1\}.$$

We call $Y = 1$ the positive class and $Y = 0$ the negative class.

A classifier produces a predicted label

$$\hat{Y} \in \{0, 1\}.$$

The four possible outcomes are summarized by the confusion matrix.

	Predicted positive	Predicted negative
True positive class	TP	FN
True negative class	FP	TN

Here, TP means true positive, FP means false positive, FN means false negative, TN means true negative.

The confusion matrix is the starting point for most threshold-based classification metrics.

2.1.2 Accuracy

Accuracy is the proportion of correctly classified observations:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}.$$

Accuracy is simple and useful when:

- the classes are reasonably balanced,
- false positives and false negatives have similar costs,
- the decision threshold is already fixed and meaningful.

However, accuracy can be misleading.

Suppose only 1% of observations are positive. A classifier that always predicts negative has 99% accuracy, but it never detects a positive case. In many applications, that classifier is useless.



Accuracy is not enough

Accuracy implicitly treats all mistakes as equally costly.

When classes are imbalanced or error costs are asymmetric, accuracy can hide the most important behavior of the classifier.

2.1.3 Recall (sensitivity)

Recall is defined as

$$\text{Recall} = \frac{TP}{TP + FN}.$$

Recall answers the question:

Among all truly positive observations, how many did we detect?

In probability notation,

$$\text{Recall} = P(\hat{Y} = 1 \mid Y = 1).$$

Recall is also called sensitivity, or true positive rate (TPR).

High recall means few false negatives. Low recall means many true positive cases were missed.

i Recall and Statistical Power

Recall has a direct connection to hypothesis testing.

Suppose we think of the classification problem as testing

$$H_0 : Y = 0 \quad \text{versus} \quad H_1 : Y = 1.$$

Predicting positive is like rejecting H_0 . Predicting negative is like failing to reject H_0 . Under this interpretation:

- a false positive is a Type I error,
- a false negative is a Type II error,
- recall plays a role analogous to statistical power.

Thus

$$\text{Recall} = 1 - \text{Type II error rate}.$$

Recall measures the ability to detect true signals.

2.1.4 Precision

Precision is defined as

$$\text{Precision} = \frac{TP}{TP + FP}.$$

Precision answers the question:

Among all observations predicted positive, how many are truly positive?

In probability notation,

$$\text{Precision} = P(Y = 1 \mid \hat{Y} = 1).$$

Precision is also called positive predictive value.

High precision means positive predictions are trustworthy. Low precision means many predicted positives are false alarms.

i Precision vs Recall

Precision and recall condition on different events.

Recall conditions on the truth:

$$P(\hat{Y} = 1 \mid Y = 1).$$

Precision conditions on the prediction:

$$P(Y = 1 \mid \hat{Y} = 1).$$

This distinction is very important.

Recall asks whether we find the positives. Precision asks whether our positive discoveries are reliable. Precision depends strongly on class prevalence.

2.1.5 F1 Score

The F1 score combines precision and recall:

$$F_1 = \frac{2PR}{P + R},$$

where P is precision and R is recall.

Equivalently,

$$F_1 = \frac{2TP}{2TP + FP + FN}.$$

The F1 score is the harmonic mean of precision and recall.

The harmonic mean strongly penalizes imbalance. If precision is high but recall is near zero, then F1 is near zero. If recall is high but precision is near zero, then F1 is also near zero.

F1 is useful when we want a single number that balances precision and recall. However, it assumes precision and recall are equally important. In applications where one type of error is more costly, a different metric or an explicit cost function may be better.

i Note

F1 ignores true negatives entirely.

This can be useful for rare event detection, where the number of true negatives may dominate the dataset and make accuracy misleading.

2.1.6 Recall vs Precision

2.1.6.1 Bayes' Rule and Class Imbalance

Precision and recall are connected by Bayes' rule.

Let $\pi = P(Y = 1)$ be the prevalence of the positive class.

Proposition 2.1.

$$\text{Precision} = P(Y = 1 \mid \hat{Y} = 1) = \frac{TPR \cdot \pi}{TPR \cdot \pi + FPR(1 - \pi)}.$$

This formula shows why precision can be low for rare events: when π is small, even a modest false positive rate can create many false positives.

Click to expand.

Let

$$TPR = P(\hat{Y} = 1 \mid Y = 1) = \text{Recall}$$

and

$$FPR = P(\hat{Y} = 1 \mid Y = 0).$$

Then by Bayes' Rule,

$$\begin{aligned} \text{Precision} &= P(Y = 1 \mid \hat{Y} = 1) \\ &= \frac{P(\hat{Y} = 1 \mid Y = 1)P(Y = 1)}{P(\hat{Y} = 1 \mid Y = 1)P(Y = 1) + P(\hat{Y} = 1 \mid Y = 0)P(Y = 0)} \\ &= \frac{TPR \cdot \pi}{TPR \cdot \pi + FPR \cdot (1 - \pi)}. \end{aligned}$$

Example 2.1 (Rare Event Example).

Click to expand.

Suppose there are 10,000 observations:

- 100 positives,
- 9,900 negatives.

Assume the classifier has:

$$TPR = 0.90, \quad FPR = 0.05.$$

Then:

$$TP = 90, \quad FP = 495.$$

The recall is high:

$$\text{Recall} = 0.90.$$

But precision is only

$$\text{Precision} = \frac{90}{90 + 495} \approx 0.154.$$

Most predicted positives are false positives. This is why precision-recall analysis is especially important for imbalanced data.

2.1.6.2 Classification Thresholds

Many classifiers first produce a score or estimated probability:

$$\hat{p}(x) = P(Y = 1 \mid X = x).$$

To make a hard classification, we choose a threshold c :

$$\hat{Y} = \begin{cases} 1, & \hat{p}(x) > c, \\ 0, & \hat{p}(x) \leq c. \end{cases}$$

The default decision threshold is often $c = 0.5$, but this is not always the optimal choice. Changing the threshold changes the confusion matrix, and therefore also changes metrics such as accuracy, precision, recall, false positive rate (FPR), and F1 score.

To better understand the precision–recall tradeoff, let us examine how altering the decision threshold changes the behavior of a classifier.

Decision Thresh- old	TP	TN	FP	FN	Recall	Precision	Classifier Behavior
Lower threshold	Non- decreasing	Non- increasing	Non- decreasing	Non- increasing	Non- decreasing	May decrease	
Higher threshold	Non- increasing	Non- decreasing	Non- increasing	Non- decreasing	Non- increasing	May increase	

Lowering the threshold classifies more observations as positive. Therefore, both true positives (TP) and false positives (FP) may increase, while false negatives (FN) decrease.

Raising the threshold classifies more observations as negative. Therefore, both true negatives (TN) and false negatives (FN) may increase, while true positives (TP) decrease.

Example 2.2 (Example: Precision Is Not Strictly Monotone).

Click to expand.

Consider a small validation dataset with five observations. The model outputs predicted probabilities $\hat{P}(Y = 1 | X)$, ordered from highest to lowest.

Instance	Predicted Probability	True Label
A	0.95	1
B	0.90	0
C	0.85	1
D	0.80	1
E	0.70	0

As the threshold is raised, fewer observations are classified as positive.

Threshold	Predicted Positives	TP	FP	Precision
0.75	A, B, C, D	3	1	$3/4 = 0.75$
0.82	A, B, C	2	1	$2/3 \approx 0.67$
0.88	A, B	1	1	$1/2 = 0.50$

Threshold	Predicted Positives	TP	FP	Precision
0.92	A	1	0	1/1 = 1.00

In this example, raising the threshold first decreases precision because some true positives are removed while the false positive remains. Precision increases only after the false positive is also removed.

This shows that precision can move up or down as the threshold changes, depending on the labels of the observations crossing the threshold.

i Thresholds are decisions

- The fitted model may estimate probabilities.
- The threshold determines the action.

Model fitting and decision making are related, but they are not the same thing.

2.1.6.3 Decision Theory and Error Costs

Suppose a false positive has cost C_{FP} and a false negative has cost C_{FN} .

If a model estimates

$$p(x) = P(Y = 1 \mid X = x),$$

then the expected cost of predicting positive is

$$\text{Cost}(1 \mid x) = C_{FP}P(Y = 0 \mid X = x) = C_{FP}(1 - p(x)).$$

The expected cost of predicting negative is

$$\text{Cost}(0 \mid x) = C_{FN}P(Y = 1 \mid X = x) = C_{FN}p(x).$$

We should predict positive when

$$C_{FP}(1 - p(x)) < C_{FN}p(x).$$

Solving this inequality gives

$$p(x) > \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

Thus the optimal threshold is

$$c^* = \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

This threshold minimizes the expected classification cost under the assumed probability model.

If false negatives are very costly, then C_{FN} becomes large, which lowers the threshold. In this case, the classifier predicts the positive class more aggressively in order to reduce missed detections.

Example 2.3 (Medical Screening). In medical screening, missing a serious disease may be much worse than sending a healthy patient for an additional test. Therefore false negatives are costly.

The threshold should often be lower, so the classifier prioritizes recall.

Example 2.4 (Spam Filtering). In spam filtering, putting an important legitimate email into spam may be very costly to the user. Therefore false positives may be more costly.

The threshold should often be higher, so the classifier prioritizes precision.

2.2 Ranking quality

Ranking quality evaluates whether positive observations tend to receive higher scores than negative observations.

This perspective is important when the classification threshold is not fixed or when the main goal is prioritization rather than direct classification. ROC curves, AUC, and precision–recall curves are commonly used to evaluate ranking performance.

2.2.1 ROC

ROC stands for receiver operating characteristic. An ROC curve plots:

$$TPR = \frac{TP}{TP + FN} \quad \text{against} \quad FPR = \frac{FP}{FP + TN}$$

as the probability threshold varies from 0 to 1.

2.2.2 ROC and Hypothesis Testing

ROC curves have a natural hypothesis testing interpretation. As discussed before,

- TPR is power,
- FPR is Type I error rate.

Therefore an ROC curve shows

power versus Type I error rate

across all possible thresholds.

The ideal classifier reaches the upper-left corner:

$$FPR = 0, \quad TPR = 1.$$

A classifier close to the diagonal line behaves almost like random guessing.

2.2.3 AUC

AUC refers to the Area Under the ROC Curve. It summarizes ranking performance across all possible classification thresholds.

Let $s(X^+)$ be the score for a randomly selected positive observation, and let $s(X^-)$ be the score for a randomly selected negative observation. Then

$$AUC = P(s(X^+) > s(X^-)),$$

up to small corrections for ties.

Thus AUC has a clean probabilistic interpretation:

AUC is the probability that a randomly selected positive observation receives a higher score than a randomly selected negative observation.

This explains why AUC is threshold-free. It evaluates ranking quality rather than the quality of a particular classification decision.

Importantly, AUC evaluates relative ordering rather than absolute probability accuracy.

AUC is not calibration

AUC can be excellent even when predicted probabilities are numerically inaccurate. AUC only asks whether positive observations tend to receive higher scores than negative observations. It does not ask whether the predicted probabilities themselves are trustworthy or well calibrated.

2.2.4 Precision-Recall Curve

A precision-recall curve plots precision against recall as the classification threshold changes.

It is especially useful when the positive class is rare. In imbalanced datasets, an ROC curve may look strong because the false positive rate is divided by the large number of true negatives. However, precision directly measures how many predicted positives are actually positive.

Average precision summarizes the precision-recall curve as a single number. Like AUC, it evaluates ranking behavior across thresholds, but it focuses more directly on performance for the positive class.

2.3 Probability Quality

Probability quality asks whether predicted probabilities can be interpreted as actual probabilities.

For example, suppose a model predicts $\hat{p}(x) = 0.8$. If the model is well calibrated, then among many observations with predicted probability near 0.8, roughly 80% should truly belong to the positive class.

Thus probability quality concerns whether predicted probabilities are numerically trustworthy.

Unlike decision quality or ranking quality, probability quality focuses on the accuracy of the probability estimates themselves. Calibration curves, Brier score, and log loss are commonly used to evaluate probability quality.

Note

In many practical machine learning applications, probability quality is less important than classification accuracy or ranking performance.

Therefore, in this course, we will mainly focus on threshold-based classification metrics such as accuracy, precision, recall, and F1 score. This section is included primarily to provide a broader view of classification evaluation.

2.3.1 Calibration Curve

A calibration curve, also called a reliability diagram, is constructed as follows:

1. Divide observations into bins according to predicted probability.
2. For each bin, compute the average predicted probability.
3. Compute the observed positive proportion in each bin.
4. Plot observed proportion against average predicted probability.

For a perfectly calibrated model, the curve follows the diagonal line.

- If the curve lies below the diagonal, the model is overconfident.
- If the curve lies above the diagonal, the model is underconfident.

2.4 Model Tuning versus Threshold Tuning

In classification, model fitting and decision making should be viewed as separate problems.

First, the model estimates a score or probability:

$$\hat{p}(x) \approx \Pr(Y = 1 \mid X = x).$$

Second, we choose a threshold to convert this probability into an action.

Model tuning affects the quality of $\hat{p}(x)$ itself. This is where ideas such as bias, variance, regularization, and model complexity matter. A very flexible classifier may capture nonlinear structure, but its probability estimates may be unstable or poorly calibrated.

Threshold tuning does not change $\hat{p}(x)$. It only changes the decision rule built on top of the fitted probabilities. Lower thresholds usually increase recall and false positives; higher thresholds usually increase precision and reduce false positives.

Thus:

- model tuning controls the fitted scores or probabilities;
- threshold tuning controls the action taken from those scores;
- calibration checks whether the probabilities are numerically trustworthy.

A good classifier therefore involves not only learning accurate scores, but also choosing appropriate decision rules for the application.

2.5 Python Example: Breast Cancer Classification

We now use the [breast cancer dataset from sklearn](#). The goal is to classify tumors as malignant or benign. This example shows how to compute threshold-based metrics, ROC and PR curves, and calibration curves.

In this example, please mainly focus on the evaluation metrics. Other part like modeling, splitting, etc. will be introduced in later sections.

2.5.1 Load the dataset

First load the data and split it into training and test sets. By default, the dataset uses 0 for malignant and 1 for benign. Since many classification metrics interpret label 1 as the positive class, we redefine malignant as 1. Therefore we need to modify the y value.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

cancer = load_breast_cancer(as_frame=True)

X = cancer.data
y_original = cancer.target
y = 1 - y_original

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=1
)
```

We may want to see the distribution of the test set.

```
import pandas as pd
pd.Series(y_test).value_counts().sort_index()
```

```
target
0      107
1       64
Name: count, dtype: int64
```

2.5.2 Fit models

We fit logistic regression model as an example, which is a typical strong baseline model.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logit = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5000),
)

logit.fit(X_train, y_train)

logit_prob = logit.predict_proba(X_test)[: , 1]
```

Note that `logit.predict_proba(X_test)` returns 2 columns:

- column 0: $\Pr(Y = 0 \mid X)$,
- column 1: $\Pr(Y = 1 \mid X)$.

Since we want the predicted probability for the positive class ($y=1$), we select the second column.

2.5.3 Confusion Matrix and Basic Metrics

All decision quality metrics can be computed using functions from `sklearn.metrics`. They are used to compare two list-like objects, while the true labels are passed first and the predicted labels second.

By default, `predict()` in `sklearn` uses threshold 0.5 for binary classification.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

y_pred = logit.predict(X_test)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

We can also compute the confusion matrix and extract the values of TP, TN, FP, and FN from it.

```
from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, y_pred)
```

```
array([[106,   1],
       [  6,  58]])
```

```
TN, FP, FN, TP = confusion_matrix(y_test, y_pred).ravel()
```

They are listed in this particular order since 0 stands for negative and 1 stands for positive in this example.

2.5.4 Modifying Thresholds

The default model use 0.5 as the threshold in `sklearn`. If you want to change the threshold, you may manually write the code.

```
threshold = 0.3
logit_prob = logit.predict_proba(X_test)[:, 1]
y_pred = (logit_prob >= threshold).astype(int)
```

You may also use `FixedThresholdClassifier` from `sklearn.model_selection` when you want the threshold to be part of the estimator workflow.

```
from sklearn.model_selection import FixedThresholdClassifier

threshold = 0.3
model_with_threshold = FixedThresholdClassifier(
    make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=5000),
    ),
    threshold=threshold,
)

model_with_threshold.fit(X_train, y_train)
y_pred_wrapper = model_with_threshold.predict(X_test)
```

Note

`FixedThresholdClassifier` is useful when the threshold is fixed in advance. However, there might be cases that you want to adjust the threshold according to the dataset. In this case you may want to use `TunedThresholdClassifierCV`. We will talk about it in details later in the Logistic regression section.

2.5.5 Tuning Thresholds

In order to make the tuning process easier, we build an interface to quickly display the results.

```
def classification_summary(y_true, prob, threshold=0.5):
    pred = (prob >= threshold).astype(int)
    TN, FP, FN, TP = confusion_matrix(y_true, pred).ravel()

    return {
        "threshold": threshold,
        "TP": TP,
        "FP": FP,
        "FN": FN,
        "TN": TN,
        "accuracy": accuracy_score(y_true, pred),
        "precision": precision_score(y_true, pred, zero_division=0), ①
        "recall": recall_score(y_true, pred, zero_division=0),
        "f1": f1_score(y_true, pred, zero_division=0),
    }
```

- ① `zero_division=0` means that if the denominator is 0 the return value will be set to 0. This always happens in this case since some of the counts might be 0.

Now examine how metrics change as the threshold changes. To save space I only show the first few rows of the table.

```
import numpy as np
import pandas as pd

thresholds = np.linspace(0.05, 0.95, 19)
threshold_results = []

for threshold in thresholds:
    row = classification_summary(y_test, logit_prob, threshold=threshold)
    threshold_results.append(row)
```

```
threshold_df = pd.DataFrame(threshold_results)
```

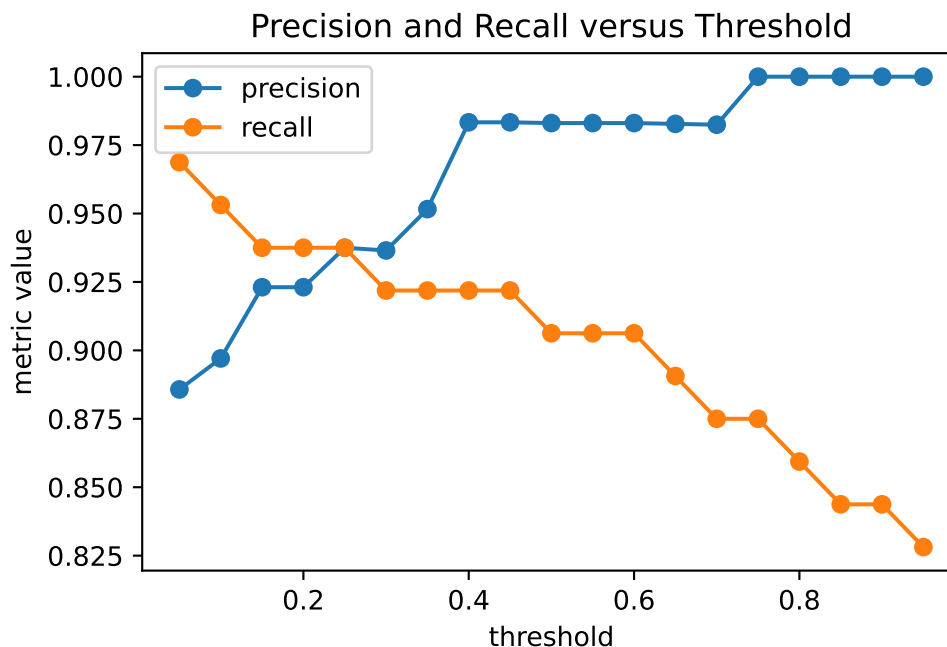
```
threshold_df.head()
```

	threshold	TP	FP	FN	TN	accuracy	precision	recall	f1
0	0.05	62	8	2	99	0.941520	0.885714	0.968750	0.925373
1	0.10	61	7	3	100	0.941520	0.897059	0.953125	0.924242
2	0.15	60	5	4	102	0.947368	0.923077	0.937500	0.930233
3	0.20	60	5	4	102	0.947368	0.923077	0.937500	0.930233
4	0.25	60	4	4	103	0.953216	0.937500	0.937500	0.937500

Plot precision and recall as functions of the threshold.

```
import matplotlib.pyplot as plt
```

```
plt.plot(threshold_df["threshold"], threshold_df["precision"], marker="o", label="precision")
plt.plot(threshold_df["threshold"], threshold_df["recall"], marker="o", label="recall")
plt.xlabel("threshold")
plt.ylabel("metric value")
plt.title("Precision and Recall versus Threshold")
plt.legend()
```



As the threshold increases, the classifier becomes more conservative in predicting the positive class. This often reduces false positives and increases precision, but may also increase false negatives and reduce recall.

2.5.6 Choosing a Threshold from Costs

In a medical setting, false negatives may be especially important because they correspond to missed malignant cases. Suppose a false negative is 5 times as costly as a false positive:

$$C_{FN} = 5, \quad C_{FP} = 1.$$

The decision-theoretic threshold is

$$c^* = \frac{C_{FP}}{C_{FP} + C_{FN}} = \frac{1}{1 + 5} \approx 0.167.$$

Then the corresponding result is

```
cost_fp = 1
cost_fn = 5

cost_threshold = cost_fp / (cost_fp + cost_fn)

classification_summary(
    y_test,
    logit_prob,
    threshold=cost_threshold,
)
```

```
{'threshold': 0.16666666666666666,
 'TP': np.int64(60),
 'FP': np.int64(5),
 'FN': np.int64(4),
 'TN': np.int64(102),
 'accuracy': 0.9473684210526315,
 'precision': 0.9230769230769231,
 'recall': 0.9375,
 'f1': 0.9302325581395349}
```

This lower threshold prioritizes recall. It predicts malignant more aggressively because missing a malignant tumor is assumed to be more costly. Note that this threshold formula assumes that the predicted probabilities are reasonably calibrated so that they can be interpreted as approximate event probabilities.

2.5.7 ROC Curve and AUC

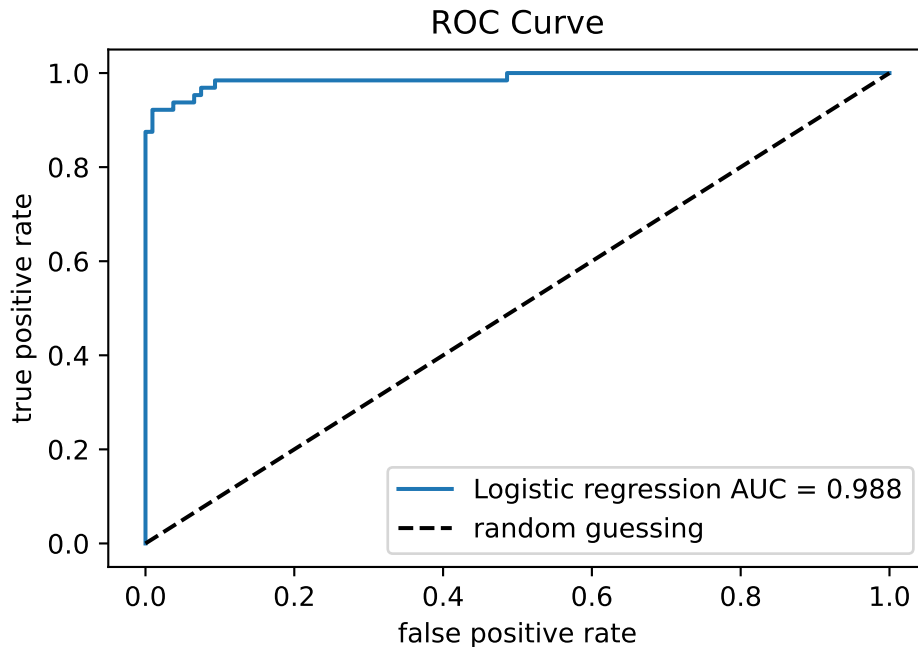
Unlike accuracy or precision, ROC analysis does not depend on a single fixed threshold. Instead, it studies model performance across all possible thresholds using the predicted probabilities or scores.

ROC curve and AUC can be computed using `roc_curve` and `roc_auc_score` from `sklearn.metrics`.

```
from sklearn.metrics import roc_curve, roc_auc_score

logit_fpr, logit_tpr, _ = roc_curve(y_test, logit_prob)
logit_auc = roc_auc_score(y_test, logit_prob)

plt.plot(logit_fpr, logit_tpr, label=f"Logistic regression AUC = {logit_auc:.3f}")
plt.plot([0, 1], [0, 1], "k--", label="random guessing")
plt.xlabel("false positive rate")
plt.ylabel("true positive rate")
plt.title("ROC Curve")
plt.legend()
```



We may fit another model and draw their ROC curve and AUC together in one plot to compare them.

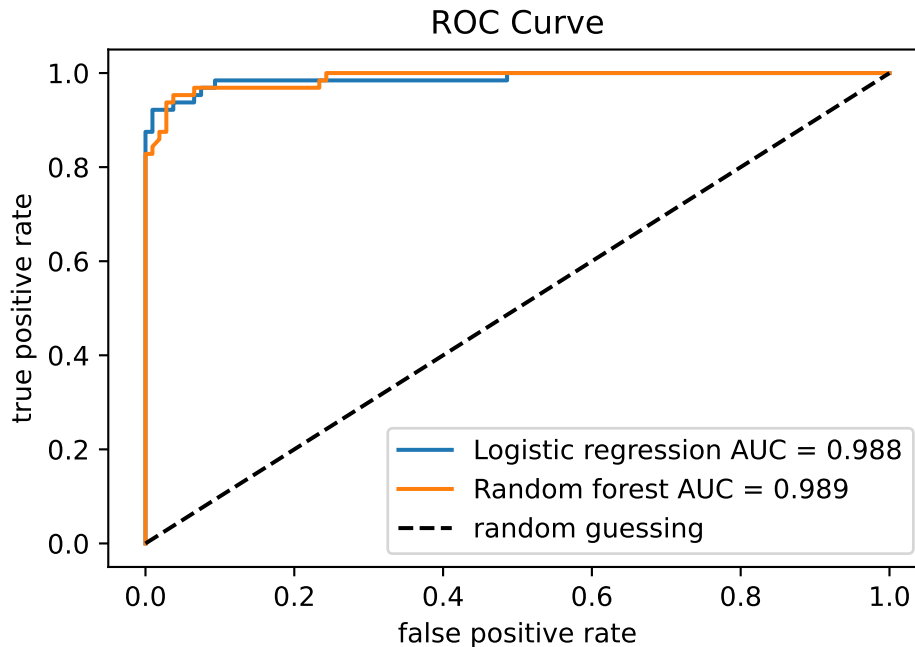
```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=500, max_features="sqrt", random_state=1, n_jobs=-1
)

rf.fit(X_train, y_train)
rf_prob = rf.predict_proba(X_test)[: , 1]

rf_fpr, rf_tpr, _ = roc_curve(y_test, rf_prob)
rf_auc = roc_auc_score(y_test, rf_prob)

plt.plot(logit_fpr, logit_tpr, label=f"Logistic regression AUC = {logit_auc:.3f}")
plt.plot(rf_fpr, rf_tpr, label=f"Random forest AUC = {rf_auc:.3f}")
plt.plot([0, 1], [0, 1], "k--", label="random guessing")
plt.xlabel("false positive rate")
plt.ylabel("true positive rate")
plt.title("ROC Curve")
plt.legend()
```

The ROC curve summarizes ranking performance across thresholds. AUC may be interpreted as the probability that a randomly chosen positive observation receives a higher score than a randomly chosen negative observation. Then a larger AUC means the model more often ranks malignant cases above benign cases.

2.5.8 Precision-Recall Curve

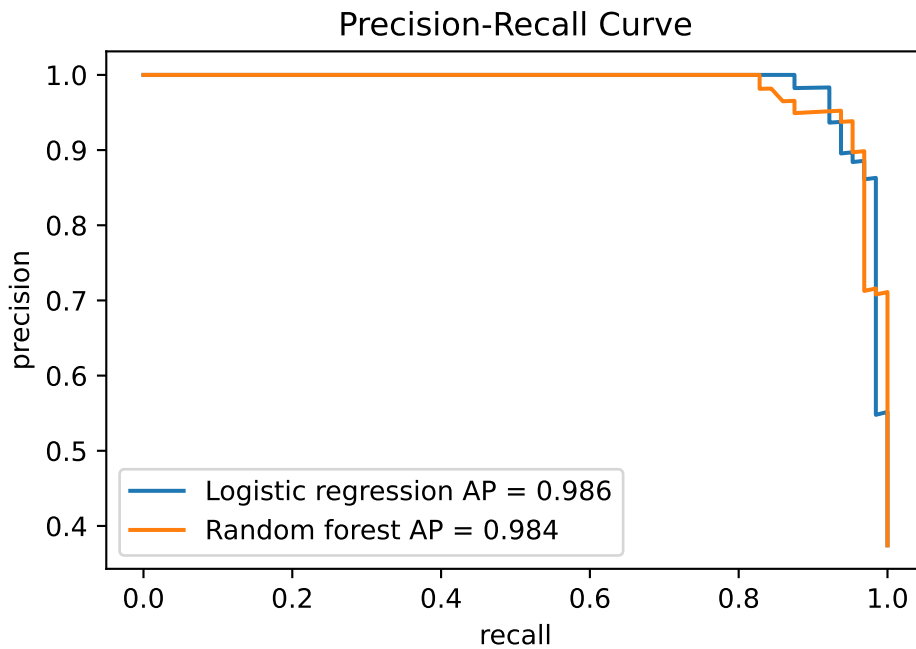
The precision-recall curve can be produced in a similar way.

```
from sklearn.metrics import precision_recall_curve, average_precision_score

logit_precision, logit_recall, _ = precision_recall_curve(y_test, logit_prob)
rf_precision, rf_recall, _ = precision_recall_curve(y_test, rf_prob)

logit_ap = average_precision_score(y_test, logit_prob)
rf_ap = average_precision_score(y_test, rf_prob)

plt.plot(logit_recall, logit_precision, label=f"Logistic regression AP = {logit_ap:.3f}")
plt.plot(rf_recall, rf_precision, label=f"Random forest AP = {rf_ap:.3f}")
plt.xlabel("recall")
plt.ylabel("precision")
plt.title("Precision-Recall Curve")
plt.legend()
```



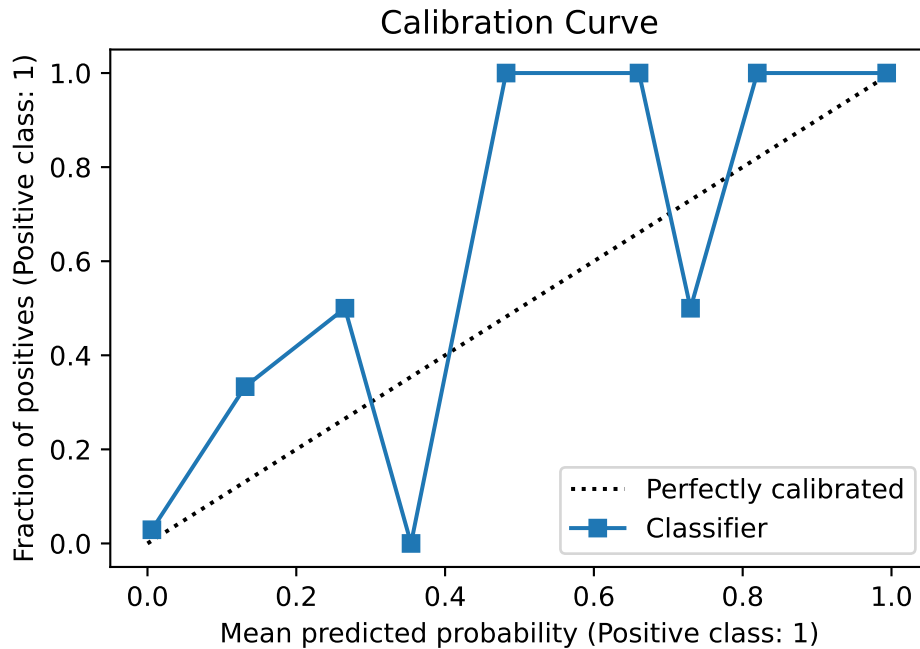
For highly imbalanced datasets, precision–recall curves are often more informative than ROC curves because ROC curves may appear overly optimistic when the negative class dominates.

2.5.9 Calibration Curves

Calibration focuses on probability quality rather than threshold-based decision quality or ranking quality. Calibration curves can be produced using `CalibrationDisplay` from `sklearn.calibration`.

```
from sklearn.calibration import CalibrationDisplay

CalibrationDisplay.from_predictions(y_test, logit_prob, n_bins=10)
plt.title("Calibration Curve");
```

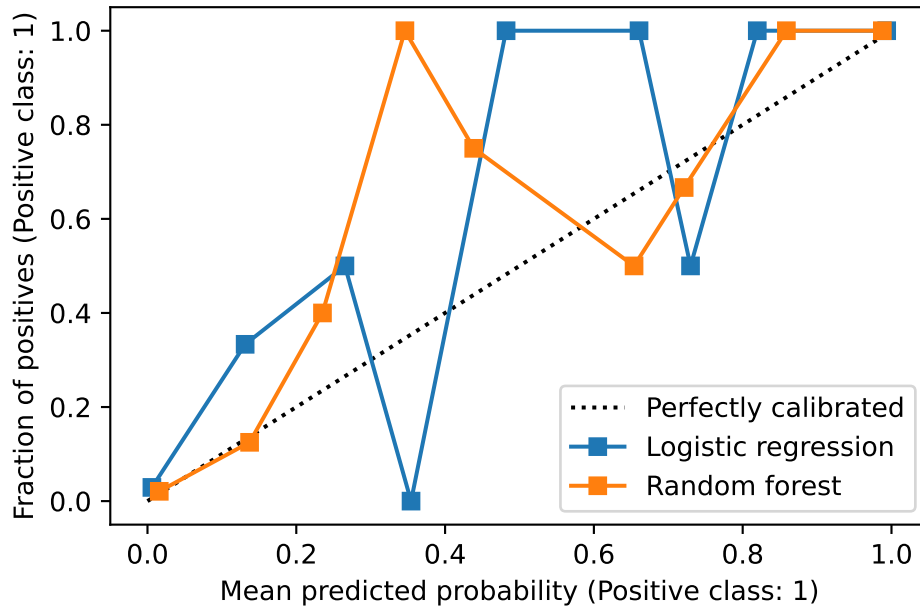


This is a packaged function provided by `sklearn`. If we want to plot multiple curves together in one plot, we can put the axis in the argument of this function.

```
fig, ax = plt.subplots()

CalibrationDisplay.from_predictions(
    y_test, logit_prob, n_bins=10, name="Logistic regression", ax=ax
)

CalibrationDisplay.from_predictions(
    y_test, rf_prob, n_bins=10, name="Random forest", ax=ax
)
```



The calibration curve compares predicted probabilities with observed frequencies.

Calibration curves diagnose probability quality; they do not by themselves recalibrate the model. Recalibration methods such as Platt scaling or isotonic regression can change the probability scale, but they do not necessarily improve AUC because AUC mainly depends on ranking.

Part II

Parametric Supervised Learning

3 Linear Regression

This section offers a brief review of linear regression, with the main emphasis on its Python implementation through examples.

3.1 Quick Overview

Linear regression is the fundamental supervised learning method for a quantitative response. It models the conditional mean of the response as a linear function of the predictors:

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \varepsilon.$$

In matrix notation, this can be written as

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon.$$

Here, $\mathbf{X}$ is the design matrix, whose first column is a column of ones representing the intercept.

3.2 The Ordinary Least Squares Estimator

The most common fitting method is ordinary least squares (OLS). It chooses the coefficient vector β that minimizes the residual sum of squares (RSS):

$$\text{RSS}(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \|\mathbf{y} - \mathbf{X}\beta\|_2^2.$$

Theorem 3.1 (OLS Estimator). *The coefficient vector that minimizes RSS is*

$$\hat{\beta}_{\text{OLS}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y},$$

provided that $\mathbf{X}^\top \mathbf{X}$ is invertible. This estimator is called the ordinary least squares (OLS) estimator.

Once the coefficients are estimated, the fitted value for observation i is

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}.$$

Interpretability is one of the main advantages of linear regression compared with many more complex models.

1. The coefficient $\hat{\beta}_j$ describes how the predicted response changes when x_j increases by one unit, while the other predictors are held fixed.
2. When interaction terms are present, a main effect cannot be interpreted in isolation. The effect of a predictor depends on the values of the variables with which it interacts.
3. If predictors are centered so that 0 corresponds to the average of the original variable, the intercept represents the predicted response at the average predictor levels.

i Note

Classical regression primarily focuses on inference and model assumptions, such as normality, constant variance, and independence of errors.

As discussed earlier in the statistical learning basics section, statistical learning places greater emphasis on predictive performance and generalization to unseen data. As a result, estimated test error and model validation often play a more central role than formal inference procedures.

3.3 Linear Regression in Python

There are many Python libraries for fitting linear regression models. In this course, we use [scikit-learn](#) as the primary modeling tool, with support from [pandas](#) and [numpy](#) for data manipulation, and [matplotlib](#) and [seaborn](#) for visualization.

To demonstrate the code, we generate a dataset with 2 predictors and 1 response variable, where $\beta_0 = 1$, $\beta_1 = 2$, and $\beta_2 = 3$.

```
import numpy as np
import pandas as pd

np.random.seed(1)

n = 100

x1 = np.random.normal(1, 0.1, size=n)
x2 = np.random.normal(0, 1, size=n)
```

```
y = 1 + 2 * x1 + 3 * x2 + np.random.normal(0, 0.5, size=n)

X = pd.DataFrame({'x1': x1, 'x2': x2})
```

When using `sklearn`, a dataset is typically represented by features `X` and a target variable `y`.

- `X` usually has 2 dimensions (number of samples $\times$ features).
- `y` is usually 1-dimensional, though it may also be represented as a 2-dimensional array with a single column.

Both `X` and `y` can be stored as either `numpy` arrays or `pandas` objects.

In this example we construct `X` as a `DataFrame`, since column names make it easier to modify variables during the model-building stage, e.g. when adding interactions or transformations.

3.3.1 Linear regression models

We fit a linear regression model using `LinearRegression`.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, y)

print(f"intercept: {model.intercept_}")
print(f"coefficients: {model.coef_}")
```

```
intercept: 1.2995891152586192
coefficients: [1.69051379 3.10918568]
```

We can then use the fitted model to generate predictions and compute residuals.

```
y_pred = model.predict(X)
res = y - y_pred
```

3.3.2 Measure model performance

`sklearn` provides many performance measures, most of which can be found in `sklearn.metrics`. A common syntax is `metric(y_true, y_pred)`.


```
from sklearn.metrics import mean_squared_error, r2_score, root_mean_squared_error

y_pred = model.predict(X)

print(f"MSE: {mean_squared_error(y, y_pred)}")
print(f"R2: {r2_score(y, y_pred)}")
print(f"RMSE: {root_mean_squared_error(y, y_pred)}")
```

Each model also provides a built-in scoring method. For a specific model, you can check the official documentation to see which metric is used, but the general idea is:

- regressors use `r2_score`, and
- classifiers use `accuracy`

```
model.score(X, y)
```

0.9723249582312465

3.3.3 Model Diagnostics

In traditional regression, diagnostic plots are powerful tools for detecting potential problems in a fitted model. However, statistical learning primarily focuses on prediction performance rather than model assumptions. Therefore, diagnostics are not our primary focus in this course, and this section is optional.

[Click for more details.](#)

In Python there is no single function that automatically produces all diagnostic plots. Instead, we usually combine tools from several libraries. In these notes we focus on basic tools from `statsmodels`, `scipy`, `seaborn`, and `matplotlib`.

We can also use `seaborn`'s theme to improve the appearance of plots produced by `matplotlib`-based tools.

```
import seaborn as sns
sns.set_theme()
```

We compute several quantities needed for residual analysis. Although our primary modeling tool is `sklearn`, residual diagnostics are easier with `statsmodels`. Therefore we construct a `statsmodels` model that holds the same data.

Since we already have the feature matrix and the target vector, the fastest way to use `statsmodels` is through `statsmodels.api` (as opposed to the R-style interface `statsmodels.formula.api`).

```
import statsmodels.api as sm
X_sm = sm.add_constant(X)
model = sm.OLS(y, X_sm).fit()
model.params
```

```
const    1.299589
x1        1.690514
x2        3.109186
dtype: float64
```

The fitted values, residuals, leverage, Cook's distance, and other quantities can be extracted directly from this `model` object. We wrap them into `DataFrames` for easier plotting.

```
y_pred = model.fittedvalues
res = model.resid

model_output = pd.DataFrame({
    "y_true": y,
    "y_pred": y_pred,
    "res": res
})
```

```
influence = model.get_influence()

std_resid = influence.resid_studentized_internal
sqrt_std_resid = np.sqrt(np.abs(std_resid))
leverage = influence.hat_matrix_diag
cooks = influence.cooks_distance[0]

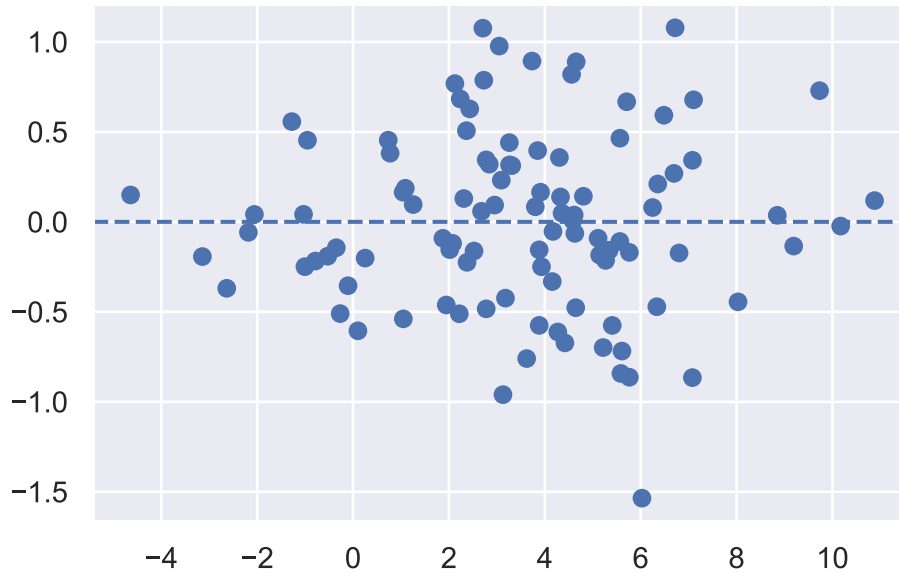
df_diag = pd.DataFrame({
    "leverage": leverage,
    "std_resid": std_resid,
    "sqrt_std_resid": sqrt_std_resid,
    "cooks": cooks
})
```

i Residual plots

The most basic residual plot is simply residuals versus fitted values.

```
import matplotlib.pyplot as plt

plt.scatter(y_pred, res)
plt.axhline(0, linestyle="--")
```

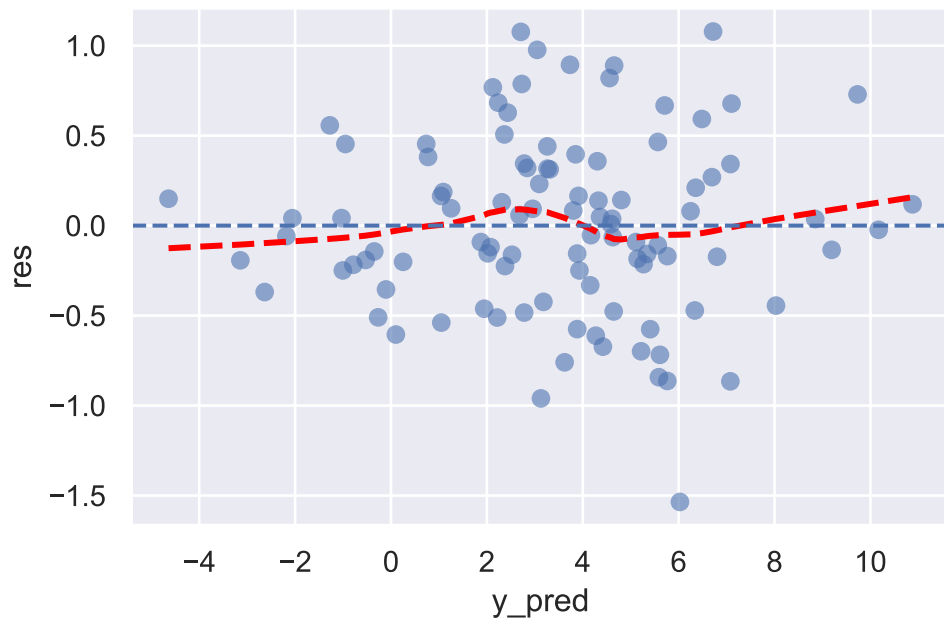


If we use `seaborn`, additional features are computed automatically. For example, setting `lowess=True` will display a smooth trend curve.

```
import seaborn as sns

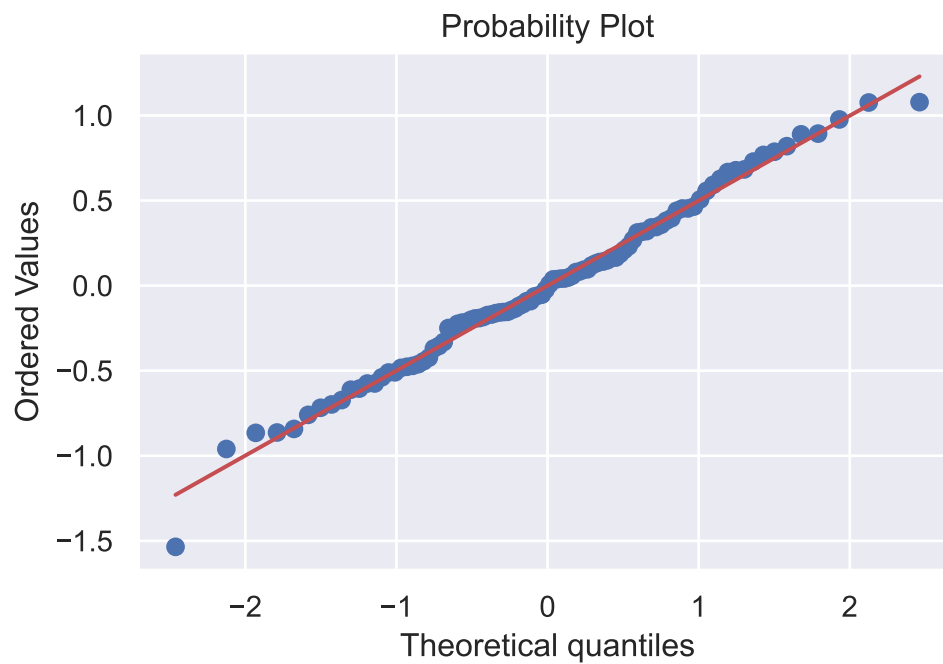
sns.regplot(
    data=model_output,
    x="y_pred",
    y="res",
    lowess=True,
    scatter_kws={"alpha": 0.6},
    line_kws={"color": "red", "linestyle": "--"},
)

plt.axhline(0, linestyle="--")
```



i Q-Q plot

```
import scipy.stats as stats  
  
stats.probplot(res, dist="norm", plot=plt);
```



i Scale-location plot

```
import numpy as np

sns.scatterplot(x=y_pred, y=sqrt_std_resid, data=df_diag)

sns.regplot(
    x=y_pred,
    y=sqrt_std_resid,
    data=df_diag,
    lowess=True,
    scatter=False,
    line_kws={"color": "red"}
)

plt.xlabel("Fitted values")
plt.ylabel(" $\sqrt{|\text{Standardized residuals}|}$ ")
plt.title("Scale-Location");
```

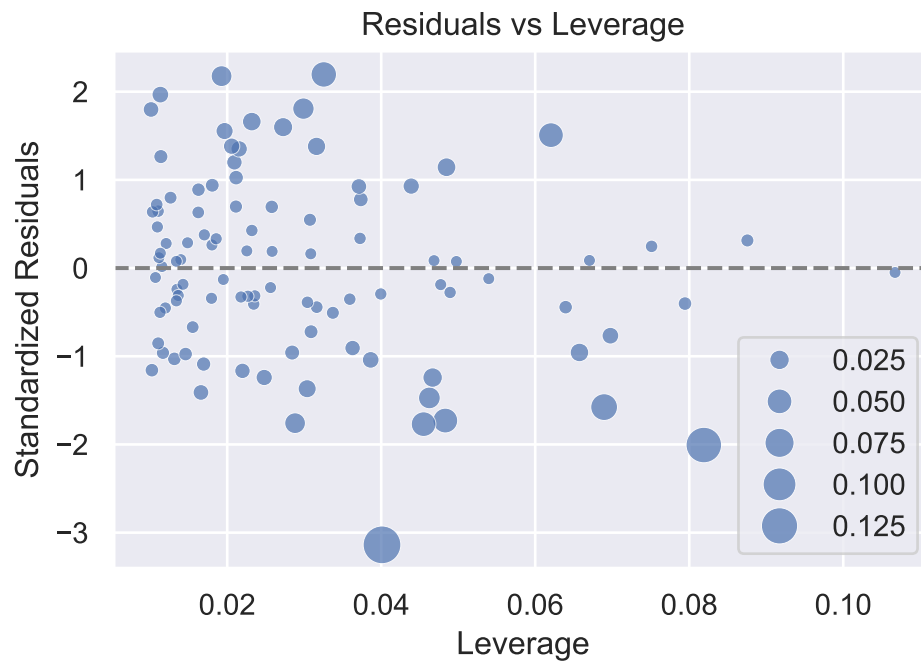


i Leverage with Cook's distance

```
sns.scatterplot(
    x=leverage,
    y=std_resid,
    data=df_diag,
    size=cooks,
    sizes=(20, 200),
    alpha=0.7
)

plt.axhline(0, linestyle="--", color="gray")

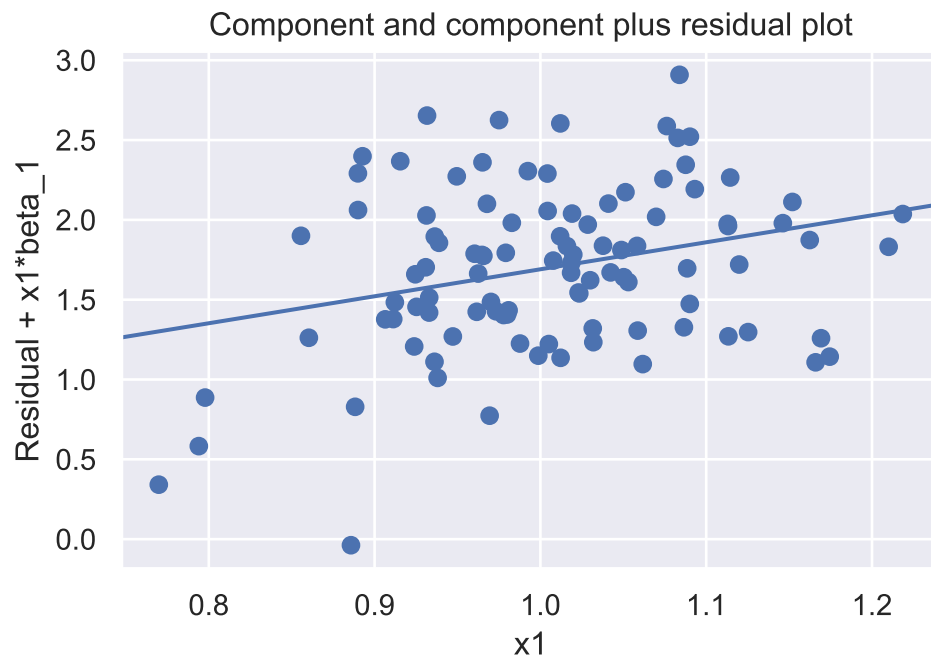
plt.xlabel("Leverage")
plt.ylabel("Standardized Residuals")
plt.title("Residuals vs Leverage");
```



i Partial residual plots

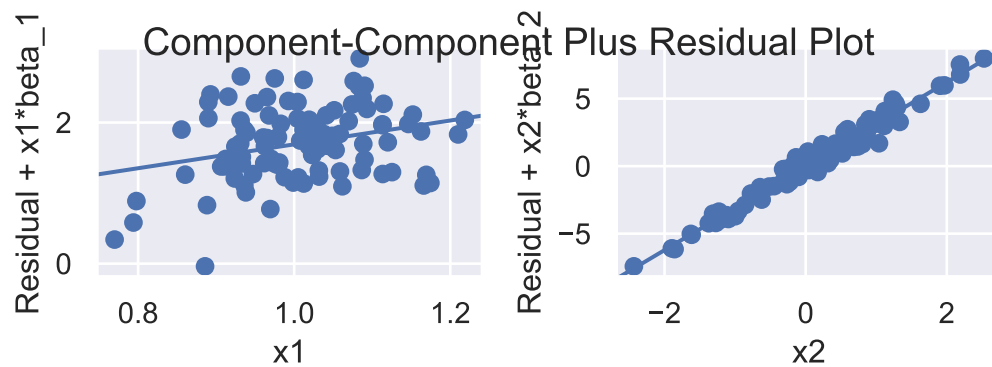
Partial residual plots help visualize the relationship between a predictor and the response after adjusting for the effects of other predictors. `plot_ccpr()` from `sm.graphics` can be used to generate partial residual plots.

```
sm.graphics.plot_ccpr(model, "x1");
```



To generate plots for all predictors at once, we can use

```
sm.graphics.plot_ccpr_grid(model);
```



The default `plot_ccpr()` does not include a smooth curve. If a smooth trend line is desired, we can compute it using the `lowess()` function and add it manually.


```

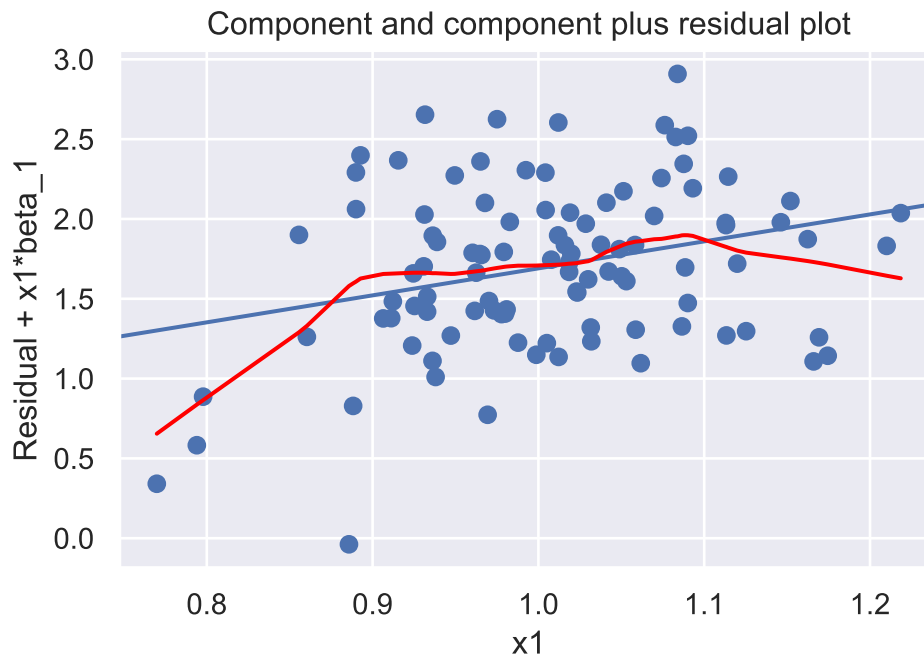
from statsmodels.nonparametric.smoothers_lowess import lowess

fig = sm.graphics.plot_ccpr(model, "x1")

ax = fig.axes[0]
x = ax.lines[0].get_xdata()
y = ax.lines[0].get_ydata()

smooth = lowess(y, x, frac=0.5)
ax.plot(smooth[:, 0], smooth[:, 1], color="red")

```



The same idea can be applied to the grid version.

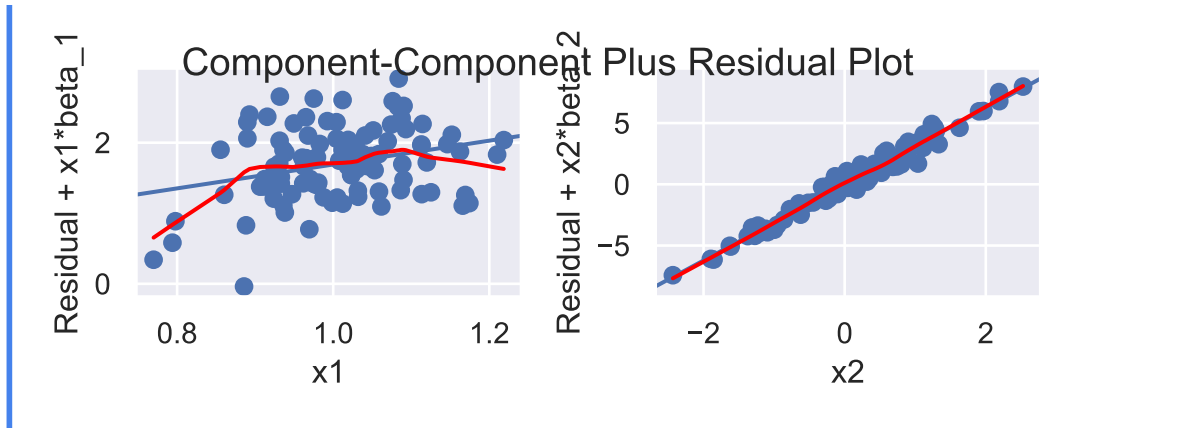
```

fig = sm.graphics.plot_ccpr_grid(model)

for ax in fig.axes:
    if len(ax.lines) != 0:
        x = ax.lines[0].get_xdata()
        y = ax.lines[0].get_ydata()

        smooth = lowess(y, x, frac=0.5)
        ax.plot(smooth[:, 0], smooth[:, 1], color="red")

```



3.4 Model evaluation

3.4.1 Train-Test Split

Instead of estimating test performance indirectly from the training data, we can evaluate a model more directly by using data that were not used to fit the model. This leads to the idea of data splitting:

The dataset is divided into two parts:

- Training set: used to fit the model.
- Test set: used to evaluate prediction performance.

After fitting the model on the training set, predictions are made on the test set, and performance measures such as MSE are computed. This provides a more realistic estimate of how the model performs on new data.

In many situations we also need to choose between several competing models or tuning parameters. To avoid using the test data for model selection, a third subset is often introduced.

- Training set: used to fit models.
- Validation set: used to compare models and select tuning parameters.
- Test set: used only once at the end to estimate the final model's performance.

This separation helps prevent overly optimistic performance estimates.

In `sklearn`, the function `train_test_split()` from `sklearn.model_selection` is commonly used to split the data into a training set and a test set.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

n = 10

rng = np.random.default_rng(1)
x1 = rng.normal(loc=1, scale=0.1, size=n)
x2 = rng.normal(loc=0, scale=1, size=n)
y = 1 + 2 * x1 + 3 * x2 + rng.normal(0, 0.5, size=n)

X = pd.DataFrame({'x1': x1, 'x2': x2})

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

The argument `test_size=0.2` means that 20% of the data are used as the test set, while the remaining 80% are used as the training set. The argument `random_state=0` ensures reproducibility.

3.4.2 Cross-Validation

When the dataset is not large, splitting the data into separate training, validation, and test sets may leave too little data for model fitting. In such cases, cross-validation is commonly used to estimate validation performance more efficiently.

The most common form is k -fold cross-validation.

1. The available training data are divided into k roughly equal parts (called folds).
2. For each fold:
 - The model is trained on $k - 1$ folds.
 - The remaining fold is used as a validation set.
3. This process is repeated k times so that each fold serves once as the validation set.
4. The validation errors from all folds are averaged to estimate the model's expected test error.

Cross-validation is primarily used to estimate test error or to compare competing models. When it is used for model selection, two additional steps are typically performed. Note that cross-validation uses only the training data and does not involve the test set.

5. After the best model configuration has been selected, the model is retrained on the entire training data.
6. The final model is then evaluated once on the test set, which serves as an unbiased estimate of its performance on new data.

In `sklearn`, there are several ways to perform k -fold cross-validation. From `KFold` to `cross_validate` and then `cross_val_score`, the interface becomes easier to use, but also less flexible.

The three approaches perform essentially the same task, but at different levels of abstraction.

KFold

`KFold` from `sklearn.model_selection` is used to split the dataset into k groups, called folds. In each iteration, one fold is used as the validation set and the remaining folds are used as the training set.

```
from sklearn.model_selection import KFold

kf = KFold(n_splits=5)

for train_idx, val_idx in kf.split(range(10)):
    print(train_idx, val_idx)
```

```
[2 3 4 5 6 7 8 9] [0 1]
[0 1 4 5 6 7 8 9] [2 3]
[0 1 2 3 6 7 8 9] [4 5]
[0 1 2 3 4 5 8 9] [6 7]
[0 1 2 3 4 5 6 7] [8 9]
```

- Here I use `range(10)` inside `kf.split()` because only the indices are needed. If a dataset is passed instead, the output is still the indices for the training set and validation set.
- If you want the split to be randomized, set `shuffle=True` when creating `KFold`. In that case, you may also specify `random_state` to make the result reproducible.

Let us now apply `KFold` to our simulated dataset.

```

from sklearn.model_selection import KFold
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

kf = KFold(n_splits=5, shuffle=True, random_state=1)

cv_scores = []
for train_idx, val_idx in kf.split(X):
    X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
    y_train, y_val = y[train_idx], y[val_idx]

    model = LinearRegression()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_val)
    mse = r2_score(y_val, y_pred)

    cv_scores.append(mse)

print(cv_scores)
print(sum(cv_scores) / len(cv_scores))

```

```

[-1.473916411368759, 0.3859310045149441, -0.7579033227389274, 0.7043154482687257, 0.882478
-0.051818894734030785

```

This gives the validation MSE for each fold, together with their average.

i cross_validate

Using `KFold` directly is somewhat manual. We may use `cross_validate()` to automate the cross-validation process. Depending on the arguments supplied, `cross_validate()` may internally use `KFold`.

You may specify which scoring methods using `scoring=[]` argument. The default choice will be the default choice from the model. In `LinearRegression` case, the default one is `r2_score`.

```
from sklearn.model_selection import cross_validate
```

```
model = LinearRegression()  
cv_result = cross_validate(model, X, y, cv=5)
```

```
cv_result
```

```
{'fit_time': array([0.00106692, 0.00075722, 0.0006671 , 0.00069594, 0.00063801]),  
 'score_time': array([0.00054526, 0.00048923, 0.00047827, 0.00045753, 0.00047588]),  
 'test_score': array([-0.39366753,  0.10843993,  0.60326202,  0.73431811,  0.91493437])}
```

If you are only interested in the validation scores, you may extract them directly.

```
cv_result['test_score']
```

```
array([-0.39366753,  0.10843993,  0.60326202,  0.73431811,  0.91493437])
```

- You may choose different scoring methods. More info can be found in [the document](#).
- Here the MSE values are negative because `sklearn` follows the convention that larger scores are better.
- If `cv=5`, then `KFold(5, shuffle=False)` is used by default. If you want randomized folds, you may pass a `KFold` object explicitly.

```
cv_result = cross_validate(  
    model, X, y, cv=KFold(5, shuffle=True, random_state=1)  
)
```

```
cv_result["test_score"]
```

```
array([-1.47391641,  0.385931 , -0.75790332,  0.70431545,  0.88247881])
```

You may compare these scores with the ones obtained in the `KFold` section. Usually, the overall cross-validation score is taken to be the mean of the fold scores.

```
cv_result['test_score'].mean()
```

```
np.float64(-0.051818894734030764)
```

i cross_val_score

`cross_val_score()` is a shorter way to obtain `cv_result["test_score"]` from the `cross_validate()` section. The arguments `cv` and `scoring` work in the same way.

```
from sklearn.model_selection import cross_val_score
```

```
cv_scores = cross_val_score(  
    model, X, y, cv=KFold(5, shuffle=True, random_state=1)  
)
```

```
cv_scores
```

```
array([-1.47391641,  0.385931  , -0.75790332,  0.70431545,  0.88247881])
```

```
cv_scores.mean()
```

```
np.float64(-0.051818894734030764)
```

So `cross_val_score()` is convenient when you only need the validation scores and do not need the extra information returned by `cross_validate()`.

Cross-validation is often used to compare competing models. One important application is hyperparameter tuning. In this setting, we compare models from the same family under different choices of hyperparameters. In `sklearn`, this process is commonly carried out with `GridSearchCV`.

The CV procedure is summarized (again) as follows:

1. Split the training data into several folds.
2. For each hyperparameter combination, hold out one fold as the validation set and use the remaining folds as the training set.
3. Fit the model on the training folds and evaluate its performance on the validation fold.
4. Repeat this process so that each fold serves once as the validation set. The average validation score is the cross-validation score for that hyperparameter combination.
5. Repeat Steps 2–4 for all hyperparameter combinations, and select the combination with the best average cross-validation score.
6. **Final step:** After the best hyperparameter combination is identified, the model is refit on the entire training set using these hyperparameters.

`GridSearchCV` automates this process. To illustrate the idea, suppose we want to decide whether a linear regression model should be fit with or without an intercept.

First, specify the hyperparameters to be tuned:

```
params = {"fit_intercept": [True, False]}
```

! Pipeline and ColumnTransformer

If the model is wrapped inside a `Pipeline` or a `ColumnTransformer` (discussed later), hyperparameters inside a step must be referred to using the syntax `step__param` with two underscores.

For example, suppose a pipeline contains a step named `linear`, and that step has a hyperparameter called `fit_intercept`. Then the parameter grid should be written as

```
params = {"linear__fit_intercept": [True, False]}
```

Next, construct the grid search.

- The argument `cv` is specified in the same way as in ordinary **cross-validation**.
- The argument `refit=True` controls whether the model is refit on the entire training set using the best hyperparameters. The default value is `True`.

```
from sklearn.model_selection import GridSearchCV

gs = GridSearchCV(model, param_grid=params, cv=5, refit=True)
gs.fit(X, y)
```

estimator estimator: estimator object	LinearRegression()
--	--------------------

This is assumed to implement
the scikit-learn estimator
interface.

Either estimator needs to
provide a “score” function,
or “scoring” must be passed.

<code>param_grid</code>	<code>param_grid:</code>	<code>{'fit_intercept': [True, False]}</code>
dict or list of dictionaries		

Dictionary with parameters names ('str') as keys and lists of parameter settings to try as values, or a list of such dictionaries, in which case the grids spanned by each dictionary in the list are explored. This enables searching over any sequence of parameter settings.

scoring scoring: str, callable, None
list, tuple or dict,
default=None

Strategy to evaluate the performance of the cross-validated model on the test set.

If ‘scoring’ represents a single score, one can use:

- a single string (see :ref:‘scoring_string_names’);
- a callable (see :ref:‘scoring_callable’) that returns a single value;
- ‘None’, the ‘estimator’s :ref:‘default evaluation criterion ‘ is used.

If ‘scoring’ represents multiple scores, one can use:

- a list or tuple of unique strings;
- a callable returning a dictionary where the keys are the metric names and the values are the metric scores;
- a dictionary with metric names as keys and callables as values.

See :ref:‘multimetric_grid_search’ for an example.

<code>n_jobs</code>	<code>n_jobs: int,</code> <code>default=None</code>	None
---------------------	--	------

Number of jobs to run in parallel.
“None“ means 1 unless in a `:obj:‘joblib.parallel_backend‘` context.
“-1“ means using all processors. See `:term:‘Glossary ‘` for more details.

.. versionchanged:: v0.20
‘n_jobs‘ default changed from 1 to None

refit refit: bool, str, or
callable, default=True

True

Refit an estimator using the
best found parameters on the
whole
dataset.

For multiple metric
evaluation, this needs to be a
'str' denoting the
scorer that would be used to
find the best parameters for
refitting
the estimator at the end.

Where there are
considerations other than
maximum score in
choosing a best estimator,
"refit" can be set to a
function which
returns the selected
"best_index_" given
"cv_results_". In that
case, the "best_estimator_"
and "best_params_" will be
set
according to the returned
"best_index_" while the
"best_score_"
attribute will not be
available.

The refitted estimator is
made available at the
"best_estimator_"
attribute and permits using
"predict" directly on this
"GridSearchCV" instance.

Also for multiple metric
evaluation, the attributes
"best_index_",
"best_score_" and
"best_params_"⁶⁸ will only be
available if
"refit" is set and all of them
will be determined w.r.t this
specific
scorer.

⁶⁸ Since "best_index_" is not available if "refit" is not set, it is not possible to determine the best parameters for a specific scorer if "refit" is not set.

`cv` `cv`: int, cross-validation generator or an iterable, default=None

5

Determines the cross-validation splitting strategy.
Possible inputs for `cv` are:

- None, to use the default 5-fold cross validation,
- integer, to specify the number of folds in a ‘(Stratified)KFold’,
- :term:‘CV splitter’,
- An iterable yielding (train, test) splits as arrays of indices.

For integer/None inputs, if the estimator is a classifier and “y” is either binary or multiclass, :class:‘StratifiedKFold’ is used. In all other cases, :class:‘KFold’ is used. These splitters are instantiated with ‘shuffle=False’ so the splits will be the same across calls.

Refer :ref:‘User Guide ‘ for the various cross-validation strategies that can be used here.

.. versionchanged:: 0.22
“cv” default value if None changed from 3-fold to 5-fold.

verbose verbose: int

0

Controls the verbosity: the higher, the more messages.

- >1 : the computation time for each fold and parameter candidate is displayed;
- >2 : the score is also displayed;
- >3 : the fold and candidate parameter indexes are also displayed together with the starting time of the computation.

`pre_dispatch pre_dispatch: '2*n_jobs'`
`int, or str,`
`default='2*n_jobs'`

Controls the number of jobs that get dispatched during parallel execution. Reducing this number can be useful to avoid an explosion of memory consumption when more jobs get dispatched than CPUs can process. This parameter can be:

- None, in which case all the jobs are immediately created and spawned. Use this for lightweight and fast-running jobs, to avoid delays due to on-demand spawning of the jobs
- An int, giving the exact number of total jobs that are spawned
- A str, giving an expression as a function of `n_jobs`, as in `'2*n_jobs'`

<code>error_score</code>	<code>error_score:</code>	<code>nan</code>
<code>'raise' or numeric,</code>		
<code>default=np.nan</code>		

Value to assign to the score if an error occurs in estimator fitting.

If set to 'raise', the error is raised. If a numeric value is given,

`FitFailedWarning` is raised.

This parameter does not affect the `refit` step, which will always raise the error.

<pre>return_train_score return_train_score: bool, default=False</pre> If “False“, the “cv_results_“ attribute will not include training scores. Computing training scores is used to get insights on how different parameter settings impact the overfitting/underfitting trade-off. However computing the scores on the training set can be computationally expensive and is not strictly required to select the parameters that yield the best generalization performance. <pre>.. versionadded:: 0.19</pre> <pre>.. versionchanged:: 0.21</pre> Default value was changed from “True“ to “False“	False
<pre>fit_intercept fit_intercept: bool, default=True</pre> Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).	True

<code>copy_X copy_X: bool,</code> <code>default=True</code>	True
If True, X will be copied; else, it may be overwritten.	
<code>tol tol: float, default=1e-6</code>	1e-06
The precision of the solution (<code>'coef_'</code>) is determined by <code>'tol'</code> which specifies a different convergence criterion for the <code>'lsqr'</code> solver. <code>'tol'</code> is set as <code>'atol'</code> and <code>'btol'</code> of <code>:func:'scipy.sparse.linalg.lsqr'</code> when fitting on sparse training data. This parameter has no effect when fitting on dense data.	
<code>.. versionadded:: 1.7</code> <code>n_jobs n_jobs: int,</code> <code>default=None</code>	None
The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly <code>'n_targets > 1'</code> and secondly <code>'X'</code> is sparse or if <code>'positive'</code> is set to <code>'True'</code>. “None“ means 1 unless in a <code>:obj:'joblib.parallel_backend'</code> context. “-1“ means using all processors. See <code>:term:'Glossary '</code> for more details.	

positive positive: bool, default=False When set to “True“, forces the coefficients to be positive. This option is only supported for dense arrays. For a comparison between a linear regression model with positive constraints on the regression coefficients and a linear regression without such constraints, see :ref:‘sphx_glr_auto_ex- amples_lin- ear_model_plot_nnl.py‘. .. versionadded:: 0.24	False
---	-------

After fitting, the best estimator and other information about the search can be obtained.

```
gs.best_estimator_
```

fit_intercept fit_intercept: bool, default=True Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered). copy_X copy_X: bool, default=True If True, X will be copied; else, it may be overwritten.	True
---	------

tol	tol: float, default=1e-6	1e-06
The precision of the solution ('coef_') is determined by 'tol' which specifies a different convergence criterion for the 'lsqr' solver. 'tol' is set as 'atol' and 'btol' of :func:'scipy.sparse.linalg.lsqr' when fitting on sparse training data. This parameter has no effect when fitting on dense data.		
.. versionadded::	1.7	
n_jobs	n_jobs: int, default=None	None
The number of jobs to use for the computation. This will only provide speedup in case of sufficiently large problems, that is if firstly 'n_targets > 1' and secondly 'X' is sparse or if 'positive' is set to 'True'. "None" means 1 unless in a :obj:'joblib.parallel_backend' context. "-1" means using all processors. See :term:'Glossary' for more details.		

positive positive: bool,
default=False

False

When set to “True“, forces
the coefficients to be positive.
This
option is only supported for
dense arrays.

For a comparison between a
linear regression model with
positive constraints
on the regression coefficients
and a linear regression
without such constraints,
see :ref:‘sphx_glr_auto_ex-
amples_lin-
ear_model_plot_nmls.py‘.

.. versionadded:: 0.24

```
gs.best_params_
```

```
{'fit_intercept': True}
```

```
gs.best_score_
```

```
np.float64(0.3934573792950554)
```

`gs.cv_results_` stores the results for all folds. Itself is a `dict`. Usually we convert it into a `DataFrame`.

```
import pandas as pd

df_result = pd.DataFrame(gs.cv_results_)
df_result
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_fit_intercept	params
0	0.000779	0.000159	0.000502	0.000041	True	{'fit_interc
1	0.000700	0.000122	0.000587	0.000164	False	{'fit_interc

Then we could get access to the mean test score or other metrics to see how scores are changed along the parameters.

```
df_result["mean_test_score"]
```

```
0    0.393457
1    0.366843
Name: mean_test_score, dtype: float64
```

n_jobs

You may assign `n_jobs` for parallel computing. You may leave it `None` right now. We will come back to it later when we really need it.

3.4.3 Modern Hyperparameter Tuning

Historically, one of the most common approaches to hyperparameter tuning was exhaustive grid search, which is introduced above. This approach is conceptually simple and works well when the number of hyperparameters is small. As a result, `GridSearchCV` became one of the standard tools in early machine learning workflows.

However, as models became more complicated, researchers observed that exhaustive grid search often wastes substantial computation. In many problems, only a few hyperparameters strongly affect performance, while others have relatively small influence.

A particularly influential paper [1] by Bergstra and Bengio showed that random search can often outperform grid search in high-dimensional hyperparameter spaces. The main idea is that random search explores more distinct values along important hyperparameter directions instead of spending excessive computation on unimportant dimensions.

This led to the widespread use of methods such as:

- `RandomizedSearchCV`
- Bayesian optimization
- TPE (Tree-structured Parzen Estimator)
- Optuna
- AutoML frameworks

Modern frameworks such as **Optuna** use adaptive search strategies. Instead of testing hyperparameters independently, they use information from previous trials to guide future searches toward more promising regions of the hyperparameter space. These methods have become increasingly important for large and computationally expensive models.

Nevertheless, in this course we will mainly use ordinary grid search because it is simple, transparent, and easy to understand. For most models in this course, naive grid search is already sufficient.

Only for more computationally expensive models such as XGBoost will we occasionally use more practical workflows such as:

- random search followed by refined grid search
- stagewise tuning
- early stopping

3.5 Feature Engineering and Preprocessing

Feature engineering refers to modifying or creating predictors in order to better represent the relationship between the predictors and the response. Here we mainly cover the **sklearn** way to perform feature engineering.

Modeling Perspective

In statistical learning (commonly implemented using **sklearn**), the primary goal is predictive performance. Models are therefore typically constructed by building a feature matrix, and modifying a model usually means modifying the feature matrix rather than changing a model formula.

In contrast, traditional regression analysis (commonly implemented using **statsmodels**) allows explicit specification of statistical models through a formula language. This approach is convenient when precise control of model terms and statistical inference are required.

3.5.1 Creating Features Manually

Before introducing automated tools, we can always create new predictors directly by modifying the original dataset.

```
import pandas as pd

X = pd.DataFrame({'x1': [0, 2, 4], 'x2': [1, 3, 5]})
X
```

	x1	x2
0	0	1
1	2	3
2	4	5

For example, we can add the squared term of x_2 and the interaction term x_1x_2 to form a new feature matrix X2.

```
X2 = X.copy()
X2["x2_sq"] = X["x2"] ** 2
X2["x1_x2"] = X["x1"] * X["x2"]
X2
```

	x1	x2	x2_sq	x1_x2
0	0	1	1	0
1	2	3	9	6
2	4	5	25	20

This corresponds to fitting a model of the form

$$y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \beta_3x_2^2 + \beta_4x_1x_2 + \varepsilon.$$

Tip

Although the model is called linear regression, it can represent nonlinear relationships by including transformed predictors such as x^2 , $\log x$, or $\sqrt{x}$. The model remains linear because it is linear in the coefficients β .

3.5.2 Polynomial Features

Click to expand.

When many predictors are present, manually creating polynomial and interaction terms can become tedious. In such cases we can use `PolynomialFeatures` from `sklearn.preprocessing`, which automatically generates polynomial and interaction terms up to the specified degree.

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, interaction_only=False, include_bias=True)
poly.fit_transform(X)
```



```
array([[ 1.,  0.,  1.,  0.,  0.,  1.],
       [ 1.,  2.,  3.,  4.,  6.,  9.],
       [ 1.,  4.,  5., 16., 20., 25.]])
```

The argument `interaction_only=False` means that both squared terms and interaction terms are included. For example, if the original variables are x_1 and x_2 , then the transformed matrix contains terms such as

$$1, x_1, x_2, x_1^2, x_1x_2, x_2^2.$$

```
poly2 = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
poly2.fit_transform(X)
```

```
array([[ 0.,  1.,  0.],
       [ 2.,  3.,  6.],
       [ 4.,  5., 20.]])
```

Here `interaction_only=True` means that only interaction terms are added, while powers such as x_1^2 and x_2^2 are excluded. In addition, the argument `include_bias=False` removes the constant column of ones.

3.5.3 Categorical Variables

Dummy Variable Trap

Including all dummy variables together with an intercept creates perfect multicollinearity because the dummy variables sum to one. Therefore, when creating dummy variables with k categories, we typically include only $k - 1$ variables.

pandas version

We could use `pd.get_dummies()` to directly modify the dataset.

```
df = pd.DataFrame({"color": ["red", "blue", "green", "red"]})
pd.get_dummies(df, drop_first=True)
```

	color_green	color_red
0	False	True
1	False	False

	color_green	color_red
2	True	False
3	False	True

The argument `drop_first=True` removes the base category column to avoid the dummy variable trap described above. If you want to keep the base column, you may set it to `False` (which is the default).

`pd.get_dummies()` automatically detects categorical variables and leaves numerical variables unchanged.

```
X = pd.DataFrame({"x": [1, 2, 3, 4, 5, 6], "group": ["A", "A", "B", "B", "C", "C"]})
pd.get_dummies(X, drop_first=True)
```

	x	group_B	group_C
0	1	False	False
1	2	False	False
2	3	True	False
3	4	True	False
4	5	False	True
5	6	False	True

If you want to convert numeric variables into dummy variables, you may manually cast them to categorical variables.

```
X["x"] = X["x"].astype("category")
pd.get_dummies(X, drop_first=True)
```

	x_2	x_3	x_4	x_5	x_6	group_B	group_C
0	False	False	False	False	False	False	False
1	True	False	False	False	False	False	False
2	False	True	False	False	False	True	False
3	False	False	True	False	False	True	False
4	False	False	False	True	False	False	True
5	False	False	False	False	True	False	True

sklearn version

`OneHotEncoder` from `sklearn` is another tool for converting categorical variables into dummy variables. It follows the standard `sklearn` API. Compared with `pd.get_dummies()`, which is convenient for quick data analysis, `OneHotEncoder` is better suited for statistical learning pipelines because it can be fitted on training data and then applied consistently to new data.

```
from sklearn.preprocessing import OneHotEncoder

df = pd.DataFrame({"color": ["red", "blue", "green", "red"]})
encoder = OneHotEncoder(drop="first", sparse_output=False)

encoder.fit_transform(df)
```

```
array([[0., 1.],
       [0., 0.],
       [1., 0.],
       [0., 1.]])
```

The argument `sparse_output=False` controls the output format. If `False`, the output is a regular matrix. If `True`, the output is a sparse matrix.

```
from sklearn.preprocessing import OneHotEncoder

X = pd.DataFrame({"x": [1, 2, 3, 4, 5, 6], "group": ["A", "A", "B", "B", "C", "C"]})
encoder = OneHotEncoder(drop="first", sparse_output=False)

encoder.fit_transform(X)
```

```
array([[0., 0., 0., 0., 0., 0., 0.],
       [1., 0., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 1., 0.],
       [0., 0., 1., 0., 0., 1., 0.],
       [0., 0., 0., 1., 0., 0., 1.],
       [0., 0., 0., 0., 1., 0., 1.]])
```

When applied to a `DataFrame` with multiple columns, unlike the `pandas` version, `OneHotEncoder` converts each column into dummy variables. In the example above, the first five columns correspond to `x` and the last two correspond to `group`.

If you want to apply `OneHotEncoder` only to certain columns, you need to use a `ColumnTransformer`, which will be introduced later.

3.5.4 Scaling Variables

Centralization, standardization and normalization

In regression and statistical learning, these transformations are typically applied to the predictors X to control their location and scale. They are rarely applied to the response y , unless there is a specific modeling reason.

- Centralization: Subtract the sample mean so that the variable has mean 0.
- Standardization: Subtract the sample mean and divide by the sample standard deviation so that the variable has mean 0 and standard deviation 1.
- Normalization: Rescale the variable so that its values lie between 0 and 1.

Standardization is especially useful when predictors are measured on very different scales. It is commonly used in statistical learning, particularly for methods such as ridge regression and lasso.

```
import numpy as np
import pandas as pd

np.random.seed(1)

X = pd.DataFrame({"x1": np.random.normal(10, 2, size=8), "x2": np.random.normal(100, 20, size=8)})
X
```

	x1	x2
0	13.248691	106.380782
1	8.776487	95.012592
2	8.943656	129.242159
3	7.854063	58.797186
4	11.730815	93.551656
5	5.396923	92.318913
6	13.489624	122.675389
7	8.477586	78.002175

Standardization:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit_transform(X)
```

```
array([[ 1.32733855,  0.43959384],
       [-0.36436724, -0.09299622],
       [-0.3011319 ,  1.51063   ],
       [-0.71329383, -1.78965739],
       [ 0.75316998, -0.16143987],
       [-1.64275917, -0.21919283],
       [ 1.41847649,  1.20298239],
       [-0.47743287, -0.88991991]])
```

Centralization (standardization without scaling):

```
scaler = StandardScaler(with_std=False)
scaler.fit_transform(X)
```

```
array([[ 3.50896013,  9.38317551],
       [-0.96324342, -1.98501392],
       [-0.7960741 , 32.24455233],
       [-1.88566784, -38.2004206 ],
       [ 1.99108467, -3.44595049],
       [-4.34280799, -4.6786935 ],
       [ 3.74989294, 25.67778244],
       [-1.26214439, -18.99543176]])
```

Normalization:

```
from sklearn.preprocessing import MinMaxScaler

minmax = MinMaxScaler()
minmax.fit_transform(X)
```

```
array([[0.97022838, 0.67547185],
       [0.41760651, 0.51409498],
       [0.43826331, 1.         ],
       [0.30362424, 0.         ],
       [0.78266733, 0.49335628],
       [0.         , 0.47585691],
       [1.         , 0.90678157],
       [0.38067187, 0.27262398]])
```

Note that scaling is not required for ordinary least squares (OLS). However, it is essential for regularization methods such as ridge and lasso because the penalty depends on the scale of the coefficients.

3.5.5 ColumnTransformer

Click to expand.

To apply different transformations to different columns, we can use `ColumnTransformer`. `ColumnTransformer` consists of multiple transformations. Each transformation is specified by a triple containing:

- a name (identifier),
- the transformer,
- the columns to which it applies.

```
from sklearn.compose import ColumnTransformer

prep = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(drop="first", sparse_output=False), ["group"]),
        ("x", "passthrough", ["x"]),
        ("x_std", StandardScaler(), ["x"]),
    ]
)

X = pd.DataFrame({"x": [1, 2, 3, 4, 5, 6], "group": ["A", "A", "B", "B", "C", "C"]})
prep.fit_transform(X)
```

```
array([[ 0.          ,  0.          ,  1.          , -1.46385011],
       [ 0.          ,  0.          ,  2.          , -0.87831007],
       [ 1.          ,  0.          ,  3.          , -0.29277002],
       [ 1.          ,  0.          ,  4.          ,  0.29277002],
       [ 0.          ,  1.          ,  5.          ,  0.87831007],
       [ 0.          ,  1.          ,  6.          ,  1.46385011]])
```

You may use the id to get into each transformer.

```
prep.transformers_

[('cat', OneHotEncoder(drop='first', sparse_output=False), ['group']),
 ('x',
  FunctionTransformer(accept_sparse=True, check_inverse=False,
                      feature_names_out='one-to-one'),
  ['x']),
 ('x_std', StandardScaler(), ['x'])]
```

```
prep.named_transformers_['x_std']
```

<code>copy</code>	<code>copy: bool,</code> <code>default=True</code>	True
If False, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.		
<code>with_mean</code>	<code>with_mean: bool,</code> <code>default=True</code>	True
If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.		
<code>with_std</code>	<code>with_std: bool,</code> <code>default=True</code>	True
If True, scale the data to unit variance (or equivalently, unit standard deviation).		

💡 Estimator, Predictor and Transformer

In `sklearn`, all models are called estimators. An estimator is any object that can be trained using the `.fit()` method.

Once an estimator is fitted,

- if it can transform input data using `.transform()`, it is called a transformer;
- if it can generate predictions using `.predict()`, it is called a predictor.

Typical examples include

- Transformers: `StandardScaler`, `MinMaxScaler`, `PolynomialFeatures`, `OneHotEncoder`, etc.
- Predictors: `LinearRegression` and most other machine learning models.

Usually a model is applied in two steps:

1. `.fit()` learns the parameters from the training data.
2. `.transform()` or `.predict()` applies the model to data.

For transformers, the combined method `.fit_transform()` performs both steps in one call.

3.6 Pipelines and transformations of y

In many modeling workflows, several steps must be performed before fitting the model. These steps may include

- feature transformations,
- scaling variables,
- generating polynomial features.

A **pipeline** combines these steps into a single object. Using pipelines ensures that the same transformations are applied consistently during training, cross-validation, and prediction, and helps prevent data leakage.

Here is a typical **pipeline**. Each step is indicated by a pair consisting of

- a name (identifier) of the step
- the estimator


```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

steps = [
    ("scaler", StandardScaler()),
    ("linear_model", LinearRegression()),
]
pipe = Pipeline(steps=steps)

```

This `pipe` object behaves like a `model` object. You may use the step names to access the internal components.

```
pipe.steps
```

```
[('scaler', StandardScaler()), ('linear_model', LinearRegression())]
```

```
pipe.named_steps['scaler']
```

`copy` `copy`: bool,
default=True

True

If False, try to avoid a copy
and do inplace scaling
instead.
This is not guaranteed to
always work inplace; e.g. if
the data is
not a NumPy array or
scipy.sparse CSR matrix, a
copy may still be
returned.

`with_mean` `with_mean`: bool, True
default=True

If True, center the data before scaling.
This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.

`with_std` `with_std`: bool, True
default=True

If True, scale the data to unit variance (or equivalently, unit standard deviation).

In a pipeline, you normally need to manually name all components. To make this easier, you may use `make_pipeline()`, which automatically generates default names for each step.

In addition, `make_pipeline()` does not require a list of (name, step) pairs. Instead, you simply provide the steps one by one.

```
from sklearn.pipeline import make_pipeline

pipe_easy = make_pipeline(
    StandardScaler(),
    LinearRegression()
)
```

It will automatically name each step by its class name.

```
pipe_easy.named_steps
```

```
{'standardscaler': StandardScaler(), 'linearregression': LinearRegression()}
```

Note

Pipelines are particularly useful when combined with cross-validation and hyperparameter tuning, since they ensure that preprocessing steps are applied within each training fold.

3.6.1 TransformedTargetRegressor

`TransformedTargetRegressor` from `sklearn.compose` is a wrapper that applies a transformation to the response variable y during model fitting and automatically applies the inverse transformation when making predictions. In other words, the regressor is trained on the transformed response, while predictions are converted back to the original scale.

Here is a simple example of a `TransformedTargetRegressor`. The transformation of y is `centralizer`, which is essentially `standardizer` without rescaling.

```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(
    regressor=LinearRegression(),
    transformer=StandardScaler(with_std=False)
)
```

3.6.2 Full Example

Click to expand.

In this example we build a more complicated pipeline.

```
import numpy as np
import pandas as pd

np.random.seed(1)

n = 100

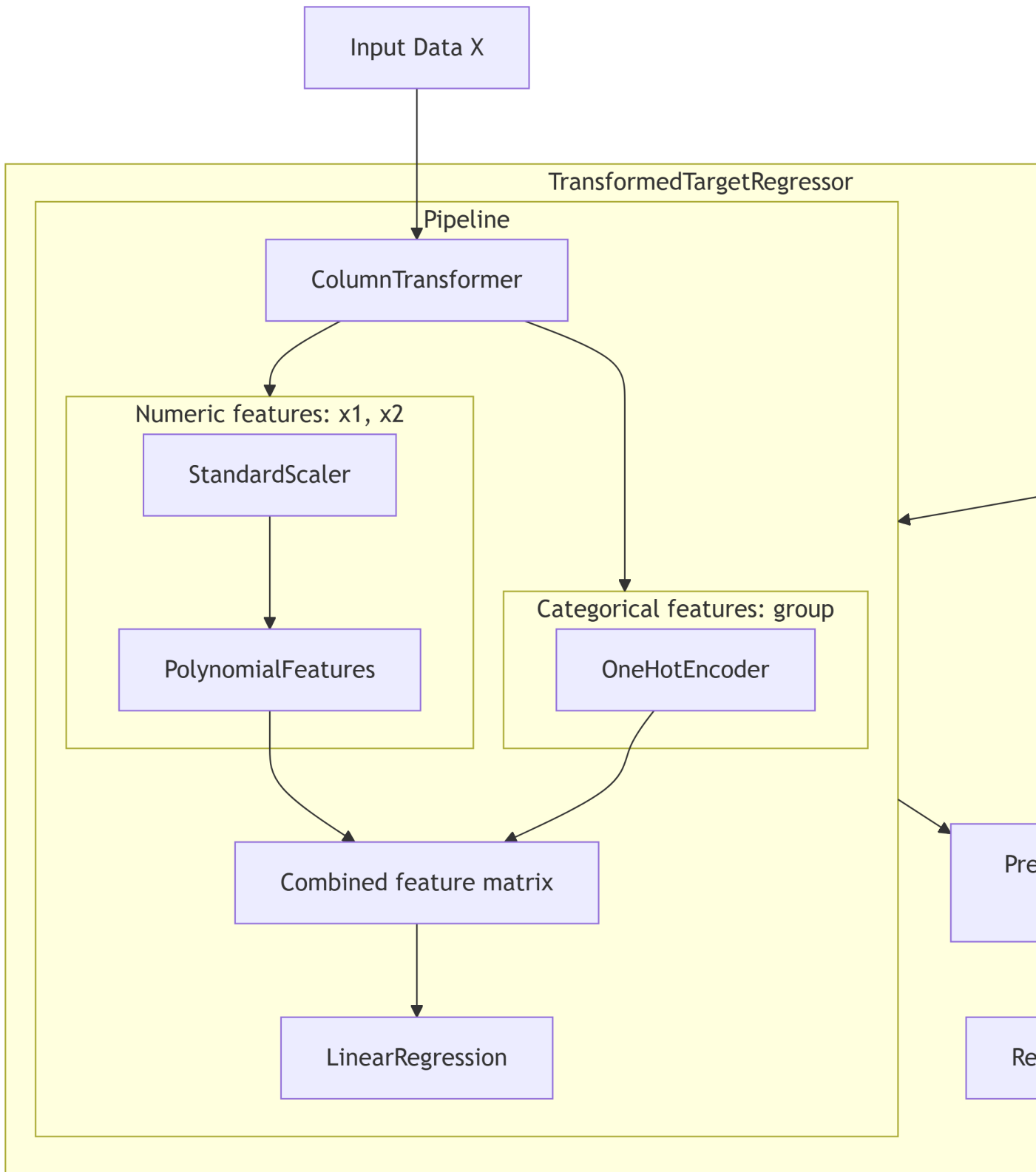
X = pd.DataFrame(
    {
        "x1": np.random.normal(0, 1, n),
        "x2": np.random.normal(5, 2, n),
        "group": np.random.choice(["A", "B", "C"], size=n),
    }
)
```

```
)
```

```
y = 1 + 2 * X["x1"] - 1.5 * X["x2"] + 3 * (X["group"] == "B") - 2 * (X["group"] == "C") + np
```

The pipeline is illustrated as follows.

1. `ColumnTransformer` separates numeric and categorical features.
2. Numeric features go through: `StandardScaler` $\rightarrow$ `PolynomialFeatures`.
3. Categorical features go through `OneHotEncoder`.
4. The transformed features are combined into a single feature matrix.
5. The model `LinearRegression` is fitted to the transformed data.
6. The response variable y is centralized.
7. The fitted model can then be used to generate predictions, while the transformation of y is automatically reversed.



```

from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.linear_model import LinearRegression

numeric_features = ["x1", "x2"]
categorical_features = ["group"]

numeric_pipe = Pipeline([("scaler", StandardScaler()), ("poly", PolynomialFeatures(degree=2,

preprocessor = ColumnTransformer(
    [("num", numeric_pipe, numeric_features), ("cat", OneHotEncoder(drop="first"), categorical
)

pipe = Pipeline([("preprocess", preprocessor), ("model", LinearRegression())])
model = TransformedTargetRegressor(regressor=pipe, transformer=StandardScaler(with_std=False)

```

The entire pipeline is trained on the training set and then evaluated on the validation set, where the training and validation sets are determined by cross-validation.

```

from sklearn.model_selection import cross_validate

cv_result = cross_validate(
    model, X, y, cv=KFold(5, shuffle=True, random_state=1), scoring=["r2", "neg_mean_squared
)

print(f"test R2: {cv_result['test_r2'].mean()}")
print(f"test neg mse: {cv_result['test_neg_mean_squared_error'].mean()}")

```

```

test R2: 0.9092762998061948
test neg mse: -1.0197334955585524

```

4 Regularization

4.1 Motivation

The OLS estimator is attractive because, under the Gauss–Markov assumptions, it is BLUE: the best linear unbiased estimator. However, when some predictors are nearly collinear, the matrix $X^\top X$ becomes nearly singular. In that case, the variance of $\hat{\beta}_{\text{OLS}}$ can be very large, so the estimated coefficients may become unstable and unreliable.

This leads to the bias-variance trade-off. By allowing a small amount of bias, we may substantially reduce the variance and improve predictive performance. A common way to do this is through regularization.

Two of the most important regularization methods are ridge regression and lasso. Both methods tend to shrink the coefficients toward zero, so they are also called shrinkage methods.

4.2 Ridge Regression

4.2.1 OLS estimator review

Consider a linear model

$$y = X\beta + \varepsilon.$$

Given the dataset X and y , the OLS estimator $\hat{\beta}_{\text{OLS}}$ finds the coefficient vector β that minimizes the residual sum of squares

$$\text{RSS}(\beta) = \|y - X\beta\|_2^2.$$

In other words,

$$\hat{\beta}_{\text{OLS}} = \arg \min_{\beta} \left\{ \|y - X\beta\|_2^2 \right\}.$$

When $X^\top X$ is invertible, the solution is

$$\hat{\beta}_{\text{OLS}} = (X^\top X)^{-1} X^\top y.$$

4.2.2 Ridge estimator

Ridge regression modifies the OLS loss function by adding a quadratic penalty.

Definition 4.1 (Ridge Regression). For a response vector $y \in \mathbb{R}^n$ and a design matrix $X \in \mathbb{R}^{n \times p}$, the Ridge estimator $\hat{\beta}_{\text{ridge}}$ is defined as:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \right\}$$

where

- $\lambda \geq 0$ is the regularization parameter that controls the trade-off between the fit to the data and the penalty on the coefficient size.
- $\|\beta\|_2^2 = \sum_{j=1}^p \beta_j^2$ is the L_2 norm of the coefficient vector

The penalty shrinks the coefficients toward zero. Larger values of λ produce stronger shrinkage. However, ridge regression does not perform variable selection, since it rarely sets any coefficient exactly to zero.

Similar to the OLS estimator, the Ridge estimator can be solved analytically.

Theorem 4.1 (The Closed-Form of Ridge estimator). *The closed-form solution of $\hat{\beta}_{\text{ridge}}$ is*

$$\hat{\beta}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y.$$

Click for proof.

Consider the ridge regression objective

$$Q(\beta) = (y - X\beta)^\top (y - X\beta) + \lambda \beta^\top \beta, \quad \text{where } \lambda \geq 0.$$

We derive the ridge estimator by minimizing $Q(\beta)$ with respect to β .

First expand the quadratic form:

$$\begin{aligned} Q(\beta) &= (y^\top - \beta^\top X^\top)(y - X\beta) + \lambda \beta^\top \beta \\ &= y^\top y - y^\top X\beta - \beta^\top X^\top y + \beta^\top X^\top X\beta + \lambda \beta^\top \beta. \end{aligned}$$

Since $y^\top X\beta$ is a scalar, it equals its transpose:

$$y^\top X\beta = \beta^\top X^\top y.$$

Thus

$$Q(\beta) = y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X \beta + \lambda \beta^\top \beta.$$

Combine the last two quadratic terms:

$$Q(\beta) = y^\top y - 2\beta^\top X^\top y + \beta^\top (X^\top X + \lambda I) \beta.$$

Now differentiate with respect to β :

$$\nabla_\beta Q(\beta) = -2X^\top y + 2(X^\top X + \lambda I)\beta.$$

Set the gradient equal to zero:

$$-2X^\top y + 2(X^\top X + \lambda I)\beta = 0.$$

So

$$(X^\top X + \lambda I)\beta = X^\top y.$$

Assuming $X^\top X + \lambda I$ is invertible, we obtain

$$\hat{\beta}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y.$$

Therefore the ridge estimator is

$$\hat{\beta}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y.$$

In this formulation, the term λI (where I is the identity matrix) is added to the cross-product matrix $X^\top X$ before inversion. This ensures that the matrix is non-singular and well-conditioned, even in the presence of multicollinearity or when $p > n$.

4.2.3 About intercept

Linear regression models are commonly written in the form

$$y = X\beta + \varepsilon.$$

This notation can represent two different model structures depending on whether an intercept is included.

- Model with intercept: $\beta = [\beta_0, \beta_1, \dots, \beta_p]^\top$ where β_0 is the intercept. The design matrix X contains a leading column of ones representing the constant term.
- Model without intercept: $\beta = [\beta_1, \dots, \beta_p]^\top$ and the design matrix contains only the predictor variables. In this case the fitted regression surface is forced to pass through the origin.

In most practical applications, models with an intercept are preferred because they allow the regression function to shift vertically rather than being constrained to go through the origin.

i Centering and the No-Intercept Form

A model with an intercept can be rewritten as a no-intercept model by centering both the response and the predictors:

$$y^c = y - \bar{y}, \quad X_j^c = X_j - \bar{X}_j.$$

After centering, all variables have mean zero. In ordinary least squares, the intercept estimate is

$$\hat{\beta}_0 = \bar{y} - \sum_{j=1}^p \hat{\beta}_j \bar{X}_j.$$

When the data are centered, this quantity becomes zero automatically. As a result, fitting a regression without an intercept on centered data produces the same fitted model as fitting a regression with an intercept on the original data.

For OLS, the intercept formulation and the centered no-intercept formulation are mathematically equivalent. However, the situation is slightly different in ridge regression. The ridge estimator introduces the penalty

$$\lambda \sum_{j=1}^p \beta_j^2,$$

which is intended to shrink the slope coefficients. In practice the intercept is not penalized, because penalizing it would shrink the overall level of the model toward zero and make the result depend on the arbitrary origin of the response variable.

For this reason ridge regression is usually formulated in the no-intercept form after centering the data, so that the penalty naturally applies only to the slope coefficients.

i Standardization in Practice

Because the ridge penalty depends on the sizes of the coefficients, ridge regression is usually implemented after standardizing the predictors. This puts the predictors on a common scale, so that the penalty treats them fairly.

In practice, the predictors are typically standardized, whereas the response is usually only centered rather than standardized.

For details see Section 3.5.4.

i Implementation in `scikit-learn`

In `scikit-learn`, ridge and lasso regression automatically center the data internally when fitting the model (while leaving the final predictions on the original scale). The intercept is therefore handled separately and excluded from the penalty.

Because of this internal preprocessing, users typically do not need to manually center the response when using `Ridge` or `Lasso`. However, it is still common to standardize the predictors explicitly using tools such as `StandardScaler`, especially when building modeling pipelines.

4.2.4 Alternative Formulation

By the theory of Lagrange multipliers, ridge regression can also be written in a constrained optimization form:

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \|y - X\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_2^2 \leq s.$$

Here $s \geq 0$ controls the size of the coefficient vector and is directly related to the penalty parameter λ . The two parameters have an inverse relationship:

- Large s corresponds to small λ . The constraint is weak, and β approaches the OLS estimator.
- Small s corresponds to large λ . The constraint is strong, and β shrinks toward 0.

This formulation highlights that ridge regression limits the magnitude of the coefficient vector rather than directly penalizing prediction error. We will later use this interpretation to explain the difference between ridge regression and lasso.

i The Constrained Optimization Problem

Ridge regression can be formulated as a constrained optimization problem, where we minimize RSS subject to a constraint s on the squared L_2 norm of the coefficients:

$$\min_{\beta} \|y - X\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_2^2 \leq s.$$

To solve this, we define the Lagrangian function $\mathcal{L}(\beta, \lambda)$:

$$\mathcal{L}(\beta, \lambda) = \|y - X\beta\|_2^2 + \lambda(\|\beta\|_2^2 - s)$$

where $\lambda \geq 0$ is the Lagrange multiplier. Taking the partial derivative with respect to β and setting it to zero gives:

$$\frac{\partial \mathcal{L}}{\partial \beta} = -2X^\top(y - X\beta) + 2\lambda\beta = 0$$

Rearranging the terms yields the standard ridge estimator:

$$(X^\top X + \lambda I)\beta = X^\top y \implies \hat{\beta}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y.$$

This shows that for every $\lambda > 0$, there exists an equivalent constrained problem with some constraint s .

i Relation Between λ and s

The connection between the penalty parameter λ and the constraint s can be explained as follows. The Lagrange multiplier method implies that at the optimal solution

$$\lambda(\|\beta\|_2^2 - s) = 0$$

This leads to two scenarios:

- Non-binding constraint: If the OLS solution already satisfies $\|\hat{\beta}_{\text{ols}}\|_2^2 < s$, then $\lambda = 0$. The constraint has no effect, and the estimator reduces to the OLS estimator $\hat{\beta}_{\text{OLS}}$.
- Binding constraint: If $\|\hat{\beta}_{\text{ols}}\|_2^2 > s$, the constraint is active. In this case, $\lambda > 0$ and the solution must lie exactly on the boundary $\|\hat{\beta}_{\text{ridge}}\|_2^2 = s$.

Therefore, the parameters λ and s are related through

$$\|(X^\top X + \lambda I)^{-1} X^\top y\|_2^2 = s, \quad \text{when } \lambda > 0.$$

λ is effectively the “price” paid to satisfy the constraint. As s decreases, λ must increase (heavier penalty) to pull the estimate further away from the OLS solution and toward the

origin. Because the norm of the ridge estimator is a strictly decreasing function of λ , for any $s < \|\hat{\beta}_{\text{OLS}}\|_2^2$, there exists a unique $\lambda > 0$ such that $\|\hat{\beta}_{\text{ridge}}\|_2^2 = s$.

4.2.5 SVD and Tikhonov Regularization

Click to expand.

Recall the OLS estimator

$$\hat{\beta}_{\text{OLS}} = (X^\top X)^{-1} X^\top y.$$

Let the singular value decomposition (SVD) of X be

$$X = UDV^\top,$$

where

- U is an $n \times n$ orthogonal matrix,
- V is a $p \times p$ orthogonal matrix,
- D is an $n \times p$ diagonal matrix whose nonzero entries $d_1, \dots, d_r$ are the singular values of X .

Using the SVD,

$$X^\top X = VD^\top DV^\top.$$

Since $D^\top D = \text{diag}(d_1^2, \dots, d_r^2)$, the eigenvalues of $X^\top X$ are d_i^2 . Hence the OLS estimator can be written as

$$\hat{\beta}_{\text{OLS}} = V(D^\top D)^{-1} D^\top U^\top y.$$

Since $D^\top D = \text{diag}(d_1^2, \dots, d_r^2)$, we have

$$(D^\top D)^{-1} = \text{diag}\left(\frac{1}{d_1^2}, \dots, \frac{1}{d_r^2}\right).$$

Let u_i and v_i denote the columns of U and V . Then the estimator can be written as

$$\hat{\beta}_{\text{OLS}} = \sum_{i=1}^r \frac{1}{d_i} v_i u_i^\top y.$$

If some singular values d_i are very small, the factors $1/d_i$ become very large, which leads to unstable coefficient estimates. Therefore this expression reveals the instability of OLS.

Now turn to Ridge regression. We replace the OLS estimator with

$$\hat{\beta}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y.$$

Substituting the SVD gives

$$\hat{\beta}_{\text{ridge}} = V(D^\top D + \lambda I)^{-1} D^\top U^\top y.$$

Because $D^\top D$ is diagonal, the estimator can be written componentwise as

$$\hat{\beta}_{\text{ridge}} = \sum_{i=1}^r \frac{d_i}{d_i^2 + \lambda} v_i u_i^\top y.$$

This representation shows that ridge regression applies a shrinkage factor

$$\frac{d_i^2}{d_i^2 + \lambda}$$

to each principal component direction.

- When d_i is large, the factor is close to 1, so the component is almost unchanged.
- When d_i is small, the factor becomes small, shrinking the unstable directions.

Thus ridge regression stabilizes the estimator by shrinking the contributions of directions associated with small singular values.

Tikhonov regularization

Tikhonov regularization is a method used to solve ill-posed problems and handle multicollinearity by adding a penalty term, $\alpha \|L\beta\|_2^2$. When the Tikhonov matrix L is the identity matrix I , it simplifies to ridge regression. Therefore, ridge regression is a special case of Tikhonov regularization [2, 3].

4.2.6 Bias-Variance Tradeoff

One of the main motivations for regularization methods is the bias-variance tradeoff. A more flexible model can fit the training data very well, but it may also be highly sensitive to random noise in the sample. In contrast, a more constrained model is usually more stable, but may systematically miss part of the true signal.

This tradeoff can be described mathematically through a decomposition of the expected prediction error.

Suppose the true model is

$$Y = f(x) + \varepsilon, \quad \mathbb{E}(\varepsilon) = 0, \quad \text{Var}(\varepsilon) = \sigma^2.$$

Let $\hat{f}(x)$ be an estimator of $f(x)$ constructed from a training sample. Since the training sample is random, $\hat{f}(x)$ is also random. We study the expected prediction error at a fixed point x :

$$\mathbb{E} \left[(Y - \hat{f}(x))^2 \right].$$

Here, the expectation is taken over both the randomness in the training data and the randomness in the new response Y .

Definition 4.2 (Bias and Variance).

- $\text{Bias}(\hat{f}(x)) = \mathbb{E}[\hat{f}(x)] - f(x)$.
- $\text{Var}(\hat{f}(x)) = \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right]$.

Theorem 4.2 (Bias-Variance Decomposition).

$$\mathbb{E} \left[(Y - \hat{f}(x))^2 \right] = \text{Bias}(\hat{f}(x))^2 + \text{Var}(\hat{f}(x)) + \sigma^2.$$

Click to expand.

Since $Y = f(x) + \varepsilon$,

$$\begin{aligned} \mathbb{E} \left[(Y - \hat{f}(x))^2 \right] &= \mathbb{E} \left[(f(x) + \varepsilon - \hat{f}(x))^2 \right] \\ &= \mathbb{E} \left[(f(x) - \hat{f}(x))^2 + 2\varepsilon(f(x) - \hat{f}(x)) + \varepsilon^2 \right] \\ &= \mathbb{E} \left[(f(x) - \hat{f}(x))^2 \right] + 2\mathbb{E} \left[\varepsilon(f(x) - \hat{f}(x)) \right] + \mathbb{E}(\varepsilon^2). \end{aligned}$$

Since the new noise ε is independent of the training data and has mean 0,

$$\mathbb{E} \left[\varepsilon(f(x) - \hat{f}(x)) \right] = 0.$$

Also,

$$\mathbb{E}(\varepsilon^2) = \text{Var}(\varepsilon) = \sigma^2.$$

Therefore,

$$\mathbb{E} \left[(Y - \hat{f}(x))^2 \right] = \mathbb{E} \left[(f(x) - \hat{f}(x))^2 \right] + \sigma^2.$$

Thus the prediction error is the sum of:

- the estimation error $\mathbb{E}[(f(x) - \hat{f}(x))^2]$,
- the irreducible error σ^2 .

Now we further decompose

$$\mathbb{E} \left[(f(x) - \hat{f}(x))^2 \right].$$

Let

$$m(x) = \mathbb{E}[\hat{f}(x)],$$

where the expectation is taken over the randomness in the training sample. Then

$$f(x) - \hat{f}(x) = (f(x) - m(x)) + (m(x) - \hat{f}(x)).$$

Squaring,

$$(f(x) - \hat{f}(x))^2 = (f(x) - m(x))^2 + (m(x) - \hat{f}(x))^2 + 2(f(x) - m(x))(m(x) - \hat{f}(x)).$$

Taking expectation,

$$\mathbb{E} \left[(f(x) - \hat{f}(x))^2 \right] = (f(x) - m(x))^2 + \mathbb{E} \left[(m(x) - \hat{f}(x))^2 \right] + 2(f(x) - m(x)) \mathbb{E}[m(x) - \hat{f}(x)].$$

But

$$\mathbb{E}[m(x) - \hat{f}(x)] = m(x) - \mathbb{E}[\hat{f}(x)] = 0.$$

So the cross term vanishes, and we obtain

$$\mathbb{E} \left[(f(x) - \hat{f}(x))^2 \right] = (f(x) - \mathbb{E}[\hat{f}(x)])^2 + \mathbb{E} \left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2 \right].$$

The first term is the squared bias:

$$\text{Bias}(\hat{f}(x))^2 = (\mathbb{E}[\hat{f}(x)] - f(x))^2.$$

The second term is the variance:

$$\text{Var}(\hat{f}(x)) = \mathbb{E} \left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2 \right].$$

Therefore,

$$\mathbb{E}[(f(x) - \hat{f}(x))^2] = \text{Bias}(\hat{f}(x))^2 + \text{Var}(\hat{f}(x)).$$

Combining this with the earlier decomposition gives

$$\mathbb{E}[(Y - \hat{f}(x))^2] = \text{Bias}(\hat{f}(x))^2 + \text{Var}(\hat{f}(x)) + \sigma^2.$$

This formula shows that the expected prediction error has three parts:

- Squared bias: error caused by systematic deviation of the estimator from the truth.
- Variance: error caused by sensitivity of the estimator to the particular training sample.
- Irreducible error: the noise level σ^2 , which cannot be removed by any method.

A very flexible model often has small bias but large variance. A more constrained model often has larger bias but smaller variance. The goal of regularization is not necessarily to minimize bias or variance alone, but to balance them so that the total prediction error is as small as possible.

Note

Ordinary least squares is often unbiased, but its variance can be very large when predictors are highly correlated or when the model is too flexible. Ridge and lasso (which will be discussed later) introduce bias by shrinking the coefficients toward 0, but this often substantially reduces variance.

As a result, although ridge or lasso may fit the training data less perfectly than OLS, they can achieve better prediction performance on new data. This is the central idea behind the bias-variance tradeoff and one of the main reasons regularization methods are useful in statistical learning.

4.3 Ridge Regression in Python

We use the following simulated dataset to demonstrate ridge regression.

The covariance matrix has a very high correlation, so the two predictors x_1 and x_2 are highly correlated.

```
import numpy as np

rng = np.random.default_rng(2)
n = 30

Sigma = np.array([[1, 0.99], [0.99, 1]])
```

```
mean = np.array([0, 0])

X = rng.multivariate_normal(mean, Sigma, size=n)
y = rng.normal(loc=X[:, 0] + X[:, 1], scale=1)
```

The data set is generated from a model with $E(y \mid x_1, x_2) = x_1 + x_2$. Therefore the true coefficients are $\beta_1 = \beta_2 = 1$.

4.3.1 Ridge estimator

We use `Ridge` from `sklearn.linear_model` to fit a ridge regression model. Its API is very similar to that of OLS. The argument `alpha` is the regularization parameter λ .

For simplicity, we use a no-intercept model, so we set `fit_intercept=False`.

```
from sklearn.linear_model import Ridge

ridge = Ridge(alpha=5, fit_intercept=False).fit(X, y)
print(f"Ridge beta = {ridge.coef_}")
```

```
Ridge beta = [0.85769874 0.84964704]
```

For comparison, we also fit the OLS estimator.

```
from sklearn.linear_model import LinearRegression

ols = LinearRegression(fit_intercept=False).fit(X, y)
print(f"OLS beta = {ols.coef_}")
```

```
OLS beta = [0.99640827 0.85969191]
```

In this example, you may notice two things:

- The ridge coefficients are smaller in magnitude than the OLS coefficients.
- The OLS coefficients may happen to be closer to the true values in this single sample.

The second point is due to randomness in the simulated data. To see the general pattern, we repeat the simulation many times.

```

import pandas as pd

res = []
for _ in range(100):
    X = rng.multivariate_normal(mean, Sigma, size=n)
    y = rng.normal(loc=X[:, 0] + X[:, 1], scale=1)
    ols = LinearRegression(fit_intercept=False).fit(X, y)
    ridge = Ridge(alpha=5, fit_intercept=False).fit(X, y)
    res.append(
        {
            "ols_b1": ols.coef_[0],
            "ols_b2": ols.coef_[1],
            "ridge_b1": ridge.coef_[0],
            "ridge_b2": ridge.coef_[1],
        }
    )
res = pd.DataFrame(res)

```

The first five results are shown below. We can see that the OLS estimates vary substantially from sample to sample.

```
res.head(5)
```

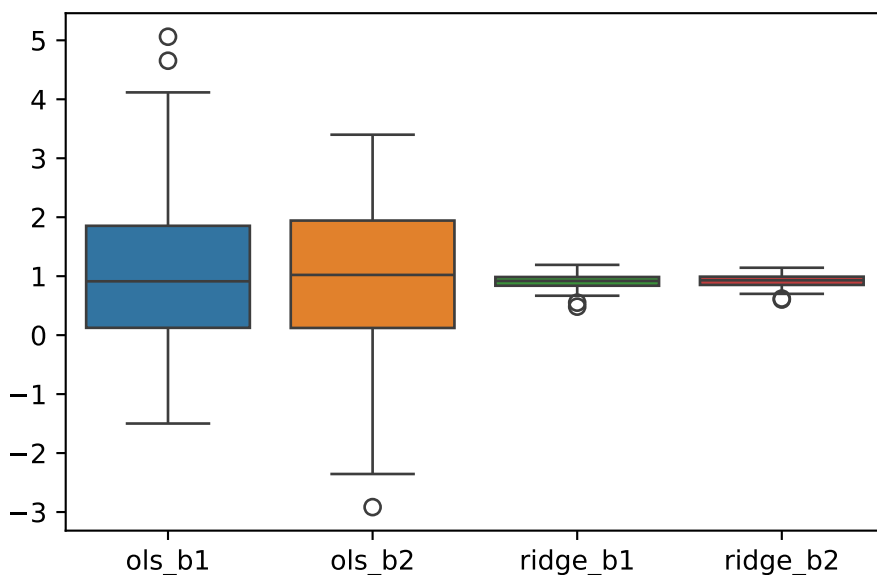
	ols_b1	ols_b2	ridge_b1	ridge_b2
0	1.642753	0.343583	0.908249	0.883823
1	1.105216	0.774858	0.885627	0.858433
2	-0.978268	3.091366	0.797745	1.092525
3	1.301118	0.325411	0.781313	0.728571
4	-0.626684	2.741871	0.926826	1.008491

This becomes even clearer in the boxplot below, which shows that the ridge estimator has much smaller variance than the OLS estimator.

```

import seaborn as sns
sns.boxplot(res)

```



4.3.2 Contour map

The variance of both $\hat{\beta}_1$ and $\hat{\beta}_2$ is quite large under OLS. This is because the variance of $\hat{\beta}$ is $\sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$. Since the columns of $\mathbf{X}$ are highly correlated, the smallest eigenvalue of $\mathbf{X}^\top \mathbf{X}$ is close to zero, making the largest eigenvalue of $(\mathbf{X}^\top \mathbf{X})^{-1}$ very large.

To see things in more details, we first build the RSS function. For each β_1 and β_2 in the range, we form a model $\hat{y}_\beta = \beta_1 x_1 + \beta_2 x_2$ and use it to compute $\text{RSS} = \sum (y - \hat{y})^2$.

```
beta1_vals = np.linspace(-1, 3, 200)
beta2_vals = np.linspace(-1, 3, 200)
B1, B2 = np.meshgrid(beta1_vals, beta2_vals)

betas = np.column_stack([B1.ravel(), B2.ravel()])
preds = X @ betas.T
rss_term = np.sum((y[:, None] - preds) ** 2, axis=0)
rss_value0 = rss_term.reshape(B1.shape)
```

We then construct the contour map for the RSS surface, plotting the isolines that correspond to the 1%, 2.5%, 5%, 20%, 50%, and 75% quantiles of the distribution. The true coefficient ($\beta_1 = 1, \beta_2 = 1$) is displayed as the red dot, as well as the estimated $\hat{\beta}$ as a blue square in the contour map.

```

import matplotlib.pyplot as plt

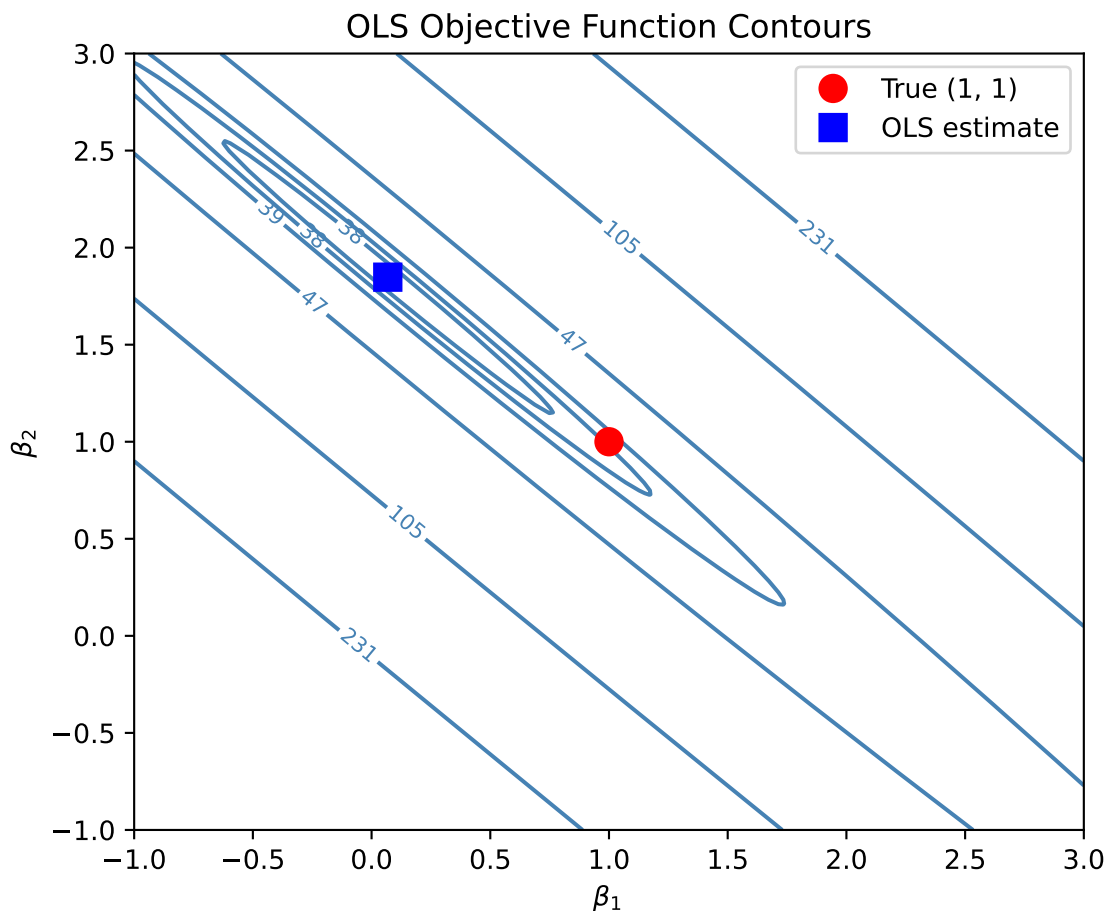
quant_levels = np.quantile(rss_value0, [0.01, 0.025, 0.05, 0.2, 0.5, 0.75])

fig, ax = plt.subplots(figsize=(6, 5))
cs = ax.contour(B1, B2, rss_value0, levels=quant_levels, colors="steelblue")
ax.clabel(cs, inline=True, fontsize=8, fmt="%.0f")

ax.plot(1, 1, "ro", markersize=10, label="True (1, 1)")
ax.plot(ols.coef_[0], ols.coef_[1], "bs", markersize=10, label="OLS estimate")

ax.set_xlabel(r"$\beta_1$")
ax.set_ylabel(r"$\beta_2$")
ax.set_title("OLS Objective Function Contours")
ax.legend()
plt.tight_layout()

```



The contour map reveals a narrow, elongated valley in the RSS objective function. While the surface is highly sensitive to movement perpendicular to the valley, it remains relatively flat along the valley floor. Because the surface is particularly flat in this longitudinal direction, numerous parameter combinations yield nearly identical RSS values. Consequently, the data provides insufficient information to distinguish between points along this axis, making the RSS minimum highly unstable. This lack of identifiability is the fundamental cause of high variance in the estimated coefficients.

When ridge regression is applied, the RSS objective function is modified by adding a penalty term. As a result, the objective surface no longer exhibits a long, flat valley floor. Instead, the minimum becomes more well-defined, and the estimate is pulled toward the origin. Although the true parameter may no longer lie at the bottom of this modified valley, the ridge estimate has substantially lower variance. In this way, ridge regression stabilizes the estimation by controlling the region along the valley floor where the OLS solution would otherwise vary widely.

Here we show the RSS objective function contours for $\lambda = 0.1, 1, 10$ and 100 .

```

from sklearn.linear_model import Ridge

beta1_vals = np.linspace(-1, 3, 200)
beta2_vals = np.linspace(-1, 3, 200)
B1, B2 = np.meshgrid(beta1_vals, beta2_vals)

lambda_vals = [0.1, 1, 10, 100]

fig, axes = plt.subplots(2, 2, figsize=(10, 8))

for ax, reg_lambda in zip(axes.ravel(), lambda_vals):
    betas = np.column_stack([B1.ravel(), B2.ravel()])
    preds = X @ betas.T
    rss_term = np.sum((y[:, None] - preds) ** 2, axis=0)
    penalty_term = reg_lambda * np.sum(betas**2, axis=1)
    rss_value = (rss_term + penalty_term).reshape(B1.shape)

    quant_levels = np.quantile(rss_value, [0.01, 0.025, 0.05, 0.2, 0.5, 0.75])

    cs = ax.contour(B1, B2, rss_value, levels=quant_levels, colors="steelblue")
    ax.clabel(cs, inline=True, fontsize=7, fmt="%.0f")

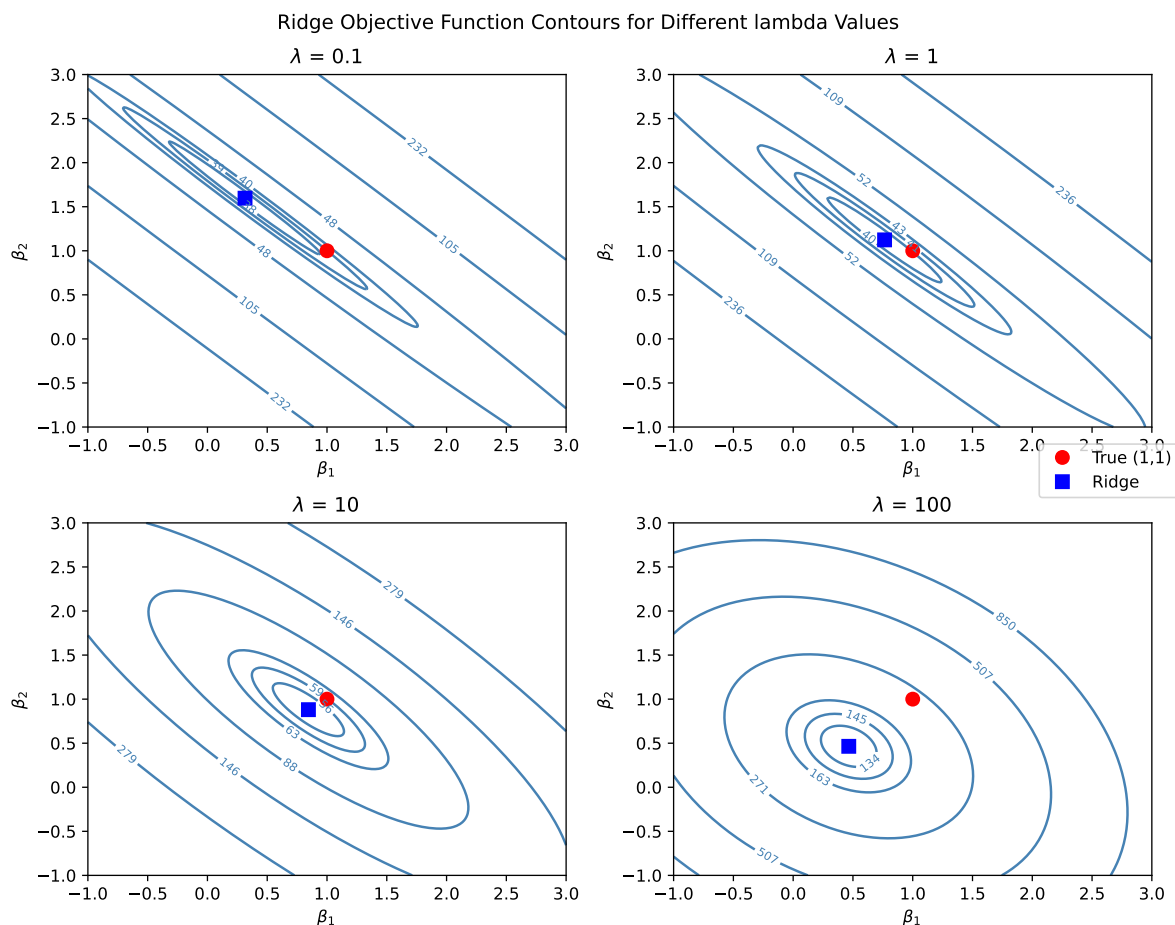
    ridge = Ridge(alpha=reg_lambda, fit_intercept=False).fit(X, y)

    ax.plot(1, 1, "ro", markersize=8, label="True (1,1)")
    ax.plot(ridge.coef_[0], ridge.coef_[1], "bs", markersize=8, label="Ridge")

    ax.set_title(r"$\lambda$ = {}".format(reg_lambda))
    ax.set_xlabel(r"$\beta_1$")
    ax.set_ylabel(r"$\beta_2$")

handles, labels = axes[0, 0].get_legend_handles_labels()
fig.suptitle("Ridge Objective Function Contours for Different lambda Values")
fig.legend(handles, labels, loc="right")
plt.tight_layout()

```



As λ increases, the bottom of the valley gradually becomes more rounded, and the objective surface rises more steeply as the coefficients move away from the origin. This additional curvature removes the long flat valley floor and makes the minimum more clearly defined. At the same time, the penalty pulls the estimated coefficients toward the origin, so the ridge estimate is effectively dragged toward zero.

Geometrically, this shrinkage prevents the estimator from drifting freely along the nearly flat valley direction that appears in the OLS objective. As a result, the ridge estimator is much more stable and typically has substantially lower variance than the OLS estimator, although this improvement comes at the cost of introducing some bias.

i Gram matrix and Covariance matrix

The Gram matrix $X^T X$ represents the raw inner products of predictors. In the context of RSS map, it is the direct source of the surface geometry.

The Covariance matrix is essentially the Gram matrix of the centered data, usually scaled

by $n - 1$.

$$\mathbf{S} = \frac{1}{n-1} \sum (x_i - \bar{x})(x_i - \bar{x})^top.$$

Therefore if the data is already centered, the Gram matrix and the covariance matrix are directly related: $\mathbf{X}^T \mathbf{X} = (n-1) \text{Cov}$.

Mathematically, these two directions (the longitudinal axis and the perpendicular axis) correspond to the eigenvectors of the $\mathbf{X}^T \mathbf{X}$ matrix (or, equivalently, the principal components of the predictor space). The flatness or steepness in those directions is governed by their corresponding eigenvalues.

```
XTX = X.T @ X
eigenvalues, eigenvectors = np.linalg.eigh(XTX)
for i in range(len(eigenvalues)):
    print(f'eigenvalue: {eigenvalues[i]: .2f}, eigenbasis: {eigenvectors[i]}')
```

```
eigenvalue:  0.26, eigenbasis: [ 0.70320986 -0.71098234]
eigenvalue: 95.19, eigenbasis: [-0.71098234 -0.70320986]
```

Then mark the eigenvectors in the contour map.

```

fig, ax = plt.subplots(figsize=(6, 5))

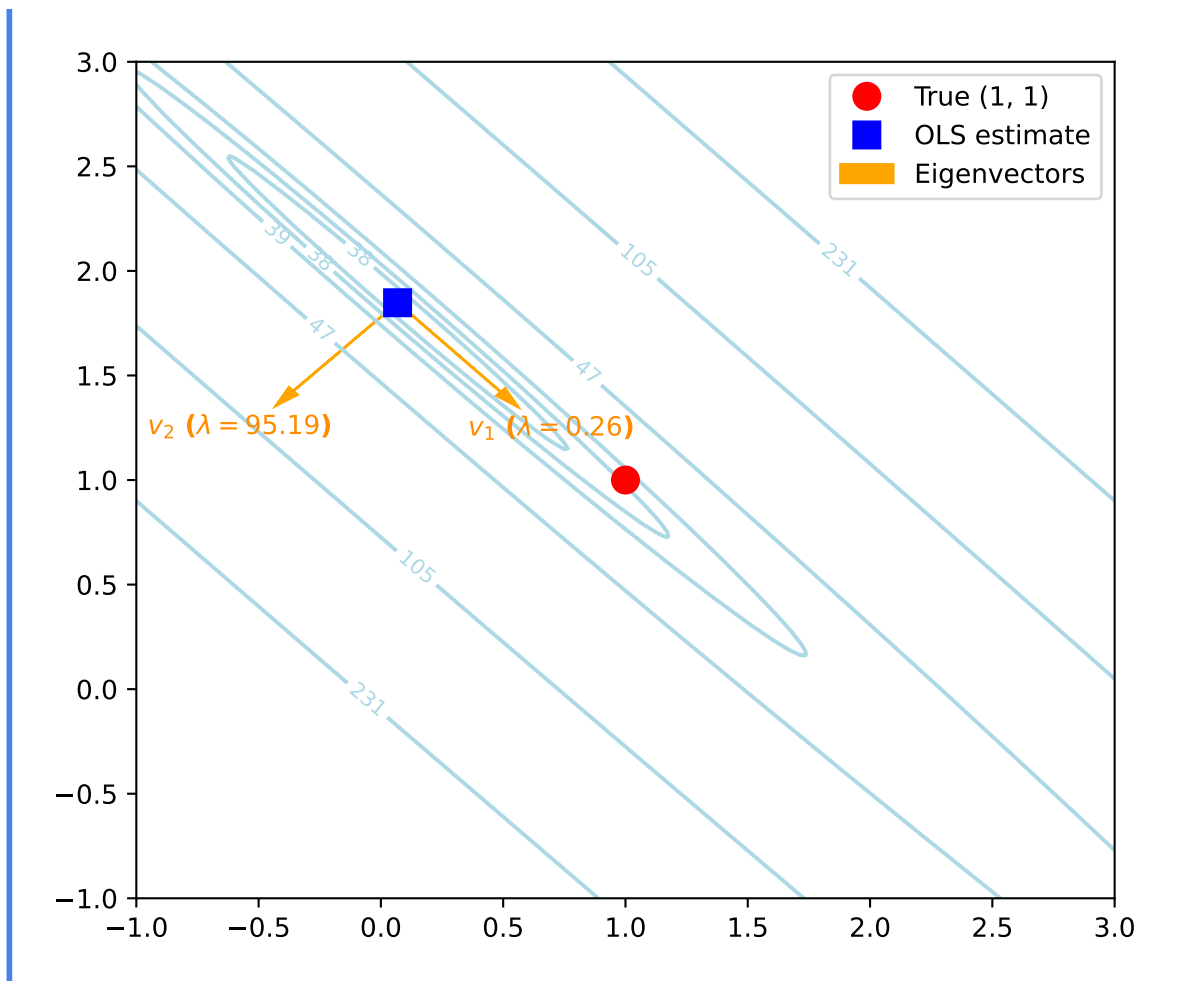
quant_levels = np.quantile(rss_value0, [0.01, 0.025, 0.05, 0.2, 0.5, 0.75])
cs = ax.contour(B1, B2, rss_value0, levels=quant_levels, colors="lightblue")
ax.clabel(cs, inline=True, fontsize=8, fmt="%.0f")

ax.plot(1, 1, "ro", markersize=10, label="True (1, 1)")
ax.plot(ols.coef_[0], ols.coef_[1], "bs", markersize=10, label="OLS estimate")

scale = 0.6
for i in range(len(eigenvalues)):
    v = eigenvectors[:, i]
    ax.arrow(
        ols.coef_[0],
        ols.coef_[1],
        v[0] * scale,
        v[1] * scale,
        head_width=0.05,
        head_length=0.1,
        fc="orange",
        ec="orange",
        label="Eigenvectors" if i == 0 else "",
    )

    ax.text(
        ols.coef_[0] + v[0] * scale * 1.5,
        ols.coef_[1] + v[1] * scale * 1.5,
        r"$v_{i}$ ($\lambda={:.2f}$)".format(i + 1, eigenvalues[i]),
        color="darkorange",
        fontweight="bold",
        ha="center",
    )
ax.legend()
plt.tight_layout()

```



4.3.3 Choosing λ

The regularization parameter λ determines how much shrinkage is applied.

In practice, the modern standard is to choose λ by cross-validation. Similar to the model selection procedure discussed in the linear regression case, the process works as follows:

1. Fix a value of λ . Split the dataset into k folds. Each time, choose one fold as the validation set and use the remaining folds as the training set to fit the model and compute the validation score.
2. Repeat this process so that each fold serves once as the validation set. The average validation score is the cross-validation score for this λ .
3. Repeat the above steps for each candidate value of λ , and select the λ with the best cross-validation score.
4. Finally, refit the model on the entire training set using the selected λ .

Since this procedure is very common in ridge regression, sklearn provides `RidgeCV`, which has built-in cross-validation. Alternatively, the same task can be performed using `GridSearchCV`.

i Leave-one-out cross-validation

`GridSearchCV` typically uses k -fold cross-validation, where the training set is split into k folds and each fold is used once as the validation set.

If each observation is treated as its own fold (that is, the validation set contains exactly one observation), the procedure is called leave-one-out cross-validation (LOOCV). In other words, LOOCV is equivalent to N -fold cross-validation, where N is the number of observations.

`RidgeCV` uses leave-one-out cross-validation by default.

The arguments of `RidgeCV` are similar to those of `Ridge`. The main differences are:

- `alphas` should be a list of candidate values for α . In the example below, α is selected from values ranging from $1e-4$ to $1e4$.
- `cv` works in the same way as in ordinary cross-validation and specifies the number of folds used in the cross-validation procedure. If it is not set, LOOCV will be used.

```
from sklearn.linear_model import RidgeCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

alphas = np.logspace(-4, 4, 100)

ridge_cv_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", RidgeCV(alphas=alphas)),
    ]
)

ridge_cv_pipe.fit(X, y)

best_alpha_ridge = ridge_cv_pipe.named_steps["model"].alpha_
best_alpha_ridge
```

1.3219411484660315

4.3.4 Validation curve and Coefficient paths

RidgeCV returns only the final fitted model at the selected value of `alpha`, rather than the entire validation curve or coefficient path. If we want to examine how the validation score changes with `alpha`, we may fit ordinary Ridge models over a grid of `alpha` values and then add a vertical line at the value selected by RidgeCV.

`sklearn` provides `validation_curve()` to streamline the process.

```
from sklearn.model_selection import validation_curve

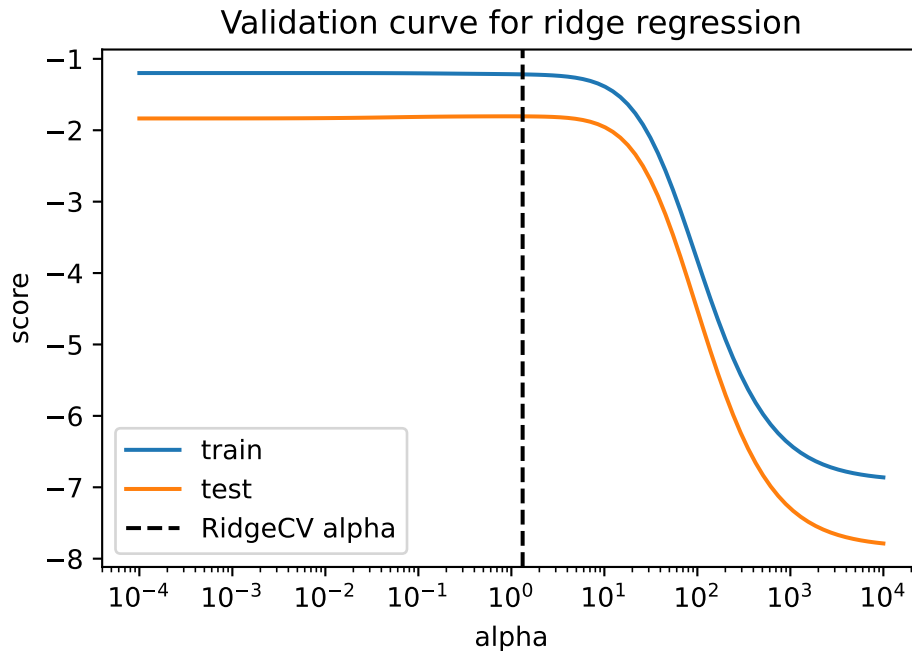
alphas = np.logspace(-4, 4, 100)

ridge_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", Ridge()),
    ]
)
train_scores, test_scores = validation_curve(
    ridge_pipe,
    X,
    y,
    param_name="model__alpha",
    param_range=alphas,
    cv=5,
    scoring="neg_mean_squared_error",
)

plt.plot(alphas, train_scores.mean(axis=1), label="train")
plt.plot(alphas, test_scores.mean(axis=1), label="test")
plt.axvline(
    best_alpha_ridge, color="black", linestyle="--", label="RidgeCV alpha"
)
plt.xscale("log")
plt.xlabel("alpha")
plt.ylabel("score")
plt.title("Validation curve for ridge regression")
plt.legend()
```

①

- ① The validation curve can also be drawn by `GridSearchCV.cv_results_`. There are no major difference between the two methods.



Similarly, if we want to examine how the coefficients change with `alpha`, we fit ordinary Ridge repeatedly over a grid of `alpha` values and record the fitted coefficients. In this case, cross-validation is not needed, since the goal is only to visualize the coefficient path rather than to compare models.

```
from sklearn.linear_model import Ridge
import matplotlib.pyplot as plt

alphas = np.logspace(-4, 4, 100)
coefs = []
for a in alphas:
    pipe = Pipeline(
        [
            ("scaler", StandardScaler()),
            ("model", Ridge(alpha=a)),
        ]
    )
    pipe.fit(X, y)
    coefs.append(pipe.named_steps["model"].coef_)
coefs = np.array(coefs)
for j in range(coefs.shape[1]):
    plt.plot(alphas, coefs[:, j], label=f"x{j + 1}")
plt.axvline(best_alpha_ridge, linestyle="--", color="black", label="RidgeCV alpha")
```

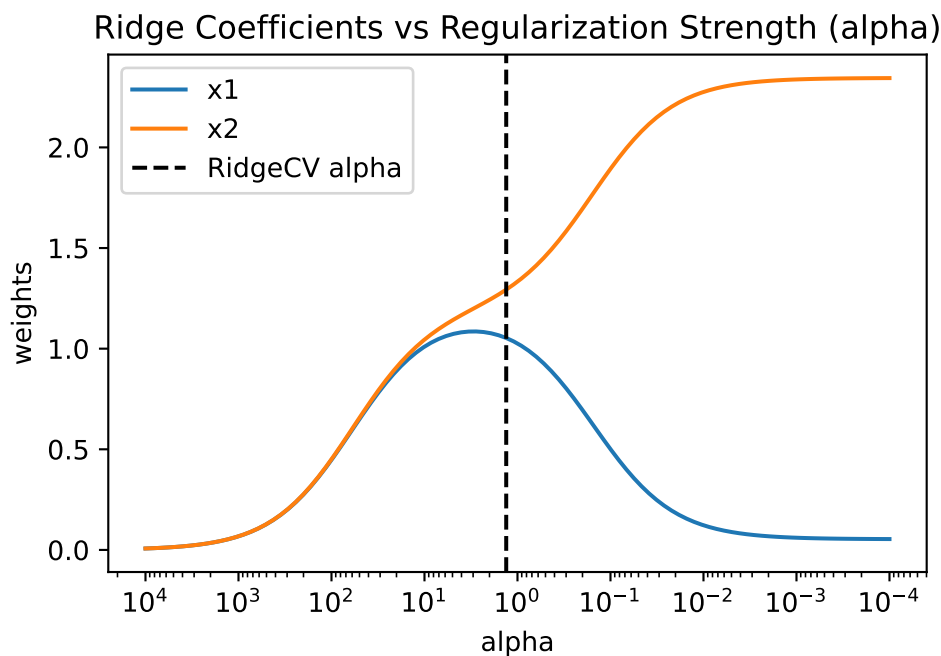
```

ax = plt.gca()
ax.set_xlim(ax.get_xlim()[::-1])
plt.xscale("log")
plt.xlabel("alpha")
plt.ylabel("weights")
plt.title("Ridge Coefficients vs Regularization Strength (alpha)")
plt.axis("tight")
plt.legend()

```

①

- ① In the coefficient path plot, the horizontal axis is shown in reverse order so that the coefficients move from heavily regularized models on the left to weakly regularized models on the right.



4.3.5 LOOCV

Naively, LOOCV would require fitting the model N times. Each time one observation is removed and the model is refit. However, ridge regression admits an analytic formula that allows the LOOCV error to be computed without refitting the model repeatedly. This is one reason ridge regression works very well with LOOCV. RidgeCV from `sklearn` is also implemented using LOOCV formula instead of refitting the model multiple times.

Theorem 4.3 (LOOCV). *The LOOCV error is*

$$CV(\lambda) = \frac{1}{N} \sum_{i=1}^N \left(\frac{y_i - \hat{y}_i}{1 - H_{\lambda,ii}} \right)^2.$$

Click for proof.

Recall that the ridge estimator is

$$\hat{\beta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y.$$

The fitted values can be written as

$$\hat{y} = X\hat{\beta}_\lambda = H_\lambda y,$$

where

$$H_\lambda = X(X^\top X + \lambda I)^{-1} X^\top$$

is the ridge hat matrix.

Let

$$r_i = y_i - \hat{y}_i$$

be the ordinary residual from the ridge regression fitted using the entire dataset.

Suppose we remove observation i and refit the ridge regression model. Let $\hat{y}_{(i)}$ denote the predicted value for y_i from this leave-one-out model. The corresponding leave-one-out residual is

$$r_{(i)} = y_i - \hat{y}_{(i)}.$$

By the PRESS residual formula [4], this residual can be computed directly from the full-data fit:

$$r_{(i)} = \frac{r_i}{1 - H_{\lambda,ii}},$$

where $H_{\lambda,ii}$ is the i -th diagonal element of the ridge hat matrix.

Therefore the LOOCV error is

$$CV(\lambda) = \frac{1}{N} \sum_{i=1}^N \left(\frac{y_i - \hat{y}_i}{1 - H_{\lambda,ii}} \right)^2.$$

i Note

This formula means that the leave-one-out residuals can be obtained by simply adjusting the ordinary residuals using the leverage term $H_{\lambda,ii}$. Consequently, the LOOCV error can be computed without refitting the model N times.

4.4 Lasso Regression

4.4.1 Definition

Lasso regression (short for least absolute shrinkage and selection operator) modifies the OLS objective by adding an L_1 penalty.

Definition 4.3 (Lasso Regression). For a response vector $y \in \mathbb{R}^n$ and a design matrix $X \in \mathbb{R}^{n \times p}$, the lasso estimator $\hat{\beta}_{\text{lasso}}$ is defined by

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \left\{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 \right\}$$

where

- $\lambda \geq 0$ is the regularization parameter,
- $\|\beta\|_1 = \sum_{j=1}^p |\beta_j|$ is the L_1 norm of the coefficient vector.

i Solving lasso

Lasso is solved using numerical optimization algorithms. The most widely used methods include

- coordinate descent
- LARS (Least Angle Regression)
- proximal gradient methods

These algorithms iteratively update the coefficients until convergence.

Unlike OLS or ridge regression, the lasso estimator does not admit a closed-form solution because the L_1 penalty is not differentiable at zero.

The lasso penalty also shrinks the coefficients toward zero, and therefore behaves similarly to ridge regression. In practice, the penalty is typically applied only to the slope coefficients, so the intercept is not penalized. For this reason, it is common to standardize the predictors and center the response variable before fitting the model.

However, lasso has an important difference from ridge regression, which we discuss in the next section.

4.4.2 Geometry and sparsity

Similar to ridge regression, by the theory of Lagrange multipliers, lasso can also be written in constrained form:

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \|y - X\beta\|_2^2 \quad \text{subject to} \quad \|\beta\|_1 \leq s.$$

Here $s \geq 0$ controls the size of the coefficient vector. This constrained formulation is useful because it makes the geometry of lasso easier to visualize.

In $\mathbb{R}^2$, the ridge constraint region is based on the L_2 norm and has a circular boundary, whereas the lasso constraint region is based on the L_1 norm and has a diamond-shaped boundary. The solution is obtained where the smallest residual RSS contour first touches the constraint region.

For ridge regression, the circular boundary is smooth, so the point of tangency typically occurs away from the coordinate axes. As a result, ridge usually shrinks coefficients continuously toward zero, but does not make them exactly zero.

For lasso, the diamond-shaped boundary has sharp corners lying on the coordinate axes. Because of these corners, the point of tangency is much more likely to occur at a location where one coordinate is zero. Consequently, lasso can produce solutions with some coefficients exactly equal to zero.

Therefore, ridge is mainly a shrinkage method, whereas lasso is both a shrinkage method and a variable-selection method.

```
import numpy as np
import matplotlib.pyplot as plt

mu = np.array([2.0, 0.35])

theta = np.deg2rad(32)
R = np.array([[np.cos(theta), -np.sin(theta)], [np.sin(theta), np.cos(theta)]])

D = np.diag([7.0, 0.45])
A = R @ D @ R.T

def rss(beta1, beta2):
```

```

B = np.stack([beta1 - mu[0], beta2 - mu[1]], axis=-1)
return np.einsum("...i,ij,...j->...", B, A, B)

x = np.linspace(-0.5, 3.2, 600)
y = np.linspace(-2.0, 2.0, 600)
X, Y = np.meshgrid(x, y)
Z = rss(X, Y)

levels = [0.15, 0.4, 0.8, 1.4, 2.4, 3.8, 5.8]

ridge_r = 1.7
lasso_t = 2.0

fig, axs = plt.subplots(1, 2, figsize=(10, 5))

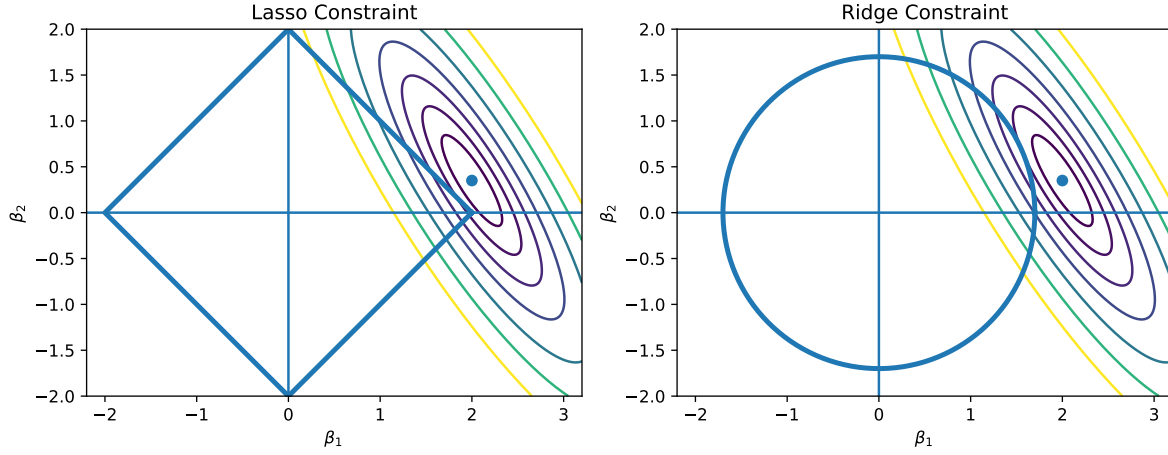
axs[0].contour(X, Y, Z, levels=levels)
diamond = np.array(
    [[lasso_t, 0], [0, lasso_t], [-lasso_t, 0], [0, -lasso_t], [lasso_t, 0]]
)
axs[0].plot(diamond[:, 0], diamond[:, 1], linewidth=3)
axs[0].scatter(*mu)
axs[0].axhline(0)
axs[0].axvline(0)
axs[0].set_title("Lasso Constraint")

axs[1].contour(X, Y, Z, levels=levels)
t = np.linspace(0, 2 * np.pi, 400)
axs[1].plot(ridge_r * np.cos(t), ridge_r * np.sin(t), linewidth=3)
axs[1].scatter(*mu)
axs[1].axhline(0)
axs[1].axvline(0)
axs[1].set_title("Ridge Constraint")

for ax in axs:
    ax.set_xlim(-2.2, 3.2)
    ax.set_ylim(-2, 2)
    ax.set_aspect("equal")
    ax.set_xlabel(r"$\beta_1$")
    ax.set_ylabel(r"$\beta_2$")

plt.tight_layout()

```



4.4.3 One-variable lasso and soft-thresholding

To understand how lasso sets coefficients to 0, it is useful to first look at the one-variable case.

Suppose we want to solve

$$\min_{\beta} \left\{ \|y - X\beta\|_2^2 + \lambda|\beta| \right\}.$$

Note that

$$\hat{\beta}_{\text{OLS}} = \arg \min_{\beta} \left\{ \|y - X\beta\|_2^2 \right\}$$

denotes the OLS solution for this one-variable problem.

Theorem 4.4 (Soft-thresholding Rule).

$$\hat{\beta}_{\text{lasso}} = \begin{cases} \hat{\beta}_{\text{OLS}} + \frac{\lambda}{2a} & \text{if } \hat{\beta}_{\text{OLS}} < -\frac{\lambda}{2a}, \\ 0 & \text{if } |\hat{\beta}_{\text{OLS}}| \leq \frac{\lambda}{2a}, \\ \hat{\beta}_{\text{OLS}} - \frac{\lambda}{2a} & \text{if } \hat{\beta}_{\text{OLS}} > \frac{\lambda}{2a}, \end{cases}$$

Click to expand.

Since there is only one β ,

$$\text{RSS}(\beta) = \|y - X\beta\|_2^2 + \lambda|\beta| = \left(\sum x_i^2 \right) \beta^2 - 2 \left(\sum x_i y_i \right) \beta + \sum y_i^2 + \lambda|\beta|.$$

Let $a = \sum x_i^2$, $c = \sum x_i y_i$. Then

$$\text{RSS}(\beta) = a\beta^2 - 2c\beta + \lambda|\beta| + \text{constant}.$$

Note that

$$\hat{\beta}_{\text{OLS}} = (X^\top X)^{-1} X^\top y = \frac{\sum x_i y_i}{\sum x_i^2} = \frac{c}{a}.$$

Now due to absolute value we split into different cases.

If $\beta \geq 0$: Then $|\beta| = \beta$. Therefore $\text{RSS}(\beta) = a\beta^2 - 2c\beta + \lambda\beta + \text{constant}$. Let $\frac{d\text{RSS}(\beta)}{d\beta} = 2a\beta - 2c + \lambda = 0$. So $\beta = \frac{c}{a} - \frac{\lambda}{2a} = \hat{\beta}_{\text{OLS}} - \frac{\lambda}{2a}$ is the critical point.

In this case since $\beta \geq 0$,

- if $\hat{\beta}_{\text{OLS}} + \frac{\lambda}{2a} > 0$, then the minimizer is $\hat{\beta}_{\text{OLS}} + \frac{\lambda}{2a}$.
- Otherwise, the minimum over the region $\beta \geq 0$ is attained at 0.

Therefore

$$\hat{\beta}_{\text{lasso}} = \begin{cases} \hat{\beta}_{\text{OLS}} - \frac{\lambda}{2a} & \text{if } \hat{\beta}_{\text{OLS}} > \frac{\lambda}{2a}, \\ 0 & \text{if } \hat{\beta}_{\text{OLS}} \leq \frac{\lambda}{2a}. \end{cases}$$

If $\beta \leq 0$: Then $|\beta| = -\beta$. Therefore $\text{RSS}(\beta) = a\beta^2 - 2c\beta - \lambda\beta + \text{constant}$. Let $\frac{d\text{RSS}(\beta)}{d\beta} = 2a\beta - 2c - \lambda = 0$. So $\beta = \frac{c}{a} + \frac{\lambda}{2a} = \hat{\beta}_{\text{OLS}} + \frac{\lambda}{2a}$ is the critical point.

In this case since $\beta \leq 0$,

- if $\hat{\beta}_{\text{OLS}} + \frac{\lambda}{2a} < 0$, then the minimizer is $\hat{\beta}_{\text{OLS}} + \frac{\lambda}{2a}$.
- Otherwise, the minimum over the region $\beta \leq 0$ is attained at 0.

In other words,

$$\hat{\beta}_{\text{lasso}} = \begin{cases} \hat{\beta}_{\text{OLS}} + \frac{\lambda}{2a} & \text{if } \hat{\beta}_{\text{OLS}} < -\frac{\lambda}{2a}, \\ 0 & \text{if } \hat{\beta}_{\text{OLS}} \geq -\frac{\lambda}{2a} \end{cases}$$

To sum up,

$$\hat{\beta}_{\text{lasso}} = \begin{cases} \hat{\beta}_{\text{OLS}} + \frac{\lambda}{2a} & \text{if } \hat{\beta}_{\text{OLS}} < -\frac{\lambda}{2a}, \\ 0 & \text{if } |\hat{\beta}_{\text{OLS}}| \leq \frac{\lambda}{2a}, \\ \hat{\beta}_{\text{OLS}} - \frac{\lambda}{2a} & \text{if } \hat{\beta}_{\text{OLS}} > \frac{\lambda}{2a}, \end{cases}$$

This formula shows the essential behavior of lasso:

- if the OLS coefficient is large enough in magnitude, it is shrunk toward zero;
- if the OLS coefficient is small enough, it is shrunk all the way to zero.

i Orthogonal design

If the design matrix satisfies $X^\top X = nI$, then the lasso problem separates into p one-variable problems. In that case, each lasso coefficient is obtained by applying soft-thresholding to the corresponding OLS coefficient.

This is the clearest way to see why lasso performs variable selection.

4.5 Lasso in Python

In `scikit-learn`, `Lasso` and `LassoCV` from `linear_model` are used in a way very similar to `Ridge` and `RidgeCV`.

- The argument `alpha` is the regularization parameter.
- `fit_intercept=True` means the intercept is handled automatically.
- If `fit_intercept=False`, the data is expected to be centered beforehand.
- It is recommended to standardize the features X .

A key computational difference from ridge is that lasso does not have a closed-form estimator. Likewise, unlike ridge regression, there is no closed-form LOOCV formula available for selecting the tuning parameter. Therefore, `LassoCV` uses ordinary K -fold cross-validation to choose the best value of `alpha`.

4.5.1 A simulated example

To illustrate lasso and ridge in code, we start with a simulated dataset. The data are constructed so that:

- there are 100 predictors and 150 observations,
- many predictors are almost collinear (that they are obtained from a rank 12 latent structure),
- but only 10 predictors truly contribute to the response.

```

import numpy as np

rng = np.random.default_rng(0)

n = 150
p = 100
k = 12

Z = rng.normal(size=(n, k))
A = rng.normal(size=(k, p))
X = Z @ A + 0.8 * rng.normal(size=(n, p))

beta = np.zeros(p)
beta[:10] = [8, 7, 6, 5, 4, 3, 2, 1.5, 1, 0.5]

y = X @ beta + 12 * rng.normal(size=n)

```

We split the data into training and test sets.

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

```

4.5.2 Initial model fitting

Before tuning the regularization parameter, we first fit OLS, ridge, and lasso with simple default or manually chosen settings. This gives a first look at how the methods behave on the same dataset.

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

ols_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", LinearRegression()),
    ]
)

```

```

)

ols_pipe.fit(X_train, y_train)
y_pred_ols = ols_pipe.predict(X_test)

print("OLS")
print(f"Test RMSE: {mean_squared_error(y_test, y_pred_ols)}")
print(f"Test R^2 : {r2_score(y_test, y_pred_ols)}")

```

OLS
Test RMSE: 1212.515485649412
Test R² : 0.49121698288112325

```

from sklearn.linear_model import Ridge

ridge_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", Ridge(alpha=1.0)),
    ]
)
ridge_pipe.fit(X_train, y_train)
y_pred_ridge = ridge_pipe.predict(X_test)

print("Ridge")
print(f"Test RMSE: {mean_squared_error(y_test, y_pred_ridge)}")
print(f"Test R^2 : {r2_score(y_test, y_pred_ridge)}")

```

Ridge
Test RMSE: 249.13745912332723
Test R² : 0.8954595552549109

```

from sklearn.linear_model import Lasso

lasso_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", Lasso(alpha=0.1, max_iter=10000)),
    ]
)

```



```

lasso_pipe.fit(X_train, y_train)
y_pred_lasso = lasso_pipe.predict(X_test)

print("\nLasso")
print(f"Test RMSE: {mean_squared_error(y_test, y_pred_lasso)}")
print(f"Test R^2 : {r2_score(y_test, y_pred_lasso)}")

```

```

Lasso
Test RMSE: 204.53280796906878
Test R^2 : 0.9141760906397303

```

One important difference between ridge and lasso is that lasso can set some coefficients exactly to zero, while ridge typically only shrinks coefficients toward zero without eliminating them.

```

ridge_coef = ridge_pipe.named_steps["model"].coef_
lasso_coef = lasso_pipe.named_steps["model"].coef_

print("Number of nonzero ridge coefficients:", np.sum(ridge_coef != 0))
print("Number of nonzero lasso coefficients:", np.sum(lasso_coef != 0))

```

```

Number of nonzero ridge coefficients: 100
Number of nonzero lasso coefficients: 64

```

You can see that in the lasso case, many coefficients are exactly 0. This is one reason lasso is often viewed as performing both regularization and variable selection.

4.5.3 Coefficient paths

To better understand how the regularization parameter affects the estimates, we now plot the coefficient paths.

- In the lasso plot, some coefficient curves hit exactly 0 as **alpha** increases.
- In the ridge plot, all coefficients are shrunk continuously toward 0, but usually remain nonzero.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import lasso_path, Ridge
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

fig, axs = plt.subplots(1, 2, figsize=(10, 5))

alphas_lasso, coefs_lasso, _ = lasso_path(X_train_scaled, y_train)

axs[0].plot(alphas_lasso, coefs_lasso.T)
axs[0].set_title("Lasso coefficient paths")

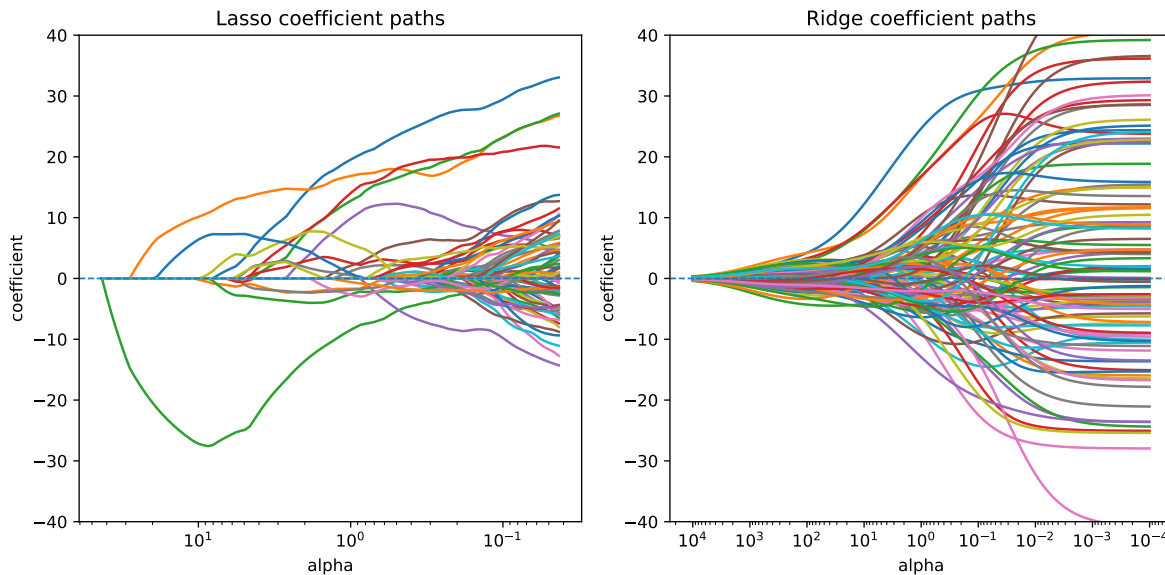
alphas_ridge = np.logspace(-4, 4, 100)
coefs_ridge = []
for a in alphas_ridge:
    ridge = Ridge(alpha=a)
    ridge.fit(X_train_scaled, y_train)
    coefs_ridge.append(ridge.coef_)
coefs_ridge = np.array(coefs_ridge)

axs[1].plot(alphas_ridge, coefs_ridge)
axs[1].set_title("Ridge coefficient paths")

for ax in axs:
    ax.set_xscale("log")
    ax.axhline(0, linestyle="--", linewidth=1)
    ax.invert_xaxis()
    ax.set_ylim(-40, 40)
    ax.set_xlabel("alpha")
    ax.set_ylabel("coefficient")

plt.tight_layout()

```



4.5.4 Tuning with cross-validation

The choice of `alpha` strongly affects the fitted model. Therefore in practice we usually tune it by cross-validation.

We first use `LassoCV` to select the best value of `alpha`.

```
from sklearn.linear_model import LassoCV

lasso_cv_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LassoCV(cv=5, max_iter=10000, random_state=0)),
])

lasso_cv_pipe.fit(X_train, y_train)
y_pred_lasso_cv = lasso_cv_pipe.predict(X_test)

best_alpha_lasso = lasso_cv_pipe.named_steps["model"].alpha_

print("Best lasso alpha:", best_alpha_lasso)
print("LassoCV Test RMSE:", mean_squared_error(y_test, y_pred_lasso_cv))
print("LassoCV Test R^2 :", r2_score(y_test, y_pred_lasso_cv))
```

```
Best lasso alpha: 0.4608014833934263
LassoCV Test RMSE: 221.76483520224215
LassoCV Test R^2 : 0.9069453683021322
```

For comparison, we also fit ridge regression with RidgeCV.

```
from sklearn.linear_model import RidgeCV

ridge_cv_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", RidgeCV()),
    ]
)

ridge_cv_pipe.fit(X_train, y_train)
y_pred_ridge_cv = ridge_cv_pipe.predict(X_test)

best_alpha_ridge = ridge_cv_pipe.named_steps["model"].alpha_

print("Best lasso alpha:", best_alpha_ridge)
print("LassoCV Test RMSE:", mean_squared_error(y_test, y_pred_ridge_cv))
print("LassoCV Test R^2 :", r2_score(y_test, y_pred_ridge_cv))
```

```
Best lasso alpha: 10.0
LassoCV Test RMSE: 274.2521785548484
LassoCV Test R^2 : 0.8849211803824286
```

After obtaining the best regularization parameters, we mark them on the coefficient path plots. This helps connect the selected model to the whole regularization path.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import lasso_path, Ridge
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
```

```

alphas_lasso, coefs_lasso, _ = lasso_path(X_train_scaled, y_train)

axs[0].plot(alphas_lasso, coefs_lasso.T)
axs[0].axvline(
    best_alpha_lasso, linestyle="--", color="black", label="LassoCV alpha"
)
axs[0].set_title("Lasso coefficient paths")

alphas_ridge = np.logspace(-4, 4, 100)
coefs_ridge = []

for a in alphas_ridge:
    ridge = Ridge(alpha=a)
    ridge.fit(X_train_scaled, y_train)
    coefs_ridge.append(ridge.coef_)

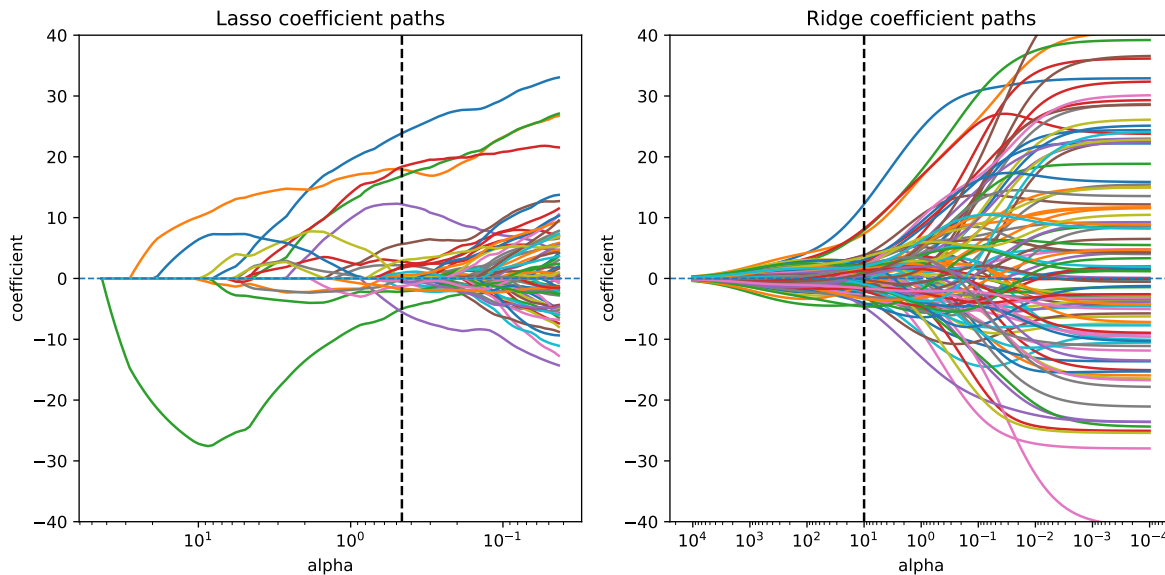
coefs_ridge = np.array(coefs_ridge)

axs[1].plot(alphas_ridge, coefs_ridge)
axs[1].axvline(
    best_alpha_ridge, linestyle="--", color="black", label="RidgeCV alpha"
)
axs[1].set_title("Ridge coefficient paths")

for ax in axs:
    ax.set_xscale("log")
    ax.axhline(0, linestyle="--", linewidth=1)
    ax.invert_xaxis()
    ax.set_ylim(-40, 40)
    ax.set_xlabel("alpha")
    ax.set_ylabel("coefficient")

plt.tight_layout()

```



4.5.5 Validation curves

Another way to visualize tuning is through the validation curve. Here we plot the mean training and validation scores across a grid of **alpha** values. Note that the score is negative MSE. Therefore the higher the better.

- When **alpha** is too small, the model may overfit.
- When **alpha** is too large, the model may underfit.
- A good value of **alpha** balances these two effects.

```
from sklearn.model_selection import validation_curve
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge, Lasso
import numpy as np
import matplotlib.pyplot as plt

alphas = np.logspace(-4, 4, 100)

fig, axs = plt.subplots(1, 2, figsize=(10, 4.5))

lasso_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", Lasso(max_iter=20000)),
```

```

    ]
)

train_scores_lasso, test_scores_lasso = validation_curve(
    lasso_pipe,
    X,
    y,
    param_name="model__alpha",
    param_range=alphas,
    cv=5,
    scoring="neg_mean_squared_error",
)

axs[0].plot(alphas, train_scores_lasso.mean(axis=1), label="train")
axs[0].plot(alphas, test_scores_lasso.mean(axis=1), label="test")
axs[0].axvline(
    best_alpha_lasso, color="black", linestyle="--", label="best alpha"
)
axs[0].set_title("Validation curve for lasso")

ridge_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("model", Ridge()),
    ]
)

train_scores_ridge, test_scores_ridge = validation_curve(
    ridge_pipe,
    X,
    y,
    param_name="model__alpha",
    param_range=alphas,
    cv=5,
    scoring="neg_mean_squared_error",
)

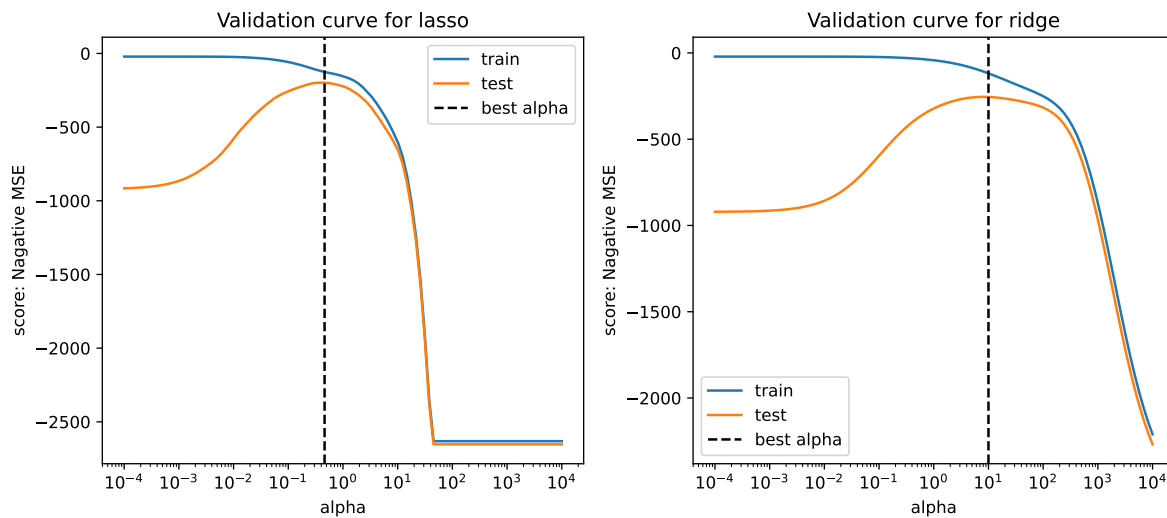
axs[1].plot(alphas, train_scores_ridge.mean(axis=1), label="train")
axs[1].plot(alphas, test_scores_ridge.mean(axis=1), label="test")
axs[1].axvline(
    best_alpha_ridge, color="black", linestyle="--", label="best alpha"
)

```

```
)
axs[1].set_title("Validation curve for ridge")

for ax in axs:
    ax.set_xscale("log")
    ax.set_xlabel("alpha")
    ax.set_ylabel("score: Negative MSE")
    ax.legend()

plt.tight_layout()
```



4.5.6 Summary

Finally, we summarize the test performance and the number of nonzero coefficients for each model.

```
import pandas as pd

results = []

models = {
    "OLS": ols_pipe,
    "RidgeCV": ridge_cv_pipe,
    "LassoCV": lasso_cv_pipe,
```



```

}

for name, model in models.items():
    y_pred = model.predict(X_test)
    rmse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    n_nonzero = np.sum(model.named_steps["model"].coef_ != 0)
    alpha = model.named_steps["model"].alpha_ if name != "OLS" else None

    results.append({
        'Model': name,
        'Best alpha': alpha,
        'Test RMSE': rmse,
        'Test R^2': r2,
        'Nonzero coefs': n_nonzero
    })

pd.DataFrame(results)

```

	Model	Best alpha	Test RMSE	Test R ²	Nonzero coefs
0	OLS	NaN	1212.515486	0.491217	100
1	RidgeCV	10.000000	274.252179	0.884921	100
2	LassoCV	0.460801	221.764835	0.906945	27

This example shows the main practical difference between ridge and lasso:

- Ridge shrinks coefficients continuously and is especially useful when predictors are strongly correlated.
- Lasso also shrinks coefficients, but can force some of them to be exactly zero, producing a sparse model.

In high-dimensional settings with collinearity and only a subset of truly relevant predictors, lasso often provides a more interpretable model, while ridge often gives more stable coefficient estimates.

5 Dimension reduction

When the number of predictors is large or when predictors are highly correlated, directly fitting a regression model using the original predictors may lead to unstable estimates and poor predictive performance.

One approach to address this problem is dimension reduction. Instead of using the original predictors directly, we first construct a smaller number of derived variables (called components) and then perform regression using these components.

Two commonly used methods are

- Principal Component Regression (PCR)
- Partial Least Squares (PLS)

Both methods construct new variables that are linear combinations of the original predictors, but they differ in how those combinations are chosen.

5.1 Principal Component Regression (PCR)

PCR first applies principal component analysis (PCA) to the predictor matrix X , and then uses the leading principal components as predictors in a regression model.

The key idea is that most of the variation in the predictors often lies in a lower-dimensional subspace.

Definition 5.1 (Principal Component Regression). Let X be the $n \times p$ predictor matrix.

The principal components are defined as

$$Z_k = Xv_k,$$

where v_k is the k -th eigenvector of the Gram matrix $X^\top X$.

Principal component regression fits a linear model using the first m components:

$$y = \theta_0 + \theta_1 Z_1 + \theta_2 Z_2 + \cdots + \theta_m Z_m + \varepsilon.$$

Here

- $Z_1, \dots, Z_m$ are the first m principal components
- $m < p$ is chosen by cross-validation

Thus PCR replaces the original predictors with a smaller set of orthogonal components.

i Key feature of PCR

PCR constructs components using only the predictor matrix X , without considering the response y .

As a result, the directions that explain the most variation in X are not necessarily the directions that are most predictive of y .

5.1.1 Data matrix and its geometric views

Let $X \in \mathbb{R}^{n \times p}$ be a data matrix with n observations and p predictors. We assume columns are centered ($\sum_{i=1}^n x_{ij} = 0$) so that the origin represents the sample mean.

Column (variable) view: observation space $\mathbb{R}^n$

Each column of X_j is a vector in n -dimensional space, which represents one predictor across all observations.

- This space is often called the observation space.
- Geometry in this space describes relationships between predictors:
 - Inner products encode covariance.
 - After standardization, angles encode correlation.
 - Orthogonality corresponds to zero correlation.

Row (observation) view: feature space $\mathbb{R}^p$

Each row of $x_i^\top$ is a vector in p -dimensional space, which represents one observation across all predictors.

- This space is called the feature space.
- Geometry in this space describes relationships between observations:
 - Euclidean distance measures dissimilarity.
 - Clustering and nearest-neighbor methods operate in this space.

Example 5.1.

Click to expand.

To demonstrate the idea, we generate the following dataset. It contains 2 variables.

```

import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)

n = 100

Sigma = np.array([[0.5, -0.65], [-0.65, 1.0]])
mu = np.array([1, 2])

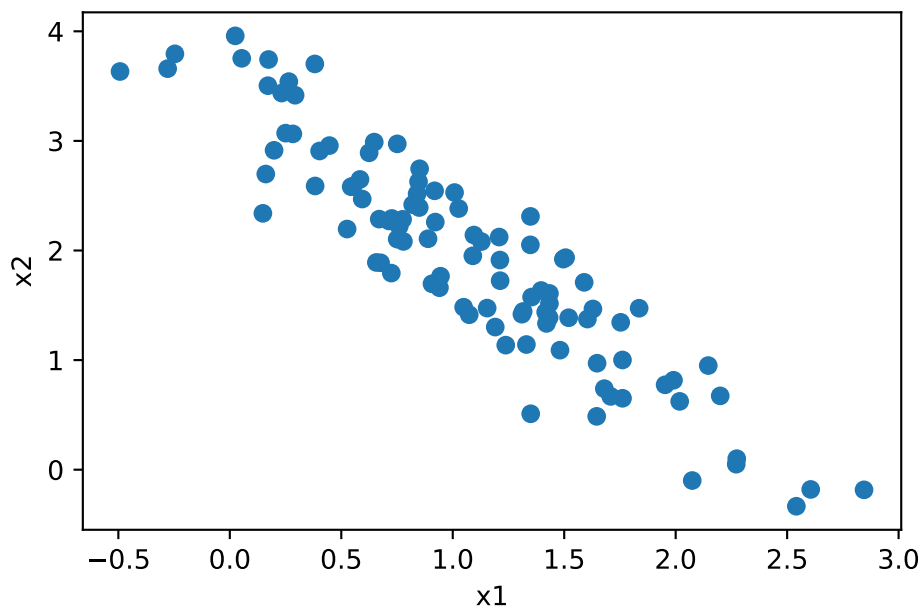
X = rng.multivariate_normal(mean=mu, cov=Sigma, size=n)

plt.scatter(X[:, 0], X[:, 1])

plt.xlabel("x1")
plt.ylabel("x2")

```

Text(0, 0.5, 'x2')



Each point is an observation and is represented by a point in the feature space.

5.1.2 Singular Value Decomposition (SVD)

The SVD provides the most direct link between the two geometric perspectives.

For a matrix with $\text{rank}(X) = r$

$$X = UDV^\top,$$

where

- $U \in \mathbb{R}^{n \times r}$, $U^\top U = I$
 - $D = \text{diag}(d_1, \dots, d_r)$, $d_1 \geq \dots \geq d_r > 0$
 - $V \in \mathbb{R}^{p \times r}$, $V^\top V = I$
-

5.1.3 Principal components (PC)

Principal component analysis (PCA) constructs new variables as linear combinations of the original predictors. It builds upon SVD.

Let $Z = XV = UD = [Z_1, \dots, Z_r] = [z_{ij}]$ denote the principal components. Each principal component has the form

$$Z_j = Xv_j = v_{1j}X_1 + v_{2j}X_2 + \dots + v_{pj}X_p$$

where $v_j = [v_{1j} \ \dots \ v_{pj}]^\top \in \mathbb{R}^p$ is a unit vector.

Loadings (directions)

- Each PC is a linear combination of the columns of X .
- **Loadings** (v_{ij}): The coefficients used to form PC Z_j are called loadings.
- Loadings describe directions in feature space.

Scores (coordinates)

- As a vector, each PC lies in $\mathbb{R}^n$ (observation space).
- **Scores** (z_{ij}): The coordinate of observation i along direction v_j is called the score.

Thus:

- rows of Z = transformed observations,
- columns of Z = principal components.

Example 5.2 (Centering and Scaling).

Click to expand.

We use the same previous dataset X , centering it as X_c and standarizing it as X_s .

```
Xc = X - X.mean(axis=0)
Xs = Xc / Xc.std(axis=0)
```

Now we plot them together with the princial compoent. `pca()` from `sklearn` will automatically centering columns. In order to show the sceanario, we use `svd()` from `numpy.linalg` to do the computation.

```
import matplotlib.pyplot as plt

def pca_svd(X):
    U, D, Vt = np.linalg.svd(X, full_matrices=False)
    return Vt.T, D

def plot_svd(ax, X, V, title, scale=3, length=5):
    ax.scatter(X[:, 0], X[:, 1])
    t = np.linspace(-length, length, 100)
    line = np.outer(t, V[:, 0])
    ax.plot(line[:, 0], line[:, 1], linewidth=2, color="red")

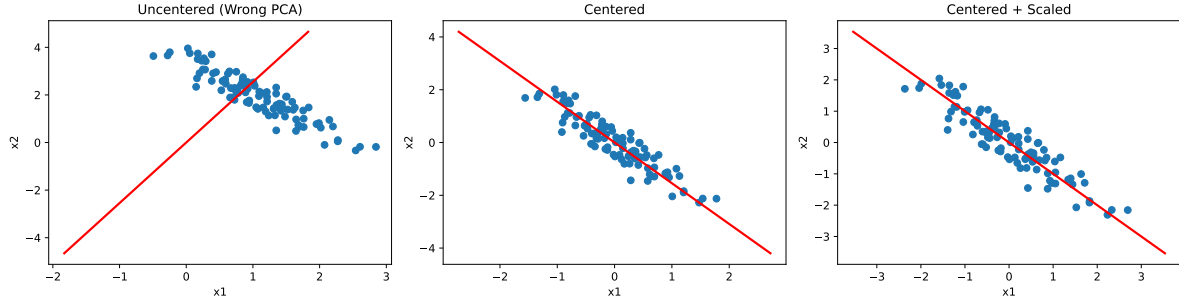
    ax.set_title(title)
    ax.set_xlabel("x1")
    ax.set_ylabel("x2")

V_raw = np.linalg.svd(X, full_matrices=False)[2].T
V_c = np.linalg.svd(Xc, full_matrices=False)[2].T
V_s = np.linalg.svd(Xs, full_matrices=False)[2].T

fig, axs = plt.subplots(1, 3, figsize=(15, 4))

plot_svd(axs[0], X, V_raw, "Uncentered (Wrong PCA)")
plot_svd(axs[1], Xc, V_c, "Centered")
plot_svd(axs[2], Xs, V_s, "Centered + Scaled")

plt.tight_layout()
```



The principal component finds the direction that maximizes the variance of the projected data.

If the data are not centered, the optimization is no longer based purely on variance. In this case, the objective involves both the covariance structure and the mean of the data.

As a result, when the mean is large, the first principal direction may be strongly influenced by the location of the data, and can point roughly toward the data cloud rather than reflecting its intrinsic variation.

Note that the last two plots may appear similar at first glance. However, their coordinate scales are different, so this visual similarity can be misleading.

5.1.4 Covariance and Optimality

The sample covariance matrix is $S = \frac{1}{n-1} X^\top X$. By SVD,

$$S = \frac{1}{n-1} X^\top X = V \frac{D^2}{n-1} V^\top$$

- V : eigenvectors of covariance matrix
- $d_j^2/(n-1)$: eigenvalues of S
- $\text{Var}(Z_j) = d_j^2/(n-1)$

Therefore

- First PC: Direction v_1 that maximizes $\text{Var}(Xv_1)$ subject to $\|v_1\| = 1$.
 - Subsequent PCs: Maximize remaining variance subject to orthogonality (that they are orthogonal to all previous PCs $v_k^\top v_j = 0$ for $j < k$).
-

5.1.5 Summary

Component	Matrix	Space	Interpretation
Loadings	V	Feature ($\mathbb{R}^p$)	Principal directions (rotation of axes)
Scores	$Z = UD$	Observation ($\mathbb{R}^n$)	Transformed coordinates of observations
Normalized scores	U	Observation ($\mathbb{R}^n$)	Directions of score vectors (unit length)
Singular values	D	—	Scaling of each component (spread)
Variance	$D^2/(n-1)$	—	Importance of each principal component

- Columns of X : variables in $\mathbb{R}^n$ (relationships via angles)
- Rows of X : observations in $\mathbb{R}^p$ (relationships via distance)
- PCA finds orthogonal directions in feature space
- V gives directions, Z gives coordinates
- SVD unifies everything:

$$X = UDV^\top, \quad Z = XV = UD$$

5.1.6 Choosing the number of components (m)

Let $\text{rank}(X) = r$. PCA produces r components, but in practice we keep only the first $m \leq r$.

Define the truncated SVD:

$$X \approx X_m = U_m D_m V_m^\top,$$

where

- $U_m \in \mathbb{R}^{n \times m}$
- $D_m \in \mathbb{R}^{m \times m}$
- $V_m \in \mathbb{R}^{p \times m}$

Correspondingly,

$$Z_m = XV_m = U_m D_m$$

contains the first m principal components.

Keeping the m largest singular values gives the best rank- m approximation of X in least squares sense.

5.1.6.1 Variance explained

The total variance is

$$\sum_{j=1}^r \frac{d_j^2}{n-1},$$

and the proportion explained by the first m components is

$$\frac{\sum_{j=1}^m d_j^2}{\sum_{j=1}^r d_j^2}.$$

5.1.6.2 Choosing m for prediction (PCR)

In predictive settings, m is typically selected via cross-validation:

1. Compute principal components.
2. Fit regression models using $m = 1, 2, \dots, r$ components.
3. Evaluate validation error.
4. Choose m with the lowest validation error.

i Scree plot (Elbow method)

The scree plot (elbow method) plots d_j^2 against j and selects m at the point where the marginal gain in variance explained drops sharply (the “elbow”). This identifies the number of components that captures most of the variation in X .

However, PCA is unsupervised: directions that explain high variance in X do not necessarily have strong predictive power for y . Therefore, in predictive settings such as PCR, m is typically chosen via cross-validation rather than the elbow method.

For unsupervised tasks (e.g., representation or clustering), the scree plot remains a reasonable and commonly used choice.

5.2 PCR in Python

5.2.1 PCA

`scikit-learn` provides PCA for computing principal components. To demonstrate the code we use the same dataset from lasso.

```

import numpy as np
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)

n = 150
p = 100
k = 12

Z = rng.normal(size=(n, k))
A = rng.normal(size=(k, p))
X = Z @ A + 0.8 * rng.normal(size=(n, p))

beta = np.zeros(p)
beta[:10] = [8, 7, 6, 5, 4, 3, 2, 1.5, 1, 0.5]

y = X @ beta + 12 * rng.normal(size=n)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

```

pca is an unsupervised learning method. Therefore to perform it we only need X.

```

from sklearn.decomposition import PCA

pca = PCA().fit(X_train)

```

All important information can be accessed from the returned object.

```

V = pca.components_.T           # loadings (V)
Z = pca.transform(X_train)      # scores (Z)
D = pca.singular_values_        # singular values (D)
var = pca.explained_variance_    # variance that are explained
ratio = pca.explained_variance_ratio_ # the ratio of variance that are explained

```

With `var` and `ratio`, we may plot the scree plot (although we won't really use it to select the number of components).

```

k = np.arange(1, len(var) + 1)

plt.figure()
plt.plot(k, ratio, marker="o")
plt.xlabel("Component index")
plt.ylabel("Proportion of variance explained")
plt.title("Scree Plot (Variance Ratio)")

cum = np.cumsum(ratio)

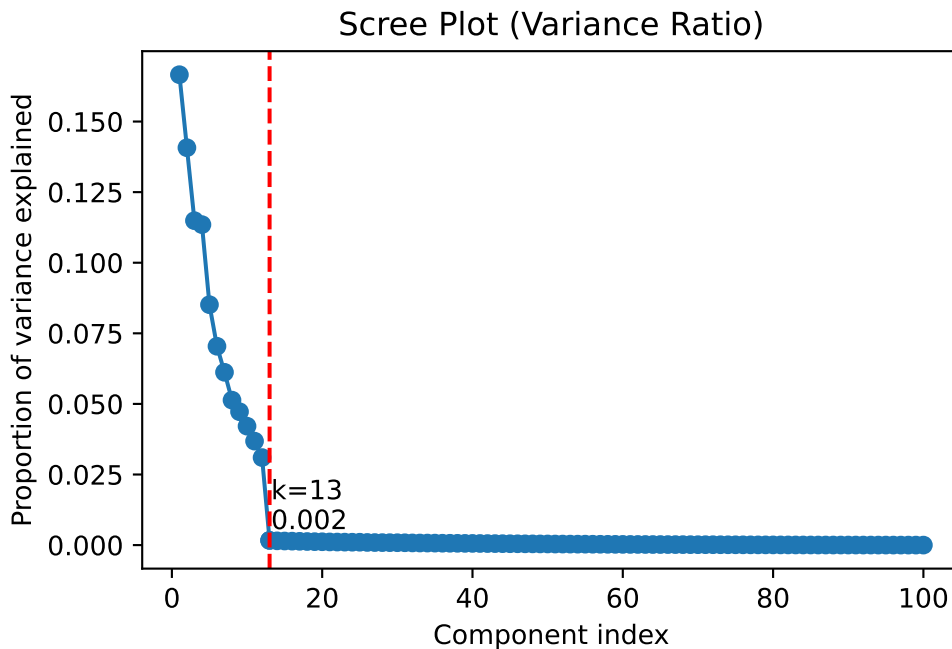
diff2 = np.diff(cum, 2)
elbow_idx = np.argmin(diff2) + 2
elbow_k = k[elbow_idx]
elbow_val = ratio[elbow_idx]
plt.axvline(elbow_k, linestyle="--", color="red")

plt.scatter(elbow_k, elbow_val)
plt.text(elbow_k + 0.2, elbow_val + 0.004, f"k={elbow_k}\n{elbow_val:.3f}")

```

① Use 2nd order difference to find the elbow.

Text(13.2, 0.005666697882320329, 'k=13\n0.002')



5.2.2 PCR

Using PCA to do regression, we have to standarize the data. Therefore we set up the pipeline.

We first use 13 components (which comes from the scree plot) as our starting point. We will tune it later using cross-validation.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

pcr_pipe = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("pca", PCA(n_components=13)),
        ("model", LinearRegression()),
    ]
)

pcr_pipe.fit(X_train, y_train)
y_pred_pcr = pcr_pipe.predict(X_test)

print(f"Test RMSE: {mean_squared_error(y_test, y_pred_pcr)}")
print(f"Test R^2 : {r2_score(y_test, y_pred_pcr)}")
```

Test RMSE: 348.7698605548008

Test R² : 0.8536528530700276

You may compare it with the results we get from lasso and ridge.

5.2.3 Choosing number of components by cross-validation

We use K -fold cross validation to find the best number of components. Our criterion is negative MSE.

```
from sklearn.model_selection import GridSearchCV

n_splits = 5
n_train_fold = int(X_train.shape[0] * (n_splits - 1) / n_splits)
max_components = min(n_train_fold, X_train.shape[1])
```

①

```

param_grid = {"pca__n_components": list(range(1, max_components + 1))}

gs_pcr = GridSearchCV(
    pcr_pipe, param_grid, cv=n_splits, scoring="neg_mean_squared_error"
)
gs_pcr.fit(X_train, y_train)
gs_pcr.best_params_["pca__n_components"]

```

- ① The maximal possible number of components depends on the shape of X. However for cross-validation, the number of rows of X is reduced due to the split. Therefore we have to estimate this number in order to avoid “n_components out of bounds” warning.

19

It shows that 19 is the best number. Then we check the test result.

```

y_pred_pcr = gs_pcr.predict(X_test)

print(f"Test RMSE: {mean_squared_error(y_test, y_pred_pcr)}")
print(f"Test R^2 : {r2_score(y_test, y_pred_pcr)}")

```

```

Test RMSE: 321.0762521471908
Test R^2 : 0.8652733542572647

```

It is slightly better than 13, but still worse than tuned **ridge** and **lasso**.

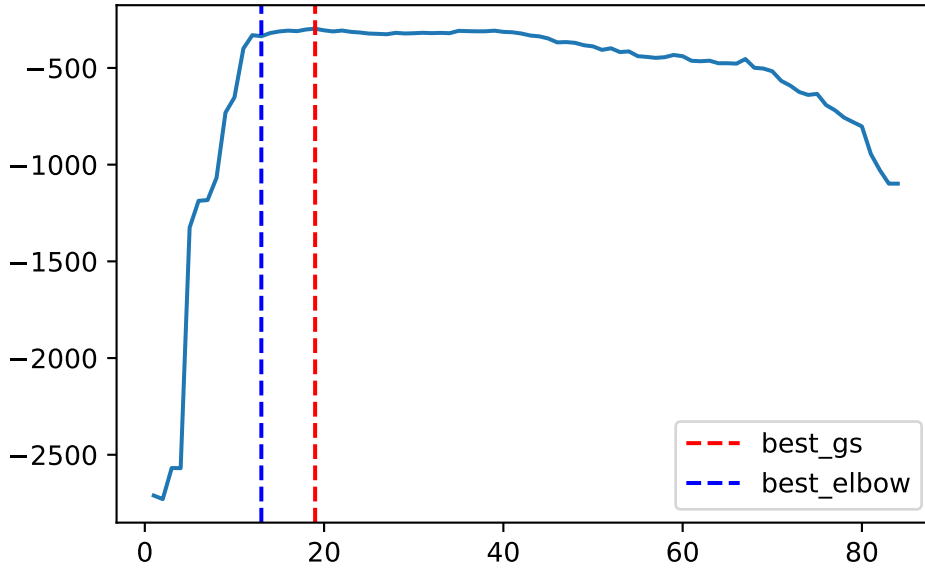
We could also look at the validation curve on the mean test score.

```

import matplotlib.pyplot as plt

plt.plot(param_grid["pca__n_components"], gs_pcr.cv_results_["mean_test_score"])
plt.axvline(19, linestyle="--", color="red", label="best_gs")
plt.axvline(13, linestyle="--", color="blue", label="best_elbow")
plt.legend()

```



5.3 Partial Least Squares (PLS)

One limitation of Principal Component Regression (PCR) is that it is unsupervised: the principal components depend only on X and ignore the response y . As a result, directions with large variance in X may not be relevant for predicting y .

Partial Least Squares (PLS) addresses this by constructing components that are guided by the response. It seeks directions in the predictor space that are both:

- informative about X , and
- strongly related to y

Definition 5.2 (Partial Least Squares). Partial least squares constructs a sequence of components

$$Z_k = Xw_k$$

where the weight vector w_k is chosen to maximize the covariance between the component and the response:

$$w_k = \arg \max_{\|w\|=1} \text{Cov}(Xw, y).$$

After constructing the component Z_k , both the predictor matrix X and the response y are deflated, and the procedure is repeated to construct additional components.

5.3.1 Algorithm intuition

PLS constructs components sequentially, with each step explicitly targeting predictive power for y , rather than variance in X .

Step 1: Find the direction. Choose a direction $w_k \in \mathbb{R}^p$ such that the component $Z_k = X_k w_k$ maximizes covariance with y_k .

$$w_k = \arg \max_{\|w\|=1} \text{Cov}(X_k w, y_k)$$

Step 2: Define the k -th component (also called the score vector):

$$Z_k = X_k w_k \in \mathbb{R}^n$$

- This is a linear combination of predictors
- Represents the direction in X most aligned with y_k

Step 3: Fit a simple regression of y_k on Z_k :

$$y_k = c_k Z_k + \text{residual}$$

This captures how strongly the component explains y_k .

Step 4: Deflation: remove the variation explained by Z_k from both X_k and y_k .

- Update response: $y_{k+1} = y_k - c_k Z_k$
- Update predictors: $X_{k+1} = X_k - Z_k p_k^\top$, where $p_k = \frac{X_k^\top Z_k}{Z_k^\top Z_k}$.
 - p_k are the loadings for reconstructing X
 - Deflation ensures the next components capture on new information

Step 5: Repeat the process for $k = 1, 2, \dots, m$.

5.3.2 Key properties

PLS has several important structural properties:

- The components $\{Z_1, Z_2, \dots\}$ are constructed sequentially.
- Each component explains the remaining variation in y .
- After deflation, the components are orthogonal.
- The ordering reflects predictive importance.

At each step, the main quantities have distinct roles:

Quantity	Interpretation
w_k	Direction in feature space used to combine predictors
$Z_k = X_k w_k$	Score (component), representing projected data
p_k	Loading, describing how the component explains X

Together, these quantities describe how information is extracted from X and transferred to explain y .

5.3.3 Comparison with PCA

The key difference between PCA and PLS lies in how the directions are chosen:

Method	Criterion	Interpretation
PCA	maximize $\text{Var}(Xw)$	captures structure of X
PLS	maximize $\text{Cov}(Xw, y)$	targets prediction of y

PCA captures the internal structure of X , while PLS focuses on directions that are useful for predicting y .

5.3.4 Geometric meaning (Krylov Subspace)

The matrix X defines a cloud of points in $\mathbb{R}^p$ and the response y defines a direction in $\mathbb{R}^n$.

PLS finds directions w_k such that the projections $Z_k = Xw_k$ are maximally aligned with y in the observation space. In this sense, PLS can be understood geometrically as searching for directions in predictor space that align with the response.

To be more precise, it can be explained using Krylov subspaces.

Click to expand.

The Krylov subspace provides a structured way to generate directions for solving regression problems. Instead of searching the entire feature space $\mathbb{R}^p$, it builds a sequence of low-dimensional subspaces that are tailored to the problem.

Given a matrix $A \in \mathbb{R}^{p \times p}$ and a vector $b \in \mathbb{R}^p$, the Krylov subspace of order k is defined as

$$\mathcal{K}_k(A, b) = \text{span}\{b, Ab, A^2b, \dots, A^{k-1}b\}.$$

The subspaces are nested, meaning $\mathcal{K}_1 \subseteq \mathcal{K}_2 \subseteq \dots$, and the dimension cannot exceed $\text{rank}(A)$. Therefore, the sequence stabilizes after at most $\text{rank}(A)$ steps.

5.3.4.1 Krylov Subspace in Regression

In linear regression, we take

$$A = X^\top X, \quad b = X^\top y.$$

Then the Krylov subspace becomes

$$\mathcal{K}_k(X^\top X, X^\top y) = \text{span}\{X^\top y, (X^\top X)X^\top y, (X^\top X)^2 X^\top y, \dots\}.$$

The vector $X^\top y$ represents directions in the predictor space that are most correlated with the response, while $X^\top X$ encodes the covariance structure of the predictors. As a result, the Krylov subspace captures directions that are both aligned with y and consistent with the geometry of X .

5.3.4.2 Connection to PLS

Partial Least Squares constructs its directions sequentially within the Krylov subspace:

$$w_k \in \mathcal{K}_k(X^\top X, X^\top y).$$

This means that PLS does not search over all possible directions in $\mathbb{R}^p$. Instead, it expands a problem-dependent subspace step by step, starting from $X^\top y$ and incorporating higher-order interactions through $X^\top X$.

The number of components is therefore bounded by

$$m \leq \dim(\mathcal{K}_k) \leq \text{rank}(X).$$

Note

The Krylov subspace can be viewed as a guided search path inside $\text{span}(X)$. The first direction is determined by $X^\top y$, and subsequent directions refine this information by incorporating the covariance structure of the predictors.

In this sense, the Krylov subspace represents the portion of the feature space that is relevant for predicting y . Methods such as PLS operate within this space, focusing on directions that carry predictive signal rather than exploring the entire feature space.

5.4 PLS in Python

Although the algorithm is different, the code is very similar to PCR. PLS is implemented by `PLSRegression` from `sklearn.cross_decomposition`. The argument `scale=True` is by default so we don't need to attach standardizer to it.

We choose the default number of components (which is 2) here. The specific hyperparameter will be tuned later by cross-validation.

```
from sklearn.cross_decomposition import PLSRegression

pls = PLSRegression()

pls.fit(X_train, y_train)
```

We could use the following code to get access to the parameters.

```
W = pls.x_weights_    # weights, directions (W)
Z = pls.x_scores_     # scores, components (Z)
P = pls.x_loadings_   # loadings (P)
```

5.4.1 Example

We use the same simulated dataset as before.

```
import numpy as np
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)

n = 150
p = 100
k = 12

Z = rng.normal(size=(n, k))
A = rng.normal(size=(k, p))
X = Z @ A + 0.8 * rng.normal(size=(n, p))

beta = np.zeros(p)
beta[:10] = [8, 7, 6, 5, 4, 3, 2, 1.5, 1, 0.5]
```

```

y = X @ beta + 12 * rng.normal(size=n)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

```

Similar to PCR, we need to choose the number of components. Since PLS is fit inside cross-validation, the maximum number of components should be no larger than the number of observations in each training fold.

```

from sklearn.model_selection import GridSearchCV
from sklearn.cross_decomposition import PLSRegression

n_splits = 5
n_train_fold = int(X_train.shape[0] * (n_splits - 1) / n_splits)
max_components = min(n_train_fold, X_train.shape[1])

param_grid = {"n_components": list(range(1, max_components + 1))}

gs_pls = GridSearchCV(
    PLSRegression(), param_grid, cv=n_splits, scoring="neg_mean_squared_error"
)
gs_pls.fit(X_train, y_train)
gs_pls.best_params_["n_components"]

```

4

After selecting the number of components by cross-validation, we evaluate the fitted model on the test set.

```

y_pred_pls = gs_pls.predict(X_test)

print(f"Test RMSE: {mean_squared_error(y_test, y_pred_pls)}")
print(f"Test R^2 : {r2_score(y_test, y_pred_pls)}")

```

```

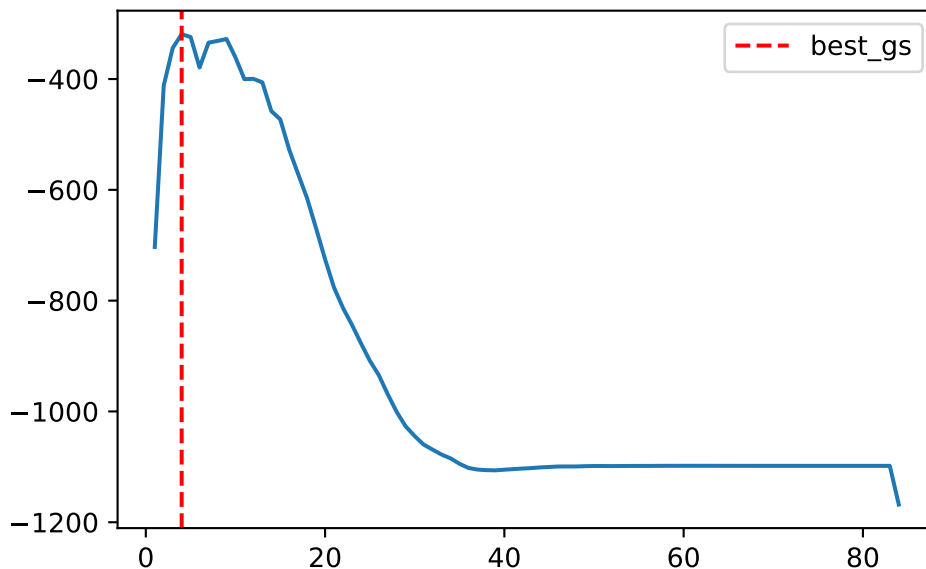
Test RMSE: 333.9484272722302
Test R^2 : 0.8598720672844287

```

The validation curve shows how the cross-validation error changes as the number of PLS components increases.

```
import matplotlib.pyplot as plt

plt.plot(param_grid["n_components"], gs_pls.cv_results_["mean_test_score"])
plt.axvline(
    gs_pls.best_params_["n_components"],
    linestyle="--",
    color="red",
    label="best_gs",
)
plt.legend()
```



i Important Notes

- **PLSRegression** automatically centers and scales the predictors by default.
- Unlike PCR, PLS uses both X and y when constructing components.
- The number of components should be chosen by cross-validation.
- Very large numbers of components are usually unnecessary and may lead to numerical warnings.

The results for this dataset are summarized in the following table.

```
<>:90: SyntaxWarning: "is not" with 'str' literal. Did you mean "!="?
```

```
<>:90: SyntaxWarning: "is not" with 'str' literal. Did you mean "!="?
```

```
C:\Users\xiaox\AppData\Local\Temp\ipykernel_32136\4187596780.py:90: SyntaxWarning: "is not" v
```

```
if (ols_in is True) or ((ols_in is False) and (name is not "OLS")):
```

	Test RMSE	Test R ²	Best param
Model			
OLS	1212.5155	0.491217	NaN
RidgeCV	274.2522	0.884921	alpha=10.0
LassoCV	221.7648	0.906945	alpha=0.4608014833934263
PCR	321.0763	0.865273	n_components=19
PLS	333.9484	0.859872	n_components=4

Note that PCR and PLS do not perform as well as ridge and lasso in this example. A key reason lies in how the dataset is generated.

- The coefficient vector β is sparse: only a small number of predictors (10 out of 100) are truly relevant.
- The noise level is relatively high, making signal recovery more difficult.
- More importantly, the variation in X is driven by a low-dimensional latent structure (ZA), while the true signal depends on a sparse set of original predictors. As a result, the directions of largest variance in X are not necessarily aligned with the directions that predict y .

Since PCR relies only on variance in X , and PLS is still influenced by this structure, both methods may fail to recover the true signal efficiently. In contrast, ridge and lasso operate directly on the original predictors, with lasso particularly well-suited for identifying sparse signals.

5.4.2 Another example

Now let us look at another example. In this setting, we construct X so that most of its variance comes from irrelevant noise, while the true signal has low variance but determines y . Specifically,

$$y = Z_1 b + \varepsilon_Y, \quad X = Z_1 A_1 + Z_2 A_2 + \varepsilon_X.$$

This setup has the following properties:

- Z_1 (signal) has low variance, while Z_2 (noise) has high variance.
- y depends only on Z_1 .
- Therefore, the predictive signal lies in low-variance but structured (low-rank) directions of X .
- For simplicity, we take Z_1 to be one-dimensional in the following example.

```

import numpy as np

rng = np.random.default_rng(42)

n = 120
p = 300

Z_1 = rng.normal(size=(n, 1))
A_1 = rng.normal(size=(1, p))
X_signal = Z_1 @ A_1 * 0.15

k_2 = 10
Z_2 = rng.normal(size=(n, k_2))
A_2 = rng.normal(size=(k_2, p))
X_noise = Z_2 @ A_2 * 15

X = X_noise + X_signal + 0.5 * rng.normal(size=(n, p))

y = 12 * Z_1.ravel() + 0.5 * rng.normal(size=n)

```

	Test RMSE	Test R ²	Best param
Model			
RidgeCV	34.7296	0.682587	alpha=0.1
LassoCV	22.8245	0.791395	alpha=0.0022916910232695206
PCR	17.2454	0.842385	n_components=51
PLS	16.2460	0.851518	n_components=12

In this example, PCR and especially PLS tend to outperform ridge and lasso.

- PCR can recover the signal if enough components are included, even though the signal lies in low-variance directions.
- PLS performs particularly well because it uses y when constructing components, allowing it to directly identify directions associated with the response.
- Ridge and lasso shrink coefficients based on the observed predictors. Since the signal is weak (low variance) and spread across many features through A_1 , it is harder for them to isolate and recover it.

Caution

Important predictive directions do not have to correspond to directions of large variance.

6 Logistic Regression

Logistic regression is a classification method for binary outcomes. It estimates the conditional probability of the positive class and then converts that probability into a class decision only after a threshold is chosen. Many of the materials were already covered in Chapter 2. We provide more details in this section.

6.1 Binary Response

6.1.1 Probability

Suppose $Y \in \{0, 1\}$. We usually call $Y = 1$ the positive class, success class, or class 1, and $Y = 0$ the negative class, failure class, or class 0.

For a given predictor vector x , define

$$p(x) = \Pr(Y = 1 \mid X = x).$$

Then

$$\Pr(Y = 0 \mid X = x) = 1 - p(x).$$

Note

Any probability is within $[0, 1]$. Therefore it is bounded.

6.1.2 Odds

If an event occurs with probability p , its odds are $\frac{p}{1-p}$. Odds compare the probability that the event occurs with the probability that it does not occur.

For binary classification,

$$\text{odds of class 1} = \frac{P(Y = 1 \mid X = x)}{P(Y = 0 \mid X = x)} = \frac{p(x)}{1 - p(x)}.$$

i Note

Odds take values in $(0, \infty)$. Comparing to probability, odds remove the upper bound of probability, but they are still restricted to be positive.

6.1.3 Logit and Log-Odds

To remove the positivity restriction of odds, we take a logarithm.

Definition 6.1 (Logit). The **logit** of a probability p is

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right).$$

It is also called the **log-odds**.

The inverse of the logit function is the logistic (sigmoid) function. It will be discussed next.

i Note

The logit transformation maps probabilities from $(0, 1)$ to the entire real line $(-\infty, \infty)$.

Example 6.1.

p	Odds	Logit	Interpretation
0.10	1/9	$\log(1/9) \approx -2.197$	Failure is much more likely than success
0.50	1	0	Success and failure are equally likely
0.75	3	$\log(3) \approx 1.099$	Success is three times as likely as failure
0.90	9	$\log(9) \approx 2.197$	Success is nine times as likely as failure

6.2 The Logistic Regression Model

Logistic regression assumes that the log-odds are linear in the predictors:

$$\text{logit}(p(x)) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

Equivalently,

$$\log \left(\frac{p(x)}{1 - p(x)} \right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p.$$

This is the main equation of logistic regression.

6.2.1 From Log-Odds Back to Probability

Starting from

$$\log \left(\frac{p(x)}{1 - p(x)} \right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = x^\top \beta,$$

exponentiating both sides gives

$$\frac{p(x)}{1 - p(x)} = \exp(x^\top \beta).$$

Solving for $p(x)$ gives

$$p(x) = \frac{\exp(x^\top \beta)}{1 + \exp(x^\top \beta)} = \frac{1}{1 + \exp(-x^\top \beta)}.$$

Definition 6.2 (The Sigmoid function). Define

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

This is the Sigmoid function, also called the logistic function.

With this definition, the logistic regression can be written as

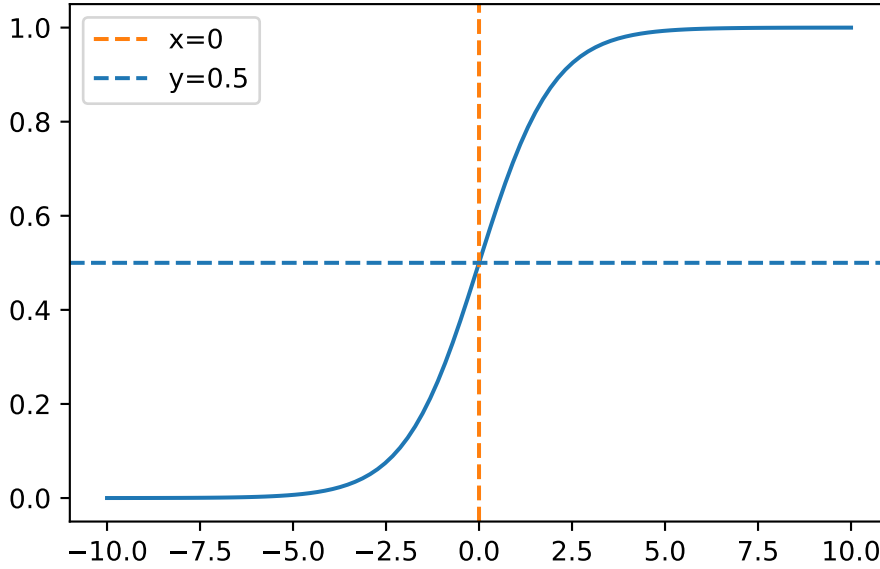
$$p(x) = \sigma(x^\top \beta) = \sigma(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = x^\top \beta).$$

The logistic function has several important properties.

1. $0 < \sigma(z) < 1$. So it always produces a valid probability for every real number z .
2. It is increasing: $z_1 < z_2 \Rightarrow \sigma(z_1) < \sigma(z_2)$.
3. $p(x)$ is

$$\begin{cases} \text{close to 0,} & \text{if } x^\top \beta \ll 0, \\ 0.5, & \text{if } x^\top \beta = 0, \\ \text{close to 1,} & \text{if } x^\top \beta \gg 0. \end{cases}$$

The curve is displayed as follows.



i Historical Note

The logistic curve was introduced by Pierre-François Verhulst in 1838 [5] while studying population growth.

The term **logit** was introduced by Joseph Berkson in 1944 [6]. Berkson used the word logit for the logarithm of the odds and advocated the logistic function in bioassay problems. Logistic regression later became a central example of generalized linear models. In GLM terminology, logistic regression uses:

- Bernoulli response distribution;
- logit link function;
- linear predictor $x^\top \beta$.

6.2.2 Logistic Regression as a Linear Classifier

Logistic regression produces probabilities, therefore it can also be used for classification.

Using the common threshold $t = 0.5$, we predict

$$\hat{y} = \begin{cases} 1, & p(x) \geq 0.5, \\ 0, & p(x) < 0.5. \end{cases}$$

Since

$$p(x) \geq 0.5 \iff x^\top \beta \geq 0,$$

the classification rule becomes

$$\hat{y} = \begin{cases} 1, & x^\top \beta \geq 0, \\ 0, & x^\top \beta < 0. \end{cases}$$

Therefore, the decision boundary is

$$x^\top \beta = 0.$$

i Geometric Interpretation

The decision boundary $x^\top \beta = 0$ is a line in two dimensions, a plane in three dimensions, and a hyperplane in higher dimensions.

Thus logistic regression is a linear classifier: Even though the probability curve is nonlinear, the decision boundary is linear in the original predictor space.

6.2.3 Interpreting the Coefficients

Consider the model

$$p(x) = \sigma(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p),$$

where $\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p = x^\top \beta$ is the linear predictor.

Suppose all other predictors are held fixed. Increasing x_j by one unit changes the linear predictor by β_j . Because the linear predictor represents the log-odds, increasing x_j by one unit changes the log-odds by β_j .

Thus:

- if $\beta_j > 0$, increasing x_j increases the log-odds of class 1;
- if $\beta_j < 0$, increasing x_j decreases the log-odds of class 1;
- if $\beta_j = 0$, increasing x_j does not change the log-odds.

For a one-unit increase in x_j , holding all other predictors fixed, the linear predictor increases by β_j . Since β_j is a change in log-odds, exponentiating gives a multiplicative change in odds, then the odds are expected to be multiplied by $\exp(\beta_j)$:

Consider

$$\log(\text{odds}_1) - \log(\text{odds}_2) = \beta_j.$$

Then

$$\frac{\text{odds}_1}{\text{odds}_2} = e^{\beta_j}.$$

Definition 6.3 (Odds Ratio). The quantity $\exp(\beta_j)$ is called the **odds ratio**.

Example 6.2 (Example). Suppose $\beta_j = 0.08$. Since

$$\exp(0.08) \approx 1.083,$$

Then a one-unit increase in x_j multiplies the odds by approximately 1.083. Equivalently, the odds increase by approximately 8.3%.

i Note

The coefficient β_j is not a direct change in probability since the relation between the logit and the probability is a sigmoid function (which is non-linear). So the probability change depends on the current value of $x^\top \beta$. For the same value of β_j , the probability change can be small when $p(x)$ is already close to 0 or close to 1, and larger when $p(x)$ is near 0.5.

6.3 Maximum Likelihood Estimation

Logistic regression is usually fit using maximum likelihood estimation.

Assume we are given a dataset $\{(x_i, y_i)\}_{i=1}^n$, where $y_i \in \{0, 1\}$. In logistic regression, we model the conditional probability that the response belongs to class 1:

$$p_i = \Pr(Y_i = 1 \mid X_i = x_i) = \sigma(x_i^\top \beta).$$

Since Y_i only takes the values 0 and 1, it is natural to model its conditional distribution using a Bernoulli distribution.

$$Y_i \mid X_i = x_i \sim \text{Bernoulli}(p_i).$$

The probability mass function of a Bernoulli random variable is

$$\Pr(Y_i = y_i) = p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Assuming the observations are independent and conditional on the predictors, the likelihood function is

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}.$$

Taking the logarithm, the log-likelihood is

$$\ell(\beta) = \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)].$$

The maximum likelihood estimate is

$$\hat{\beta} = \arg \max_{\beta} \ell(\beta).$$

i Gradient descent

Unlike ordinary least squares, logistic regression does not have a simple closed-form solution for $\hat{\beta}$. In practice, numerical optimization methods are used.

Gradient descent is one of the most widely used optimization methods in machine learning. Although linear regression and logistic regression use different models and different loss functions, their gradient expressions have a similar matrix form. As a result, their gradient descent update formulas also look very similar.

Gradient descent is beyond the scope of this course, although it plays an important role in modern machine learning.

6.3.1 Binary Cross-Entropy and a Single Neuron

The negative log-likelihood is

$$-\ell(\beta) = - \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)].$$

In machine learning, this is called binary cross-entropy loss or log loss. It is simply the negative of the log-likelihood mentioned in the previous section. The reason of this negative is that in machine learning it is usually preferred to lower the loss function.

Therefore logistic regression is not just a classification model. It is also a probabilistic model trained by minimizing a loss function that rewards accurate probability estimates.

What is more, a logistic regression model can be viewed as a single sigmoid neuron, since the model has the form

$$x \mapsto x^\top \beta \mapsto \sigma(x^\top \beta) \mapsto p(x).$$

This is essentially a neural network with no hidden layers and one output unit, with sigmoid activation and binary cross-entropy loss.

This connects logistic regression to modern machine learning methods, including neural networks, which often use cross-entropy loss for classification.

6.4 Regularized Logistic Regression

When there are many predictors, logistic regression can overfit. A common solution is to add regularization, like ridge and lasso for linear regression.

1. Ridge logistic regression penalizes large coefficients using an L^2 penalty:

$$-\ell(\beta) + \lambda \sum_{j=1}^p \beta_j^2.$$

This shrinks coefficients toward zero but usually does not set them exactly to zero.

2. Lasso logistic regression uses an L^1 penalty:

$$-\ell(\beta) + \lambda \sum_{j=1}^p |\beta_j|.$$

This can shrink some coefficients exactly to zero, producing a simpler model.

6.5 Logistic Regression in sklearn

We usually use `LogisticRegression` from `sklearn.linear_model` to implement logistic regression. The most commonly used solver in modern versions of scikit-learn is `lbfgs`, which is the default choice for `LogisticRegression()`. It is a quasi-Newton optimization method that usually converges much faster than simple gradient descent. In addition it is automatically extended to multiclass cases.

The basic code is as usual:

```

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logit = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5000),
)

logit.fit(X_train, y_train)

test_prob = logit.predict_proba(X_test)
test_pred = logit.predict(X_test)

```

Logistic regression is very similar to linear regression, that the scales of features matters. Therefore we need to set up a pipeline to standarize al features first.

The model actually has two types of outputs:

- `predict_proba()` returns estimated class probabilities. The second column is the estimated probability of the positive class.
- `predict()` returns class labels using the default threshold 0.5.

6.5.1 Regularization

The general idea of regularization in `LogisticRegression()` is based on the elastic-net style penalty, which is controlled by two arguments `l1_ratio` and `C`. Conceptually, the penalty behaves like

$$-\ell + \frac{1}{C}(\text{l1_ratio}\|\beta\|_1 + (1 - \text{l1_ratio})\|\beta\|_2^2).$$

- `l1_ratio` is to control the type of regularization:
 - `l1_ratio = 0`: pure L2 regularization;
 - `l1_ratio = 1`: pure L1 regularization;
 - `0 < l1_ratio < 1`: elastic-net regularization.
- `C`, which is $1/\lambda$, is to control the regularization strength:
 - `C=np.inf` means no penalty.

The exact formula used inside `sklearn` is more complicated which incorprates other aspects such as number of observations.

Note that some combinations of regularization and solvers are not compatible. For example,

- elastic-net regularization requires the `saga` solver;
- the default solve `lbfgs` requires `l2` so `l1_ratio` has to be 0.

The default setting for `LogisticRegression()` is that `l1_ratio=0`, `C=1` and `solver="lbfgs"`.

6.6 Example 1: Binary classification

We first consider a binary classification problem.

The Titanic dataset records information about passengers aboard the Titanic. Our goal is to predict whether a passenger survived. We load the dataset from the database from `seaborn`.

```
import seaborn as sns
from sklearn.model_selection import train_test_split

titanic = sns.load_dataset("titanic")
data = titanic[["survived", "age", "fare", "pclass"]].dropna()
X = data[["age", "fare", "pclass"]]
y = data["survived"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=1
)
```

Here we only use 3 features `age`, `fare` and `pclass` to predict `survived`.

We quickly fit the model with default setting.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

logit = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
logit.fit(X_train, y_train)
test_prob = logit.predict_proba(X_test)[:, 1]
test_pred = logit.predict(X_test)
```

- ① For binary classification, the output of `.predict_proba()` contains two columns, each for a class. We want to test 1, so we choose column 1.

We focus on tuning `C` and the threshold. Note that the two hyperparameters take effects in different stage. `C` is about the training of the model, while the threshold is about prediction. Therefore for logistic regression we usually tune `C` first and then find the best threshold afterwards.

6.6.1 Tuning C

We use cross-validation with the F1 score as the evaluation metric. Since the F1 score combines precision and recall, it is useful when we want to balance false positives and false negatives. In this example, we use it to select a model that balances the two types of classification errors when predicting passenger survival.

```
import numpy as np
from sklearn.model_selection import GridSearchCV

param_grid = {"logisticregression__C": np.logspace(-3, 3, 20)}
grid = GridSearchCV(logit, param_grid=param_grid, cv=5, scoring="f1")
grid.fit(X_train, y_train)
grid.best_params_
```

```
{'logisticregression__C': np.float64(2.976351441631316)}
```

6.6.2 Tuning the threshold

To tune the threshold, we use the helper function defined in Chapter 2.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix

def classification_summary(y_true, prob, threshold=0.5):
    pred = (prob >= threshold).astype(int)
    TN, FP, FN, TP = confusion_matrix(y_true, pred).ravel()

    return {
        "threshold": threshold,
        "TP": TP,
        "FP": FP,
        "FN": FN,
        "TN": TN,
        "accuracy": accuracy_score(y_true, pred),
        "precision": precision_score(y_true, pred, zero_division=0),
        "recall": recall_score(y_true, pred, zero_division=0),
        "f1": f1_score(y_true, pred, zero_division=0),
    }
```

We may use the regular split-validation method. The model use the best C from the cross-validation from the previous section.

```

from sklearn.model_selection import train_test_split
import pandas as pd

X_model, X_valid, y_model, y_valid = train_test_split(
    X_train, y_train, test_size=0.25, stratify=y_train, random_state=1
)

best_C = grid.best_params_['logisticregression__C']

candidate_thresholds = np.linspace(0.05, 0.95, 91)

results = []
for t in candidate_thresholds:
    best_logit = make_pipeline(
        StandardScaler(),
        LogisticRegression(C=best_C, max_iter=5000),
    )
    best_logit.fit(X_model, y_model)
    valid_prob = best_logit.predict_proba(X_valid)[:, 1]
    results.append(classification_summary(y_valid, valid_prob, threshold=t))

threshold_df = pd.DataFrame(results)

best_row = threshold_df.loc[threshold_df["f1"].idxmax()]
best_row

```

① `best_row` is a Series object. Therefore although it is a row, it is displayed as a column.

```

threshold    0.360000
TP           39.000000
FP           17.000000
FN           12.000000
TN           57.000000
accuracy     0.768000
precision    0.696429
recall       0.764706
f1           0.728972
Name: 31, dtype: float64

```

We can fix the threshold and treat it as part of the prediction rule.

```

from sklearn.model_selection import FixedThresholdClassifier

final_logit = make_pipeline(
    StandardScaler(), LogisticRegression(C=best_C, max_iter=5000)
)
model_with_threshold = FixedThresholdClassifier(
    final_logit, threshold=best_row["threshold"]
)
model_with_threshold.fit(X_train, y_train)
model_with_threshold.score(X_test, y_test)

```

0.6465116279069767

Note

Notice that changing the threshold does not change the fitted model. The estimated probabilities remain the same. The threshold only changes how those probabilities are converted into class labels.

6.7 Multiclass Extension

The basic logistic regression model handles binary outcomes. For problems with more than two classes, a common extension is multinomial logistic regression.

Suppose

$$Y \in 1, 2, \dots, K.$$

Instead of modeling the probability of a single class, multinomial logistic regression models the probability of every class simultaneously.

For each class k , define a linear predictor

$$\eta_k(x) = x^\top \beta_k = \beta_{0,k} + \beta_{1,k}x_1 + \dots + \beta_{p,k}x_p.$$

The predicted probability of class k is then given by the softmax function:

$$\Pr(Y = k \mid X = x) = \frac{\exp(\eta_k(x))}{\sum_{\ell=1}^K \exp(\eta_\ell(x))}.$$

The softmax function converts the collection of scores

$$\eta_1(x), \eta_2(x), \dots, \eta_K(x)$$

into probabilities that satisfy

$$0 \leq \Pr(Y = k \mid X = x) \leq 1$$

and

$$\sum_{k=1}^K \Pr(Y = k \mid X = x) = 1.$$

The softmax function plays the same role in multiclass classification that the sigmoid function plays in binary classification.

i Note

For K=2 classes, softmax reduces to the sigmoid model.

For prediction, the observation is assigned to the class with the largest predicted probability:

$$\hat{y} = \arg \max_k P(Y = k \mid X = x).$$

6.7.1 Cross-Entropy Loss

Suppose the true class labels are represented using one-hot encoding:

$$y_{ik} = \begin{cases} 1, & \text{if observation } i \text{ belongs to class } k, \\ 0, & \text{otherwise.} \end{cases}$$

Let

$$p_{ik} = P(Y_i = k \mid X_i = x_i)$$

denote the predicted probability from the softmax model.

The multiclass cross-entropy loss is

$$L = - \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(p_{ik}).$$

Because exactly one value of y_{ik} equals 1 for each observation, this loss simply penalizes the model when it assigns a low probability to the true class.

Minimizing the multiclass cross-entropy loss is equivalent to maximizing the likelihood of the multinomial logistic regression model, just as binary cross-entropy corresponds to maximum likelihood estimation in ordinary logistic regression.

i One-vs-Rest (OvR)

Another approach to multiclass classification is One-vs-Rest (OvR), also called One-vs-All. Instead of fitting a single multinomial model, OvR trains one binary classifier for each class. For example, with three classes A, B, and C, three separate models are fit:

- A versus not A
- B versus not B
- C versus not C

The final prediction is usually made using the classifier with the largest predicted probability.

Although OvR is simple and often works well in practice, multinomial logistic regression provides a more natural probabilistic model because all class probabilities are estimated simultaneously and automatically sum to one.

i solver

There are many algorithms for fitting logistic regression models. In `scikit-learn`, these optimization algorithms are specified through the `solver` argument. Examples include `"lbfgs"`, `"newton-cg"`, `"newton-cholesky"`, `"sag"`, `"saga"`, and `"liblinear"`.

Different solvers use different optimization techniques and may have different computational properties. Some are better suited for large datasets, while others are designed for smaller problems or support additional features such as regularization.

Historically, some solvers handled multiclass classification using a One-vs-Rest (OvR) strategy, while others directly fit a multinomial logistic regression model using the softmax formulation.

In this course, we focus on the multinomial (softmax) approach because it provides a natural extension of binary logistic regression and is widely used in modern machine learning. The details of optimization algorithms are beyond the scope of this course.

6.8 Example 2: Multinomial Logistic Regression

The Titanic example involved two classes. We now consider a multiclass classification problem using the wine dataset.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

wine = load_wine(as_frame=True)

X = wine.data[["alcohol", "flavanoids", "color_intensity"]]
y = wine.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=1
)
```

We fit a multinomial logistic regression model.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

wine_logit = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=5000),
)

wine_logit.fit(X_train, y_train)

test_prob = wine_logit.predict_proba(X_test)
test_pred = wine_logit.predict(X_test)
```

6.8.1 Probabilities and Softmax

For multiclass problems, logistic regression uses the softmax formulation discussed earlier.

```
test_prob[:5]
```

```
array([[3.32849861e-02, 4.58338942e-02, 9.20881120e-01],
```

```
[7.81045388e-02, 3.03513972e-02, 8.91544064e-01],  
[1.29661355e-02, 5.23981366e-05, 9.86981466e-01],  
[7.03100543e-03, 9.90470483e-01, 2.49851168e-03],  
[3.96520174e-02, 9.59921408e-01, 4.26574383e-04]])
```

Each row contains the estimated probabilities for all classes. The probabilities always sum to one.

```
test_prob[:,5].sum(axis=1)
```

```
array([1., 1., 1., 1., 1.])
```

The predicted class is the class with the largest probability.

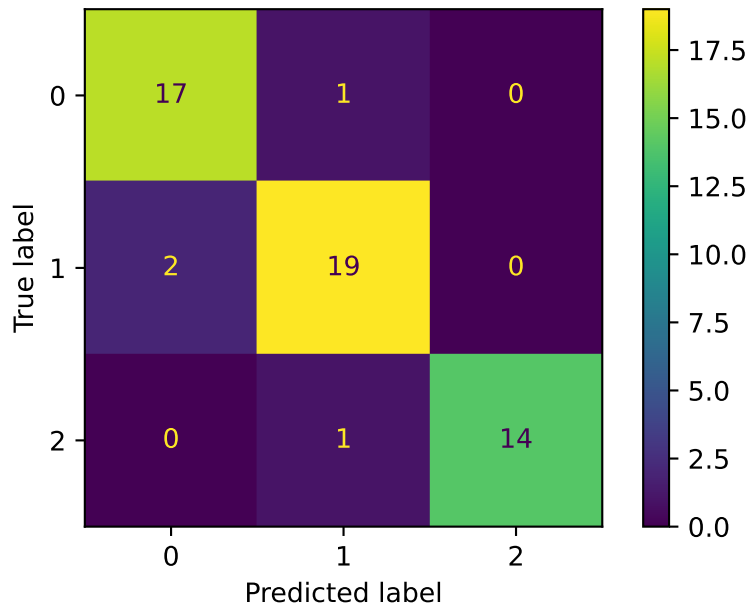
```
test_pred[:,5]
```

```
array([2, 2, 2, 1, 1])
```

Unlike binary logistic regression, there is usually no threshold parameter to tune. The model simply predicts the class with the highest probability.

6.8.2 Confusion Matrix

```
from sklearn.metrics import ConfusionMatrixDisplay  
  
ConfusionMatrixDisplay.from_predictions(y_test, test_pred)
```



We could also directly generate the classification report.

```
from sklearn.metrics import classification_report
print(classification_report(y_test, test_pred))
```

	precision	recall	f1-score	support
0	0.89	0.94	0.92	18
1	0.90	0.90	0.90	21
2	1.00	0.93	0.97	15
accuracy			0.93	54
macro avg	0.93	0.93	0.93	54
weighted avg	0.93	0.93	0.93	54

The report provides precision, recall, and F1 scores for each class separately.

i Note

For multiclass logistic regression, the regularization parameter **C** can be tuned in the same way as in the binary classification example.

However, there is usually no single threshold parameter to tune. The standard prediction rule is to choose the class with the largest predicted probability.

In this example, we skip the tuning step for $\mathcal{C}$ because the procedure is the same as before.

Part III

Nonparametric Supervised Learning

7 K-Nearest Neighbors

K-nearest neighbors (KNN) is a simple nonparametric prediction method. It does not estimate a fixed formula like linear regression. Instead, it predicts from nearby training observations. The method is local: points close to the new observation have influence, and distant points usually have none.

7.1 The KNN Model

KNN can be used for both regression and classification. The ideas are the same: use nearby observations to predict.

7.1.1 KNN Regression

Suppose the training data are

$$\{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^p, y_i \in \mathbb{R}.$$

We want to estimate the regression function

$$f(x) = \mathbb{E}[Y \mid X = x].$$

Definition 7.1 (KNN regressor). For a new point x_0 , let $N_K(x_0)$ be the set of indices corresponding to the K training points closest to x_0 . The KNN regression estimate is

$$\hat{f}(x_0) = \frac{1}{K} \sum_{i \in N_K(x_0)} y_i.$$

That is, the predicted value is the average response among the K nearest neighbors of x_0 .

KNN regression estimates the conditional mean locally by averaging the responses of nearby observations.

Sometimes neighbors are assigned different weights. In that case, a weighted KNN regressor has the form

$$\hat{f}(x_0) = \sum_{i=1}^n w_i(x_0) y_i,$$

where

$$w_i(x_0) \geq 0, \quad \sum_{i=1}^n w_i(x_0) = 1.$$

Ordinary KNN regression uses

$$w_i(x_0) = \begin{cases} 1/K, & i \in N_K(x_0), \\ 0, & i \notin N_K(x_0). \end{cases}$$

7.1.2 KNN Classification

For classification, the response is a class label rather than a numerical response. Given a new point x_0 , KNN classification finds the K nearest training observations and predicts by majority vote.

Definition 7.2 (KNN classifier). For a new point x_0 , let $N_K(x_0)$ be the set of indices of the K nearest training observations. The KNN classifier predicts

$$\hat{G}(x_0) = \arg \max_g \sum_{i \in N_K(x_0)} I(y_i = g).$$

In words, KNN predicts the class receiving the largest number of votes among the K nearest neighbors.

For binary classification, KNN naturally provides a local estimate of the probability that an observation belongs to class 1:

$$\hat{p}(x_0) = \frac{1}{K} \sum_{i \in N_K(x_0)} I(y_i = 1).$$

Then predict class 1 if $\hat{p}(x_0)$ is larger than a chosen threshold, usually 0.5.

This estimate can be interpreted as the proportion of class-1 observations among the nearest neighbors. The threshold part is related to logistic regression.

i Note

Weighted KNN classification assigns larger weights to some neighbors, typically those closer to the query point. The predicted class is then determined by a weighted majority vote.

i Lazy learner

Based on the definition, KNN does not estimate model parameters during training. It mainly stores the training observations and waits until prediction time to use them. For this reason, KNN is often called a lazy learner.

The training phase is extremely simple because KNN does not estimate any model parameters. The algorithm mainly stores the training observations for later use. If the training data have n observations and p predictors, storing the data costs about $O(np)$. The prediction cost can be much higher. For a single new observation, a brute-force KNN method computes its distance to all n training observations. This costs about $O(np)$. After computing the distances, the algorithm must identify the K nearest observations. A full ranking of all distances requires $O(n \log n)$ operations, although more efficient algorithms may reduce this cost. Taking all these into consideration, if we predict for m new observations, the brute-force distance computation costs about $O(mnp)$.

Thus, KNN is simple to train but can be relatively slow at prediction time, especially when the training set is large or the number of predictors is large.

i Comparison with Linear Models

Linear models are global models. A single fitted equation is used throughout the entire predictor space.

KNN is local. The prediction at x_0 is determined by nearby training observations.

Because of this difference:

- Linear models are simple and interpretable but may fail to capture complex nonlinear relationships.
- KNN can adapt to nonlinear patterns without specifying a functional form.

However, KNN may become unstable when nearby data are sparse or noisy, while linear models often provide more stable predictions.

7.1.3 Distance

The most common distance is Euclidean distance:

$$d(x, x') = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}.$$

KNN depends heavily on this distance. If one feature has a much larger scale than the others, it can dominate the distance calculation. This is why scaling is usually important.

Although Euclidean distance is the most common choice, other distances such as Manhattan distance can also be used.

i Scaling Matters

KNN relies entirely on distances between observations.

If one predictor is measured on a much larger scale than another, it can dominate the distance calculation and effectively determine which observations are considered neighbors. For example, a variable measured in thousands may overwhelm another variable measured between 0 and 1.

Therefore, KNN is usually applied after feature scaling, such as standardization.

In practice, KNN is often combined with `StandardScaler()` in a pipeline.

7.1.4 The Role of K

The tuning parameter K controls the smoothness of the fitted model.

- Small K : uses very local information and can be sensitive to noise.
- Large K : averages over more observations and is more stable, but can be biased.

This is the usual bias-variance tradeoff. Smaller K produces a more flexible fit. Larger K produces a smoother fit.

In practice, K is typically chosen using cross-validation.

i Curse of Dimensionality

KNN can work well in low-dimensional problems, but it often struggles when the number of features is large.

In high dimensions, data become sparse. As the number of features increases, the volume of the feature space grows rapidly. As a result, observations tend to be far apart from one another. Therefore local averaging becomes less meaningful.

This is called the curse of dimensionality.

7.2 KNN in sklearn

sklearn.neighbors provides:

- `KNeighborsRegressor` for regression,
- `KNeighborsClassifier` for classification.

The estimator interface is the same as other sklearn models:

1. Create the model.
2. Fit the model using `.fit()`.
3. Predict using `.predict()`.
4. Usually coupled with `StandardScaler()` to create a pipeline.

The arguments here are straightforward. In most cases the default choice (other than `n_neighbors` of course) is good enough. For details please check the official documents.

7.3 Example 1: One-Dimensional Regression

We first simulate a one-dimensional regression problem based on a noisy sine curve.

```
import numpy as np
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(1)

N = 100
X = rng.uniform(0, 2 * np.pi, size=N).reshape(-1, 1)
y = 2 * np.sin(X[:, 0]) + rng.normal(size=N)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

We then generate a grid of values and apply the fitted model to the grid in order to estimate the underlying regression function.

```
X_grid = np.linspace(0, 2 * np.pi, 1001).reshape(-1, 1)
```

Fit a KNN regressor with $K = 3$.

```
from sklearn.neighbors import KNeighborsRegressor
```

```
knn = KNeighborsRegressor(n_neighbors=3)  
knn.fit(X_train, y_train)
```

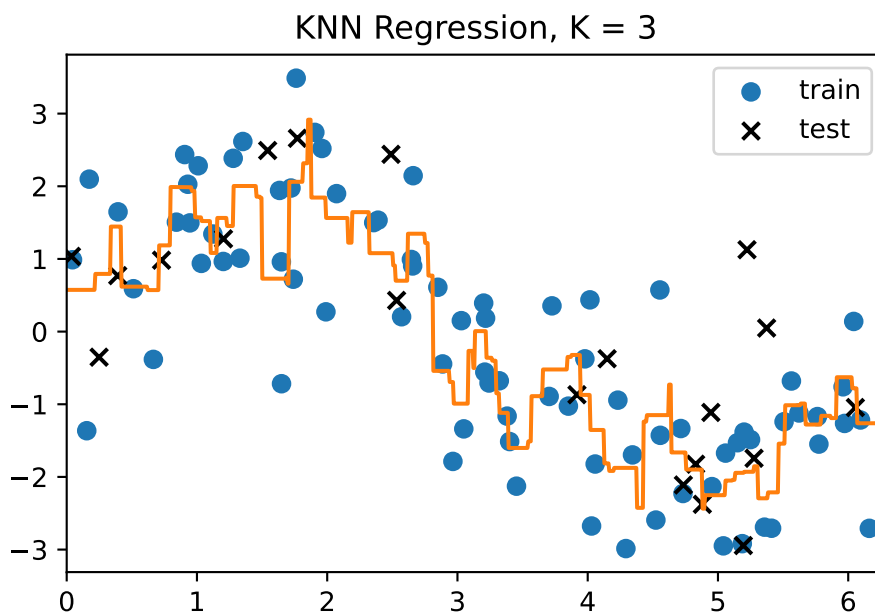
```
y_grid = knn.predict(X_grid)
```

Plot the fitted curve.

```
import matplotlib.pyplot as plt
```

```
plt.scatter(X_train.ravel(), y_train, s=40, label="train")  
plt.scatter(X_test.ravel(), y_test, s=40, marker="x", color="black", label="test")  
plt.plot(X_grid.ravel(), y_grid, color="C1")
```

```
plt.title("KNN Regression, K = 3")  
plt.xlim(0, 2 * np.pi)  
plt.legend();
```



i The dimension of X and X_{grid}

Please pay attention to the dimension of X and X_{grid} . We generate them as 2D arrays by `.reshape(-1, 1)`, and change them back to 1D array in plotting by `.ravel()`.

Note that `.ravel()` is the same as `.reshape(-1)`. We choose it because it is shorter.

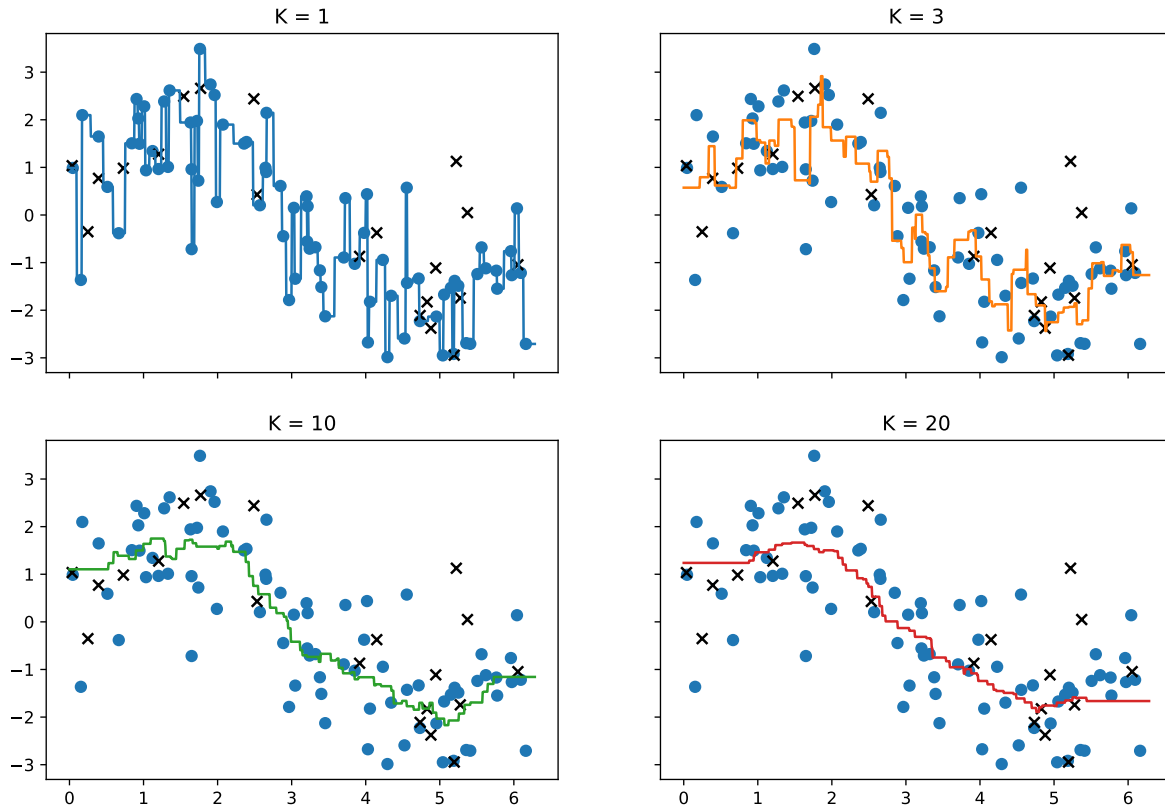
7.3.1 Comparing Different Values of K

The following plot compares several choices of K .

```
fig, axs = plt.subplots(2, 2, figsize=(12, 8), sharex=True, sharey=True)

for ax, k, color in zip(
    axs.ravel(),
    [1, 3, 10, 20],
    ["C0", "C1", "C2", "C3"],
):
    model = KNeighborsRegressor(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_grid)

    ax.scatter(X_train.ravel(), y_train, s=40, label="train")
    ax.scatter(X_test.ravel(), y_test, s=40, marker="x", color="black", label="test")
    ax.plot(X_grid.ravel(), pred, color=color)
    ax.set_title(f"K = {k}");
```



The fitted curve becomes smoother as K increases.

7.3.2 Choosing K by Cross-Validation

We now use cross-validation on the training set to select the value of K .

Since the training set contains 80 observations and we use `cv=5`, each training fold contains only 64 observations. Therefore we restrict the search range to values of K that are safely below this size.

```
from sklearn.model_selection import GridSearchCV

k_values = list(range(1, 61))

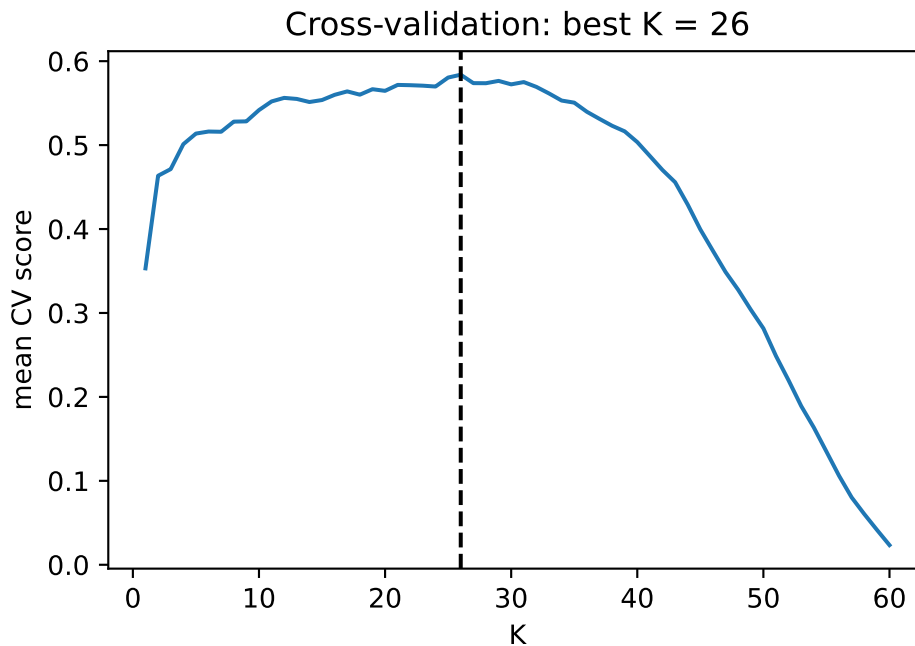
grid = GridSearchCV(KNeighborsRegressor(), param_grid={"n_neighbors": k_values}, cv=5)
grid.fit(X_train, y_train)

grid.best_params_
```

```
{'n_neighbors': 26}
```

Plot the cross-validation score.

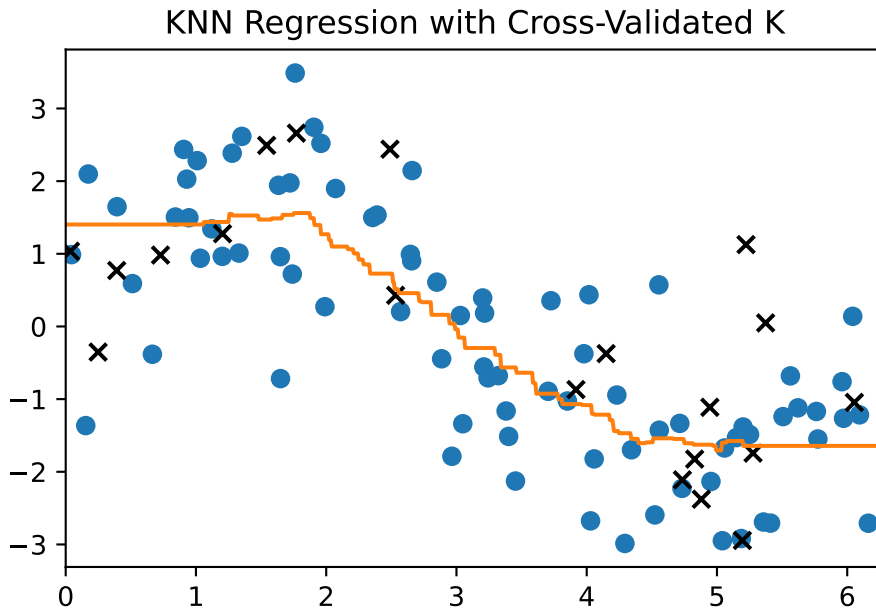
```
plt.plot(k_values, grid.cv_results_["mean_test_score"])
plt.axvline(
    x=grid.best_params_["n_neighbors"],
    color="black",
    linestyle="--",
)
plt.xlabel("K")
plt.ylabel("mean CV score")
plt.title(f"Cross-validation: best K = {grid.best_params_['n_neighbors']}");
```



Now plot the fitted curve using the selected value of K .

```
y_grid_best = grid.predict(X_grid)

plt.scatter(X_train.ravel(), y_train, s=40, label="train")
plt.scatter(X_test.ravel(), y_test, s=40, marker="x", color="black", label="test")
plt.plot(X_grid.ravel(), y_grid_best, color="C1")
plt.title("KNN Regression with Cross-Validated K")
plt.xlim(0, 2 * np.pi);
```



Finally, evaluate the selected model on the test set.

```
grid.score(X_test, y_test)
```

```
0.5552931849752354
```

7.4 Example: Iris Classification

We use the iris data from `sklearn.datasets`. To make visualization easy, we use only the first two features: sepal length and sepal width.

```
from sklearn import datasets
from sklearn.model_selection import train_test_split

iris = datasets.load_iris()

X = iris.data[:, :2]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y
)
```

Plot the training data.

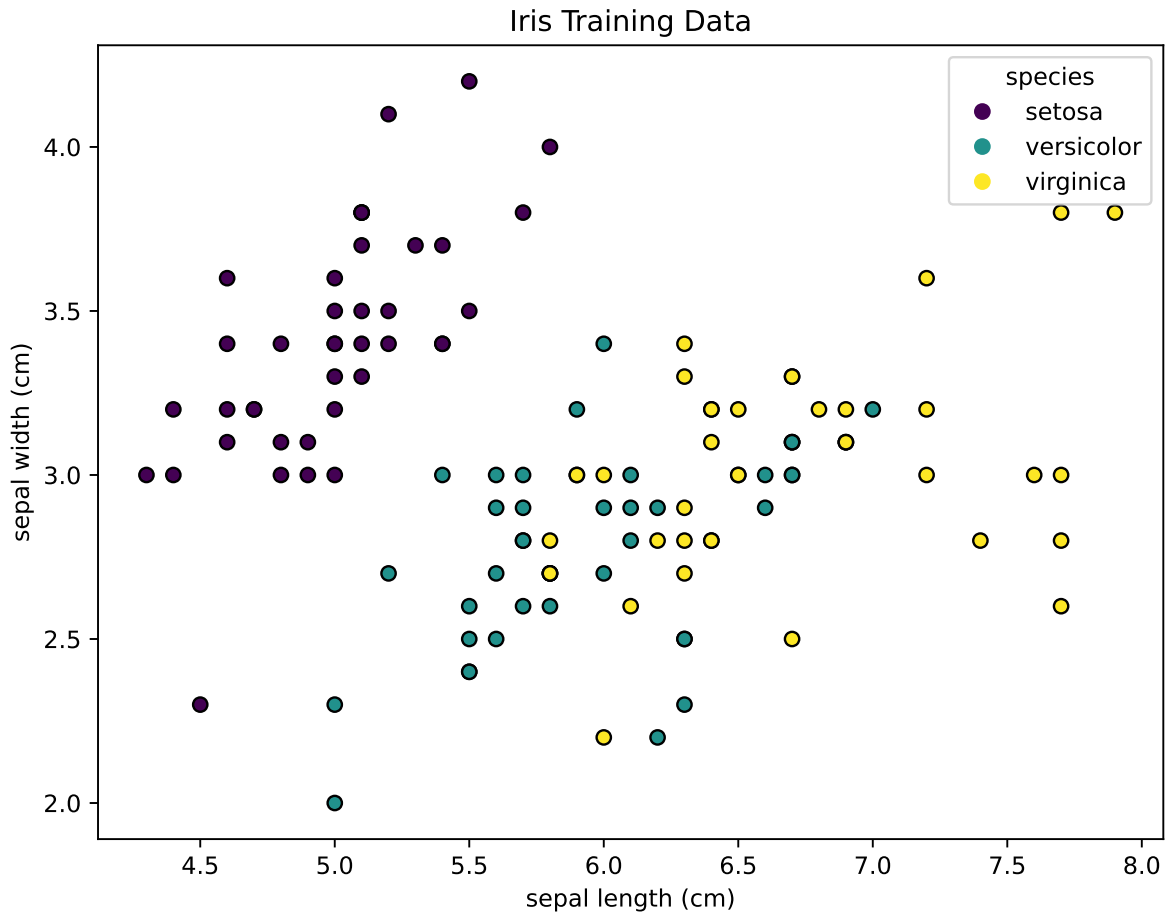
```
fig, ax = plt.subplots(figsize=(8, 6))

scatter = ax.scatter(
    X_train[:, 0],
    X_train[:, 1],
    c=y_train,
    edgecolor="k",
)

labels = ["setosa", "versicolor", "virginica"]
ax.legend(
    handles=scatter.legend_elements()[0],
    labels=labels,
    title="species",
)

ax.set_xlabel(iris.feature_names[0])
ax.set_ylabel(iris.feature_names[1])
ax.set_title("Iris Training Data")
```

```
Text(0.5, 1.0, 'Iris Training Data')
```



7.4.1 Pipeline

The usual KNN workflow includes scaling followed by the KNN model. A Pipeline keeps these steps together.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

pipe = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=10))

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
pipe.score(X_test, y_test)
```

0.7

7.4.2 Choosing K for Classification

To choose K , we tune `kneighborsclassifier__n_neighbors` inside the pipeline.

```
from sklearn.model_selection import GridSearchCV

n_values = list(range(1, 91))

param_grid = {"kneighborsclassifier__n_neighbors": n_values}

grid = GridSearchCV(pipe, param_grid=param_grid, cv=5, scoring="accuracy")

grid.fit(X_train, y_train)
grid.best_params_
```

```
{'kneighborsclassifier__n_neighbors': 5}
```

The best cross-validation score is:

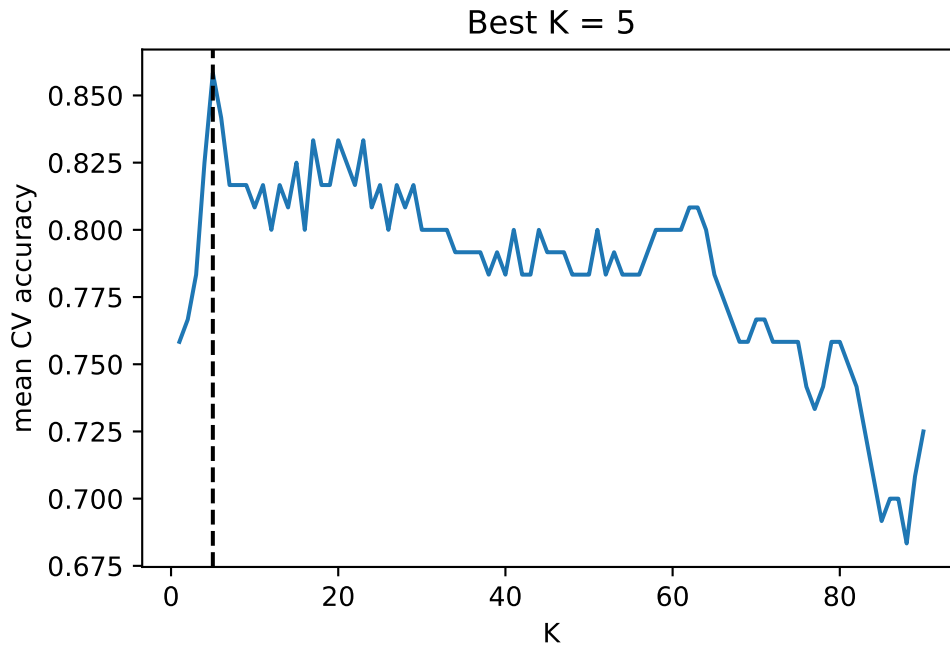
```
grid.best_score_
```

```
np.float64(0.8583333333333332)
```

Plot the validation curve over different values of K .

```
plt.plot(n_values, grid.cv_results_["mean_test_score"])
plt.axvline(
    x=grid.best_params_["kneighborsclassifier__n_neighbors"],
    color="black",
    linestyle="--",
)
plt.xlabel("K")
plt.ylabel("mean CV accuracy")
plt.title(f"Best K = {grid.best_params_['kneighborsclassifier__n_neighbors']}")
```

```
Text(0.5, 1.0, 'Best K = 5')
```



7.4.3 Decision Boundary

We can visualize the decision boundary using `DecisionBoundaryDisplay`.

```
from sklearn.inspection import DecisionBoundaryDisplay

pipe.set_params(
    kneighborsclassifier__n_neighbors=grid.best_params_[
        "kneighborsclassifier__n_neighbors"
    ]
)
pipe.fit(X_train, y_train)

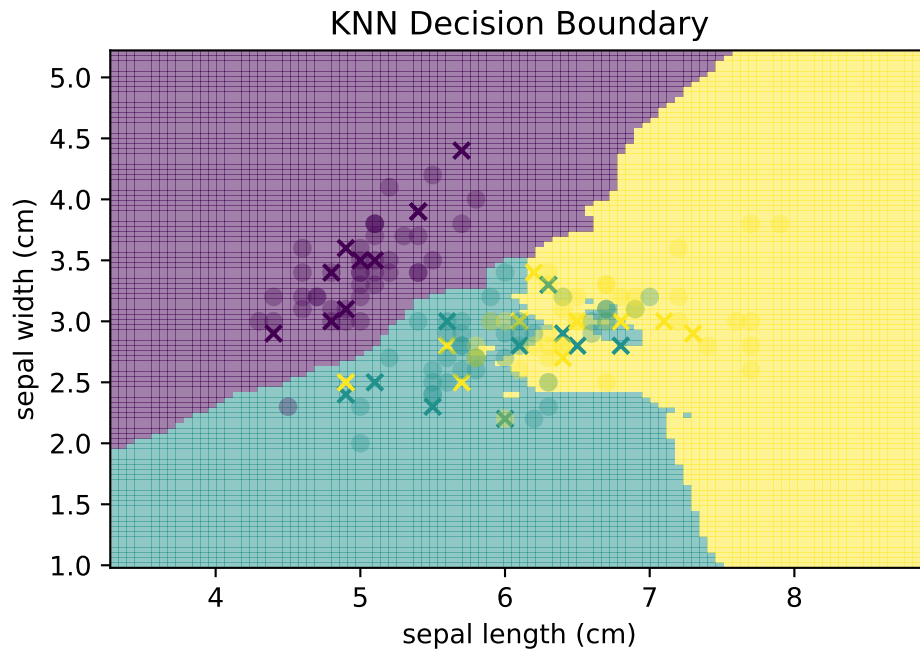
disp = DecisionBoundaryDisplay.from_estimator(
    pipe,
    X_train,
    response_method="predict",
    plot_method="pcolormesh",
    xlabel=iris.feature_names[0],
    ylabel=iris.feature_names[1],
    alpha=0.5,
)
```



```

disp.ax_.scatter(X_test[:, 0], X_test[:, 1], c=y_test, marker="x", label="test")
disp.ax_.scatter(X_train[:, 0], X_train[:, 1], c=y_train, alpha=0.3, label="train")
plt.title("KNN Decision Boundary");

```



8 Kernel Smoothing

Kernel smoothing is a family of nonparametric methods for estimating smooth functions from data. Like KNN, it is based on local information: nearby observations should have more influence than distant observations.

The difference is that KNN uses a hard neighbor set, while kernel methods usually use smoothly decaying weights.

i Note

This chapter focuses on **kernel smoothing**, including kernel density estimation and kernel regression. It does not cover kernel machines such as support vector machines or kernel ridge regression. These methods also use kernel functions, but their modeling goals are different.

Unlike many methods discussed earlier in this book, classical kernel smoothing is not fully integrated into the standard `sklearn` estimator framework. While some tools, such as `KernelDensity`, are available in `sklearn`, many classical smoothers are primarily studied through their underlying algorithms rather than through ready-made estimators. Therefore, in this section we will implement several methods directly using `numpy` in order to better understand how they work.

8.1 From KNN to Kernel Weights

KNN regression predicts by averaging the K nearest responses. This gives equal weight to all K nearest neighbors and zero weight to every other observation. In other words, KNN is a weighted average with hard weights:

$$\hat{f}_{\text{KNN}}(x_0) = \sum_{i=1}^n w_i(x_0) y_i,$$

where

$$w_i(x_0) = \begin{cases} 1/K, & i \in N_K(x_0), \\ 0, & i \notin N_K(x_0). \end{cases}$$

Kernel smoothing keeps the same local-averaging idea but changes the weights. Instead of deciding whether a point is inside or outside the neighbor set, it assigns a smooth weight based on distance:

$$w_i(x_0) \propto K\left(\frac{x_0 - x_i}{h}\right).$$

Here $K(\cdot)$ is the kernel function and $h > 0$ is the bandwidth.

A kernel function converts distance into weight. It gives larger values to points near zero and smaller values to points far from zero. For density estimation, we usually require

$$K(u) \geq 0, \quad \int_{-\infty}^{\infty} K(u) du = 1.$$

The scaled kernel is

$$K_h(x, x_i) = \frac{1}{h} K\left(\frac{x - x_i}{h}\right).$$

The bandwidth h controls the distance scale. A small h makes the kernel narrow and local; a large h makes it wide and smooth.

Common choices include:

- Gaussian kernel: $K(u) = \frac{1}{\sqrt{2\pi}} \exp(-u^2/2)$.
- Uniform kernel: $K(u) = I(|u| \leq 1/2)$.
- Epanechnikov kernel: $K(u) = \frac{3}{4}(1 - u^2)I(|u| \leq 1)$.

KNN fits are often discontinuous because the neighbor set can change abruptly as x_0 moves. Kernel smoothers, especially with smooth kernels such as the Gaussian kernel, usually produce smooth fitted curves.

In practice, the exact kernel shape is usually less important than the bandwidth. The bandwidth controls the main bias–variance tradeoff.

8.2 Kernel Density Estimation

Suppose a random variable has an unknown density function $f(x)$. Given a sample $\{x_1, \dots, x_n\}$, we would like to estimate this density from the observed data.

Kernel density estimation (KDE) is a nonparametric method for doing so. The kernel density estimator is

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^n K_h(x, x_i).$$

Equivalently,

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right).$$

The interpretation is simple: place a small smooth bump at every observation, add the bumps together, and divide by n .

The kernel function determines the shape of each bump, while the bandwidth h controls its width. As we will see, the bandwidth is usually the most important tuning parameter in kernel density estimation.

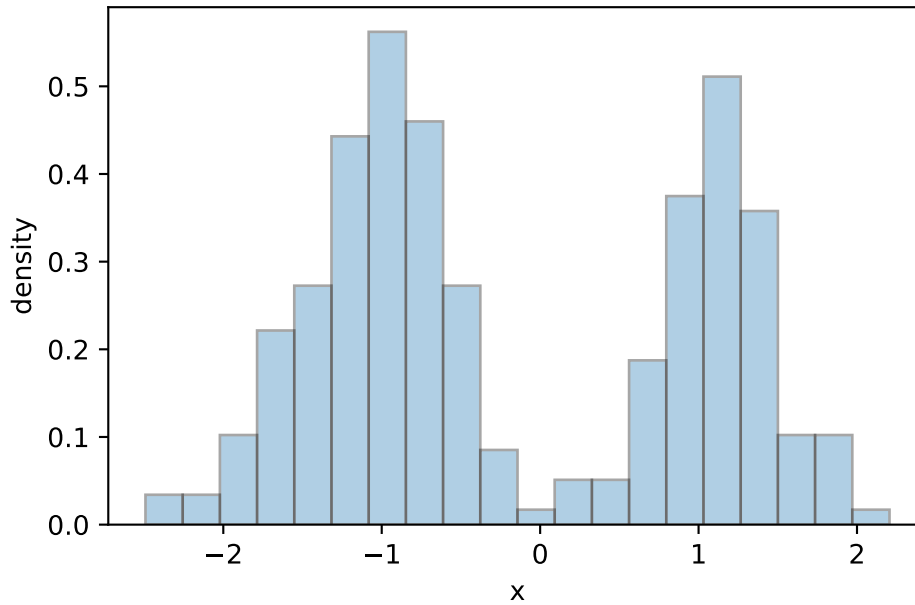
We demonstrate the idea with the following example. We use two normal distributions to generate a simple dataset. The dataset is $\mathbf{x}$, and its histogram is displayed below.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(1)

n1 = rng.normal(loc=-1.0, scale=0.55, size=150)
n2 = rng.normal(loc=1.2, scale=0.35, size=100)
x = np.concatenate([n1, n2])

plt.hist(x, bins=20, density=True, alpha=0.35, edgecolor="black")
plt.xlabel("x")
plt.ylabel("density");
```



8.2.1 Uniform Kernel and Histograms

A histogram estimates density by counting observations inside fixed bins. Kernel density estimation generalizes this idea by replacing fixed bins with a moving window.

Using the uniform kernel

$$K(u) = \begin{cases} 1, & |u| \leq 1/2, \\ 0, & \text{otherwise,} \end{cases}$$

the kernel density estimator becomes

$$\hat{f}(x) = \frac{\#\{x_i : x_i \in [x - h/2, x + h/2]\}}{nh}.$$

This can be viewed as a moving histogram. Instead of counting observations in fixed bins, we center a window of width h at the evaluation point x and count the observations that fall inside the window. The value $h=0.35$ is chosen solely for illustration. In practice, selecting an appropriate bandwidth is an important problem and often requires careful study.

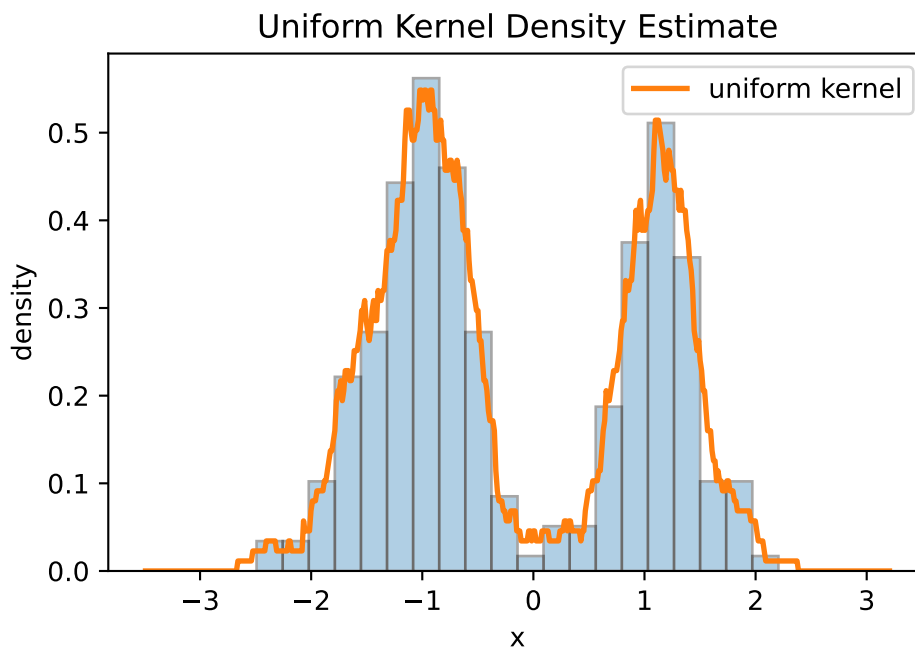
```

x_grid = np.linspace(x.min() - 1, x.max() + 1, 500)
h = 0.35

counts = [np.mean((x >= x0 - h / 2) & (x <= x0 + h / 2)) / h for x0 in x_grid]

plt.hist(x, bins=20, density=True, alpha=0.35, edgecolor="black")
plt.plot(x_grid, counts, color="C1", linewidth=2, label="uniform kernel")
plt.xlabel("x")
plt.ylabel("density")
plt.title("Uniform Kernel Density Estimate")
plt.legend();

```



8.2.2 Gaussian KDE

The uniform kernel gives equal weight to all observations inside the window and zero weight to observations outside. A Gaussian kernel uses a smoother weighting scheme in which nearby observations receive more weight than distant observations.

The Gaussian kernel is

$$K(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2}.$$

Substituting this kernel into the kernel density estimator gives

$$\hat{f}(x) = \frac{1}{nh\sqrt{2\pi}} \sum_{i=1}^n \exp\left(-\frac{1}{2}\left(\frac{x-x_i}{h}\right)^2\right).$$

This estimator can be interpreted as placing a small Gaussian density centered at each observation and then summing the contributions from all observations. The bandwidth h controls the width of these Gaussian curves.

In practice, kernel density estimation is usually performed using software rather than by evaluating the formula directly. `KernelDensity` from `sklearn.neighbors` provides an implementation of KDE with several kernel choices, including the Gaussian kernel.

```
from sklearn.neighbors import KernelDensity

kde = KernelDensity(kernel="gaussian", bandwidth=0.25 * x.std())
kde.fit(x.reshape(-1, 1))

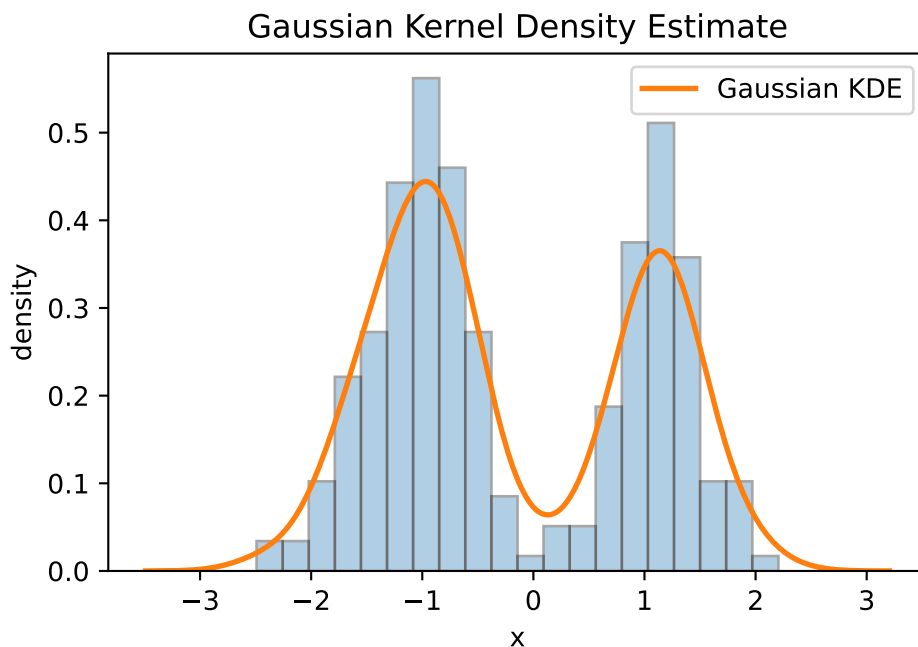
log_density = kde.score_samples(x_grid.reshape(-1, 1))
density = np.exp(log_density)
```

①

① As with most `sklearn` estimators, `KernelDensity` expects a 2D predictor matrix. Therefore the 1D array `x` is reshaped to an $n \times 1$ matrix before fitting the model.

The method `.score_samples()` returns the logarithm of the estimated density rather than the density itself. This is done for numerical stability. Applying `np.exp()` converts the values back to the density scale.

```
plt.hist(x, bins=20, density=True, alpha=0.35, edgecolor="black")
plt.plot(x_grid, density, color="C1", linewidth=2, label="Gaussian KDE")
plt.xlabel("x")
plt.ylabel("density")
plt.title("Gaussian Kernel Density Estimate")
plt.legend();
```



8.2.3 Expectation of the KDE

To better understand why kernel density estimation works, let us examine the expectation of the estimator.

Assume $x_1, \dots, x_n$ are independent samples from a density f . The estimator is

$$\hat{f}(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h} K\left(\frac{x - x_i}{h}\right).$$

Its expectation is

$$\begin{aligned} \mathbb{E}[\hat{f}(x)] &= \mathbb{E}\left[\frac{1}{h} K\left(\frac{x - X}{h}\right)\right] \\ &= \int \frac{1}{h} K\left(\frac{x - t}{h}\right) f(t) dt. \end{aligned}$$

Let

$$u = \frac{x - t}{h}, \quad t = x - hu.$$

Then

$$\mathbb{E}[\hat{f}(x)] = \int K(u)f(x - hu) du.$$

If h is small and f is smooth, then $f(x - hu)$ is close to $f(x)$. Since $\int K(u) du = 1$,

$$\mathbb{E}[\hat{f}(x)] \approx f(x).$$

This explains why KDE can recover the true density when the bandwidth is small enough, but not too small.

8.3 Kernel Regression

Kernel regression estimates

$$f(x) = \mathbb{E}[Y \mid X = x].$$

The most common kernel regression estimator is the Nadaraya-Watson estimator.

Definition 8.1 (Nadaraya-Watson estimator). Given training data (x_i, y_i) , the Nadaraya-Watson estimator is

$$\hat{f}(x) = \frac{\sum_{i=1}^n K_h(x, x_i) y_i}{\sum_{i=1}^n K_h(x, x_i)}.$$

This is a weighted average of the responses. The weights are

$$w_i(x) = \frac{K_h(x, x_i)}{\sum_{j=1}^n K_h(x, x_j)}.$$

Thus

$$\hat{f}(x) = \sum_{i=1}^n w_i(x) y_i, \quad \sum_{i=1}^n w_i(x) = 1.$$

For the Gaussian kernel,

$$K_h(x, x_i) = \frac{1}{h\sqrt{2\pi}} \exp \left\{ -\frac{(x - x_i)^2}{2h^2} \right\}.$$

Because the Nadaraya-Watson estimator normalizes the weights, the constant $1/(h\sqrt{2\pi})$ cancels out. Therefore, for regression weights, it is enough to use

$$\exp\left\{-\frac{(x-x_i)^2}{2h^2}\right\}.$$

8.3.1 Comparing KNN and Kernel Regression

The following example compares a KNN regression curve with a Gaussian kernel regression curve on the same data.

Example 8.1. We first generate a one-feature dataset and a grid for plotting predictions.

```
rng = np.random.default_rng(1)

X = rng.uniform(0, 2 * np.pi, size=40)
y = 2 * np.sin(X) + rng.normal(size=len(X))
X_grid = np.linspace(0, 2 * np.pi, 500)
```

For comparison we fit a KNN regressor with `n_neighbors=10`. Since the purpose is to show the idea, for simplicity we do not split train-test or tune K .

```
from sklearn.neighbors import KNeighborsRegressor

knn = KNeighborsRegressor(n_neighbors=10)
knn.fit(X.reshape(-1, 1), y)
y_knn = knn.predict(X_grid.reshape(-1, 1))
```

Now we define Gaussian kernel regression using the formula above.

```
def kernel_regression_predict(X, y, X_new, bandwidth):
    diff = X_new[:, None] - X[None, :]
    weights = np.exp(-0.5 * (diff / bandwidth) ** 2)
    weights = weights / weights.sum(axis=1, keepdims=True)
    return weights @ y

h = np.std(X) * 0.25
y_kernel = kernel_regression_predict(X, y, X_grid, bandwidth=h)
```

Here `diff = X_new[:, None] - X[None, :]` uses `numpy` broadcasting to create a matrix of pairwise differences. It was briefly discussed in Section C.3.3. If `X_new` contains m points and `X` contains n points, then `diff` has shape (m, n) , where the (i, j) entry is the difference between the i th prediction point and the j th training point.

The bandwidth is chosen to be 0.25 of the standard deviation of the feature. This is only a simple starting value; bandwidth tuning is discussed later.

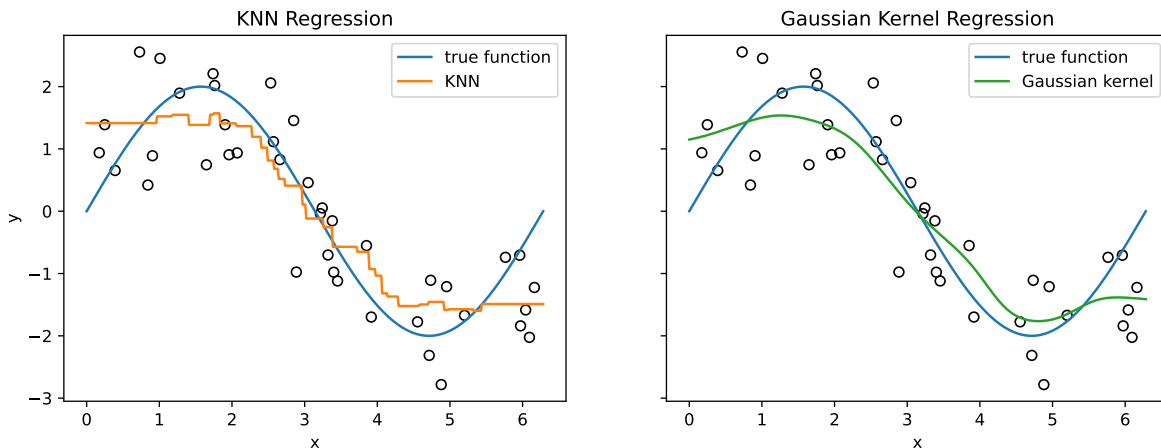
```
fig, axs = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

axs[0].scatter(X, y, s=35, facecolors="none", edgecolors="black")
axs[0].plot(X_grid, 2 * np.sin(X_grid), color="C0", label="true function")
axs[0].plot(X_grid, y_knn, color="C1", label="KNN")
axs[0].set_title("KNN Regression")

axs[1].scatter(X, y, s=35, facecolors="none", edgecolors="black")
axs[1].plot(X_grid, 2 * np.sin(X_grid), color="C0", label="true function")
axs[1].plot(X_grid, y_kernel, color="C2", label="Gaussian kernel")
axs[1].set_title("Gaussian Kernel Regression")

for ax in axs:
    ax.set_xlabel("x")
    ax.legend()

axs[0].set_ylabel("y");
```



The KNN fit is piecewise constant because the prediction is the average of a fixed set of nearest neighbors. As the target point moves, the neighbor set may suddenly change, causing abrupt jumps in the fitted curve.

The Gaussian kernel fit changes smoothly because every observation contributes to the prediction. The weights vary continuously with distance, producing a smooth estimate of the regression function.

8.3.2 Local Averaging at One Target Point

The key idea of kernel regression is local weighted averaging. In this section we use the previous example to visualize this behavior.

At a target point x_0 , observations closer to x_0 receive larger kernel weights. Recall that with a Gaussian kernel and bandwidth $h = 0.5$,

$$K_h(x_0, x_i) = \frac{1}{h\sqrt{2\pi}} \exp \left\{ -\frac{(x_0 - x_i)^2}{2h^2} \right\}.$$

The Nadaraya-Watson estimate at x_0 is the weighted average

$$\hat{f}(x_0) = \sum_{i=1}^n w_i(x_0) y_i, \quad \text{where} \quad w_i(x_0) = \frac{K_h(x_0, x_i)}{\sum_{j=1}^n K_h(x_0, x_j)}.$$

We first compute the kernel weights for the target point $x_0 = 3$. Here, raw means that the weights have not yet been normalized. The estimate $\hat{f}(x_0)$ is then computed.

```
x0 = 3.0
h = 0.5

raw_weights = np.exp(-0.5 * ((x0 - X) / h) ** 2) / h
fhat_x0 = np.sum(raw_weights * y) / np.sum(raw_weights)
```

We then use the raw weights to determine the size of each plotted point so that larger points correspond to larger kernel weights.

```
point_sizes = 200 * raw_weights / raw_weights.max()
```

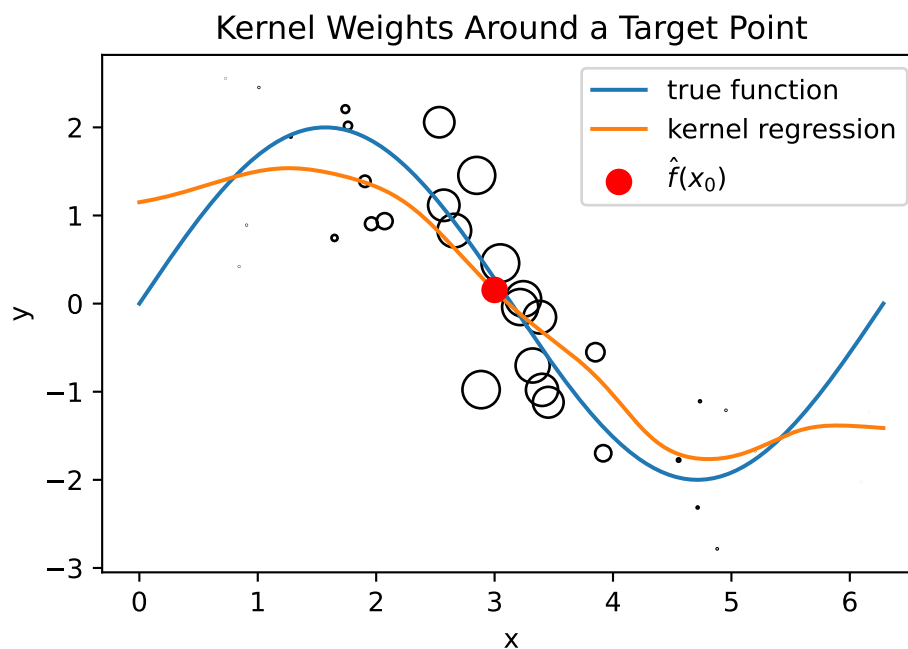
We can now plot the data together with the fitted curve, using the assigned point sizes to represent the kernel weights.

```
plt.scatter(X, y, s=point_sizes, facecolors="none", edgecolors="black")
plt.plot(X_grid, 2 * np.sin(X_grid), color="C0", label="true function")
plt.plot(X_grid, y_kernel, color="C1", label="kernel regression")

plt.scatter([x0], [fhat_x0], color="red", s=80, zorder=3, label=r"$\hat{f}(x_0)$") ①

plt.xlabel("x")
plt.ylabel("y")
plt.title("Kernel Weights Around a Target Point")
plt.legend();
```

① We specifically plot the estimated point.



The circle sizes are proportional to the kernel weights. Points near x_0 receive larger weights and therefore have a greater influence on the local average. Points farther away receive smaller weights and contribute less to the estimate.

8.4 Bandwidth Tuning

The bandwidth h is the most important tuning parameter in kernel smoothing.

- Small h : uses very local information and gives a flexible, variable estimate.

- Large h : averages over a wider region and gives a smoother, more biased estimate.

This is the same bias–variance tradeoff seen in KNN. In fact, the bandwidth plays a role similar to the parameter K in KNN: both control how local the estimate is. Choosing an appropriate bandwidth is therefore a model selection problem. In practice, cross-validation is often used to select a bandwidth that balances bias and variance.

8.4.1 Effect of Bandwidth in KDE

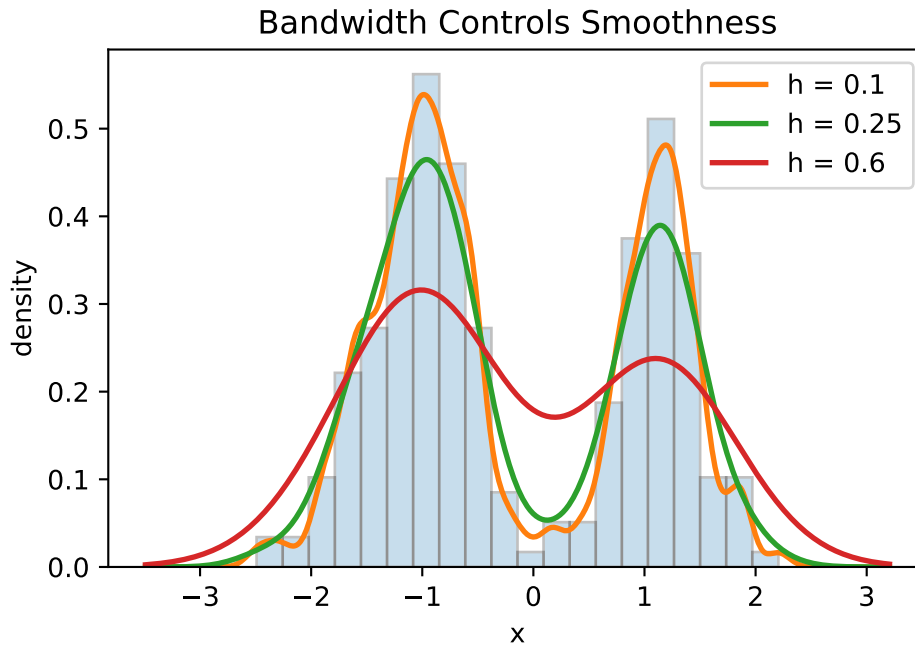
We can plot the KDE curve with different bandwidths to see the effect.

```
bandwidths = [0.1, 0.25, 0.6]

plt.hist(x, bins=20, density=True, alpha=0.25, edgecolor="black")

for bw in bandwidths:
    kde = KernelDensity(kernel="gaussian", bandwidth=bw)
    kde.fit(x.reshape(-1, 1))
    density = np.exp(kde.score_samples(x_grid.reshape(-1, 1)))
    plt.plot(x_grid, density, linewidth=2, label=f"h = {bw}")

plt.xlabel("x")
plt.ylabel("density")
plt.title("Bandwidth Controls Smoothness")
plt.legend();
```



From the plot we can see that bandwidth controls the smoothness of the estimated density.

- Small bandwidths capture local detail but may produce a noisy estimate.
- Large bandwidths produce smoother estimates but may hide important features of the distribution.

8.4.2 Effect of Bandwidth in Kernel Regression

The same idea applies to kernel regression.

```
X_grid = np.linspace(0, 2 * np.pi, 500)

fig, axs = plt.subplots(1, 3, figsize=(14, 4), sharey=True)

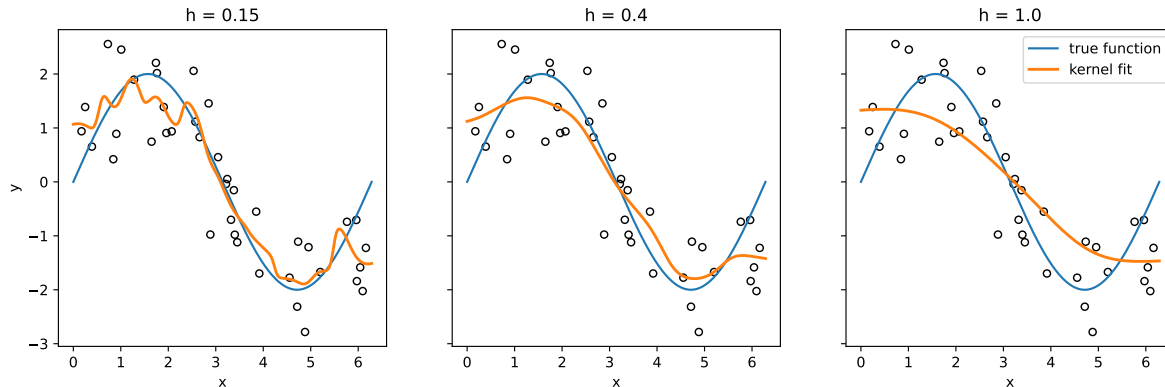
for ax, bw in zip(axs, [0.15, 0.4, 1.0]):
    y_hat = kernel_regression_predict(X, y, X_grid, bandwidth=bw)

    ax.scatter(X, y, s=25, facecolors="none", edgecolors="black")
    ax.plot(X_grid, 2 * np.sin(X_grid), color="C0", label="true function")
    ax.plot(X_grid, y_hat, color="C1", linewidth=2, label="kernel fit")
    ax.set_title(f"h = {bw}")
    ax.set_xlabel("x")
```

```

axs[0].set_ylabel("y")
axs[-1].legend()
plt.show()

```



As the bandwidth increases, variance decreases but bias increases. Choosing an appropriate bandwidth therefore requires balancing these two sources of error.

In practice, bandwidth selection is itself a statistical problem. We next introduce a simple rule-of-thumb that provides a reasonable starting value for the bandwidth.

8.4.3 Rule-of-Thumb Bandwidth Selection

Numerous bandwidth selection methods have been developed in the literature. The optimal choice often depends on the underlying distribution, the sample size, and the purpose of the analysis. In practice, bandwidth selection is usually treated as a tuning problem.

For one-dimensional smoothing, a common rule-of-thumb bandwidth is Silverman's rule:

$$h = 1.06 \hat{\sigma}_x n^{-1/5}.$$

This formula is useful as a starting value, but it depends on assumptions that may not hold in practice. Cross-validation is usually safer when prediction performance matters.

```

silverman_h = 1.06 * np.std(X, ddof=1) * len(X) ** (-1 / 5)
silverman_h

```

```

np.float64(0.918770850083262)

```


8.4.4 Bandwidth Selection by Cross-Validation

Rule-of-thumb methods are fast and easy to use, but they rely on assumptions that may not hold in practice. Cross-validation provides a more data-driven approach to bandwidth selection.

The following code uses `GridSearchCV` to select the bandwidth. Since `sklearn` doesn't provide the API for Nadaraya-Watson estimator, in order to keep the usual `sklearn` workflow, we write a custom estimator.

```
from sklearn.base import BaseEstimator, RegressorMixin

class NadarayaWatsonRegressor(BaseEstimator, RegressorMixin):
    def __init__(self, bandwidth=1.0):
        self.bandwidth = bandwidth

    def fit(self, X, y):
        self.X_train_ = np.asarray(X).reshape(-1, 1)
        self.y_train_ = np.asarray(y)
        return self

    def predict(self, X):
        X = np.asarray(X).reshape(-1, 1)
        distances = X - self.X_train_.T
        weights = np.exp(-0.5 * (distances / self.bandwidth) ** 2)
        weights = weights / weights.sum(axis=1, keepdims=True)
        return weights @ self.y_train_
```

Then use the regular grid search process.

```
from sklearn.model_selection import GridSearchCV

bandwidth_grid = {
    "bandwidth": np.linspace(0.1, 1.2, 23),
}

grid = GridSearchCV(NadarayaWatsonRegressor(), param_grid=bandwidth_grid, cv=5)

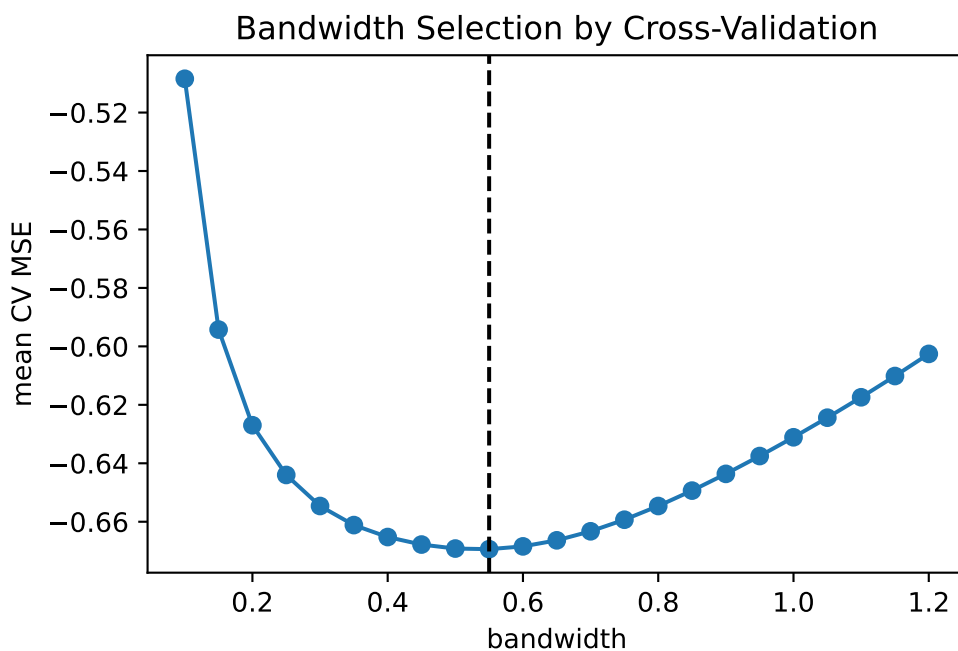
grid.fit(X.reshape(-1, 1), y)
grid.best_params_
```

```
{'bandwidth': np.float64(0.5499999999999999)}
```

Plot the cross-validation curve.

```
cv_mse = -grid.cv_results_["mean_test_score"]
bandwidth_values = bandwidth_grid["bandwidth"]

plt.plot(bandwidth_values, cv_mse, marker="o")
plt.axvline(grid.best_params_["bandwidth"], color="black", linestyle="--")
plt.xlabel("bandwidth")
plt.ylabel("mean CV MSE")
plt.title("Bandwidth Selection by Cross-Validation");
```

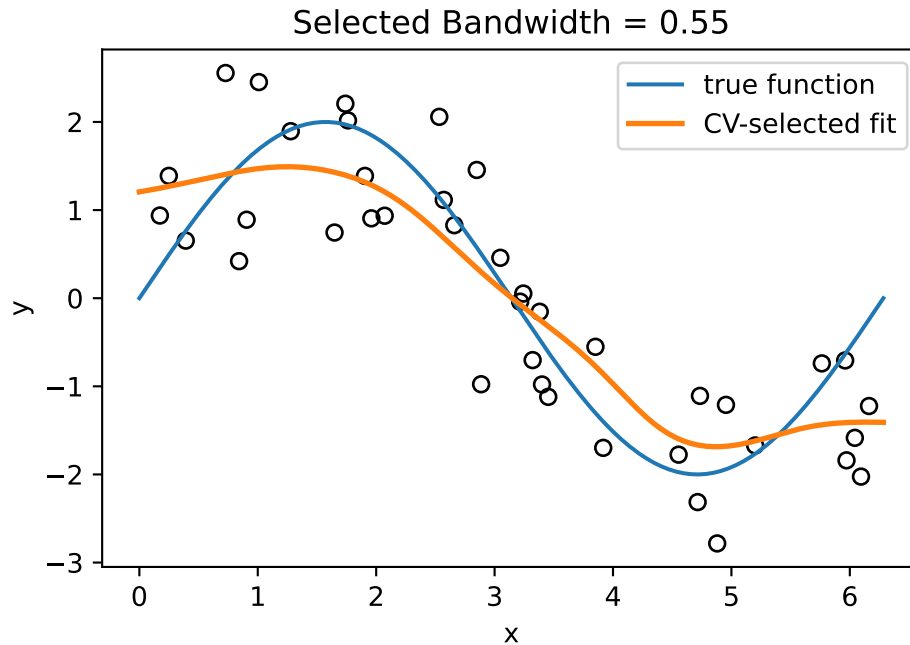


Fit the selected model.

```
best_kernel = grid.best_estimator_
y_best = best_kernel.predict(X_grid.reshape(-1, 1))

plt.scatter(X, y, s=35, facecolors="none", edgecolors="black")
plt.plot(X_grid, 2 * np.sin(X_grid), color="C0", label="true function")
plt.plot(X_grid, y_best, color="C1", linewidth=2, label="CV-selected fit")
plt.xlabel("x")
```

```
plt.ylabel("y")
plt.title(f"Selected Bandwidth = {grid.best_params_['bandwidth']:.2f}")
plt.legend()
```



8.5 Extensions

The basic Nadaraya-Watson estimator is a local constant estimator. It can be extended in several directions.

8.5.1 Local Linear Regression

Around each target point x_0 , local constant regression fits a weighted constant. Local linear regression instead fits a weighted line:

$$\min_{\beta_0, \beta_1} \sum_{i=1}^n K_h(x_0, x_i) [y_i - \beta_0 - \beta_1(x_i - x_0)]^2.$$

The fitted value at x_0 is the fitted intercept:

$$\hat{f}(x_0) = \hat{\beta}_0.$$

Local linear regression often improves boundary behavior. Near the edge of the data, a local average can be biased because there are observations on only one side of the target point. A local line can adjust for the slope.

i Boundary bias

Local averaging can be biased near the boundary of the data.

Local linear regression reduces this problem by fitting a local trend instead of a local constant.

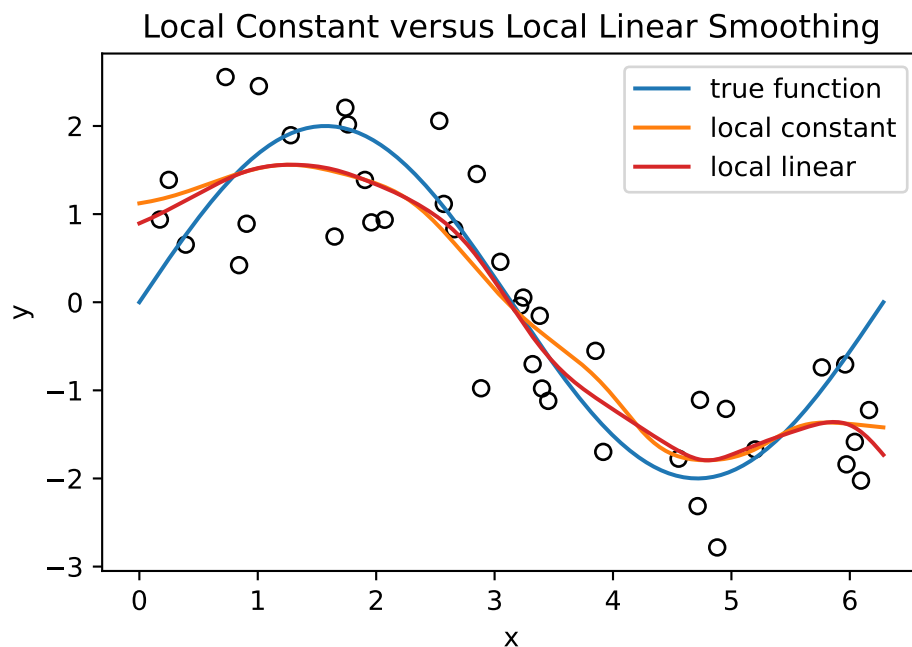
```
def local_linear_predict(X_train, y_train, X_new, bandwidth):
    X_train = np.asarray(X_train)
    y_train = np.asarray(y_train)
    preds = []

    for x0 in X_new:
        z = X_train - x0
        weights = np.exp(-0.5 * (z / bandwidth) ** 2)
        design = np.column_stack([np.ones_like(z), z])
        weighted_design = design * weights[:, None]
        beta = np.linalg.pinv(weighted_design.T @ design) @ weighted_design.T @ y_train
        preds.append(beta[0])

    return np.asarray(preds)

y_local_constant = kernel_regression_predict(X, y, X_grid, bandwidth=0.4)
y_local_linear = local_linear_predict(X, y, X_grid, bandwidth=0.4)

plt.scatter(X, y, s=35, facecolors="none", edgecolors="black")
plt.plot(X_grid, 2 * np.sin(X_grid), color="C0", label="true function")
plt.plot(X_grid, y_local_constant, color="C1", label="local constant")
plt.plot(X_grid, y_local_linear, color="C3", label="local linear")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Local Constant versus Local Linear Smoothing")
plt.legend()
plt.show()
```



8.5.2 Local Polynomial Regression

Local linear regression can be extended to local polynomial regression. At each target point x_0 , fit

$$\min_{\beta_0, \dots, \beta_d} \sum_{i=1}^n K_h(x_0, x_i) \left[y_i - \beta_0 - \sum_{r=1}^d \beta_r (x_i - x_0)^r \right]^2.$$

Then

$$\hat{f}(x_0) = \hat{\beta}_0.$$

The degree d controls the local shape:

- $d = 0$: local constant regression,
- $d = 1$: local linear regression,
- $d = 2$: local quadratic regression.

Higher-degree local polynomials can reduce bias, but they may also become unstable if the bandwidth is too small or the data are sparse.

The following code compares local constant, local linear, and local quadratic smoothing.

```

def local_polynomial_predict(X_train, y_train, X_new, bandwidth, degree):
    X_train = np.asarray(X_train)
    y_train = np.asarray(y_train)
    preds = []

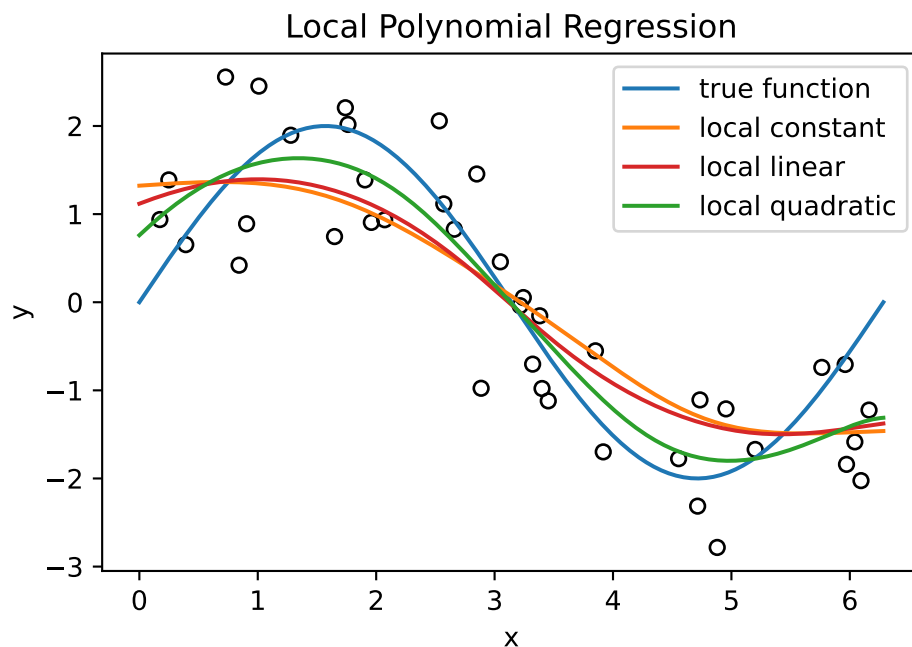
    for x0 in X_new:
        z = X_train - x0
        weights = np.exp(-0.5 * (z / bandwidth) ** 2)
        design = np.column_stack([z**d for d in range(degree + 1)])
        weighted_design = design * weights[:, None]
        beta = np.linalg.pinv(weighted_design.T @ design) @ weighted_design.T @ y_train
        preds.append(beta[0])

    return np.asarray(preds)

h_rule = silverman_h
y_constant = kernel_regression_predict(X, y, X_grid, bandwidth=h_rule)
y_linear = local_polynomial_predict(X, y, X_grid, bandwidth=h_rule, degree=1)
y_quadratic = local_polynomial_predict(X, y, X_grid, bandwidth=h_rule, degree=2)

plt.scatter(X, y, s=30, facecolors="none", edgecolors="black")
plt.plot(X_grid, 2 * np.sin(X_grid), color="C0", label="true function")
plt.plot(X_grid, y_constant, color="C1", label="local constant")
plt.plot(X_grid, y_linear, color="C3", label="local linear")
plt.plot(X_grid, y_quadratic, color="C2", label="local quadratic")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Local Polynomial Regression")
plt.legend()
plt.show()

```



8.5.3 Multiple Dimensions

For $x_i \in \mathbb{R}^p$, a common Gaussian kernel is

$$K_h(x, x_i) \propto \exp\left(-\frac{\|x - x_i\|^2}{2h^2}\right).$$

This is similar to KNN in that it relies on distances in feature space. Therefore, scaling matters.

In multiple dimensions, kernel smoothing faces the curse of dimensionality. As p increases, local neighborhoods become sparse. To get enough nearby points, the bandwidth must grow, which increases bias.

For p dimensions, bandwidth rates are often of order

$$n^{-1/(p+4)}.$$

This gets slower as p increases, which is another way to see the curse of dimensionality.

Bandwidth conventions

Different software packages may define bandwidth differently. Some use a raw scale h inside the kernel. Others use a span, a nearest-neighbor fraction, or an internally rescaled bandwidth. Always check the documentation before comparing bandwidth values across implementations.

8.6 Summary

Kernel smoothing:

- starts from the same local-information intuition as KNN,
- uses a kernel function to assign distance-based weights,
- estimates densities through KDE and regression functions through kernel regression,
- is controlled mainly by the bandwidth h ,
- produces smoother estimates than KNN when the kernel is smooth,
- has the same bias–variance tradeoff as other nonparametric methods,
- can suffer from boundary bias,
- becomes harder in high-dimensional feature spaces.

The practical workflow is:

1. Start from the KNN intuition: local observations matter most.
2. Choose a kernel, often Gaussian.
3. Use KDE for density estimation or kernel regression for conditional means.
4. Tune the bandwidth by validation, cross-validation, or a rule of thumb.
5. Consider extensions such as local linear smoothing when boundary bias matters.

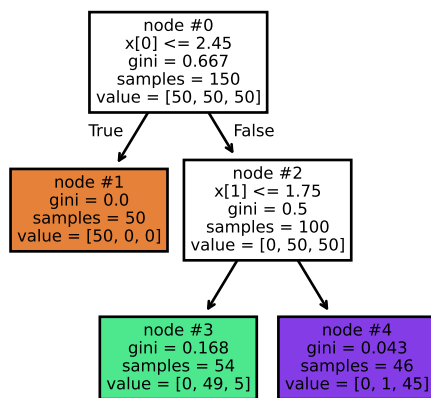
9 Tree-Based Methods

A decision tree is another nonparametric method, which learns a collection of simple if-then rules directly from the data.

The basic idea is:

1. Split the feature space into regions.
2. Continue splitting each region into smaller regions.
3. Use the observations inside each final region to make predictions.

Example 9.1 (Example: A Classification Tree).

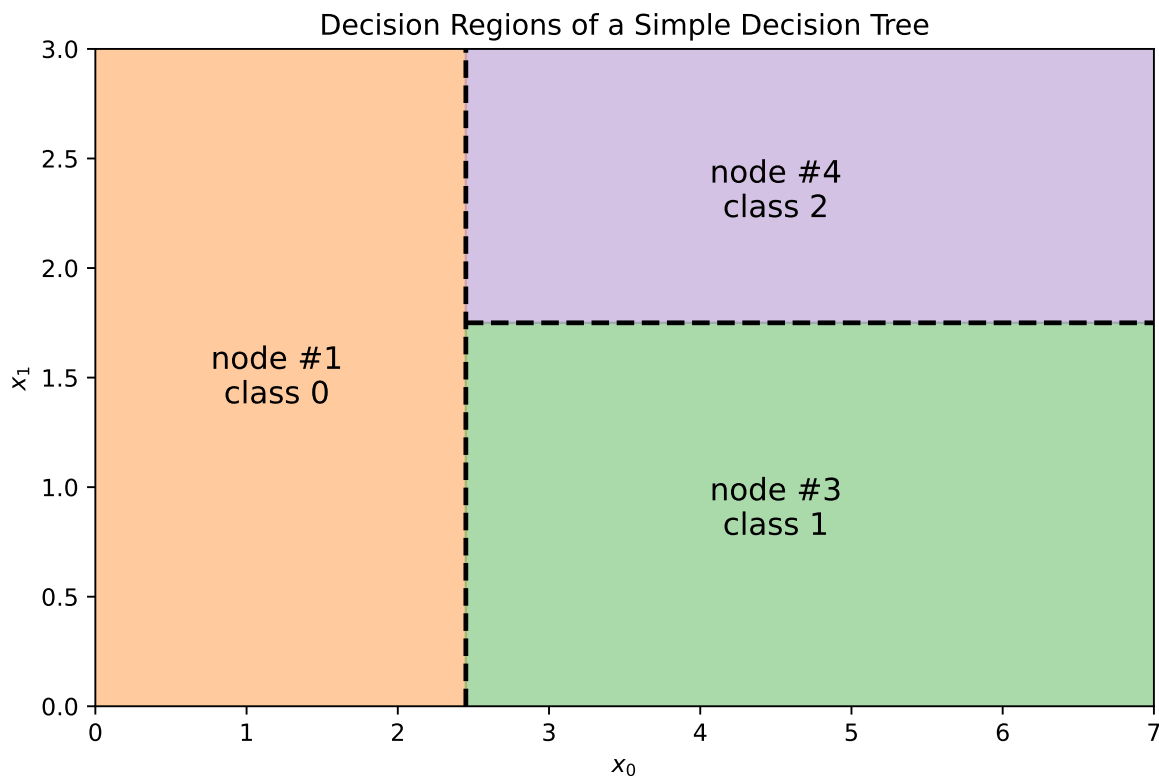


This is a classification tree. The tree repeatedly asks simple questions such as

- Is $x[0] \leq 2.45$?
- Is $x[1] \leq 1.75$?

Each split divides the feature space into smaller regions.

This tree actually create the following splits in the feature space.



Based on the idea of a decision tree, a tree makes decisions by repeatedly asking questions of the form $x_j \leq c$. Therefore, a tree has two important properties:

- It depends directly on the original features.
- The scale of a feature is usually not important.

As a result, we should expect that a tree is:

- sensitive to feature construction or feature combinations: A tree can only split on the features that are provided. If the useful signal is hidden in a combination such as $x_1 + x_2$ or $x_1 x_2$, a tree may need many splits to approximate that relationship.
- not sensitive to monotone transformations of a feature: For example, replacing x by $\log(x)$ or $2x + 5$ does not change the ordering of the observations. Since tree splits mainly depend on ordering, the tree can usually produce an equivalent split after the transformation.

9.1 CART: Classification and Regression Trees

There are many algorithms for constructing decision trees. The most common one is called CART (Classification and Regression Trees).

CART has the following properties:

- It creates only binary splits;
- It can perform both classification and regression tasks:
 - For classification, common splitting criteria include Gini impurity and entropy;
 - For regression, common splitting criteria include residual sum of squares (RSS) and mean squared error (MSE).

9.1.1 Splitting a Dataset

At each step, CART searches over possible splits and chooses the split that produces the largest reduction in impurity (or prediction error).

i Algorithm: Split the Dataset

Inputs: A dataset $S = [\text{features}, \text{target}]$

Outputs: The best split (k, t)

1. For each feature x_k :
 1. For each candidate split value t :
 1. Split the dataset into two subsets: $G_l = \{\text{data with } x_k \leq t\}$, $G_r = \{\text{data with } x_k > t\}$.
 2. Compute the split cost function $J(k, t)$
 3. Keep the best split found so far
 2. Return the split (k, t) with the smallest cost

The splitting algorithm is applied recursively.

i Algorithm: CART

Inputs: A labeled dataset S and a maximal tree depth `max_depth`

Outputs: A decision tree

1. Start with the original dataset S
2. If the node is not pure and stopping conditions are not met:
 1. Find the best split

2. Split the node into: G_l and G_r
 3. Apply the same procedure recursively to both child nodes
3. Stop when:
- the maximal depth is reached, or
 - the node becomes pure, or
 - further splitting is impossible

When making predictions, a classification tree uses the most common class label in a region as the predicted value, while a regression tree uses the mean of the response values in the region as the predicted value.

9.1.2 Classification Trees: Gini Impurity

Classification trees are used when the response variable is categorical. A classification tree partitions the feature space into piecewise constant regions, where each region predicts either a class label or class probabilities.

In these notes, we use Gini impurity to measure the quality of a split.

Assume that a node contains n observations divided into K classes. Let $p_i = \frac{n_i}{n}$ be the proportion of class i . If we classify observations according to the class proportions:

- the probability an observation belongs to class i is p_i
- the probability we incorrectly classify it is $1 - p_i$

Therefore the probability of misclassifying an observation from class i is

$$p_i(1 - p_i).$$

Summing over all classes gives

$$\sum_{i=1}^K p_i(1 - p_i).$$

Since

$$\sum_{i=1}^K p_i = 1,$$

we obtain

$$\text{Gini} = \sum_{i=1}^K p_i(1 - p_i) = 1 - \sum_{i=1}^K p_i^2.$$

Definition 9.1 (Gini Impurity). The **Gini impurity** of a node is

$$\text{Gini} = 1 - \sum_{i=1}^K p_i^2,$$

where p_i is the proportion of class i inside the node.

Example 9.2 (Pure node). Suppose a node contains observations from only one class. Then one of the probabilities equals 1 and all others equal 0.

Therefore

$$\text{Gini} = 1 - 1^2 = 0.$$

This is the minimum possible impurity.

A node with Gini impurity 0 is called a pure node.

Example 9.3 (Two-Class Example). Suppose we have two classes.

Let

$$p_2 = 1 - p_1.$$

Then

$$\text{Gini} = 1 - p_1^2 - (1 - p_1)^2 = 2p_1 - 2p_1^2.$$

When:

- $p_1 = 0$ or 1 , the impurity is 0;
- $p_1 = 0.5$, the impurity is maximized.

Therefore, impurity is largest when the classes are perfectly balanced.

The impurity of a node alone is not enough. CART evaluates a split by examining the impurity of the child nodes after splitting.

Suppose a node G is split into G_l and G_r . The split cost is

$$J = \frac{|G_l|}{|G|} \text{Gini}(G_l) + \frac{|G_r|}{|G|} \text{Gini}(G_r).$$

A good split produces child nodes with low impurity, ideally creating nodes that are close to pure.

i Entropy

Entropy is another popular impurity measure. Its formula is

$$\text{Entropy} = - \sum_{i=1}^K p_i \log(p_i).$$

Entropy and Gini impurity usually produce very similar trees.

Because Gini impurity is slightly simpler computationally and conceptually, we mainly focus on it.

9.1.3 Regression Trees: RSS

Regression trees are used when the response variable is quantitative. Instead of predicting a class label, they predict numerical values.

Suppose a terminal node contains responses $y_1, y_2, \dots, y_n$. The prediction for that region is usually the sample mean:

$$\hat{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

Therefore, regression trees approximate the regression function using piecewise constant regions.

For regression trees, CART typically minimizes the residual sum of squares (RSS). Sometimes the mean squared error (MSE) is used instead. Since MSE differs from RSS only by a constant scaling factor, minimizing one is equivalent to minimizing the other.

Suppose a split divides the data into two regions G_l and G_r . The split cost is

$$\text{RSS} = \sum_{i \in G_l} (y_i - \bar{y}_l)^2 + \sum_{i \in G_r} (y_i - \bar{y}_r)^2,$$

where $\bar{y}_l$ and $\bar{y}_r$ are the mean responses within the two regions. The best split is the one that minimizes this quantity.

9.1.4 Trees are local

A tree makes predictions using only observations inside a small region of the feature space. Suppose a tree partitions the feature space into regions. The prediction at a point x depends only on the region containing x . As a result, tree predictions are local:

- nearby regions may have very different predictions
- only observations in the same local region affect the prediction
- changing distant observations usually does not affect the prediction

This is similar in spirit to KNN, where prediction also depends primarily on nearby observations.

9.1.5 Trees are nonlinear

A tree creates many if-then rules. Because of these recursive splits:

- the prediction function can change abruptly
- variable effects depend on the region
- interactions are created automatically

The prediction surface is therefore discontinuous and piecewise constant. Therefore, trees are nonlinear models.

9.1.6 Automatic interaction effects

One important property of trees is that they automatically create interaction effects.

For example, consider a tree with depth 3. One path through the tree may look like:

- if $x_1 \leq 1$
- and $x_2 \leq 2$
- and $x_3 \leq 3$
- then predict $y = 1.1$

Notice that the prediction depends on the combination of all three conditions simultaneously. Therefore, the effect of one variable may depend on the values of earlier splitting variables. In other words, the effect of x_3 is not independent of x_1 and x_2 . The split on x_3 is only relevant inside the region defined by the earlier splits.

Therefore, trees naturally model interactions between variables.

This is very different from a standard additive linear model such as

$$f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3,$$

where each variable contributes independently unless explicit interaction terms like $x_1 x_2$ are manually added.

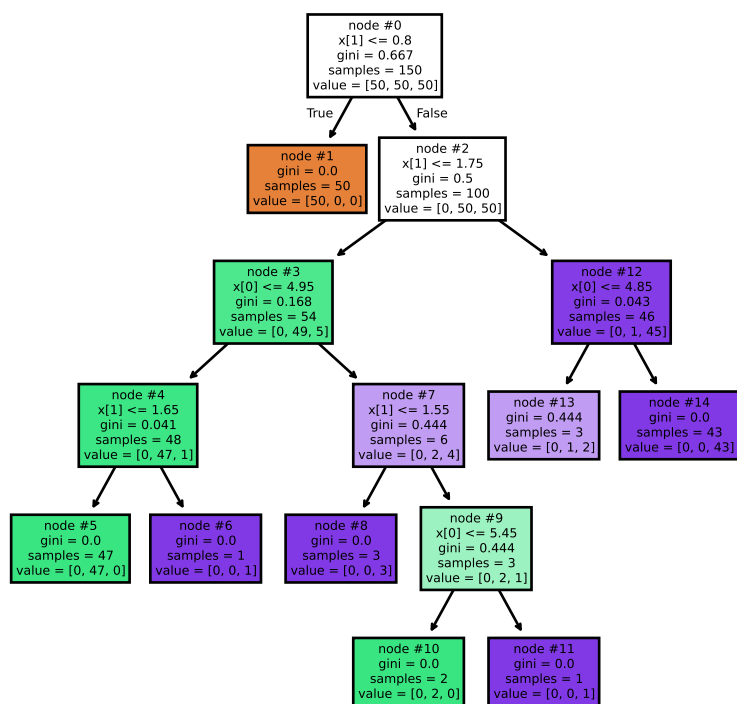
9.2 Tuning hyperparameters

Building a decision tree is the same as splitting the training dataset. If we are allowed to keep splitting it, it is possible to get to a case that each end node is pure: the Gini impurity is 0, or every element has the same response variable locally.

Example 9.4 (ALMOST all end nodes are pure).

Click to expand.

In this example, we let the tree grow as further as possible. It only stops when (almost) all end nodes are pure, even if the end nodes only contain ONE element (like #6 and #11).



In this example there is one exception. #13 node is not pure. We could list all data in node #13.

	x[0]	x[1]	y
0	4.8	1.8	1
1	4.8	1.8	2
2	4.8	1.8	2

All the data points in this node has the same feature while their labels are different. They cannot be split further purely based on features.

As the tree continues splitting the feature space into smaller regions, it learns increasingly fine details of the training set. Although this increases flexibility, it can also make the model overly sensitive to noise. Therefore, we usually do not want the tree to grow indefinitely.

There are two common ways to control a single tree growth:

1. Pruning: first grow a large tree, then remove unnecessary branches afterward.
2. Pre-pruning: stop the tree early by imposing restrictions during training.

9.2.1 Pruning

Pruning means removing branches from a tree. The pruning method introduced in CART is called Minimal Cost-Complexity Pruning (MCCP).

The idea is very similar to regularization in ridge or lasso regression: we balance goodness of fit against model complexity by adding a penalty term. In MCCP, the penalty is proportional to the number of terminal nodes (leaves) in the tree.

More specifically, the cost function is modified by adding a complexity penalty controlled by the parameter `ccp_alpha`. The parameter `ccp_alpha` is called the complexity parameter and is always ≥ 0 . Let T be a subtree, and let $|T|$ denote the number of terminal nodes. The cost-complexity criterion is

$$C_\alpha(T) = J(T) + \alpha|T|,$$

where:

- $J(T)$ measures the training error,
- $|T|$ measures tree complexity through the number of terminal nodes,
- α is corresponding to `ccp_alpha` in the notes.

This has the same general structure as regularized regression: loss + penalty.

The tuning parameter α controls how expensive it is to add another leaf. If a split reduces the training error by only a small amount, it may not be worth keeping when α is large. When $\alpha = 0$, there is no complexity penalty, so the procedure behaves like an ordinary decision tree and larger trees are generally preferred.

When applying MCCP, we typically begin with a large tree and then compute the `ccp_alpha` path, which contains the critical values where the tree structure changes. Each critical value corresponds to a different subtree produced by pruning. We then use validation or cross-validation to compare these candidate subtrees and choose the best value of `ccp_alpha`.

If several values of `ccp_alpha` achieve similar validation performance, we usually prefer the simpler tree. Therefore, among models with similar test or validation scores, a larger `ccp_alpha` is often preferred because it produces a smaller and more stable tree.

9.2.2 Pre-pruning

We often impose some early stopping conditions to limit the growth of a tree. This strategy is called pre-pruning. Some common early stopping conditions are:

- `max_depth`: maximum tree depth
- `min_samples_split`: minimum number of observations required to split a node
- `min_samples_leaf`: minimum number of observations in a terminal node
- `max_leaf_nodes`: maximum number of leaves

Smaller trees are usually more stable but may underfit. Larger trees are more flexible but may overfit. A larger tree is not automatically better, even if it achieves lower training error.

However, pre-pruning can be greedy. A split that appears only mildly useful at an early stage may later enable highly informative splits deeper in the tree. If we stop the tree too early, we may miss important structure in the data.

Therefore, in modern practice, pre-pruning is usually not the primary method for selecting the final model complexity. Instead, we mainly rely on pruning by choosing the best `ccp_alpha` through validation or cross-validation. Pre-pruning is often used only as a guardrail to prevent the initial tree from becoming excessively large. In practice, we usually set relatively mild early stopping limits to stabilize the tree-growing process and then rely on pruning to determine the final tree size.

i Tuning in practice

In classical CART, `ccp_alpha` is the primary complexity parameter and is usually selected by cross-validation after growing a large tree.

In modern practice, pre-pruning parameters and `ccp_alpha` are often tuned together, especially for computational efficiency and stability. In other words, methods such as `GridSearchCV` can be used to tune all hyperparameters simultaneously.

However, when the search space becomes too large, this approach can become computationally expensive. In that case, we often return to the more traditional CART-style strategy: we first coarsely identify a reasonable range for the early stopping hyperparameters and then fine-tune `ccp_alpha`.

9.3 Trees in Python

We use `DecisionTreeClassifier` and `DecisionTreeRegressor` from `sklearn.tree` to implement decision tree models. The API is almost identical to other `sklearn` models: `.fit()`, `.predict()`, `.score()`, and other familiar methods are used in the same way.

1. We could set `max_depth`, `min_samples_leaf`, `ccp_alpha`, etc. for the tree model where their meaning is straightforward.
2. The method `.cost_complexity_pruning_path()` can be used to find the critical `ccp_alpha` values for pruning.
3. We could also set `random_state`. The randomness in `sklearn` decision trees mainly comes from the internal feature ordering used during the split search. Even when `splitter="best"` is used, `sklearn` randomly permutes the feature order before evaluating splits. Therefore, when multiple splits produce the same impurity reduction, different trees may be generated unless `random_state` is fixed.

i Note

Decision trees are often described as methods that naturally handle categorical predictors. This is true conceptually, but **sklearn** decision trees still require numeric input. Therefore categorical predictors must be encoded before fitting the model. For nominal categorical variables, one-hot encoding is usually a safe choice.

However, one-hot encoding for the target variable is not necessary because **sklearn** classifiers can directly work with class labels.

9.3.1 Example 1: Classify iris dataset

The **iris** dataset is a classic classification dataset. The goal is to use four numeric flower measurements to classify the species of iris. The dataset contains 150 observations and 3 classes. More information can be found in the **sklearn** documentation for datasets.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

The following code shows the basic **sklearn** API.

```
from sklearn.tree import DecisionTreeClassifier

clf = DecisionTreeClassifier(random_state=2)
clf.fit(X_train, y_train)
clf.score(X_test, y_test)
```

0.9666666666666667

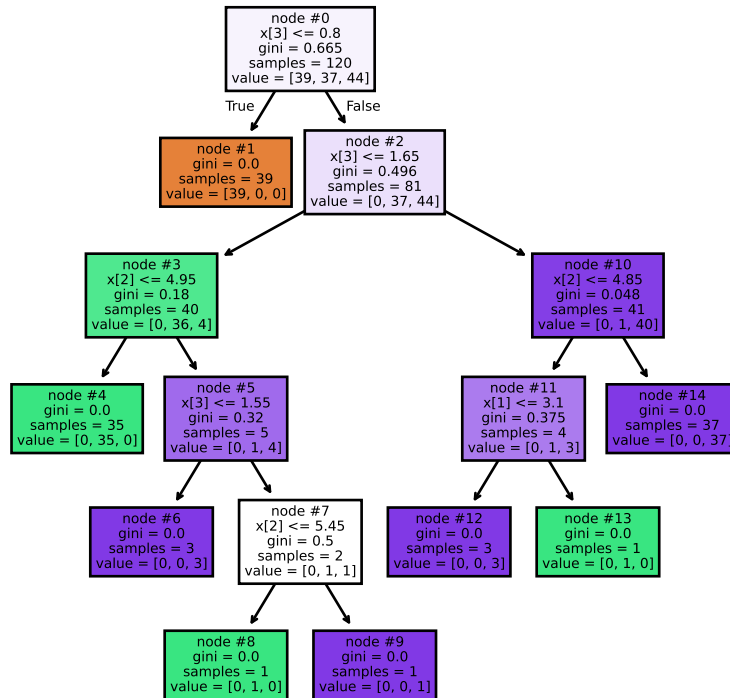
After fitting the model, we can display the tree using **plot_tree()**. In many real examples, the tree may be too large to read clearly, so we use **matplotlib** to control the size and resolution of the plot.

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(5, 5), dpi=300)
plot_tree(clf, filled=True, impurity=True, node_ids=True);
```

①

- ① `filled=True` fills the nodes with colors. `impurity=True` displays the Gini impurity. `node_ids=True` displays the node ID.



Now we prune the tree. We first find all critical `ccp_alpha` values.

```
ccp_path = clf.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = ccp_path.ccp_alphas[:-1]
```

The last `ccp_alpha` is usually removed because it is large enough to prune the tree down to a single root node. In most applications, that model is too simple to be useful.

We then train one model for each effective `ccp_alpha`. After fitting the trees, we record both predictive performance and model complexity. Here we use `accuracy` as our metric, which comes automatically from `clf.score()`.

```

from sklearn.metrics import accuracy_score

clfs = []

for ccp_alpha in ccp_alphas:
    clf = DecisionTreeClassifier(random_state=42, ccp_alpha=ccp_alpha)
    clf.fit(X_train, y_train)
    clfs.append(clf)

node_counts = [clf.tree_.node_count for clf in clfs]
train_acc = [clf.score(X_train, y_train) for clf in clfs]
test_acc = [clf.score(X_test, y_test) for clf in clfs]

```

Here we use a single train/test split. This is a simple validation strategy: the model is fitted on the training set and evaluated on the test set. In addition to accuracy, we also record the number of nodes because it is a useful measure of tree complexity.

A larger tree usually has lower bias but higher variance. A smaller tree is often more interpretable and more stable.

The results are easier to compare visually.

```

import matplotlib.pyplot as plt

best_alpha = 0.05

fig, ax = plt.subplots(2, 1, figsize=(7, 7), sharex=True)

ax[0].plot(ccp_alphas, node_counts, marker="o", drawstyle="steps-post") ①
ax[1].plot(ccp_alphas, train_acc, marker="o", drawstyle="steps-post", label="train")
ax[1].plot(ccp_alphas, test_acc, marker="o", drawstyle="steps-post", label="test")

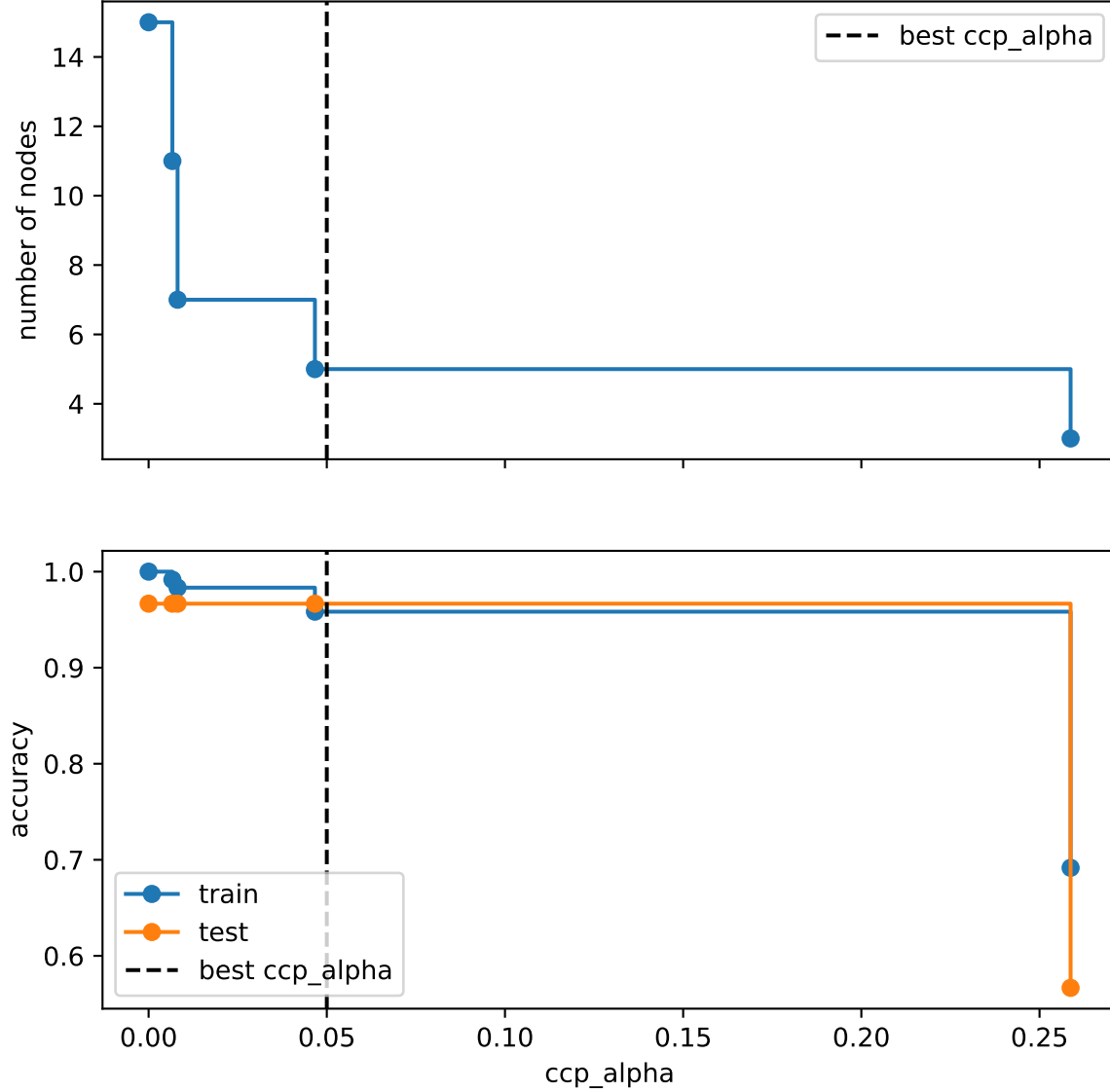
ax[0].axvline(best_alpha, color="black", linestyle="--", label="best ccp_alpha")
ax[1].axvline(best_alpha, color="black", linestyle="--", label="best ccp_alpha")

ax[1].set_xlabel("ccp_alpha")
ax[0].set_ylabel("number of nodes")
ax[1].set_ylabel("accuracy")

ax[0].legend()
ax[1].legend(loc="lower left")

```

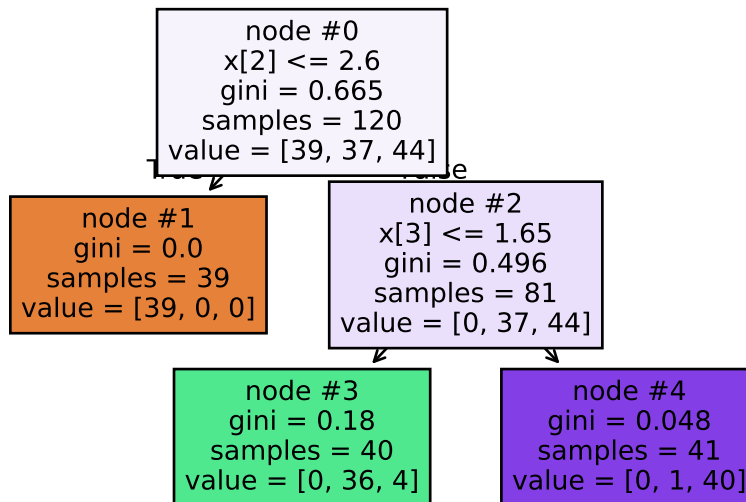
- ① For pruning-related plots, `drawstyle="steps-post"` is important because the tree structure stays the same within each interval of `ccp_alpha` values. The tree only changes after reaching the next pruning threshold. Each critical `ccp_alpha` corresponds to pruning one or more weak internal nodes, so the pruning path is piecewise constant.



Based on the plot, from $0.0 \leq \text{ccp_alpha} < 0.04666666666666667$, the test accuracy stay the same. Therefore we choose the model with the least number of nodes. So we could choose any `ccp_alpha` from $0.0 \leq \text{ccp_alpha} < 0.04666666666666667$. Therefore we would like to choose the one with the least number of nodes, which is 0.008130081300813012. Since all values in the range is the same, we hand pick `ccp_alpha=0.05` for simplicity.

We can now build the final tree and display it.

```
clf = DecisionTreeClassifier(random_state=42, ccp_alpha=best_alpha)
clf.fit(X_train, y_train)
plot_tree(clf, filled=True, impurity=True, node_ids=True);
```



i Note

Although the hyperparameters with the highest validation score are often preferred, model complexity should also be taken into consideration. This is why the number of nodes is displayed.

In this `iris` example, the dataset is very small and the differences between validation scores are negligible. In such situations, a slightly simpler tree is often preferred because it is more interpretable and potentially more stable while achieving similar predictive performance.

In larger datasets, small differences in validation scores are often more meaningful, so predictive performance may be prioritized more heavily.

9.3.2 Example 2: Regression with the diabetes dataset

We now look at a regression example. The `diabetes` dataset uses 10 baseline variables to predict a quantitative measure of disease progression one year after baseline. More information can be found in the `sklearn` documentation and the original diabetes dataset [description](#).


```

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

```

The overall workflow is similar to the `iris` example. In this example, we choose to use cross-validation instead of a single validation split in order to obtain a more stable estimate of model performance. This is not specific to regression trees; either approach can be used for both classification and regression problems.

We first create a temporary tree to compute the critical `ccp_alpha` values. The method `cost_complexity_pruning_path()` internally grows a tree in order to compute the effective pruning path, although the estimator itself is not stored as a fitted model.

```

from sklearn.tree import DecisionTreeRegressor

base_tree = DecisionTreeRegressor(random_state=1)
path = base_tree.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas[:-1]

```

This follows the classical CART pruning strategy:

1. Grow a large tree.
2. Compute the pruning path.
3. Use validation or cross-validation to select the final subtree.

Now we run `GridSearchCV` over these `ccp_alpha` values. We use R^2 as the model metric in this example.

```

from sklearn.model_selection import GridSearchCV
from sklearn.metrics import r2_score

params = param_grid = {"ccp_alpha": ccp_alphas}
grid = GridSearchCV(
    DecisionTreeRegressor(random_state=1), param_grid=params, scoring="r2"
)

grid.fit(X_train, y_train)

best_tree = grid.best_estimator_

```

```

y_pred_train = best_tree.predict(X_train)
y_pred_test = best_tree.predict(X_test)

print(f"Best ccp_alpha: {grid.best_params_['ccp_alpha']}")
print(f"Training R^2: {r2_score(y_train, y_pred_train)}")
print(f"Testing R^2: {r2_score(y_test, y_pred_test)}")

```

```

Best ccp_alpha: 105.18472145590351
Training R^2: 0.5347863093947497
Testing R^2: 0.1922738375305083

```

We can visualize the cross-validation curve. `GridSearchCV` records validation scores, but it does not automatically record tree complexity measures such as the number of nodes. Therefore, if we want to display the complexity curve, we need to refit the models for the same `ccp_alpha` values and record their node counts.

```

cv_score = grid.cv_results_["mean_test_score"]

node_counts = []
for ccp_alpha in ccp_alphas:
    tree = DecisionTreeRegressor(random_state=1, ccp_alpha=ccp_alpha)
    tree.fit(X_train, y_train)
    node_counts.append(tree.tree_.node_count)

best_alpha = grid.best_params_["ccp_alpha"]

fig, ax = plt.subplots(2, 1, figsize=(7, 7), sharex=True)

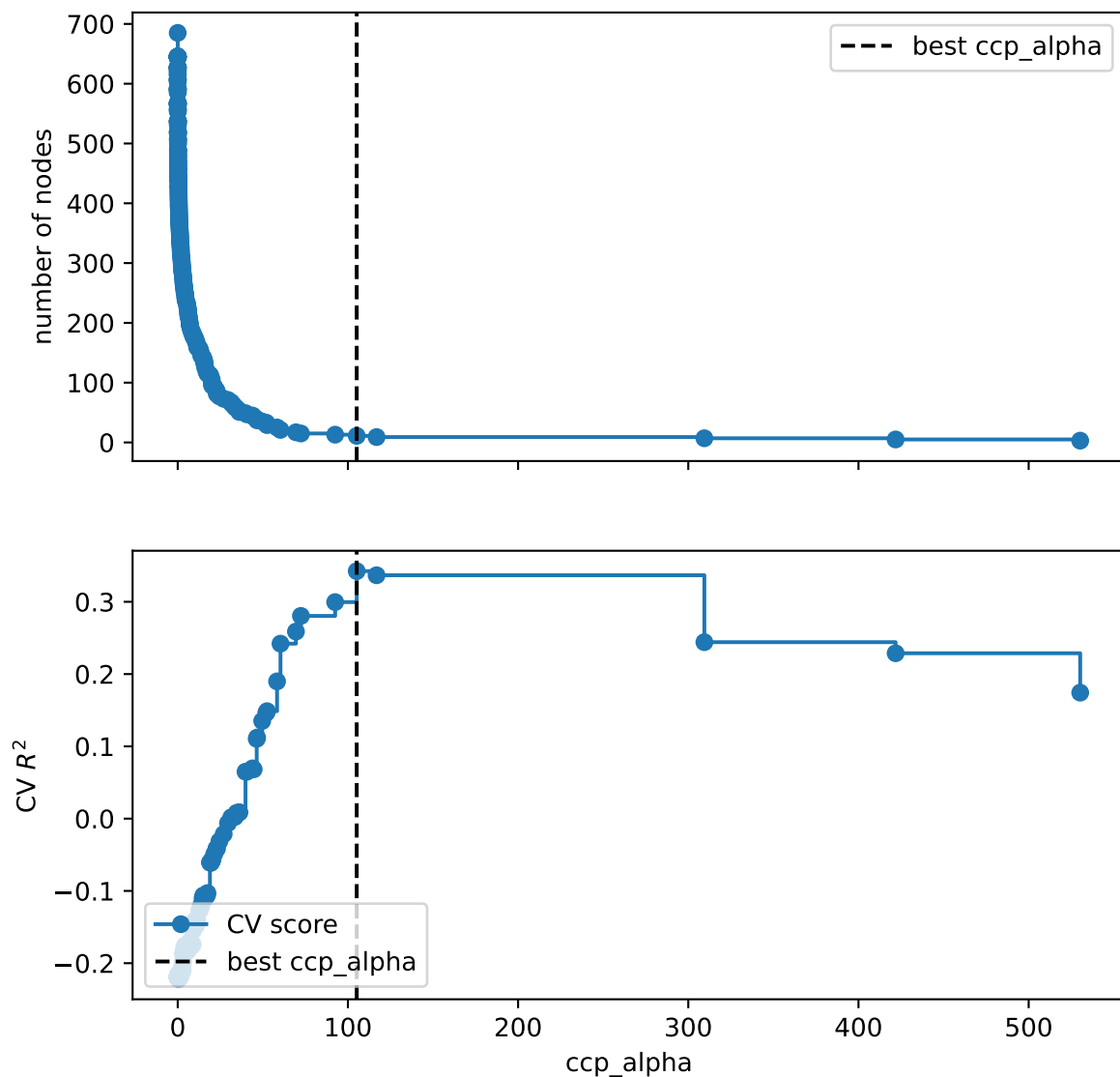
ax[0].plot(ccp_alphas, node_counts, marker="o", drawstyle="steps-post")
ax[1].plot(ccp_alphas, cv_score, marker="o", drawstyle="steps-post", label="CV score")

ax[0].axvline(best_alpha, color="black", linestyle="--", label="best ccp_alpha")
ax[1].axvline(best_alpha, color="black", linestyle="--", label="best ccp_alpha")

ax[1].set_xlabel("ccp_alpha")
ax[0].set_ylabel("number of nodes")
ax[1].set_ylabel("CV $R^2$")

ax[0].legend()
ax[1].legend(loc="lower left")

```



From the plot, our selected `ccp_alpha=np.float64(105.18472145590351)` is reasonable since it gives a relative strong cross-validation score without producing an unnecessarily large tree.

⚠ Warning

A single regression tree often has unstable predictive performance on moderate-sized noisy datasets. Therefore, the test R^2 may not look high in this example. This is one reason why ensemble methods such as random forests and boosting are often preferred in practice.

10 Bagging and Random Forests

In earlier chapters we studied single decision trees. A tree is flexible because it can capture nonlinear relationships and interactions without specifying them in advance. However, a single tree is also unstable. A small change in the training data may produce a very different tree, especially when the tree is grown deeply.

Bagging and random forests are ensemble methods designed to reduce this instability.

The central idea is:

Instead of trusting one unstable tree, build many unstable trees and average them.

This chapter first discusses bagging. Random forests will then be introduced as an improvement of bagging. Since bagging and random forests use almost the same aggregation idea for regression and classification, we will mainly use regression notation. For classification, averages are usually replaced by majority vote or averaged class probabilities.

10.1 Ensemble Averaging

Suppose we observe training data

$$\{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^p, \quad y_i \in \mathbb{R}.$$

An ensemble combines many fitted models

$$\hat{f}_1, \hat{f}_2, \dots, \hat{f}_B$$

into one final predictor. For regression, the simplest ensemble average is

$$\hat{f}_{\text{ens}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

Each individual model may be noisy. However, the average can be much more stable.

To see why, suppose each $\hat{f}_b(x)$ has variance σ^2 , and suppose for a moment that the fitted models are independent. Then

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x) \right) = \frac{\sigma^2}{B}.$$

Thus averaging can greatly reduce variance.

In practice, the fitted models are not independent. If the pairwise correlation between two fitted models is approximately ρ , then

Click to expand.

$$\text{Corr}(\hat{f}_i(x), \hat{f}_j(x)) \approx \rho, \quad i \neq j.$$

Then

$$\text{Cov}(\hat{f}_i(x), \hat{f}_j(x)) = \text{Corr}(\hat{f}_i(x), \hat{f}_j(x)) \sqrt{\text{Var}(\hat{f}_i(x)) \text{Var}(\hat{f}_j(x))} \approx \rho \sigma^2.$$

Therefore,

$$\begin{aligned} \text{Var} \left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x) \right) &= \frac{1}{B^2} \left[\sum_{b=1}^B \text{Var}(\hat{f}_b(x)) + \sum_{i \neq j} \text{Cov}(\hat{f}_i(x), \hat{f}_j(x)) \right] \\ &\approx \frac{1}{B^2} [B\sigma^2 + B(B-1)\rho\sigma^2] \\ &= \sigma^2 \left[\frac{1}{B} + \frac{B-1}{B}\rho \right] \\ &= \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \end{aligned}$$

Then

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x) \right) \approx \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

This formula is very important. Increasing B reduces the second term, but the first term remains. Therefore, if the individual trees are highly correlated, averaging has limited benefit.

10.2 Bagging

Bagging is short for bootstrap aggregation.

Assume the original training set has n data. A bootstrap sample is a sample drawn from the original training data with replacement. Each bootstrap sample has size n , but because sampling is done with replacement, some observations appear more than once and some observations do not appear at all.

Let $Z = \{(x_1, y_1), \dots, (x_n, y_n)\}$ be the original training set. Bagging constructs bootstrap samples $Z^{*1}, Z^{*2}, \dots, Z^{*B}$. For each bootstrap sample Z^{*b} , we fit a model $\hat{f}_b(x)$. The bagged regression estimate is then

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

For classification, the final prediction is usually based on majority vote:

$$\hat{G}_{\text{bag}}(x) = \text{mode}\{\hat{G}_1(x), \dots, \hat{G}_B(x)\}.$$

Equivalently, we may average estimated class probabilities and choose the class with the largest average probability.

Bias-variance view

Bagging is useful when the base learner is unstable. A decision tree is a typical unstable learner. Different bootstrap samples lead to different trees, because each tree sees a slightly different version of the data.

For a decision tree, bagging usually uses large trees rather than heavily pruned trees. The reason is that a large tree has low bias but high variance. Bagging is designed to reduce variance, so it is natural to start with a flexible base learner.

A single deep tree usually has low bias and high variance. Bagging keeps the low bias and reduces the variance by averaging many deep trees.

Bagging estimators

Although bagging is most commonly applied to decision trees, it can also be used with other types of models.

10.2.1 Out-of-Bag Error

Although a bootstrap sample has size n , it does not contain all n observations.

Definition 10.1 (Out-of-bag observations). Each bootstrap sample contains about 63.2% of the original observations as distinct observations. The remaining observations are called **out-of-bag** (OOB) observations for that bootstrap sample.

Click to expand.

For a fixed observation i , the probability that it is not selected in one draw is

$$1 - \frac{1}{n}.$$

The probability that it is never selected in n draws is

$$\left(1 - \frac{1}{n}\right)^n.$$

When n is large,

$$\left(1 - \frac{1}{n}\right)^n \rightarrow e^{-1} \approx 0.368.$$

Therefore, the probability that observation i appears at least once is approximately

$$1 - e^{-1} \approx 0.632,$$

and it is the expected proportion of distinct observations in a sample.

The OOB observations give a natural way to estimate prediction error.

For observation i , consider only the trees for which observation i was out-of-bag. Average those trees to get an OOB prediction:

$$\hat{f}_{\text{OOB},i} = \frac{1}{|B_i|} \sum_{b \in B_i} \hat{f}_b(x_i),$$

where B_i is the set of trees that did not use observation i in their bootstrap sample.

The OOB mean squared error is

$$\text{MSE}_{\text{OOB}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}_{\text{OOB},i})^2.$$

All other metrics have OOB versions in the straightforward way.

OOB error is useful because it behaves like an internal validation estimate. We do not need to set aside a separate validation set just to estimate the error of the bagged model. Therefore, for bagging, in addition to single-split validation and cross-validation, we can also use OOB validation to tune hyperparameters.

10.3 Random Forests

Random forests are a modification of bagging for decision trees. In addition to the diversity by sampling observations from Bagging, Random forests create additional diversity by sampling features during tree construction.

The random forest algorithm for regression is:

1. Draw a bootstrap sample from the training data.
2. Grow a large regression tree on this bootstrap sample.
3. **At each split, randomly choose m predictors from the full set of p predictors.**
4. **Find the best split only among those m predictors.**
5. Repeat this process for B trees.
6. Average the predictions.

The final random forest regressor is

$$\hat{f}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

The formula looks the same as bagging. The difference is how the trees are grown.

The key difference from Bagging(Trees) is the Step 3 and 4. In ordinary bagging, all predictors are available at every split. If there is one very strong predictor, many trees may split on that same predictor near the top of the tree. As a result, the trees become similar.

Similar trees are highly correlated. From the variance formula,

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2,$$

we know that high correlation limits the benefit of averaging.

Now random forests reduce this issue by forcing each split to consider only a random subset of predictors. This prevents the strongest predictors from dominating every tree. The trees become less correlated, and averaging becomes more effective.

10.3.1 `max_features`

How many predictors are considered at each split? In `sklearn`, this is controlled by `max_features`. This is a very important hyperparameter because it affects the correlations between trees.

Large `max_features` means each tree can choose from many predictors. Individual trees may become stronger, but the trees may also become more similar to each other.

Small `max_features` forces trees to explore different predictors. This usually decreases the correlations between trees, but each individual tree may become weaker.

Thus `max_features` controls an important tradeoff between tree strength and tree correlation.

Common traditional choices are:

- for classification, `max_features` = $\sqrt{p}$
- for regression, `max_features` = $p/3$

These recommendations originated from the original Random Forest literature [7] and later became popular defaults in many software implementations [3]. They are mainly based on empirical experience and the general intuition that:

- classification forests often benefit more from tree diversity
- regression forests often benefit more from stronger individual trees

In modern practice, however, `max_features` is usually treated as a tuning hyperparameter and is commonly selected using validation or cross-validation.

10.3.2 Random Forest as Adaptive Local Averaging

A random forest can also be viewed as a data-adaptive local averaging method.

For one tree, two points are close if they fall in the same terminal node. Let

$$I_b(x, x_i) = \begin{cases} 1, & \text{if } x \text{ and } x_i \text{ are in the same leaf of tree } b, \\ 0, & \text{otherwise.} \end{cases}$$

Across the whole forest, define the similarity

$$K_{\text{RF}}(x, x_i) = \frac{1}{B} \sum_{b=1}^B I_b(x, x_i).$$

This measures the proportion of trees in which x and x_i fall into the same terminal node. The quantity $K_{\text{RF}}(x, x_i)$ can be interpreted as a data-adaptive similarity measure between x and x_i .

The random forest prediction can be written as a weighted average

$$\hat{f}_{\text{RF}}(x) = \sum_{i=1}^n w_i(x) y_i,$$

where observations that frequently share terminal nodes with x receive larger weights.

More details about the adaptive nearest-neighbor interpretation of random forests can be found in [8] and [3].

Note

The adaptive-neighbor interpretation helps explain why random forests often perform much better than classical distance-based methods such as KNN in complex datasets. In KNN, neighborhoods are determined by a fixed geometric distance. In contrast, random forests learn the neighborhood structure directly from the data through recursive splitting. Two observations are considered close if the forest repeatedly places them into the same terminal nodes, so the similarity automatically adapts to the importance of variables and their relationships with the response variable.

As a result, the neighborhood automatically adapts to:

- important variables
- interaction effects
- nonlinear structures
- local heterogeneity of the feature space

This viewpoint also provides intuition for why random forests are often robust in high-dimensional settings. Instead of relying on a global distance metric, the forest constructs many local data-dependent neighborhoods and averages over them.

From a theoretical perspective, this interpretation connects random forests to classical nonparametric smoothing methods such as KNN and kernel regression, helping explain random forests using the broader language of statistical learning theory.

10.3.3 Feature Importance

There are two commonly used measures of feature importance. We briefly discuss them here. More detailed examples and implementations can be found in the [scikit-learn documentation](#).

10.3.3.1 Mean Decrease in Impurity

One common measure is impurity-based importance, also called Mean Decrease in Impurity (MDI). This importance measure is specific to tree-based methods because it depends directly on the tree splitting process.

Recall that when building a tree, each split is selected to reduce a splitting impurity (or loss), such as:

- squared error for regression
- Gini impurity for classification

Impurity-based importance measures how much a variable decreases the splitting impurity across all splits in all trees. Variables that frequently produce large impurity reductions receive larger importance values.

For a single tree, the feature importance is simply the total impurity reduction contributed by that variable across all splits in the tree. However, because a single decision tree is often unstable and has high variance, these importance values may vary substantially from one tree to another. In bagging and Random Forests, the importance values are averaged across many trees, producing a much more stable and reliable measure of variable importance.

10.3.3.2 Permutation importance

Another common measure is permutation importance. Unlike impurity-based importance, permutation importance is a general model-agnostic method and is not specific to trees or Random Forests.

The basic idea is:

1. Measure prediction error on validation data.
2. Randomly permute one predictor.
3. Measure prediction error again.

Permuting a predictor destroys the relationship between that predictor and the response while keeping the marginal distribution unchanged. If permuting a predictor substantially worsens predictive performance, then the model strongly relies on that predictor, indicating high importance.

Permutation importance is often easier to interpret because it is directly tied to predictive performance rather than the internal structure of the model.

MDI vs Permutation Importance

The two methods answer different questions:

- MDI explains how much the model used a variable during training.
- Permutation importance measures how much the model depends on that variable for prediction.

In general, if the main goal is to understand predictive performance and generalization, permutation importance is usually more relevant. However, MDI is much faster to compute because it is obtained directly during training. This advantage becomes especially important when the dataset contains many features or when the model is very large.

In many practical situations, MDI and permutation importance may produce similar rankings of variables. In addition, MDI can also help us investigate the internal structure of the model by showing how frequently and how effectively variables are used for splitting.

Limitations of MDI

Continuous predictors, predictors with many possible split points, or high-cardinality categorical variables may have more opportunities to create impurity reductions during splitting. Therefore, MDI may favor these variables.

Limitations of Permutation Importance

When there are redundant features or highly correlated predictors, permuting one variable may not substantially reduce predictive performance because other correlated variables can still provide similar information. As a result, permutation importance may underestimate the importance of correlated predictors.

10.4 Different Types of Randomization in Tree Ensembles

One of the main ideas behind ensemble tree methods is to introduce randomness in order to reduce variance. However, there are many different ways to introduce randomness. Different methods randomize different parts of the tree-building process.

The general tradeoff is:

- more randomness usually decreases variance
- but too much randomness may increase bias

The goal is therefore to make trees different while still keeping each tree reasonably strong.

10.4.1 (Almost) No randomness: Single Decision Tree

A standard CART tree contains almost no randomness.

At each node:

1. all predictors are considered
2. all candidate split points are searched
3. the best split is selected greedily

As a result:

- trees are highly adaptive
- trees usually have low bias
- but trees often have very high variance

Small changes in the training data may completely change the structure of the tree.

In reality, there is still a small amount of randomness in CART. For example, when multiple features produce identical impurity reductions, the algorithm may randomly choose among them.

In practice, in `sklearn`, this is implemented partly by randomizing the order of features. The `random_state` parameter controls this randomness.

10.4.2 Bootstrap Randomness: Bagging

Bagging introduces randomness through bootstrap resampling.

For each tree:

1. sample observations with replacement
2. grow a large tree using the bootstrap sample
3. aggregate all trees together

The splitting procedure itself is unchanged:

- all predictors are still considered
- the best split is still searched greedily

Therefore, bagging mainly creates diversity by changing the training data seen by each tree.

The individual trees are still strong learners, but averaging reduces variance.

10.4.3 Feature Randomness: Random Forest

Random Forest adds another layer of randomness. At each split:

1. randomly select a subset of predictors
2. search for the best split only among those predictors

This has several important effects:

- strong predictors cannot dominate every split
- trees become less correlated
- averaging becomes more effective

However, the split threshold is still optimized greedily. For a chosen predictor, Random Forest still searches all candidate split points.

A natural question is why not randomly choose a subset of predictors once for the entire tree. The reason is that this would usually create trees that are too weak.

Suppose an important predictor is excluded at the beginning. Then the entire tree loses access to that predictor forever.

Random Forest instead performs randomization locally:

- each node receives a new random subset of predictors
- important predictors still have many opportunities to appear
- trees remain reasonably strong

This creates a balance between tree strength and tree diversity.

10.4.4 Threshold Randomness: Extra Trees

Extra Trees (Extremely Randomized Trees) introduces even more randomness. At each split:

1. randomly choose a subset of predictors (similar to Random Forest)
2. randomly generate candidate split thresholds, typically one threshold for each selected predictor
3. select the best split among those randomly generated thresholds

Unlike Random Forest:

- the algorithm does not exhaustively search all candidate split points
- the split thresholds themselves are randomized

This additional randomness usually:

- further decreases correlation between trees

- further reduces variance
- slightly increases bias

10.5 Bagging in Python

We now discuss implementation details in `sklearn`.

There are two main approaches.

1. Use `BaggingRegressor` or `BaggingClassifier` with a chosen base estimator.
2. Use `RandomForestRegressor` or `RandomForestClassifier` when the base estimator is a decision tree and we also want random feature selection.

In the initial demo, we will only talk about Classifier since it is very similar to Regressor. We will use the following generated dataset as the demo example.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(
    n_samples=1200,
    n_features=10,
    n_informative=3,
    n_redundant=2,
    n_repeated=0,
    n_clusters_per_class=3,
    flip_y=0.03,
    random_state=1,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1, stratify=y
)
```

10.5.1 Basic Bagging classifier

The following code creates a bagged ensemble of regression trees.

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag = BaggingClassifier(
```

```

    estimator=DecisionTreeClassifier(random_state=0),
    n_estimators=300,
    max_samples=1.0,
    bootstrap=True,
    oob_score=True,
    random_state=0,
)

```

The important arguments are:

- **estimator**: the base model. For classical tree bagging, this is a decision tree.
- **n_estimators**: the number of bootstrap models.
- **max_samples**: the sample size used for each bootstrap sample.
- **bootstrap**: whether sampling is done with replacement. **True** gives bagging; **False** gives pasting.
- **oob_score**: whether to compute out-of-bag performance.

After creating the model, we use the same `.fit()` and `.predict()` and `.score()` workflow as before.

```

bag.fit(X_train, y_train)
bag.score(X_test, y_test)

```

```
0.9138888888888889
```

10.5.2 Random Forest Classifier

A random forest classifier is similar, but the random feature selection is built in.

```

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=300,
    max_features=1.0,
    bootstrap=True,
    oob_score=True,
    random_state=0
)

rf.fit(X_train, y_train)
rf.score(X_test, y_test)

```


0.9138888888888889

The most important additional argument is `max_features`.

For example:

```
RandomForestClassifier(max_features=1.0)
RandomForestClassifier(max_features="sqrt")
RandomForestClassifier(max_features=0.5)
```

- 1.0: consider all predictors at each split. In this case, it is reduced to a `BaggingRegressor`.
- "sqrt": consider about $\sqrt{p}$ predictors at each split.
- 0.5: consider half of the predictors at each split.

When `max_features=1.0`, a random forest regressor is very close to bagging with decision trees. When `max_features` is smaller, the trees become more decorrelated.

10.5.3 Useful Attributes

After fitting a bagging or random forest model, several attributes are useful. This example shows random forest, but bagging also supports these attributes.

- `.estimators_` stores the fitted trees, and we could indicate which tree we want to use by calling the index.

```
rf.estimators_[0]
```

```
criterion criterion: {"gini",      'gini'
"entropy", "log_loss"},
default="gini"
```

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see :ref:'tree_mathematical_formulation'.

splitter splitter: {"best",
"random"}, default="best" 'best'

The strategy used to choose
the split at each node.

Supported

strategies are "best" to
choose the best split and
"random" to choose
the best random split.

max_depth max_depth: int, None
default=None

The maximum depth of the
tree. If None, then nodes are
expanded until

all leaves are pure or until all
leaves contain less than
min_samples_split samples.

min_samples_split 2

min_samples_split: int or
float, default=2

The minimum number of
samples required to split an
internal node:

- If int, then consider

'min_samples_split' as the
minimum number.

- If float, then

'min_samples_split' is a
fraction and

'ceil(min_samples_split *
n_samples)' are the
minimum

number of samples for each
split.

.. versionchanged:: 0.18

Added float values for
fractions.

`min_samples_leaf`
`min_samples_leaf`: int or
float, default=1

1

The minimum number of
samples required to be at a
leaf node.

A split point at any depth
will only be considered if it
leaves at
least “`min_samples_leaf`”
training samples in each of
the left and
right branches. This may
have the effect of smoothing
the model,
especially in regression.

- If int, then consider
‘`min_samples_leaf`’ as the
minimum number.
- If float, then
‘`min_samples_leaf`’ is a
fraction and
‘`ceil(min_samples_leaf *
n_samples)`’ are the
minimum
number of samples for each
node.

.. versionchanged:: 0.18
Added float values for
fractions.

<code>min_weight_fraction_leaf</code>	<code>0.0</code>
---------------------------------------	------------------

`min_weight_fraction_leaf:`
float, default=0.0

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

`max_features` `max_features:` 1.0
int, float or {"sqrt", "log2"},
default=None

The number of features to consider when looking for the best split:

- If int, then consider 'max_features' features at each split.
- If float, then 'max_features' is a fraction and 'max(1, int(max_features * n_features_in_))' features are considered at each split.
- If "sqrt", then 'max_features=sqrt(n_features)'.
- If "log2", then 'max_features=log2(n_features)'.
- If None, then 'max_features=n_features'.

.. note::

The search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than "max_features" features.

`random_state` `random_state`: 209652396
int, RandomState instance or
None, default=None

Controls the randomness of the estimator. The features are always randomly permuted at each split, even if “splitter” is set to “best”. When “max_features < n_features”, the algorithm will select “max_features” at random at each split before finding the best split among them. But the best found split may vary across different runs, even if “max_features=n_features”. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, “random_state” has to be fixed to an integer. See :term:‘Glossary ‘ for details.

<code>max_leaf_nodes</code> <code>max_leaf_nodes: int,</code> <code>default=None</code>	None
---	------

Grow a tree with
“`max_leaf_nodes`” in
best-first fashion.
Best nodes are defined as
relative reduction in impurity.
If None then unlimited
number of leaf nodes.

min_impurity_decrease 0.0
min_impurity_decrease:
float, default=0.0

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following::

$$N_t / N * (\text{impurity} - N_{t_R} / N_t * \text{right_impurity} - N_{t_L} / N_t * \text{left_impurity})$$

where “N” is the total number of samples, “N_t” is the number of samples at the current node, “N_t_L” is the number of samples in the left child, and “N_t_R” is the number of samples in the right child.

“N”, “N_t”, “N_t_R” and “N_t_L” all refer to the weighted sum, if “sample_weight” is passed.

.. versionadded:: 0.19

`class_weight` `class_weight`: None
dict, list of dict or
"balanced", default=None

Weights associated with
classes in the form
"{class_label: weight}".
If None, all classes are
supposed to have weight one.
For
multi-output problems, a list
of dicts can be provided in
the same
order as the columns of y.

Note that for multioutput
(including multilabel) weights
should be
defined for each class of every
column in its own dict. For
example,
for four-class multilabel
classification weights should
be
[{0: 1, 1: 1}, {0: 1, 1: 5}, {0:
1, 1: 1}, {0: 1, 1: 1}] instead
of
[{1:1}, {2:5}, {3:1}, {4:1}].

The "balanced" mode uses
the values of y to
automatically adjust
weights inversely proportional
to class frequencies in the
input data
as " $n_{\text{samples}} / (n_{\text{classes}} * \text{np.bincount}(y))$ "

For multi-output, the weights
of each column of y will be
multiplied.

Note that these weights will
be multiplied with
`sample_weight` (passed
through the fit method) if
`sample_weight` is specified.

`ccp_alpha` `ccp_alpha`: 0.0
non-negative float,
default=0.0

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than “`ccp_alpha`” will be chosen. By default, no pruning is performed. See :ref:‘minimal_cost_complexity_pruning’ for details. See :ref:‘sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py’ for an example of such pruning.

.. versionadded:: 0.22

<code>monotonic_cst</code> <code>monotonic_cst</code>: array-like of int of shape (n_features), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - -1: monotonic decrease If <code>monotonic_cst</code> is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when 'n_classes > 2'), - multioutput classifications (i.e. when 'n_outputs > 1'), - classifications trained on data with missing values. The constraints hold over the probability of the positive class. Read more in the :ref:'User Guide '. .. versionadded:: 1.4	None
---	------

- `.oob_score_` gives the OOB score if `oob_score=True`. For classification, `bag.oob_score_` is the OOB accuracy. For regression, it is the OOB R^2 score.

```
rf.oob_score_
```

```
0.888095238095238
```

10.5.3.1 Feature importance

MDI can be directly computed using `.feature_importances_` after the model is fitted.

```
rf.feature_importances_
```

```
array([0.02767912, 0.05947287, 0.11152095, 0.02443874, 0.02509798,  
       0.25352125, 0.02526593, 0.03385812, 0.26526717, 0.17387786])
```

For permutation importance, we use `permutation_importance` from `sklearn.inspection`.

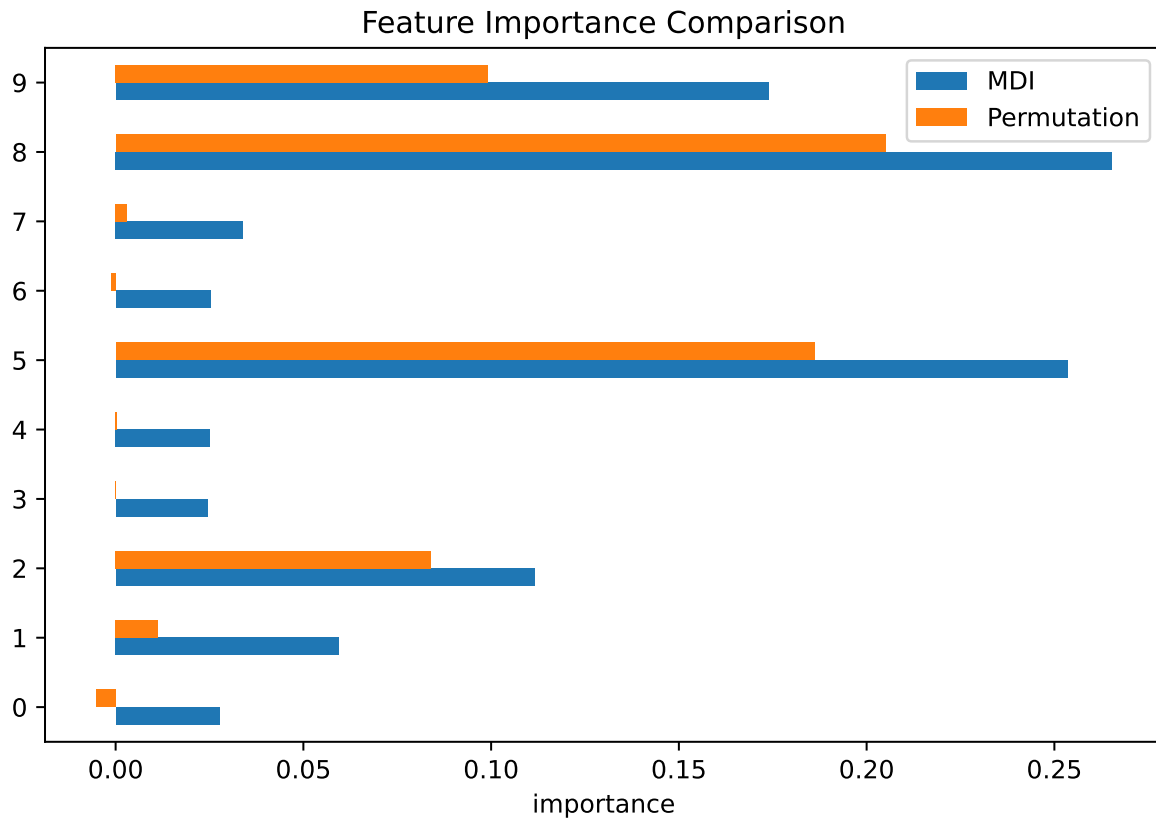
```
from sklearn.inspection import permutation_importance  
  
pi = permutation_importance(rf, X_test, y_test, n_repeats=30, random_state=1)  
  
pi.importances_mean
```

```
array([-5.27777778e-03,  1.12037037e-02,  8.38888889e-02,  9.25925926e-05,  
       4.62962963e-04,  1.86111111e-01, -1.20370370e-03,  2.96296296e-03,  
       2.05000000e-01,  9.92592593e-02])
```

In this example you may see that the two are very similar to each other.

```
import pandas as pd  
import matplotlib.pyplot as plt  
  
importance_df = pd.DataFrame(  
    {"MDI": rf.feature_importances_, "Permutation": pi.importances_mean}  
)  
importance_df.plot.barh(figsize=(8, 5))  
  
plt.xlabel("importance")  
plt.title("Feature Importance Comparison")
```

```
Text(0.5, 1.0, 'Feature Importance Comparison')
```



10.6 Tuning Hyperparameters

Bagging and random forests are usually easier to tune than boosting. Still, several parameters matter.

Parameter	Main effect
<code>n_estimators</code>	More trees reduce Monte Carlo variation, but increase computation.
<code>max_features</code>	Controls tree correlation. Smaller values create more randomness.
<code>max_samples</code>	Controls bootstrap sample size. Smaller values create more diversity.
<code>min_samples_leaf</code>	Controls smoothness of each tree. Larger values reduce overfitting.
<code>max_depth</code>	Limits tree depth. Often left as <code>None</code> , but can help noisy data.

A practical tuning strategy is:

1. Choose a reasonably large `n_estimators` (so it is highly possible to be stable).
2. Tune `max_features`, `min_samples_leaf`, `max_samples` or `max_depth` with cross-validation or OOB error.
3. Make `n_estimators` smaller to find the smallest stable number.

`n_estimators`

For bagging / random forests, `n_estimators` is usually not a regularization parameter in the usual sense. Increasing the number of trees usually stabilizes the model rather than causing overfitting. The main cost is computation. Therefore when reading the score vs number of trees plot, we are not looking at the highest score, we are looking at when the score is stabilizing.

1 and 1.0

These two represent different things.

- `max_samples=1` means that the max number of samples we choose is 1.
- `max_samples=1.0` means that we want to choose all samples that the percent is 1.0=100%.

Note

Comparing to single tree/boosting, we tend to use larger trees in bagging. Therefore we don't need `ccp_alpha`, and we could use larger `max_depth` and smaller `min_samples_leaf`.

OOB validation

We could use oob score to tune hyperparameters. However we have to manually write the tuning code.

```

from sklearn.ensemble import RandomForestClassifier

max_features_list = [0.3, 0.5, 0.8, 1.0]
best_score = -1
best_param = None

for mf in max_features_list:
    rf = RandomForestClassifier(
        n_estimators=300, max_features=mf, oob_score=True, random_state=0, n_jobs=-1
    )
    rf.fit(X_train, y_train)
    if rf.oob_score_ > best_score:
        best_score = rf.oob_score_
        best_param = mf

print(best_param)

```

0.3

i Cross-validation

We could still use cross-validation to tune a random forest classifier.

```

from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {
    "max_features": [0.3, 0.6, 1.0],
    "min_samples_leaf": [1, 3, 5],
    "max_samples": [0.6, 0.8, 1.0],
}

rf = RandomForestClassifier(n_estimators=300, random_state=0)

grid = GridSearchCV(rf, param_grid=param_grid, cv=5, n_jobs=-1)

grid.fit(X_train, y_train)

best_rf = grid.best_estimator_
grid.best_params_

```

```
{'max_features': 0.3, 'max_samples': 0.8, 'min_samples_leaf': 1}
```

i Randomized Search

If the dataset is large, a full grid can be slow. In that case, `RandomizedSearchCV` is often a better choice. In this case, only a few random combinations will be searched. In the following example, we only search 30 combinations.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

param_dist = {
    "max_features": uniform(0.2, 0.8),
    "min_samples_leaf": randint(1, 20),
    "max_samples": uniform(0.5, 0.5),
}

rf = RandomForestClassifier(n_estimators=300, random_state=0)

search = RandomizedSearchCV(
    rf, param_distributions=param_dist, n_iter=30, cv=5, random_state=0, n_jobs=-1
)

search.fit(X_train, y_train)
search.best_params_

{'max_features': np.float64(0.9220787804235238),
 'max_samples': np.float64(0.7249749949556138),
 'min_samples_leaf': 2}
```

10.6.1 `n_jobs`

When we reach this stage, you may notice that training can become quite slow. This is expected because, in `GridSearchCV`, we need to evaluate many combinations of hyperparameters. For each combination, we may need to train many trees (due to bagging or Random Forests), and the entire process must be repeated multiple times because of cross-validation (for example, `cv=5`).

Although many models need to be trained, an important advantage is that these training tasks are largely independent. Therefore, they can often be trained simultaneously using multiple CPU cores.

In contrast, boosting methods that we will discuss later are fundamentally sequential. Each tree depends on the results of the previous trees, so the trees must be trained one after another. As a result, boosting methods are generally less parallelizable at the tree level.

`sklearn` supports multi-core CPUs through the `n_jobs` argument.

- `n_jobs=1` means single-core execution
- `n_jobs=k` uses exactly `k` CPU cores
- `n_jobs=-1` uses all available CPU cores
- `n_jobs=None` uses the library default, which is currently equivalent to 1 in most `sklearn` functions

Many functions in `sklearn` support this argument. For details, consult the official documentation for the specific function being used.

One important practical issue is nested parallelism. When multiple nested functions both support `n_jobs`, it is usually best to parallelize only the outermost function.

For example, `GridSearchCV(RandomForestClassifier(n_jobs=1), n_jobs=-1)` is generally preferred over `GridSearchCV(RandomForestClassifier(n_jobs=-1), n_jobs=-1)` because allowing both levels to parallelize simultaneously may create too many worker processes or threads, leading to excessive overhead and sometimes even slower performance.

Example 10.1 (time it). We first write a helper function to compute the runtime.

```
from time import perf_counter
from contextlib import contextmanager

@contextmanager
def timer(name="runtime"):
    start = perf_counter()
    yield
    end = perf_counter()

    print(f"{name}: {end - start:.4f} seconds")
```

Then use it to test the time for different `n_jobs`.

```
param_grid = {
    "max_features": [0.3, 0.6, 1.0],
    "min_samples_leaf": [1, 3, 5],
    "max_samples": [0.6, 0.8, 1.0],
}
```

```

rf = RandomForestClassifier(n_estimators=300, random_state=0)

grid = GridSearchCV(rf, param_grid=param_grid, cv=5, n_jobs=-1)
with timer("n_jobs=-1"):
    grid.fit(X_train, y_train)

grid = GridSearchCV(rf, param_grid=param_grid, cv=5, n_jobs=1)
with timer("n_jobs=1"):
    grid.fit(X_train, y_train)

```

n_jobs=-1: 20.3990 seconds
n_jobs=1: 73.1687 seconds

10.7 Example: Diabetes dataset

We now use the diabetes regression dataset from `sklearn`. In order to compare results using trees, we use the same split with `random_state=1`.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

X, y = load_diabetes(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

```

We compare three models:

1. a single regression tree,
2. bagging with regression trees,
3. a random forest.

```

from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import BaggingRegressor, RandomForestRegressor

tree = DecisionTreeRegressor(random_state=1, ccp_alpha=105.185)

```

```

bag = BaggingRegressor(
    estimator=DecisionTreeRegressor(random_state=1),
    n_estimators=300,
    oob_score=True,
    random_state=1,
)

rf = RandomForestRegressor(
    n_estimators=300,
    max_features="sqrt",
    oob_score=True,
    random_state=1,
)

```

```

tree.fit(X_train, y_train)
tree.score(X_test, y_test)

```

0.1922738375305083

```

bag.fit(X_train, y_train)
bag.score(X_test, y_test)

```

0.2951908085625511

```

rf.fit(X_train, y_train)
rf.score(X_test, y_test)

```

0.3555401846697034

The single tree is expected to be unstable. Bagging and random forests usually improve test performance by averaging many trees.

For the bagging and random forest models, we can also check OOB performance.

```

bag.oob_score_

```

0.4567923788468702

```
rf.oob_score_
```

```
0.47245609860472904
```

OOB R^2 gives an internal estimate of predictive performance based only on the training data.

10.7.1 Tuning the Random Forest

We showed Bagging in the previous section. Since it overlaps with `RandomForest`, we will only focus on the Random Forest model in this section. We first tune `max_features`, `min_samples_leaf`, and `max_samples` by cross-validation, with a relative small number of estimators.

```
from sklearn.model_selection import GridSearchCV

rf_base = RandomForestRegressor(n_estimators=300, random_state=1)

param_grid = {
    "max_features": ["sqrt", 0.5, 1.0],
    "min_samples_leaf": [1, 3, 5, 10],
    "max_samples": [0.6, 0.8, 1.0],
}

grid = GridSearchCV(rf_base, param_grid=param_grid, cv=5, n_jobs=-1)

grid.fit(X_train, y_train)

print(f"Best parameters: {grid.best_params_}")
print(f"Best CV R^2: {grid.best_score_}")
```

```
Best parameters: {'max_features': 'sqrt', 'max_samples': 1.0, 'min_samples_leaf': 5}
Best CV R^2: 0.45220837115313606
```

Now evaluate the tuned model on the test set.

```
grid.best_estimator_.score(X_test, y_test)
```

```
0.3777186499485139
```

10.7.2 Find the Number of Trees

The number of trees controls how stable the ensemble average is. The following code fits forests with different numbers of trees and records OOB performance.

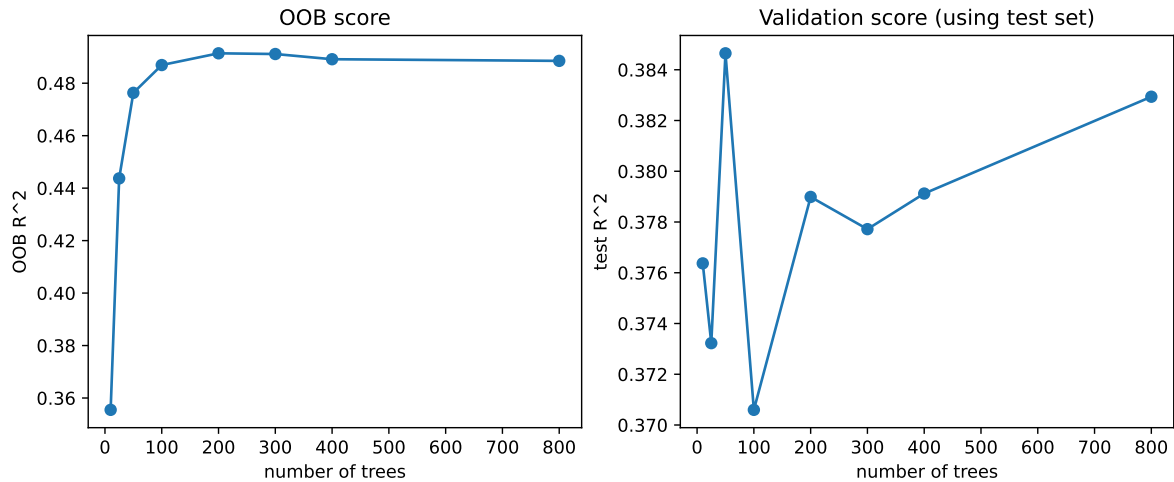
```
tree_counts = [10, 25, 50, 100, 200, 300, 400, 800]
oob_scores = []
test_r2 = []

for n_trees in tree_counts:
    model = RandomForestRegressor(
        n_estimators=n_trees,
        max_features="sqrt",
        max_samples=1.0,
        min_samples_leaf=5,
        oob_score=True,
        random_state=1,
        n_jobs=-1,
    )
    model.fit(X_train, y_train)
    oob_scores.append(model.oob_score_)
    test_r2.append(model.score(X_test, y_test))

fig, ax = plt.subplots(1, 2, figsize=(11, 4))

ax[0].plot(tree_counts, oob_scores, marker="o")
ax[0].set_xlabel("number of trees")
ax[0].set_ylabel("OOB R^2")
ax[0].set_title("OOB score")

ax[1].plot(tree_counts, test_r2, marker="o")
ax[1].set_xlabel("number of trees")
ax[1].set_ylabel("test R^2")
ax[1].set_title("Validation score (using test set)");
```



As mentioned previously, when reading these two plots, it is important not to focus on the single highest point. Instead, we look for the point where the curve becomes stable and nearly flat. In this example, around 200 trees appears to be a reasonable choice.

Therefore our best model so far is

```
best_rf = RandomForestRegressor(  
    n_estimators=200,  
    max_features="sqrt",  
    max_samples=1.0,  
    min_samples_leaf=5,  
    oob_score=True,  
    random_state=1,  
    n_jobs=-1  
)  
best_rf.fit(X_train, y_train)
```

`n_estimators` `n_estimators:` 200
`int, default=100`

The number of trees in the forest.

.. `versionchanged:: 0.22`
The default value of
“`n_estimators`“ changed from
10 to 100
in 0.22.

```
criterion criterion: 'squared_error'
{"squared_error",
 "absolute_error",
 "friedman_mse", "poisson"},
default="squared_error"
```

The function to measure the quality of a split. Supported criteria

are "squared_error" for the mean squared error, which is equal to

variance reduction as feature selection criterion and minimizes the L2

loss using the mean of each terminal node,

"friedman_mse", which uses mean squared error with Friedman's improvement score for potential

splits, "absolute_error" for the mean absolute error, which minimizes

the L1 loss using the median of each terminal node, and

"poisson" which uses reduction in Poisson deviance to find splits.

Training using "absolute_error" is significantly slower than when using "squared_error".

.. versionadded:: 0.18
Mean Absolute Error (MAE) criterion.

.. versionadded:: 1.0
Poisson criterion.

`max_depth` `max_depth`: int, None
default=None

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.
`min_samples_split` 2
`min_samples_split`: int or float, default=2

The minimum number of samples required to split an internal node:

- If int, then consider '`min_samples_split`' as the minimum number.
- If float, then '`min_samples_split`' is a fraction and '`ceil(min_samples_split * n_samples)`' are the minimum number of samples for each split.

.. versionchanged:: 0.18
Added float values for fractions.

`min_samples_leaf`
`min_samples_leaf`: int or
float, default=1

5

The minimum number of
samples required to be at a
leaf node.

A split point at any depth
will only be considered if it
leaves at
least “`min_samples_leaf`”
training samples in each of
the left and
right branches. This may
have the effect of smoothing
the model,
especially in regression.

- If int, then consider
‘`min_samples_leaf`’ as the
minimum number.
- If float, then
‘`min_samples_leaf`’ is a
fraction and
‘`ceil(min_samples_leaf *
n_samples)`’ are the
minimum
number of samples for each
node.

.. versionchanged:: 0.18
Added float values for
fractions.

<code>min_weight_fraction_leaf</code>	<code>0.0</code>
---------------------------------------	------------------

`min_weight_fraction_leaf`:
float, default=0.0

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

`max_features` `max_features`: 'sqrt'
{ "sqrt", "log2", None }, int or
float, default=1.0

The number of features to
consider when looking for the
best split:

- If int, then consider
'max_features' features at
each split.
- If float, then 'max_features'
is a fraction and
'max(1, int(max_features *
n_features_in_))' features
are considered at each
split.
- If "sqrt", then 'max_fea-
tures=sqrt(n_features)'.
- If "log2", then 'max_fea-
tures=log2(n_features)'.
- If None or 1.0, then
'max_features=n_features'.

.. note::

The default of 1.0 is
equivalent to bagged trees
and more
randomness can be achieved
by setting smaller values, e.g.
0.3.

.. versionchanged:: 1.1

The default of 'max_features'
changed from "auto" to 1.0.

Note: the search for a split
does not stop until at least
one
valid partition of the node
samples is found, even if it
requires to
effectively inspect more than
"max_features" features.

<code>max_leaf_nodes</code> <code>max_leaf_nodes: int,</code> <code>default=None</code>	None
---	------

Grow trees with
“`max_leaf_nodes`” in
best-first fashion.
Best nodes are defined as
relative reduction in impurity.
If None then unlimited
number of leaf nodes.

min_impurity_decrease 0.0
min_impurity_decrease:
float, default=0.0

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following::

$$N_t / N * (\text{impurity} - N_{t_R} / N_t * \text{right_impurity} - N_{t_L} / N_t * \text{left_impurity})$$

where “N” is the total number of samples, “N_t” is the number of samples at the current node, “N_t_L” is the number of samples in the left child, and “N_t_R” is the number of samples in the right child.

“N”, “N_t”, “N_t_R” and “N_t_L” all refer to the weighted sum, if “sample_weight” is passed.

.. versionadded:: 0.19

`bootstrap` `bootstrap: bool,` `True`
`default=True`

Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.

`oob_score` `oob_score: bool` `True`
or callable, `default=False`

Whether to use out-of-bag samples to estimate the generalization score. By default, `:func:~sklearn.metrics.r2_score` is used. Provide a callable with signature `'metric(y_true, y_pred)'` to use a custom metric. Only available if `'bootstrap=True'`.

For an illustration of out-of-bag (OOB) error estimation, see the example `:ref:'sphx_glr_auto_examples_ensemble_plot_ensemble_oob.py'`.

`n_jobs` `n_jobs`: int, -1
default=None

The number of jobs to run in parallel. `:meth:'fit'`, `:meth:'predict'`, `:meth:'decision_path'` and `:meth:'apply'` are all parallelized over the trees. “None” means 1 unless in a `:obj:'joblib.parallel_backend'` context. “-1” means using all processors. See `:term:'Glossary'` for more details.

`random_state` `random_state`: 1
int, RandomState instance or None, default=None

Controls both the randomness of the bootstrapping of the samples used when building trees (if “bootstrap=True”) and the sampling of the features to consider when looking for the best split at each node (if “max_features < n_features”). See `:term:'Glossary'` for details.

`verbose` `verbose`: int, 0
default=0

Controls the verbosity when fitting and predicting.

`warm_start` `warm_start`: False
bool, default=False

When set to “True”, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just fit a whole new forest. See

:term:‘Glossary ‘ and :ref:‘tree_ensemble_warm_start‘ for details.
`ccp_alpha` `ccp_alpha`: 0.0
non-negative float, default=0.0

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than

“`ccp_alpha`” will be chosen. By default, no pruning is performed. See :ref:‘minimal_cost_complexity_pruning‘ for details. See :ref:‘sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py‘ for an example of such pruning.

.. versionadded:: 0.22

`max_samples` `max_samples:` 1.0
int or float, default=None

If bootstrap is True, the
number of samples to draw
from X
to train each base estimator.

- If None (default), then draw
'X.shape[0]' samples.
 - If int, then draw
'max_samples' samples.
 - If float, then draw
'max(round(n_samples *
max_samples), 1)' samples.
- Thus,
'max_samples' should be in
the interval '(0.0, 1.0]'.

.. versionadded:: 0.22

<code>monotonic_cst</code> <code>monotonic_cst</code>: array-like of int of shape (n_features), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonically increasing - 0: no constraint - -1: monotonically decreasing If <code>monotonic_cst</code> is None, no constraints are applied. Monotonicity constraints are not supported for: - multioutput regressions (i.e. when <code>'n_outputs_' > 1</code>), - regressions trained on data with missing values. Read more in the :ref:'User Guide '. .. versionadded:: 1.4	None
---	------

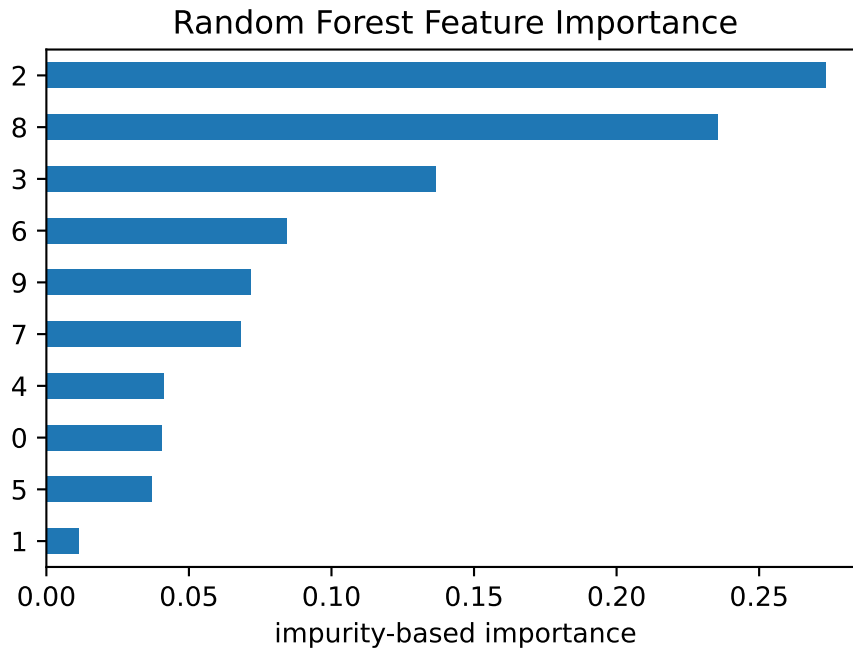
10.7.2.1 Variable Importance

We first examine impurity-based feature importance.

```
importance = pd.Series(best_rf.feature_importances_).sort_values(ascending=True)

importance.plot(kind="barh")
plt.xlabel("impurity-based importance")
plt.title("Random Forest Feature Importance")
```

```
Text(0.5, 1.0, 'Random Forest Feature Importance')
```



These values measure how much each variable reduces prediction error inside the trees. They are useful for a quick summary, but they should not be treated as causal effects.

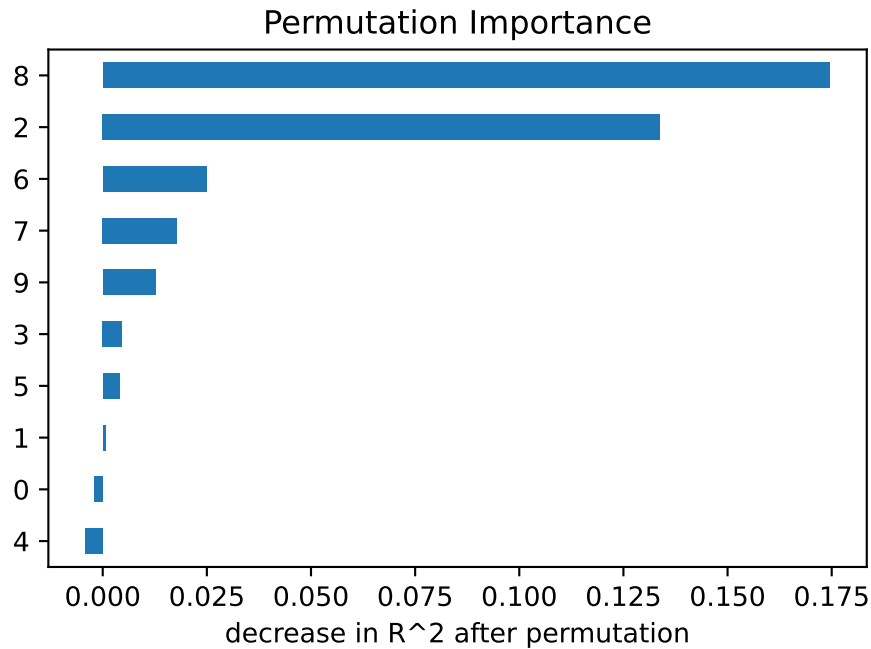
Permutation importance gives a more direct performance-based measure.

```
from sklearn.inspection import permutation_importance

perm = permutation_importance(
    best_rf, X_test, y_test, n_repeats=30, random_state=1, n_jobs=-1
)
perm_importance = pd.Series(perm.importances_mean).sort_values(ascending=True)

perm_importance.plot(kind="barh")
plt.xlabel("decrease in R^2 after permutation")
plt.title("Permutation Importance")
```

```
Text(0.5, 1.0, 'Permutation Importance')
```



If permuting a predictor substantially worsens predictions, then the fitted random forest relies on that predictor. If the importance is near zero, the predictor may not add much predictive information beyond the others.

From the two plots, feature 2 and 8 are the two most important features. If we go back to check the description of the dataset, they are `bmi`: body mass index and `s5`: `ltg`, possibly log of serum triglycerides level.

11 Boosting

Boosting is another ensemble method. Like bagging and random forests, it combines many trees into one model. The difference is how the trees are built.

Bagging and random forests build trees independently and then average them.

Boosting builds trees sequentially. Each new tree is trained to improve the current ensemble.

The central idea is:

Build many weak learners one at a time, and let each new learner focus on what the current model still gets wrong.

This chapter first discusses AdaBoost. AdaBoost is historically important and is the cleanest way to see the boosting idea. We then discuss the margin effect of AdaBoost and how to tune its hyperparameters. Finally, we discuss gradient boosting and use XGBoost as the main implementation example.

11.1 AdaBoost

AdaBoost (Adaptive Boosting) was introduced by Yoav Freund and Robert E. Schapire in 1995 [9] as one of the first highly successful boosting algorithms. Their original work showed that many weak learners, each performing only slightly better than random guessing, could be combined into a highly accurate classifier. AdaBoost became a foundational method in statistical learning, and it strongly influenced the later development of modern boosting methods such as Gradient Boosting and XGBoost.

The original AdaBoost algorithm was proposed as a classification method. Regression extensions were developed later after the success of AdaBoost classifiers. In this course, we mainly study AdaBoost classification because it introduces the core ideas of boosting in a simple and historically important setting.

For regression problems, we will move on to gradient boosting methods, which have become the dominant boosting framework in modern machine learning.

To keep the notation simple, assume the response is binary:

$$y_i \in \{-1, +1\}.$$

AdaBoost constructs a classifier of the form

$$F_M(x) = \sum_{m=1}^M \alpha_m G_m(x),$$

where:

- $G_m(x)$ is the m -th weak classifier,
- α_m is the weight assigned to that classifier,
- M is the number of boosting rounds.

The final prediction is

$$\hat{G}(x) = \text{sign}\{F_M(x)\}.$$

Thus AdaBoost is a weighted voting method. Classifiers with larger α_m have more influence in the final vote.

11.1.1 Weak Learners

AdaBoost is built from weak learners.

A weak learner is a classifier that is only slightly better than random guessing. In tree-based AdaBoost, the most common weak learner is a decision stump, which is a tree with only one split.

In `sklearn`, a decision stump is created by

```
from sklearn.tree import DecisionTreeClassifier  
  
DecisionTreeClassifier(max_depth=1)
```

A decision stump is extremely simple and usually underfits when used alone. One of the most surprising and important ideas behind AdaBoost is that by combining many such weak learners sequentially, the resulting ensemble can become a highly accurate and powerful model.

11.1.2 Weights of observations

In AdaBoost, there are two different types of weights.

- Estimator weights (α_m) determine how much influence each weak learner has in the final ensemble.
- Observation weights ($w_i^{(m)}$) determine which training observations receive more attention when fitting the next learner.

During training, AdaBoost repeatedly updates the weights of the training observations. Observations that are difficult to classify receive larger weights, so future learners focus more on them.

To understand this process, we need to discuss how a decision tree is trained on a weighted dataset.

The main idea is very simple. In an ordinary dataset, quantities such as class frequencies and node sample counts are computed using the number of observations. For a weighted dataset, these counts are replaced by weighted counts, that is, by the sum of the observation weights.

For example, suppose a node contains observations with weights $w_1, w_2, \dots, w_n$. Then the effective sample size of the node becomes $\sum_{i=1}^n w_i$. Similarly, class proportions are computed using weighted proportions rather than ordinary proportions.

Therefore, training a decision tree on weighted data is conceptually very similar to ordinary tree training. We simply replace ordinary counts by weighted counts throughout the splitting calculations.

Example 11.1. Consider the following data:

Table 11.1

x0	x1	y	Weight
1.0	2.1	+	0.5
1.0	1.1	+	0.125
1.3	1.0	-	0.125
1.0	1.0	-	0.125
2.0	1.0	+	0.125

The weighted Gini impurity is

$$\text{WeightedGini} = 1 - (0.5 + 0.125 + 0.125)^2 - (0.125 + 0.125)^2 = 0.375.$$

11.1.3 The AdaBoost Algorithm

Suppose we have training data

$$(x_1, y_1), \dots, (x_n, y_n), \quad y_i \in \{-1, +1\}.$$

Initialize observation weights:

$$w_i^{(1)} = \frac{1}{n}.$$

Then AdaBoost proceeds as follows.

1. For boosting round $m = 1, \dots, M$, fit a weak classifier G_m using the current weights.
2. Compute the weighted classification error by the m -th classifier:

$$\text{err}_m = \frac{\text{the total weights of misclassified observations}}{\text{the total weights}} = \frac{\sum_{\text{misclassified}} w_i^{(m)}}{\sum_{\text{all}} w_i^{(m)}}.$$

3. Assign a weight to the weak classifier:

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right).$$

4. Update observation weights and then normalize the weights so they sum to one:

$$w_i^{(m+1)} \xleftarrow{\text{normalize}} \hat{w}_i^{(m+1)} \leftarrow \begin{cases} w_i^{(m)} \exp(-\alpha_m) & \text{if observation } i \text{ is correctly classified by the } m\text{th weak learner} \\ w_i^{(m)} \exp(\alpha_m) & \text{if observation } i \text{ is misclassified by the } m\text{th weak learner.} \end{cases}$$

that

$$w_i^{(m+1)} = \frac{\hat{w}_i^{(m+1)}}{\sum_j \hat{w}_j^{(m+1)}}.$$

It is easy to see from the updating rule that AdaBoost puts more weight on observations that were misclassified.

5. Repeat the above process until the stop conditions are met.

The final model is then

$$F_M(x) = \sum_{m=1}^M \alpha_m G_m(x),$$

and the final prediction is

$$\hat{G}(x) = \text{sign} \{F_M(x)\}.$$

i Learning Rate and the $\frac{1}{2}$ Factor

In the classical AdaBoost derivation, the classifier weight is

$$\alpha_m = \frac{1}{2} \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right).$$

In practice, many implementations additionally introduce a learning rate $\eta > 0$ and use

$$\alpha_m = \eta \cdot \frac{1}{2} \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right).$$

Some textbooks instead write

$$\alpha_m = \eta \cdot \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right),$$

that is, without the factor $\frac{1}{2}$.

This does not fundamentally change the algorithm. The factor $\frac{1}{2}$ can simply be absorbed into the learning rate by replacing η with $\eta/2$. Therefore the two formulas differ only by a rescaling of the step size.

We mainly use the classical form with $\frac{1}{2}$ since it naturally appears in the standard derivation from exponential loss minimization and margin theory.

The learning rate controls how much each weak learner contributes to the final ensemble.

- A larger learning rate makes the model change more aggressively at each iteration.
- A smaller learning rate produces more gradual updates.

In practice, smaller learning rates are often combined with a larger number of estimators.

i Margin

$yF(x)$ is called the margin of the model F on the observation (x, y) . Since AdaBoost uses $\text{sign}\{F(x)\}$ as the final prediction rule, the margin can be interpreted as a confidence score for the prediction.

- If the margin is positive, the prediction is correct.
- If the margin is negative, the prediction is incorrect.
- A larger positive margin indicates a more confident prediction.
- A margin close to zero indicates uncertainty.

Notice that the observation weight updating formula can be rewritten as

$$w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i G_m(x_i)),$$

where $y_i G_m(x_i)$ is exactly the margin of the weak learner G_m on observation (x_i, y_i) .

Therefore, AdaBoost increases the weights of observations with small or negative margins and decreases the weights of observations with large positive margins. In this sense, AdaBoost can be viewed as an algorithm that attempts to improve the margin distribution by pushing margins toward larger positive values.

Margin theory is one of the central ideas in boosting and helps explain why boosting methods often continue to improve test performance even after the training error has already reached zero. We will discuss this topic in more detail later.

i Why the Learner Weight Makes Sense

The classifier weight is

$$\alpha_m = \eta \cdot \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right).$$

- If err_m is small, then the classifier is useful and α_m is large.
- If err_m is close to 0.5, then the classifier is close to random guessing and α_m is close to zero.
- If $\text{err}_m > 0.5$, then the classifier is worse than random guessing. In practice, this means the weak learner is not doing its job.

11.1.4 Why Does AdaBoost Work?

One of the most surprising aspects of AdaBoost is that it often performs extremely well even when each individual learner is very weak.

A useful way to understand boosting is through the bias-variance tradeoff.

- Bagging and Random Forests mainly reduce variance by averaging many independent trees.
- Boosting mainly reduces bias by sequentially correcting the errors made by the current model.

At each boosting round, the new learner focuses more on the observations that are currently difficult to classify. As more learners are added, many simple decision rules combine into a much more flexible decision boundary. This sequential correction process allows AdaBoost to gradually improve the model while still using very simple base learners such as decision stumps.

In the early boosting rounds, the model often becomes substantially more accurate. However, if boosting continues too aggressively, variance may eventually increase and overfitting may occur. Several techniques are commonly used to control this effect, like shallow trees, smaller learning

rates, subsampling and early stopping. Thus boosting can often achieve highly accurate models while still maintaining good generalization performance.

From a mathematical viewpoint, understanding *why* AdaBoost works so well mathematically became one of the major research topics in statistical learning during the late 1990s and early 2000s. Many theoretical studies showed that AdaBoost is closely connected to several important topics, including additive modeling, exponential loss minimization and margin theory.

One particularly influential result showed that AdaBoost can be interpreted as a stagewise additive optimization procedure. Important theoretical works include [10], [11], [12], and [13]. These studies greatly influenced the later development of modern boosting methods such as Gradient Boosting and XGBoost.

11.2 AdaBoost in Python

The basic APIs are as usual. We use `AdaBoostClassifier` from `sklearn.ensemble` to implement AdaBoost. All the regular APIs are the same to other models.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=800,
    learning_rate=0.1,
    random_state=1,
)

ada.fit(X_train, y_train)
```

SAMME and SAMME.R

The original AdaBoost algorithm was designed for binary classification. Later extensions such as SAMME and SAMME.R generalized AdaBoost to multiclass classification problems. In `sklearn`, multiclass classification for `AdaBoostClassifier` is implemented using the SAMME algorithm by default.

However, these variants are less important in modern statistical learning practice and will not be studied in detail in this course. One reason is that these extensions substantially increase the algorithmic complexity while introducing relatively limited new conceptual ideas beyond the original AdaBoost framework. In particular, SAMME.R relies heavily on predicted class probabilities, making the method more sensitive to the quality of probability estimation and numerical stability.

More importantly, many of the key ideas behind SAMME.R already overlap with the additive optimization viewpoint used in modern gradient boosting methods. As a result, modern frameworks such as Gradient Boosting and XGBoost provide a more general, flexible, and unified approach for both regression and classification problems. Consequently, we will focus primarily on the original AdaBoost classifier as an accessible introduction to the core ideas of boosting before moving directly to Gradient Boosting. Reflecting a similar shift in modern machine learning practice, `sklearn` has also deprecated SAMME.R in recent versions due to its reliance on probability estimation and its conceptual overlap with more general gradient boosting frameworks.

11.2.1 The Margin Effect of AdaBoost

AdaBoost is often explained as an algorithm that reduces training error. However, its behavior is more subtle than that.

For a binary classifier with labels $y_i \in \{-1, +1\}$, define the margin of observation i as

$$\text{margin}_i = y_i F_M(x_i).$$

The sign of the margin determines whether the observation is classified correctly:

- if $\text{margin}_i > 0$, the observation is correctly classified;
- if $\text{margin}_i < 0$, the observation is misclassified.

The magnitude of the margin measures confidence. A large positive margin means the ensemble strongly supports the correct class.

11.2.2 Why Margins Matter

AdaBoost often continues to improve test performance even after the training error has reached zero. This can seem surprising. If the training error is already zero, what is left to improve?

The answer is margins.

After all training observations are classified correctly, AdaBoost may still increase the margins. Larger margins mean the classifier is more confident and more stable. This can improve generalization.

i Margin view

Training error only asks whether the margin is positive.
The margin view also asks how positive it is.

This is one reason AdaBoost can work well in practice. It does not only try to classify points correctly. It also tends to push correctly classified points farther away from the decision boundary.

11.2.3 Margin study in Python

The following example uses a manually crafted dataset. We track training error, test error, and margins as the number of boosting rounds increases.

```
import numpy as np
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(1)

n = 3000
p = 10
X = rng.normal(size=(n, p))

score = (
    1.2 * (X[:, 0] > -0.6)
    + 1.0 * (X[:, 0] > 0.4)
    - 1.1 * (X[:, 1] > 0.0)
    + 0.9 * (X[:, 2] > 0.5)
    - 0.8 * (X[:, 3] > -0.2)
    + 0.7 * ((X[:, 4] > 0) & (X[:, 5] > 0))
    - 0.7 * ((X[:, 6] < 0) & (X[:, 7] > 0.3))
    + 0.5 * (X[:, 8] > 0.1)
    - 0.5 * (X[:, 9] > -0.4)
)
y = 2 * (score > np.quantile(score, 0.52)).astype(int) - 1

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, stratify=y, random_state=1
)
```

We first train the model.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=2, random_state=1),
```

```

    n_estimators=2000,
    learning_rate=0.2,
    random_state=1,
)

ada.fit(X_train, y_train)

```

estimator	estimator: object, default=None	DecisionTreeC...an- dom_state=1)
-----------	------------------------------------	-------------------------------------

The base estimator from which the boosted ensemble is built. Support for sample weighting is required, as well as proper “classes_“ and “n_classes_“ attributes. If “None“, then the base estimator is :class:~sklearn.tree.Decision-TreeClassifier‘ initialized with ‘max_depth=1‘.

.. versionadded:: 1.2
‘base_estimator‘ was renamed to ‘estimator‘.
n_estimators n_estimators: 2000
int, default=50

The maximum number of estimators at which boosting is terminated. In case of perfect fit, the learning procedure is stopped early. Values must be in the range ‘[1, inf)‘.

`learning_rate` `learning_rate:` 0.2
float, default=1.0

Weight applied to each classifier at each boosting iteration. A higher learning rate increases the contribution of each classifier. There is a trade-off between the ‘`learning_rate`’ and ‘`n_estimators`’ parameters. Values must be in the range ‘(0.0, inf)’.

`random_state` `random_state:` 1
int, RandomState instance or None, default=None

Controls the random seed given at each ‘`estimator`’ at each boosting iteration. Thus, it is only used when ‘`estimator`’ exposes a ‘`random_state`’. Pass an int for reproducible output across multiple function calls. See :term:‘Glossary ‘.

criterion criterion: {"gini",
"entropy", "log_loss"},
default="gini" 'gini'

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see :ref:'tree_mathematical_formulation'.

splitter splitter: {"best",
"random"}, default="best" 'best'

The strategy used to choose the split at each node.

Supported strategies are "best" to choose the best split and "random" to choose the best random split.

max_depth max_depth: int, 2
default=None

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

`min_samples_split` 2
`min_samples_split`: int or
float, default=2

The minimum number of
samples required to split an
internal node:

- If int, then consider
‘`min_samples_split`’ as the
minimum number.
- If float, then
‘`min_samples_split`’ is a
fraction and
‘`ceil(min_samples_split *
n_samples)`’ are the
minimum
number of samples for each
split.

.. versionchanged:: 0.18
Added float values for
fractions.

`min_samples_leaf`
`min_samples_leaf`: int or
float, default=1

1

The minimum number of
samples required to be at a
leaf node.

A split point at any depth
will only be considered if it
leaves at
least “`min_samples_leaf`”
training samples in each of
the left and
right branches. This may
have the effect of smoothing
the model,
especially in regression.

- If int, then consider
‘`min_samples_leaf`’ as the
minimum number.
- If float, then
‘`min_samples_leaf`’ is a
fraction and
‘`ceil(min_samples_leaf *
n_samples)`’ are the
minimum
number of samples for each
node.

.. versionchanged:: 0.18
Added float values for
fractions.

<code>min_weight_fraction_leaf</code>	<code>0.0</code>
<code>min_weight_fraction_leaf:</code> float, default=0.0	

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

`max_features` `max_features:` None
int, float or {"sqrt", "log2"},
default=None

The number of features to consider when looking for the best split:

- If int, then consider ‘`max_features`’ features at each split.
- If float, then ‘`max_features`’ is a fraction and ‘`max(1, int(max_features * n_features_in_))`’ features are considered at each split.
- If "sqrt", then ‘`max_features=sqrt(n_features)`’.
- If "log2", then ‘`max_features=log2(n_features)`’.
- If None, then ‘`max_features=n_features`’.

.. note::

The search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than “`max_features`” features.

`random_state` `random_state`: 1
int, `RandomState` instance or
None, default=None

Controls the randomness of the estimator. The features are always randomly permuted at each split, even if “`splitter`” is set to “`best`”. When “`max_features < n_features`”, the algorithm will select “`max_features`” at random at each split before finding the best split among them. But the best found split may vary across different runs, even if “`max_features=n_features`”. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a deterministic behaviour during fitting, “`random_state`” has to be fixed to an integer. See :term:‘Glossary’ for details.

<code>max_leaf_nodes</code> <code>max_leaf_nodes: int,</code> <code>default=None</code>	None
---	------

Grow a tree with
“`max_leaf_nodes`” in
best-first fashion.
Best nodes are defined as
relative reduction in impurity.
If None then unlimited
number of leaf nodes.

min_impurity_decrease 0.0
min_impurity_decrease:
float, default=0.0

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following::

$$N_t / N * (\text{impurity} - N_{t_R} / N_t * \text{right_impurity} - N_{t_L} / N_t * \text{left_impurity})$$

where “N” is the total number of samples, “N_t” is the number of samples at the current node, “N_t_L” is the number of samples in the left child, and “N_t_R” is the number of samples in the right child.

“N”, “N_t”, “N_t_R” and “N_t_L” all refer to the weighted sum, if “sample_weight” is passed.

.. versionadded:: 0.19

`class_weight` `class_weight`: None
dict, list of dict or
"balanced", default=None

Weights associated with
classes in the form
“{class_label: weight}”.
If None, all classes are
supposed to have weight one.
For
multi-output problems, a list
of dicts can be provided in
the same
order as the columns of `y`.

Note that for multioutput
(including multilabel) weights
should be
defined for each class of every
column in its own dict. For
example,
for four-class multilabel
classification weights should
be
[{0: 1, 1: 1}, {0: 1, 1: 5}, {0:
1, 1: 1}, {0: 1, 1: 1}] instead
of
[{1:1}, {2:5}, {3:1}, {4:1}].

The "balanced" mode uses
the values of `y` to
automatically adjust
weights inversely proportional
to class frequencies in the
input data
as “`n_samples / (n_classes *
np.bincount(y))`”

For multi-output, the weights
of each column of `y` will be
multiplied.

Note that these weights will
be multiplied with
`sample_weight` (passed
through the `fit` method) if
`sample_weight` is specified.

`ccp_alpha ccp_alpha:` 0.0
non-negative float,
default=0.0

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than “`ccp_alpha`” will be chosen. By default, no pruning is performed. See :ref:‘minimal_cost_complexity_pruning’ for details. See :ref:‘sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py’ for an example of such pruning.

.. versionadded:: 0.22

<code>monotonic_cst</code> <code>monotonic_cst</code> : array-like of int of shape (n_features), default=None	None
Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - -1: monotonic decrease	
If <code>monotonic_cst</code> is None, no constraints are applied.	
Monotonicity constraints are not supported for: - multiclass classifications (i.e. when '<code>n_classes > 2</code>'), - multioutput classifications (i.e. when '<code>n_outputs > 1</code>'), - classifications trained on data with missing values.	
The constraints hold over the probability of the positive class.	
Read more in the :ref:'User Guide '.	
.. versionadded:: 1.4	

Then since the model contains the information of all trees, we could directly read them and use them about how the model works if we only use the first several trees. Here we use `.staged_predict()` and `.staged_decision_function()` methods.

- `.staged_predict()`: returns the predictions after each boosting round.
- `.staged_decision_function()`: returns the decision scores after each boosting round.

We would like to record all the training and test scores.

```

from sklearn.metrics import accuracy_score

train_acc = []
test_acc = []

for pred_train, pred_test in zip(
    ada.staged_predict(X_train), ada.staged_predict(X_test)
):
    train_acc.append(accuracy_score(y_train, pred_train))
    test_acc.append(accuracy_score(y_test, pred_test))

```

In addition, the 10th-percentile margin is of our interesting, since we treat $\text{margin} < 10\%$ as no confidence.

```

train_margin_10 = []
test_margin_10 = []

for score_train, score_test in zip(
    ada.staged_decision_function(X_train),
    ada.staged_decision_function(X_test),
):
    train_margins = y_train * score_train
    test_margins = y_test * score_test

    train_margin_10.append(np.quantile(train_margins, 0.1))
    test_margin_10.append(np.quantile(test_margins, 0.1))

```

We also need to test when the training score becomes 1.0, and when the test score is the highest. Then we have the interesting output.

```

first_full_train = next((i + 1 for i, s in enumerate(train_acc) if s == 1.0), None)
best_test_round = np.argmax(test_acc) + 1

print("First round with training accuracy 1:", first_full_train)
print("Test accuracy at that round:", test_acc[first_full_train - 1])
print("Best test accuracy:", np.max(test_acc))
print("Best test round:", best_test_round)
print("Final training accuracy:", train_acc[-1])
print("Final test accuracy:", test_acc[-1])

```

First round with training accuracy 1: 426

Test accuracy at that round: 0.9746666666666667
Best test accuracy: 0.9806666666666667
Best test round: 1121
Final training accuracy: 1.0
Final test accuracy: 0.98

We could plot the curves. From the plot, we can see that after the training score becomes 1.0, the model is still improving and the test score reaches the highest long after.

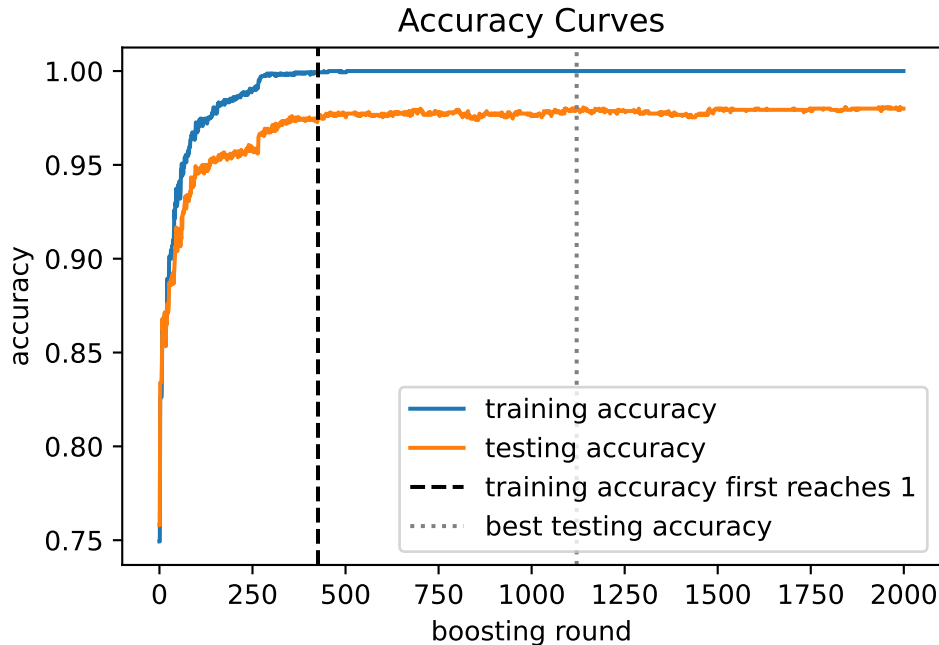
```
import matplotlib.pyplot as plt

plt.plot(train_acc, label="training accuracy")
plt.plot(test_acc, label="testing accuracy")

plt.axvline(
    first_full_train,
    color="black",
    linestyle="--",
    label="training accuracy first reaches 1",
)

plt.axvline(best_test_round, color="gray", linestyle=":", label="best testing accuracy")

plt.xlabel("boosting round")
plt.ylabel("accuracy")
plt.title("Accuracy Curves")
plt.legend();
```



We may also plot the 10th-percentile margin curve. The 10th-percentile margin represents the lower tail of the margin distribution, that is, the examples on which the classifier is least confident.

From the plot, we can see that even after the training accuracy reaches 1.0, the margins may still continue to change. This shows that AdaBoost is not merely optimizing training accuracy; it continues to adjust the additive model in order to improve the margin distribution and increase confidence.

- If the lower-percentile margin continues to increase while test performance remains stable or improves, this suggests that the model is still improving its confidence on difficult cases and is not yet clearly overfitting.
- If the lower-percentile margin decreases together with decreasing validation or test performance, this suggests that the model may be becoming less reliable on difficult cases and may be starting to overfit.

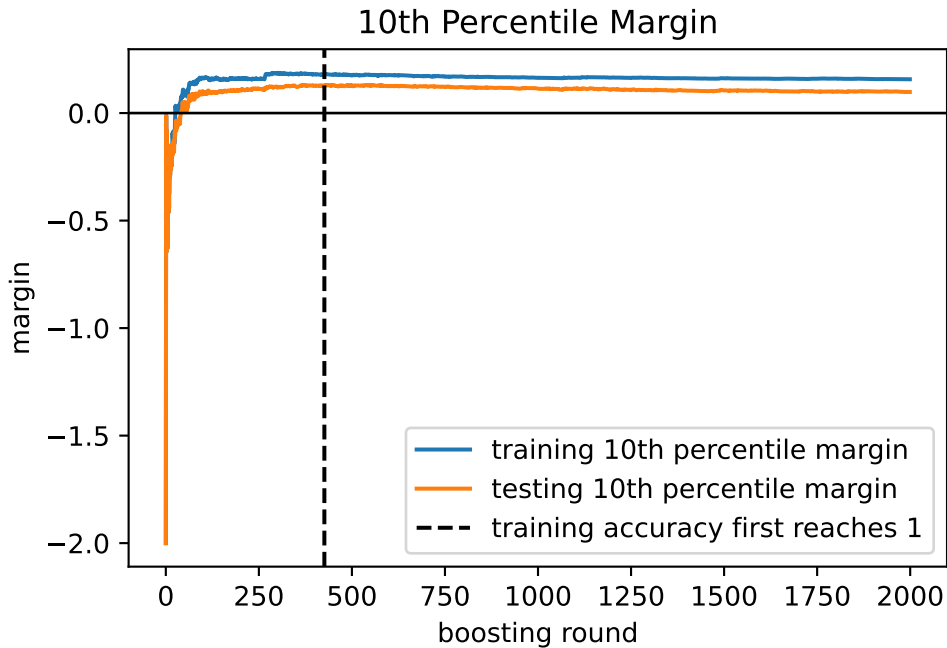
```
plt.plot(train_margin_10, label="training 10th percentile margin")
plt.plot(test_margin_10, label="testing 10th percentile margin")

plt.axvline(
    first_full_train,
    color="black",
    linestyle="--",
    label="training accuracy first reaches 1",
```

```
)

plt.axhline(0, color="black", linewidth=1)

plt.xlabel("boosting round")
plt.ylabel("margin")
plt.title("10th Percentile Margin")
plt.legend();
```



11.3 Tuning AdaBoost Hyperparameters

The most important AdaBoost hyperparameters are:

Parameter	Role
<code>n_estimators</code>	Number of weak learners.
<code>learning_rate</code>	Shrinks the contribution of each learner.
<code>estimator__max_depth</code>	Complexity of the base tree.
<code>estimator__min_samples_leaf</code>	Smoothness of each base tree.

The two most important parameters are `n_estimators` and `learning_rate`.

11.3.1 Learning Rate and Number of Trees

There is a strong tradeoff:

- smaller `learning_rate` usually needs larger `n_estimators`;
- larger `learning_rate` learns faster but can overfit or become unstable;
- smaller `learning_rate` often gives smoother and more stable models.

11.3.2 Base Tree Depth

Decision stumps use `max_depth=1`. They are common because AdaBoost was originally designed around weak learners. Using deeper trees allows each boosting round to model interactions. In most cases, AdaBoost use depths from 1 to 3. Deeper trees are stronger learners, but they also increase the risk of overfitting.

11.3.3 Grid Search for AdaBoost

The following code tunes AdaBoost by grid search cross-validation.

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(random_state=0),
    random_state=0,
)

param_grid = {
    "n_estimators": [100, 300, 600],
    "learning_rate": [0.01, 0.05, 0.1, 0.5],
    "estimator__max_depth": [1, 2, 3],
    "estimator__min_samples_leaf": [1, 5, 10],
}

grid = GridSearchCV(ada, param_grid=param_grid, cv=5, scoring="accuracy", n_jobs=-1)

grid.fit(X_train, y_train)
grid.best_params_
```


Tuning should be done using validation data or cross-validation. The test set should be saved for the final evaluation.

Noisy data

AdaBoost can be sensitive to mislabeled observations and outliers. Because it repeatedly increases the weights of difficult observations, noisy points may receive too much attention.

11.3.4 Example: AdaBoost on a Nonlinear Classification Problem

We first use a synthetic nonlinear classification problem. This example shows that many weak learners can create a flexible decision boundary.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(n_samples=1500, noise=0.25, random_state=1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=1
)
```

Fit a baseline stump and an AdaBoost model.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

stump = DecisionTreeClassifier(max_depth=1, random_state=1)

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=1),
    n_estimators=300,
    learning_rate=0.05,
    random_state=1,
)

stump.fit(X_train, y_train)
ada.fit(X_train, y_train)

print("Stump accuracy:", stump.score(X_test, y_test))
print("AdaBoost accuracy:", ada.score(X_test, y_test))
```

Stump accuracy: 0.8488888888888889
AdaBoost accuracy: 0.9222222222222223

Now tune the AdaBoost model.

```
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score

ada_base = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(random_state=1), random_state=1
)

param_grid = {
    "n_estimators": [100, 300, 600],
    "learning_rate": [0.03, 0.05, 0.1],
    "estimator__max_depth": [1, 2],
    "estimator__min_samples_leaf": [1, 5],
}

grid = GridSearchCV(
    ada_base, param_grid=param_grid, cv=5, scoring="accuracy", n_jobs=-1
)

grid.fit(X_train, y_train)
best_ada = grid.best_estimator_

print("Best parameters:", grid.best_params_)
print("Best CV accuracy:", grid.best_score_)
print("Test accuracy:", best_ada.score(X_test, y_test))
```

Best parameters: {'estimator__max_depth': 2, 'estimator__min_samples_leaf': 5, 'learning_rate': 0.05}
Best CV accuracy: 0.9466666666666667
Test accuracy: 0.94

11.4 XGBoost and Gradient Boosting

The core idea of boosting is to sequentially train models in order to correct the errors made by earlier models. A natural idea is therefore:

1. fit a model
2. compute the residuals

3. fit a new model to the residuals
4. repeat

We mainly consider regression problems at the current stage.

11.4.1 Additive model

The above idea leads directly to **Gradient Boosting**. When the models used are trees, the model is also called Gradient Boosted Decision Trees (GBDT).

Gradient Boosting builds an additive model of the form

$$F_M(x) = \sum_{m=1}^M \beta_m h_m(x)$$

where

- $h_m(x)$ is usually a small regression tree
- β_m is a coefficient
- the model is built sequentially

Unlike bagging, where trees are trained independently, boosting trains each new tree based on the current errors of the model.

11.4.2 Functional gradient viewpoint

In ordinary optimization problems, we optimize finitely many parameters. For example, in linear regression we optimize the coefficient vector

$$\beta = (\beta_1, \dots, \beta_p).$$

Gradient Boosting takes a different viewpoint. Instead of directly optimizing finitely many parameters, it attempts to optimize the prediction function itself.

The model is built additively:

$$F_{m+1}(x) = F_m(x) + f_m(x),$$

where each new function $f_m(x)$ is usually a small regression tree.

Thus each boosting step adds a new basis function to the model. Since the space of possible functions is extremely large, Gradient Boosting is often interpreted as a form of gradient descent in function space, also called functional gradient descent.

Conceptually:

- ordinary gradient descent updates parameters
- Gradient Boosting updates functions

11.4.3 Regression tree fitting

In order to find the best model $F(x)$, we would like to minimize the empirical loss function

$$L(F) = \sum_{i=1}^n l(y_i, F(x_i)),$$

where $\{(x_i, y_i)\}$ is the training dataset. One of the most common choices for the loss function is the squared-error loss

$$L(F) = \sum_{i=1}^n (y_i - F(x_i))^2.$$

Assume the current model is $F_m(x)$. After adding a new tree $f_m(x)$, the next model becomes

$$F_{m+1}(x) = F_m(x) + f_m(x).$$

The new tree $f_m(x)$ is chosen to reduce the loss function as much as possible.

Now consider the first-order Taylor expansion of the loss function:

$$l(y_i, F_m(x_i) + f_m(x_i)) \approx l(y_i, F_m(x_i)) + g_i f_m(x_i),$$

where

$$g_i = \left. \frac{\partial l(y_i, F_m(x_i))}{\partial F_m(x_i)} \right|_{F=F_m}.$$

This gives a linear approximation of the loss function in terms of the update $f_m(x_i)$.

Since the first-order Taylor approximation is linear, the negative gradient gives the direction of steepest descent. Therefore we define the pseudo-residual

$$r_i = -g_i = - \left. \frac{\partial l(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_m}.$$

The pseudo-residual plays the role of the error signal that the next tree should fit.

Assume that the new tree splits the dataset into regions $R_1, \dots, R_s$, and for each region R_j a value s_j is assigned. We would like the tree to approximate the pseudo-residuals r_i . Therefore we choose the tree to minimize

$$\sum_{j=1}^s \sum_{x_i \in R_j} (r_i - s_j)^2.$$

For a fixed region R_j , the optimal value of s_j is the mean of the observations inside that region:

$$s_j = \frac{1}{|R_j|} \sum_{x_i \in R_j} r_i.$$

Therefore, the new tree $f_m(x)$ is simply a regression tree (trained using RSS) with target values r_i .

i Pseudo-residual

r_i is called a *pseudo-residual* because it is technically a negative gradient rather than an ordinary residual. However, it plays the role of the residual in Gradient Boosting. For squared-error loss

$$l(y, F(x)) = (y - F(x))^2,$$

we have

$$r_i = 2(y_i - F_m(x_i)),$$

which is proportional to the ordinary residual.

Therefore, under L^2 loss, Gradient Boosting can literally be interpreted as fitting residuals.

11.4.4 XGBoost: Initial idea

XGBoost stands for **Extreme Gradient Boosting**. Its mathematical foundation can be viewed as a modified and improved version of GBDT.

XGBoost became one of the most successful shallow machine learning models in modern practice. In addition to its mathematical improvements, it also introduced many engineering innovations that significantly improved efficiency and scalability. After the success of XGBoost, several related models such as LightGBM and CatBoost were also developed and became widely used.

We mainly focus on the mathematical ideas behind XGBoost. The motivation is very similar to GBDT. We consider the additive model

$$F_{m+1}(x) = F_m(x) + f_m(x),$$

and would like to minimize the empirical loss

$$L(F) = \sum_{i=1}^n l(y_i, F(x_i)).$$

Assume the current model is $F_m(x)$. After adding a new tree $f_m(x)$, we obtain

$$F_{m+1}(x) = F_m(x) + f_m(x).$$

Instead of using only first-order information as in GBDT, XGBoost uses a local second-order Taylor approximation of the loss function at each boosting iteration:

$$l(y_i, F_m(x_i) + f_m(x_i)) \approx l(y_i, F_m(x_i)) + g_i f_m(x_i) + \frac{1}{2} h_i f_m(x_i)^2,$$

where

$$g_i = \left. \frac{\partial l(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_m},$$

and

$$h_i = \left. \frac{\partial^2 l(y_i, F(x_i))}{\partial F(x_i)^2} \right|_{F=F_m}.$$

Here

- g_i measures the gradient
- h_i measures the curvature of the loss function

Thus, XGBoost uses both gradient and curvature information.

11.4.5 Intuition of the quadratic approximation

The first-order Taylor expansion used in GBDT gives only a linear approximation of the loss function. In contrast, the second-order Taylor expansion gives a quadratic approximation:

$$g_i f + \frac{1}{2} h_i f^2.$$

As a function of f , its derivative is

$$g_i + h_i f.$$

Setting this equal to zero gives the critical point

$$f = -\frac{g_i}{h_i}.$$

This can be viewed as the approximately optimal update suggested by the local quadratic approximation of the loss function.

11.4.6 Tree structure in XGBoost

Assume that the new tree splits the dataset into regions

$$R_1, \dots, R_s,$$

and assigns a prediction value s_j to region R_j . Then for every observation inside region R_j , we have

$$f_m(x_i) = s_j.$$

Substituting this into the quadratic approximation gives the approximate objective

$$\sum_{j=1}^s \sum_{x_i \in R_j} \left(g_i s_j + \frac{1}{2} h_i s_j^2 \right).$$

For a fixed tree structure, we minimize this expression with respect to s_j .

Differentiating with respect to s_j gives

$$\sum_{x_i \in R_j} (g_i + h_i s_j) = 0.$$

Therefore,

$$s_j = -\frac{\sum_{x_i \in R_j} g_i}{\sum_{x_i \in R_j} h_i}.$$

If we define

$$G_j = \sum_{x_i \in R_j} g_i, \quad H_j = \sum_{x_i \in R_j} h_i,$$

then the optimal leaf value becomes

$$s_j = -\frac{G_j}{H_j}.$$

⚠ Important difference from ordinary CART

This tree is NOT an ordinary CART regression tree trained with RSS.

- In GBDT, we explicitly train a CART regression tree to fit the pseudo-residuals.
- In XGBoost, the leaf values are derived directly from the quadratic approximation of the objective function itself.

Therefore, although XGBoost trees may look structurally similar to CART trees, mathematically they are solving a different optimization problem.

11.4.7 Regularization

So far we derived the basic XGBoost objective using the second-order Taylor approximation.

However, minimizing only the training loss may produce overly complex trees and lead to overfitting. Therefore XGBoost introduces an explicit regularization term to control tree complexity.

Suppose the new tree $f_m(x)$ has

- T leaves
- leaf values $s_1, \dots, s_T$

XGBoost defines the regularization term

$$\Omega(f_m) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T s_j^2.$$

Therefore the objective becomes

$$L(F_m + f_m) + \Omega(f_m).$$

Here

- γT penalizes the number of leaves
- $\frac{1}{2} \lambda \sum s_j^2$ penalizes large leaf values

Using the second-order Taylor approximation and grouping terms by regions gives

$$\sum_{j=1}^T \left[G_j s_j + \frac{1}{2} (H_j + \lambda) s_j^2 \right] + \gamma T.$$

For a fixed tree structure, we minimize the objective with respect to s_j . Differentiating gives

$$G_j + (H_j + \lambda) s_j = 0.$$

Therefore the optimal leaf value becomes

$$s_j = -\frac{G_j}{H_j + \lambda}.$$

Compared with the non-regularized version

$$s_j = -\frac{G_j}{H_j},$$

the regularization term enlarges the denominator and shrinks the leaf predictions toward zero. This effect is similar to ridge regularization in linear regression.

Substituting the optimal leaf values back into the objective gives

$$-\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T.$$

This quantity is used by XGBoost to evaluate the quality of candidate tree splits.

Therefore, unlike ordinary CART trees which use RSS or Gini impurity, XGBoost derives its splitting criterion directly from the regularized optimization objective.

11.4.8 Learning rate

In practice, although XGBoost computes the best update tree f_m , it usually does not add the full tree update directly to the model. Instead, the update is scaled by a learning rate (also called the shrinkage parameter):

$$F_{m+1}(x) = F_m(x) + \eta f_m(x),$$

where

$$0 < \eta \leq 1.$$

Thus every new tree contributes only partially to the final model.

Equivalently, after computing the optimal leaf value

$$s_j = -\frac{G_j}{H_j + \lambda},$$

XGBoost actually uses the scaled leaf prediction

$$\eta s_j.$$

Therefore shrinkage reduces the influence of each individual tree.

The parameter η is called the **learning rate** or shrinkage parameter.

XGBoost in practice

The mathematical foundation of XGBoost is built on three main ideas:

- second-order Taylor approximation of the loss function
- explicit regularization on tree complexity
- shrinkage through the learning rate

In addition to these mathematical ideas, XGBoost also introduced many important engineering optimizations such as:

- parallel tree construction

- efficient handling of sparse data
- cache-aware memory access
- approximate split finding algorithms

These engineering improvements are one of the main reasons why XGBoost became highly successful in practical machine learning applications.

11.4.9 GBDT and XGBoost for classification

Both GBDT and XGBoost can naturally be extended from regression to classification problems. The overall framework remains the same:

- additive tree model
- boosting optimization
- sequentially adding trees
- minimizing a loss function

The main difference is the choice of the loss function.

For binary classification, boosting methods commonly use logistic loss

$$l(y, p) = -[y \log p + (1 - y) \log(1 - p)].$$

The model output $F(x)$ is converted into probabilities using the logistic function

$$p(x) = \frac{1}{1 + e^{-F(x)}}.$$

For multiclass classification, softmax-based loss functions are commonly used.

The difference between GBDT and XGBoost remains the same as in regression:

- GBDT uses first-order gradients (pseudo-residuals)
- XGBoost uses second-order Taylor approximation together with regularization

Thus both methods provide a unified framework for regression and classification problems.

i AdaBoost as Gradient Boosting with exponential loss

AdaBoost can be interpreted as a special case of Gradient Boosting using the exponential loss function

$$l(y, F(x)) = e^{-yF(x)},$$

where

$$y \in \{-1, 1\}.$$

The empirical loss becomes

$$L(F) = \sum_{i=1}^n e^{-y_i F(x_i)}.$$

This loss heavily penalizes observations with negative margins

$$y_i F(x_i).$$

In particular:

- correctly classified points with large positive margins receive very small loss
- misclassified points receive exponentially large loss

Therefore minimizing exponential loss naturally increases the weights of difficult observations, which explains the adaptive reweighting behavior of AdaBoost.

Taking derivative gives

$$\frac{\partial l(y_i, F(x_i))}{\partial F(x_i)} = -y_i e^{-y_i F(x_i)}.$$

Thus the pseudo-residual becomes

$$r_i = y_i e^{-y_i F(x_i)}.$$

Observations with small or negative margins receive much larger pseudo-residuals and therefore larger influence in the next boosting step.

From this viewpoint, AdaBoost can be interpreted as functional gradient descent under exponential loss.

Gradient Boosting generalizes this idea by allowing arbitrary differentiable loss functions rather than only exponential loss.

11.5 XGBoost in Python

XGBoost is implemented using the library **XGBoost**. You may go to the [official website](#) for more information. We will cover the basic usage in this lecture.

A typical XGBoost regression model looks like this.

```
from xgboost import XGBRegressor

xgb = XGBRegressor()
```

```

    objective="reg:squarederror",
    n_estimators=500,
    learning_rate=0.05,
    max_depth=3,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    random_state=0,
    n_jobs=-1,
)

xgb.fit(X_train, y_train)

```

The `objective="reg:squarederror"` is to indicate the loss function. The default is `"reg:squarederror"` that is regression with squared loss. More loss function can be found in the [documnet](#).

The most important hyperparameters are:

Parameter	Meaning
<code>n_estimators</code>	Number of boosting rounds.
<code>learning_rate</code>	Learning rate.
<code>max_depth</code>	Depth of each tree.
<code>subsample</code>	Fraction of observations used for each tree.
<code>colsample_bytree</code>	Fraction of predictors used for each tree.
<code>reg_lambda</code>	L2 regularization on leaf weights.
<code>reg_alpha</code>	L1 regularization on leaf weights.

Most of them are straightforward. I only discuss these since they are not from the theory.

i subsample

subsample controls the fraction of training observations randomly sampled for each boosting round. For example `subsample=0.8` means that each new tree is trained using only 80% of the observations.

This introduces additional randomness into boosting, similar to bagging in Random Forests. Smaller values of **subsample** and reduce variance and improve generalization. However, if the value is too small, the model may underfit.

i colsample_bytree

`colsample_bytree` controls the fraction of predictors randomly sampled for each tree. For example `colsample_bytree=0.7` means that each tree only considers 70% of the predictors.

This idea is similar to feature subsampling in Random Forests. It reduces correlation between trees and can improve robustness and generalization performance.

i reg_alpha

`reg_alpha` is the L1 regularization parameter on leaf weights. Previously we introduced the L2 regularization term $\frac{1}{2}\lambda \sum s_j^2$, which corresponds to ridge-style regularization. XGBoost can also include an L1 penalty

$$\alpha \sum |s_j|,$$

which is controlled by `reg_alpha`. L1 regularization tends to shrink some leaf values exactly to zero, producing sparser models and sometimes improving interpretability and robustness. This corresponds to lasso-style regularization.

We don't introduce L1 regularization in the first place because it might make the math formula too complicated.

Example grid:

```
from sklearn.model_selection import GridSearchCV
from xgboost import XGBRegressor

xgb = XGBRegressor(random_state=1, n_jobs=1)

param_grid = {
    "n_estimators": [200, 500, 800],
    "learning_rate": [0.03, 0.05, 0.1],
    "max_depth": [2, 3, 4],
    "subsample": [0.7, 0.9, 1.0],
    "colsample_bytree": [0.7, 0.9, 1.0],
    "reg_lambda": [1.0, 5.0, 10.0],
}

grid = GridSearchCV(xgb, param_grid=param_grid, cv=5, n_jobs=-1)

grid.fit(X_train, y_train)
```

i n_jobs

Although for boosting parallel computing is in the nature as in bagging, **XGBoost** successfully support it and offers **n_jobs** to accelerate. The API is similar, so we could use **n_jobs=-1** to indicate using all CPU cores.

Similar to bagging, for a nested struture like **GridSearchCV** with **XGBoost**, we only apply **n_jobs** to one of them. In general, if it is a small dataset, we let **GridSearchCV** use multiple cores. If it is a large dataset, we let **XGBoost** use multiple cores.

11.6 Tuning XGBoost

If there are enough computing resources, to tune **XGBoost** is no different from other models. However since the model becomes more complicated, it usually takes longer to go through the whole grid search. In this case, usually we make some modifications to the workflow.

11.6.1 RandomSearchCV

We already show the code of **RandomSearchCV** before. Here we give more explanations.

RandomSearchCV is to search random combinations of hyperparameters. Therefore it is used to do a rough search in a very large parameter space. Other than the **param_grid** like **GridSearchCV**, **RandomSearchCV** needs **n_iter** to indicate how many combinations one wants to test.

Since now we can search in a very large space, usually we make a really large space, by directly choosing hyperparameters based on distributions. We use distributions from the library **scipy.stats**. Here we give an examples.

```
from scipy.stats import randint, uniform

param_distributions = {
    "n_estimators": randint(100, 1000),
    "max_depth": randint(2, 8),
    "learning_rate": uniform(0.01, 0.19),
    "subsample": uniform(0.6, 0.4),
    "colsample_bytree": uniform(0.6, 0.4),
    "reg_lambda": uniform(0, 10),
}

random_search = RandomizedSearchCV(
```

```
xgb, param_distributions=param_distributions, n_iter=30, cv=5, random_state=0
)
```

11.6.2 Workflow

A practical tuning workflow is often a two-stage process.

First, run `RandomizedSearchCV` over a relatively large parameter space. This gives a rough idea of promising regions in the hyperparameter space.

Then, based on the best combination found by random search, choose a smaller and more refined grid around those values and run `GridSearchCV`.

If the refined grid is still too slow, we can tune parameters in smaller batches. A common strategy is:

1. tune model complexity parameters first, such as `max_depth`
2. tune learning rate
3. tune sampling parameters, such as `subsample` and `colsample_bytree`
4. tune regularization parameters, such as `reg_lambda` and `reg_alpha`

This workflow is not guaranteed to find the global best combination, but it is usually much more practical than running one huge exhaustive grid search.

11.6.3 Early Stopping

Boosting may eventually overfit if too many trees are added. Early stopping helps prevent this by monitoring validation performance during training.

In principle, `n_estimators` can be tuned together with other hyperparameters using grid search or similar methods. In practice, however, it is often determined separately through early stopping.

Conceptually, the procedure is simple:

1. Split the training data into a training part and a validation part.
2. Fit trees sequentially.
3. Monitor the validation error after each boosting round.
4. Stop when the validation performance no longer improves.

In XGBoost, this is commonly implemented using an evaluation set together with `early_stopping_rounds`. For example, `early_stopping_rounds=20` means that training stops if the validation score does not improve for 20 consecutive boosting rounds.


```
xgb = XGBRegressor(early_stopping_rounds=20)
xgb.fit(X_train, y_train, eval_set=[(X_val, y_val)])
```

Exact early-stopping syntax can differ across XGBoost versions, so it is useful to check the installed version when writing production code. The current syntax comes from [XGBoost 3.2.0](#).

i Modern hyperparameter tuning

In modern machine learning practice, workflows often use tools such as **Optuna** and various **AutoML** frameworks for hyperparameter optimization.

Unlike **GridSearchCV**, which exhaustively searches a fixed parameter grid, these methods use adaptive search strategies to explore promising hyperparameter regions more efficiently. This is especially useful for large models such as **XGBoost**, **LightGBM**, and neural networks, where exhaustive grid search may become computationally expensive.

Many modern tuning frameworks are related to ideas from Bayesian optimization. In practice, these methods often attempt to use information from previous trials to guide future searches toward more promising regions of the parameter space.

We only briefly mention these topics here since they have become mainstream tools in modern machine learning workflows:

- Bayesian optimization
- TPE (Tree-structured Parzen Estimator)
- acquisition functions
- Gaussian process optimization
- AutoML systems

A full discussion of topics is beyond the scope of this course.

11.7 Example: XGBoost on the Diabetes Data

We now use the diabetes regression dataset from **sklearn** introduced earlier. The response is a quantitative measure of disease progression one year after baseline.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split

X, y = load_diabetes(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

In general you should not touch test sets. However in this example for simplicity we abuse it by using test sets as validation sets.

11.7.1 Baseline model

We first build a baseline model, with all default hyperparameters.

```
from xgboost import XGBRegressor

xgb_baseline = XGBRegressor(random_state=1, n_jobs=-1)
xgb_baseline.fit(X_train, y_train)
xgb_baseline.score(X_test, y_test)
```

0.12475668653462724

11.7.2 RandomSearchCV

The first round we use random search to try to find a good enough point. Note that we choose a relatively large `n_estimators` which we will tune in the next early stopping stage.

```
from scipy.stats import randint, uniform
from sklearn.model_selection import RandomizedSearchCV

param_distributions = {
    "max_depth": randint(2, 8),
    "learning_rate": uniform(0.01, 0.19),
    "subsample": uniform(0.6, 0.4),
    "colsample_bytree": uniform(0.6, 0.4),
    "reg_lambda": uniform(0, 10),
}

random_search = RandomizedSearchCV(
    XGBRegressor(n_estimators=2000, random_state=1, n_jobs=1),
    param_distributions=param_distributions,
    n_iter=20,
    cv=5,
    random_state=0,
    n_jobs=-1,
)
```

```
random_search.fit(X_train, y_train)
random_search.best_params_
```

```
{'colsample_bytree': np.float64(0.8650107467800177),
 'learning_rate': np.float64(0.012578610766300869),
 'max_depth': 3,
 'reg_lambda': np.float64(8.379449074988038),
 'subsample': np.float64(0.6384393631575852)}
```

The purpose of this random search is to find a rough range of the best combination in order for the refined grid search. According to the result, we may construct a smaller range for refined grid search.

```
param_grid_refined = {
    "learning_rate": [0.008, 0.0125, 0.018],
    "max_depth": [2, 3, 4],
    "subsample": [0.55, 0.64, 0.72],
    "colsample_bytree": [0.80, 0.87, 0.95],
    "reg_lambda": [5.0, 8.5, 12.0],
}
```

11.7.3 Early stopping

Before we run the refined grid search, we need to determine the `n_estimators` using early stopping.

```
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=0
)
xgb_es = XGBRegressor(
    n_estimators=3000,
    **random_search.best_params_,
    early_stopping_rounds=50,
    random_state=0,
    n_jobs=-1,
)
xgb_es.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=False)
best_round = xgb_es.best_iteration + 1
best_round
```

11.7.4 Refined grid search

Now we can run refined grid search using `n_estimators=best_round` (which is 281 in this example).

```
from sklearn.model_selection import GridSearchCV

xgb_base = XGBRegressor(n_estimators=best_round, random_state=0, n_jobs=1)

param_grid_refined = {
    "learning_rate": [0.008, 0.0125, 0.018],
    "max_depth": [2, 3, 4],
    "subsample": [0.55, 0.64, 0.72],
    "colsample_bytree": [0.80, 0.87, 0.95],
    "reg_lambda": [5.0, 8.5, 12.0],
}

grid_search = GridSearchCV(xgb_base, param_grid=param_grid_refined, cv=5, n_jobs=-1)
grid_search.fit(X_train, y_train)
```

Sometimes a full refined grid search is still computationally expensive. In this case, a common practical strategy is to tune the hyperparameters in stages.

i Stage 1: tune tree complexity

```
xgb_base = XGBRegressor(n_estimators=best_round, random_state=0, n_jobs=1)

param_grid_1 = {"max_depth": [1, 2, 3]}

grid_1 = GridSearchCV(xgb_base, param_grid=param_grid_1, cv=5, n_jobs=-1)

grid_1.fit(X_train, y_train)
grid_1.best_params_

{'max_depth': 1}
```

i Stage 2: tune learning rate around the selected number of trees

```
xgb_stage2 = XGBRegressor(  
    n_estimators=best_round, random_state=0, n_jobs=1, **grid_1.best_params_  
)  
  
param_grid_2 = {"learning_rate": [0.035, 0.05, 0.07]}  
  
grid_2 = GridSearchCV(xgb_stage2, param_grid=param_grid_2, cv=5, n_jobs=-1)  
  
grid_2.fit(X_train, y_train)  
grid_2.best_params_  
  
{'learning_rate': 0.07}
```

i Stage 3: tune sampling parameters

```
xgb_stage3 = XGBRegressor(  
    n_estimators=best_round,  
    random_state=0,  
    n_jobs=1,  
    **grid_1.best_params_,  
    **grid_2.best_params_,  
)  
  
param_grid_3 = {"subsample": [0.60, 0.66, 0.75], "colsample_bytree": [0.85, 0.90, 0.95]}  
  
grid_3 = GridSearchCV(xgb_stage3, param_grid=param_grid_3, cv=5, n_jobs=-1)  
  
grid_3.fit(X_train, y_train)  
grid_3.best_params_  
  
{'colsample_bytree': 0.85, 'subsample': 0.75}
```

i Stage 4: tune regularization

```
xgb_stage4 = XGBRegressor(  
    n_estimators=best_round,  
    random_state=0,  
    n_jobs=1,  
    **grid_1.best_params_,  
    **grid_2.best_params_,  
    **grid_3.best_params_,  
)  
  
param_grid_4 = {"reg_lambda": [2.0, 3.25, 5.0]}  
  
grid_4 = GridSearchCV(xgb_stage4, param_grid=param_grid_4, cv=5, n_jobs=-1)  
  
grid_4.fit(X_train, y_train)  
grid_4.best_params_  
  
{'reg_lambda': 2.0}
```

Finally we put all hyperparameters together for a final training.

```
best_params = {  
    **grid_1.best_params_,  
    **grid_2.best_params_,  
    **grid_3.best_params_,  
    **grid_4.best_params_,  
}  
  
final_model = XGBRegressor(  
    n_estimators=best_round, random_state=0, n_jobs=-1, **best_params  
)  
  
final_model.fit(X_train, y_train)  
final_model.score(X_test, y_test)
```

0.415029538595333

It doesn't have to be exactly the same as the full grid search result. However the result will be very similar.

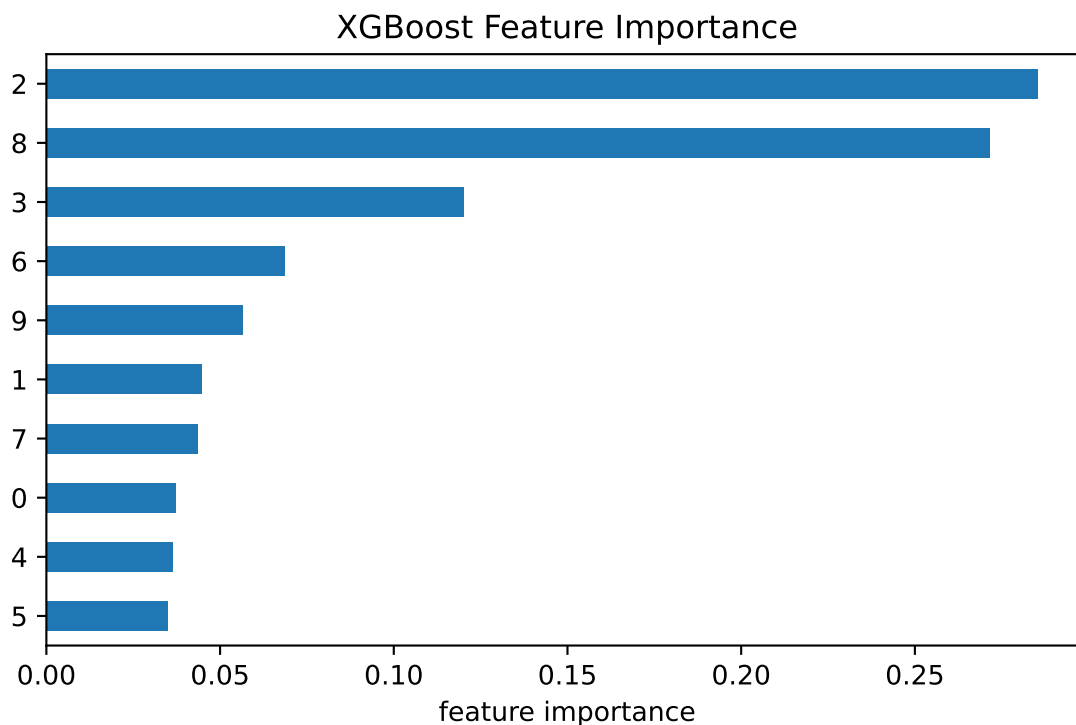
11.7.5 Feature Importance

XGBoost provides impurity-like feature importance through `.feature_importances_`. Conceptually, this is similar to MDI in Random Forests since both measure how much a feature improves the training objective during tree construction.

However, they are not exactly the same. In Random Forests, MDI is based on reductions in RSS or Gini impurity, while in XGBoost the gain is computed from reductions in the regularized boosting objective.

```
import matplotlib.pyplot as plt
importance = pd.Series(final_model.feature_importances_).sort_values(ascending=True)

plt.figure(figsize=(7, 4))
importance.plot(kind="barh")
plt.xlabel("feature importance")
plt.title("XGBoost Feature Importance");
```



Feature importance describes how much the fitted model uses each predictor for prediction. It should not be interpreted as a causal effect.

A Python IDE Setup

There are many ways to set up a Python development environment, and each has its own advantages and disadvantages. In this tutorial, we will cover one of the most popular current setups: VS Code + uv.

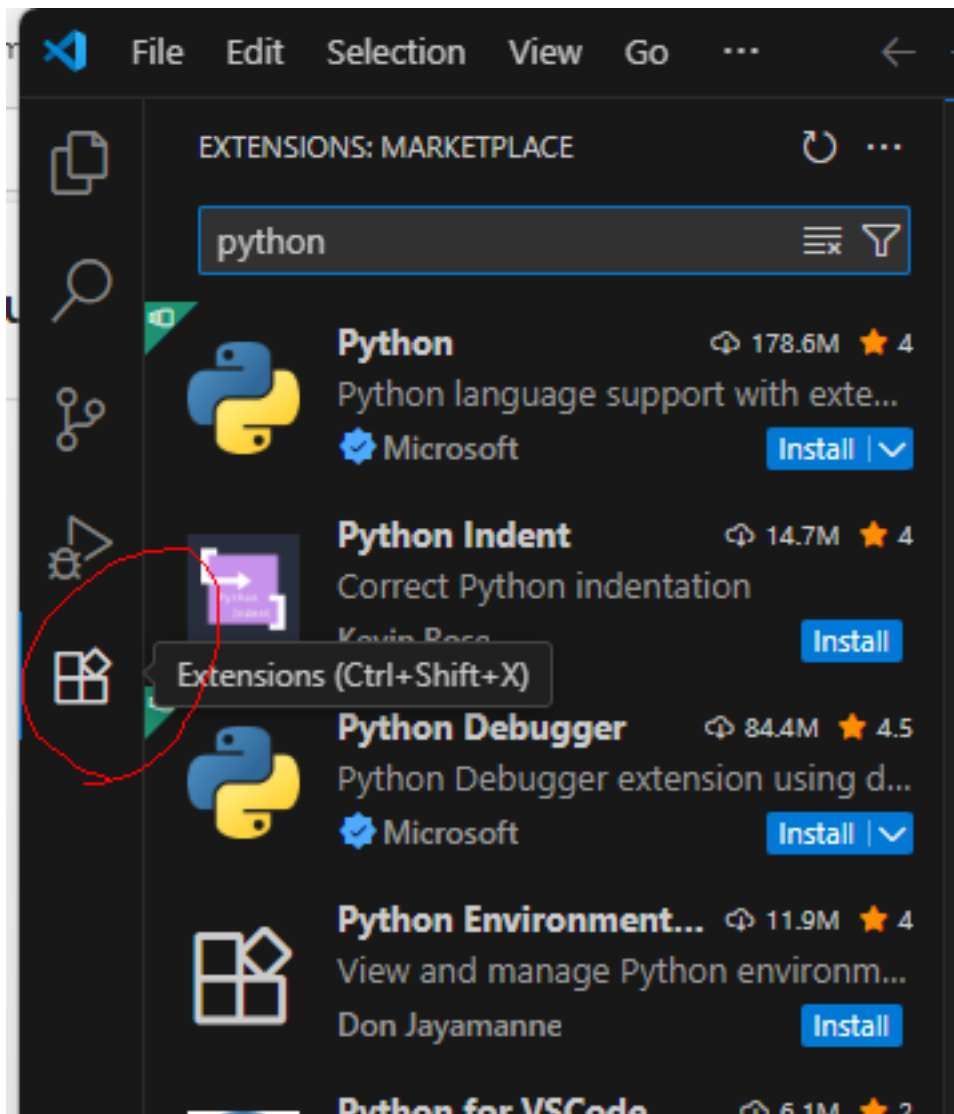
For now, we will focus only on the practical setup steps. The underlying concepts will be discussed in the later appendices.

This tutorial uses Windows 11. If you use another operating system, the steps are very similar, but you may need to make some minor but straightforward modifications. If you encounter major issues, please contact me.

A.1 VS Code

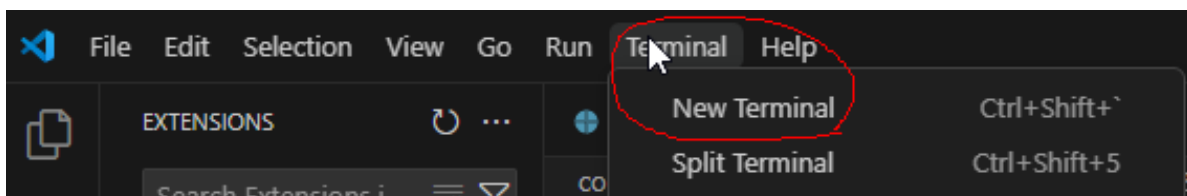
We choose VS Code as our IDE for Python. The installation is straightforward starting from the [Official website](#). After installation, we need to do a few configurations to make the experience better.

1. For development of Python, the essential plugins are the Python plugin and Jupyter plugin. You only need to install the two main ones, and all related plugins will be installed automatically.

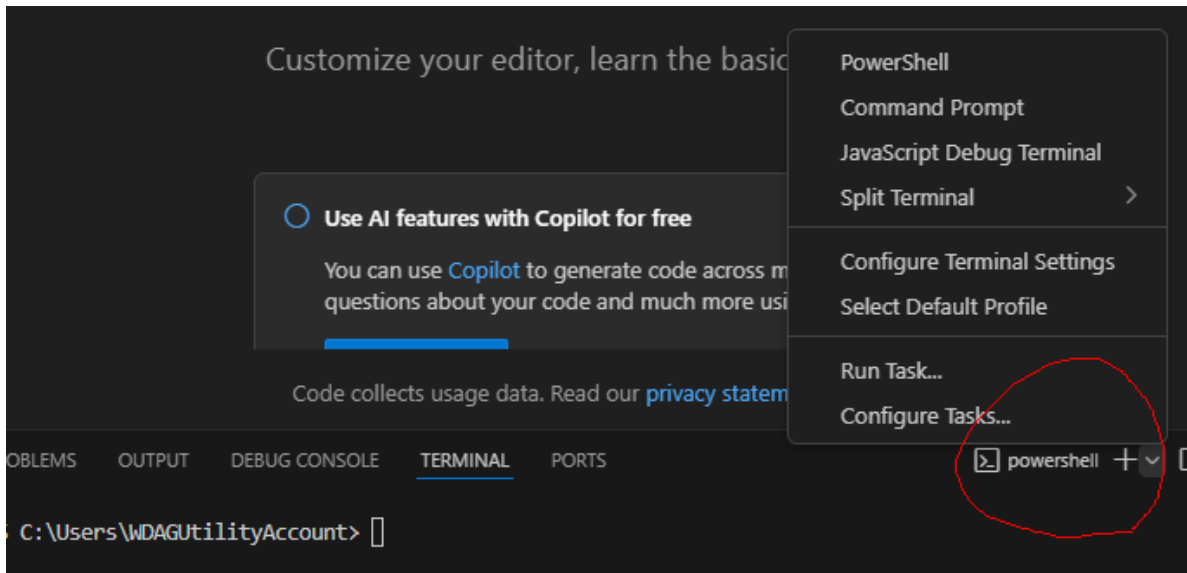


You may either search the extensions directly from VS Code extension tab (which is usually located on the left side bar), or you may install it from the link [python](#) and [jupyter](#).

2. Sometimes you may need to use the terminal inside VS Code. You may start the terminal from top menu (whose hotkey is by default `ctrl+shift+``).



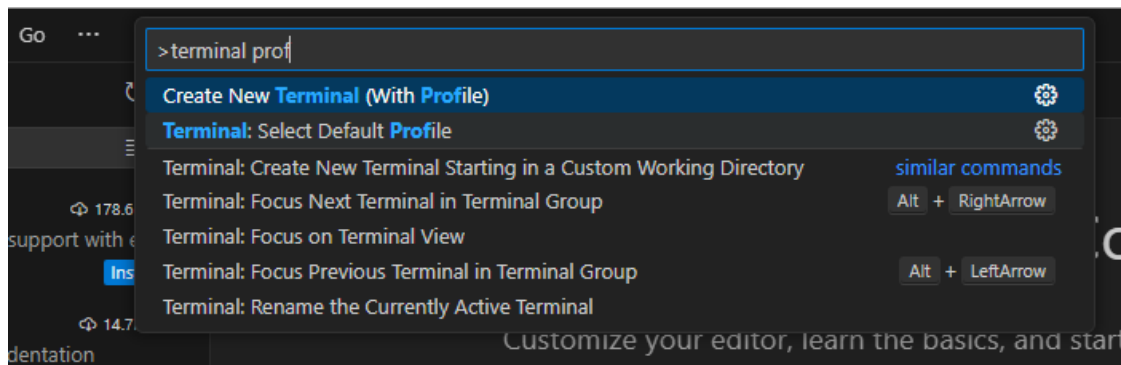
Note that there are many different choices of terminals. After you start the first default terminal, you may start any new terminals using the small + and the v mark on the right.



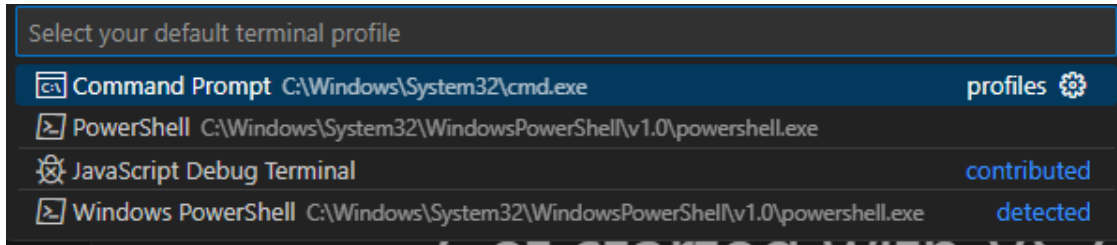
🔥 Change Terminal default profile

In Windows the default terminal is Powershell when you first install VS Code. In the past it had bugs with `conda` (and I don't know whether it is fixed now). In order to avoid headaches due to the bugs, we simply switch to other terminals, like Command Prompt. You may change the default terminal by

1. Press **Ctrl+Shift+P** or **F1** to open the top prompt menu.
2. Find the **Terminal: Select Default Profile** command.



3. Select the desired terminal as the default one.



A.2 Python Virtual Environment installation

There are many tools available for managing Python environments and packages. As of Summer 2026, `uv` has become one of the most popular choices.

`uv` is an all-in-one tool for managing Python installations, virtual environments, and project dependencies. With `uv`, you do not need to install Python separately, as it can download and manage Python versions for you. You may visit the [official documentation](#) to install `uv`.

Windows headache

There is a possibility that Smart App Control (SAC) or other Application Control policies in Windows 11 may block the execution of `uv.exe`. In this case, you may see a message similar to

```
'C:\Users\WDAGUtilityAccount\.local\bin\uv.exe'  
was blocked by your organization's Device Guard policy.  
Contact your support person for more info.
```

This may occur because Windows does not yet consider the executable sufficiently trusted under its application-control policies.

If this happens, note that the exact cause and solution can vary across machines, Windows versions, and security configurations. In addition, both Windows and `uv` are evolving rapidly, so solutions that work today may change in the future.

You are encouraged to search for the solution that best fits your situation. For this course, one workaround is to use WSL (see Section [A.4](#)).

After installation (and you may need to restart VS Code or open a new terminal window so that the `uv` command becomes available), you can use the following commands to verify that `uv` is working correctly.

Command	Purpose	Example Output
<code>uv --version</code>	Display the installed <code>uv</code> version. Useful for verifying that <code>uv</code> is installed correctly.	<code>uv 0.11.17</code>
<code>uv self update</code>	Update <code>uv</code> itself to the latest available version. Similar to updating package managers such as <code>pip</code> or <code>cargo</code> .	Upgraded uv from v0.11.4 to v0.11.17!
<code>uv python list</code>	Show all Python versions available locally and those that can be installed through <code>uv</code> .	Lists installed and downloadable Python versions.

A.2.1 Install the first Python

Since VS Code often expects to discover at least one Python interpreter before it can fully initialize its Python and Jupyter services, we first install a Python version managed by `uv`.

```
uv python install
```

This Python installation is managed entirely by `uv`; it is not intended to be used directly for course work. Instead, it serves as a bootstrap interpreter that allows VS Code to detect Python correctly and manage project-specific virtual environments more reliably. You can verify the installed Python versions using `uv python list` as mentioned above.

If you skip this step and directly jump into the virtual environment, there is a possibility that Python in VS Code doesn't work well.

A.2.2 Setup the environment for the course

To set up the environment for this course, we provide the [toml file](#) and [python version file](#) for this course.

```
[project]
name = "STAT432"
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    "jupyter>=1.1.1",
    "matplotlib>=3.10.9",
```

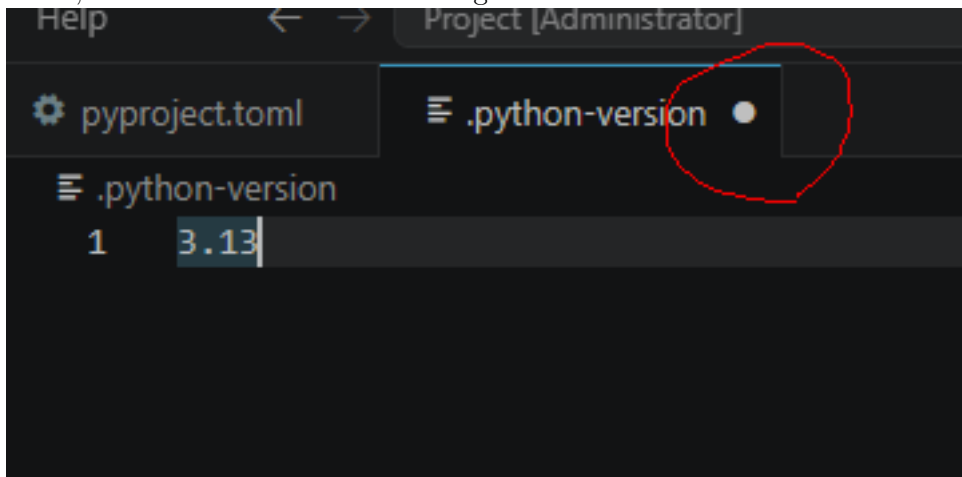
```
"pandas>=3.0.2",  
"scikit-learn>=1.8.0",  
"seaborn>=0.13.2",  
"statsmodels>=0.14.6",  
"xgboost>=3.2.0",  
]
```

The python version file only contains the version number 3.13 inside since this is the version of Python I use when I wrote the notes. You may either download the file and put it inside the folder or create a file with the name `.python-version` and write 3.13 inside.

⚠ Autosave

Before proceeding, make sure the file is saved.

By default, VS Code does not enable Auto Save. If you see a small dot next to the file name, the file contains unsaved changes.



Many beginner problems occur because they modify a file but forget to save it before running commands.

1. Create a new folder. The folder name is irrelevant. Put the files `pyproject.toml` and `.python-version` in the folder.
2. Open a terminal (no matter whether within VS Code or not) and enter the folder. Use the command `uv sync`.

After the command finishes, `uv` creates a virtual environment named `.venv` and installs all required packages specified in `pyproject.toml`. The resulting environment should closely match the one used to prepare these notes.

Now it is possible to start using the virtual environment for the course. In case that you need to modify the virtual environment, please continue to read the following optional part.

i Modifying the Environment

To install a package and update the project configuration, use `uv add <package-name>`. For example, you may need `openpyxl` to read and write Excel files:

```
uv add openpyxl
```

This command updates the project configuration in `pyproject.toml` and installs the package into the project's virtual environment `.venv`.

Most package management tasks can be handled through `uv add`.

i Jupyter kernels and modifying venv

A Jupyter kernel is the Python environment used by a notebook. When a notebook cell is executed, the code runs inside the selected kernel. Therefore, it is important to connect the notebook to the kernel associated with your course environment.

Modern versions of VS Code can usually detect a project managed by `uv` and use it directly as a notebook kernel. In many cases, no manual kernel registration is required. If VS Code does not automatically detect the environment, you can create a dedicated Jupyter kernel:

```
uv run python -m ipykernel install --user --name stat432 --display-name "(STAT432)"
```

The option `--name stat432` specifies the kernel identifier. In this example the identifier is `stat432`, but you may choose any name that is easy for you to recognize. This identifier is used internally by Jupyter and should be unique on your system.

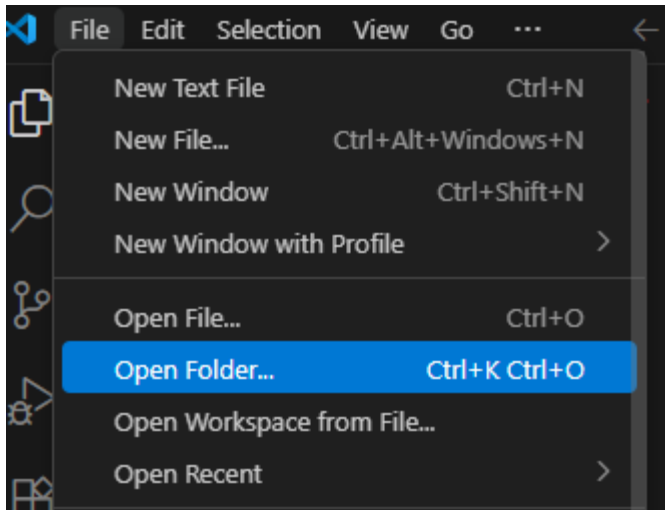
The option `--display-name "(STAT432)"` specifies the name shown in VS Code and Jupyter when selecting a kernel. Unlike the identifier, the display name does not need to be unique, although a descriptive name is recommended.

A.3 Back to VS Code and start Jupyter notebook

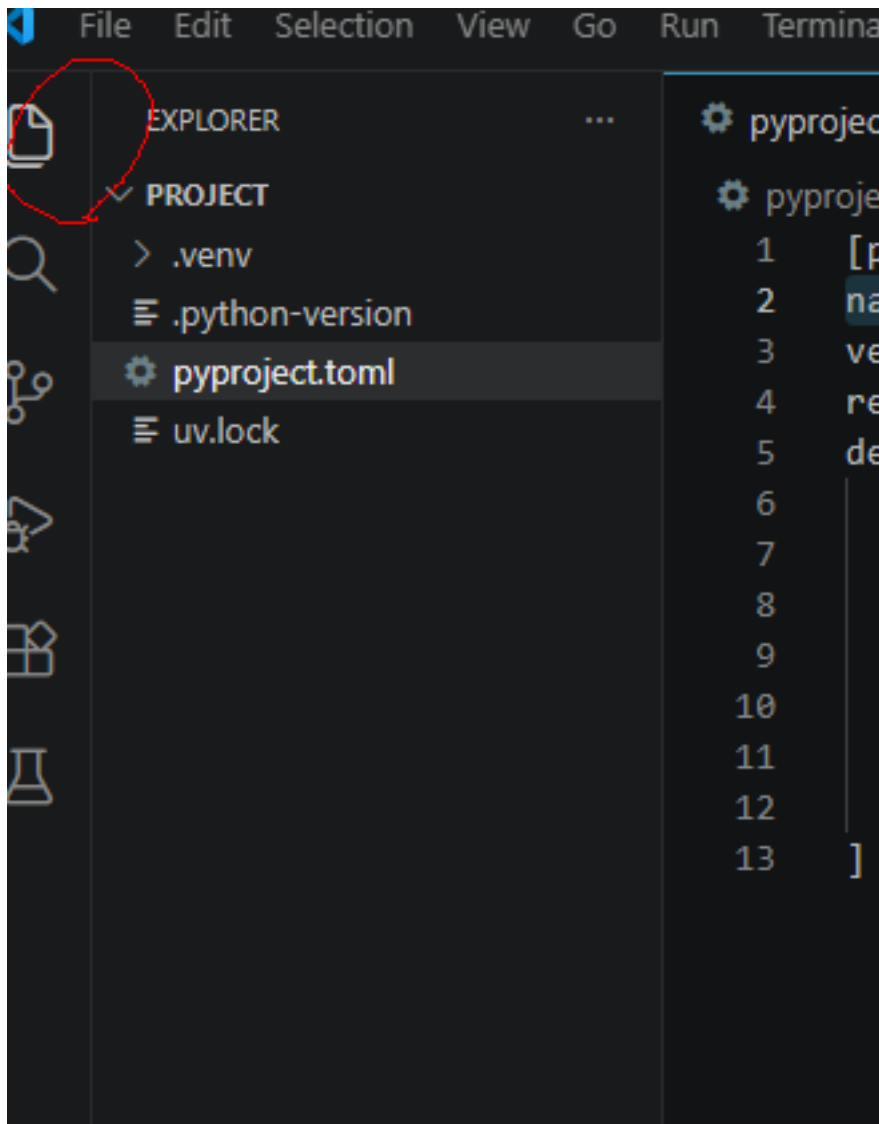
After Python is installed and virtual environment is set up, we could start to do a Hello World project.

The way VS Code organizes projects is through folders by default. So we first start a new folder (called the working folder) and all project related files should be put inside this folder. (There are ways to work with files outside the working folder, but you don't need it in most cases.)

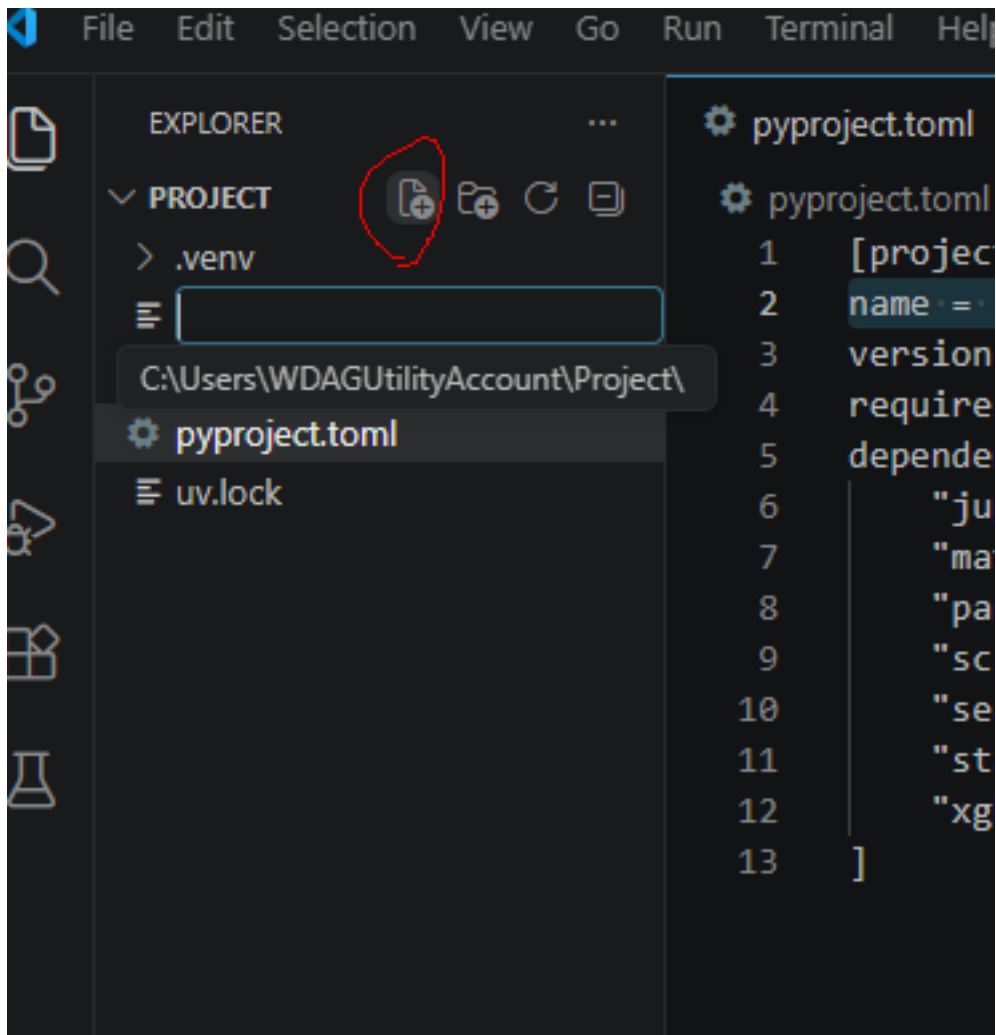
Since we already set up a folder (with a virtual environment), we open it as our working folder.



The folder already contains the project files and the environment files. You may see them from the file explorer.

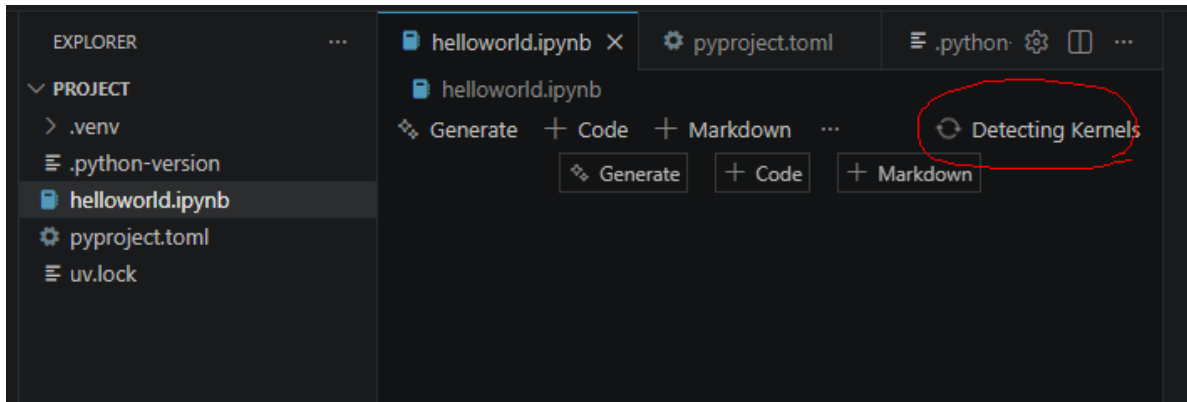


Then you may create new files/folders or copy files/folders into this working folder. For demonstration a new file called `helloworld.ipynb` is created.

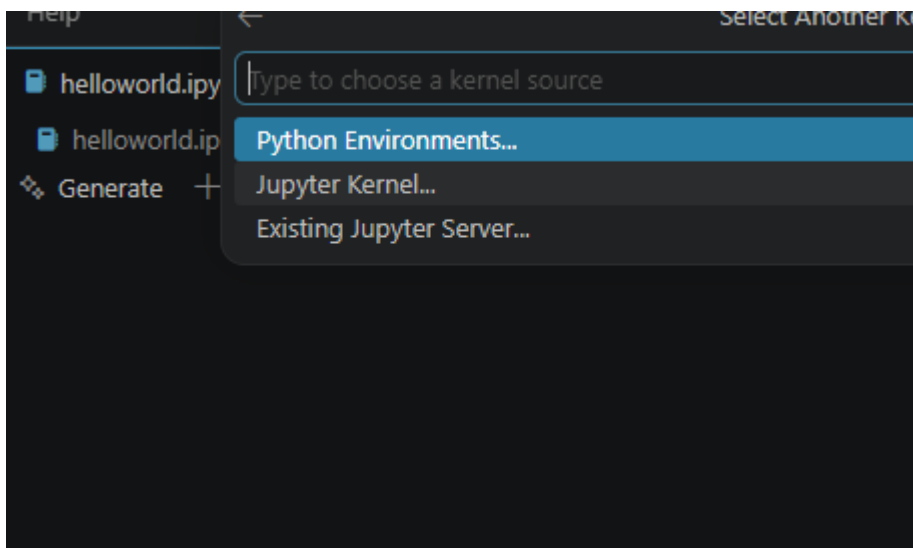


ipynb means this is a Jupyter notebook file (which is short for interactive Python notebook). This is the format for this course's homework assignment.

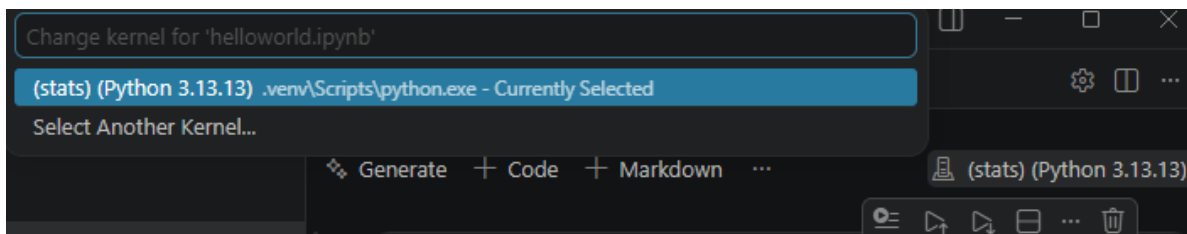
This is the appearance of an empty notebook file.



We attach the kernel we just created to this notebook. Click the **Select Kernel / Detecting Kernels** on the right upper corner and select the environment we want.

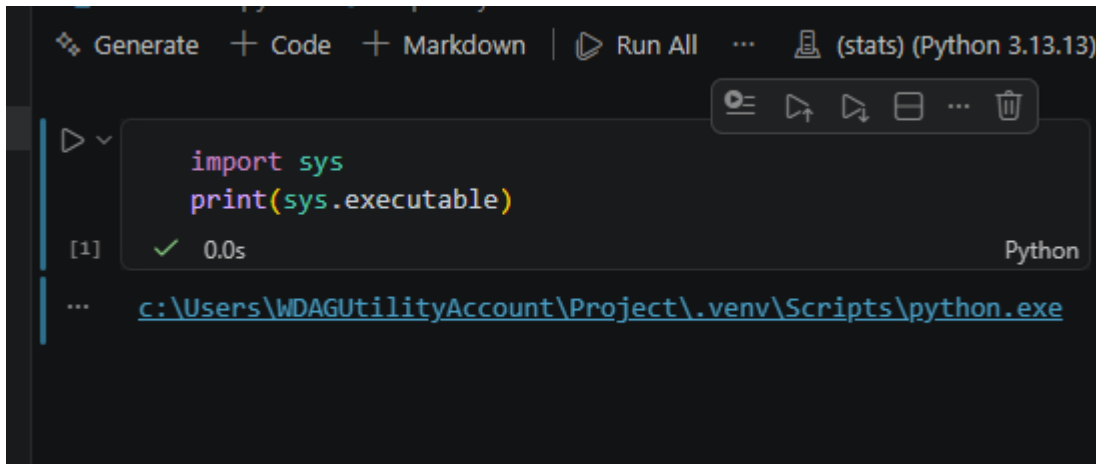


- If you created a new Jupyter kernel, you can find it from **Jupyter kernel**....
- If not, you can find your virtual environment in the working folder from **Python Environments**....



After the kernel is selected, we could start using the notebook. I prefer to use the following code as hello world. It can also tell us whether the correct python is used.

```
import sys
print(sys.executable)
```



A.3.1 Output to html and pdf

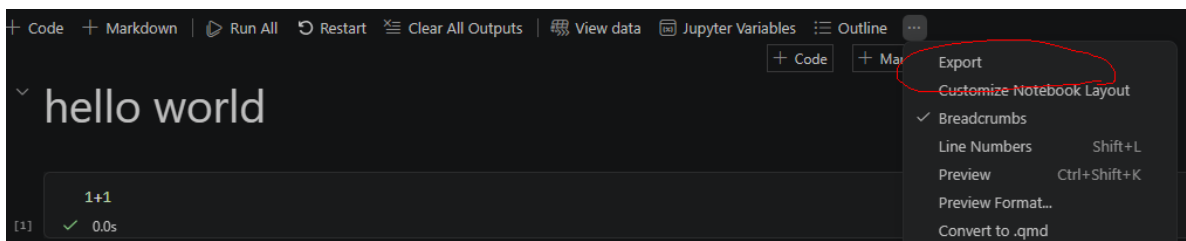
After finishing your Jupyter Notebook, you need to generate a PDF version for assignment submission.

Since we do not want to spend too much time on document formatting, we will use a simple workflow. Although there are many more sophisticated tools available (such as Pandoc, Quarto, and LaTeX-based workflows), the approach below is completely sufficient for this course.

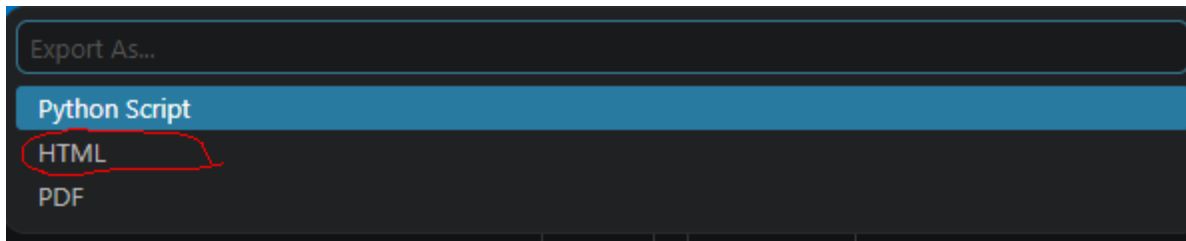
Our workflow is:

0. Restart the kernel and run all.
1. Export the notebook to HTML using VS Code.
2. Open the HTML file in a web browser.
3. Print the page to a PDF file.

First, locate the notebook toolbar and click the ... button on the right side. From the expanded menu, select **Export**.



Then choose HTML.



This will create an HTML file. Open the file in your preferred web browser and print it to PDF. Most modern browsers (such as Chrome, Edge, and Firefox) support printing directly to PDF.

For assignment submission, submit both the PDF file and the notebook file. Grading will be based primarily on the PDF file, while the notebook file serves as a backup source in case I need to inspect the code or verify specific parts of the submission.

i Restart and Run All

A Jupyter notebook keeps its internal state in memory while you work. As a result, cells can be executed in different orders, making it possible for the notebook contents and the actual execution state to become inconsistent.

For example, a variable may exist because it was created in an earlier session, even though the cell that creates it has been deleted and no longer appears before its first use in the notebook. Such notebooks may appear to run correctly during editing but fail when executed from scratch.

To avoid these issues, you are required to restart the kernel and run all cells before creating your submission PDF. Any errors that occur during this process must be fixed before submission.

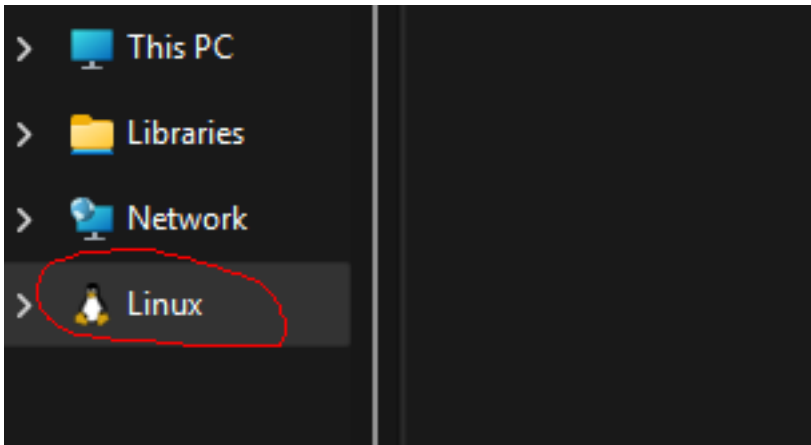
Although notebook workflows have several well-known limitations, Jupyter notebooks remain the dominant platform for data science, machine learning, and scientific computing. New alternatives such as **marimo** have attracted considerable attention in recent years, but they are still relatively young and may require several more years to reach the level of adoption enjoyed by Jupyter — if they ever do.

A.4 WSL

WSL stands for Windows subsystem for Linux. It allows you to run a Linux environment directly inside Windows without installing a separate operating system.

To install WSL, please follow the [official Microsoft guide](#).

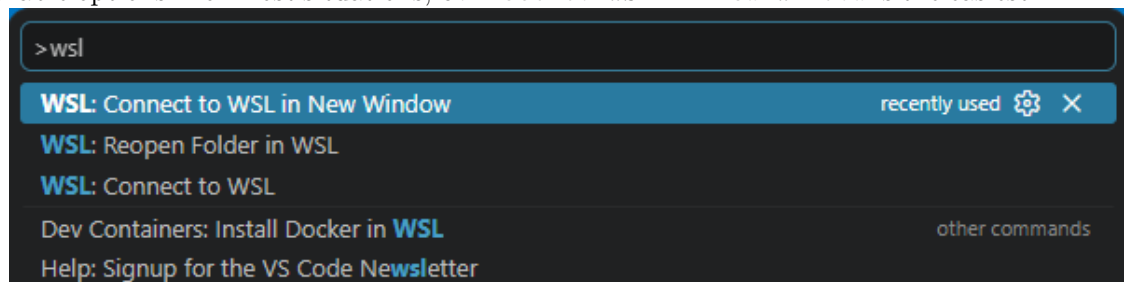
After installation, you will have a Linux system (usually Ubuntu) running alongside Windows. The Linux file system is separate from the Windows file system, although you can access it through File Explorer to copy / move files back and forth.



VS Code provides excellent support for WSL through the Remote WSL extension. To open a WSL window, you may:

- Click the button in the lower-left corner of VS Code 
- Or open the Command Palette (**Ctrl+Shift+P** or **F1**), search for WSL, and select one of the available options. For most situations, **Connect to WSL in New Window** is the easiest

choice.



Once connected to WSL, you are working inside a Linux environment rather than Windows.

Because WSL is a separate system, you will need to:

- Open your project folder again.
- Install **uv**, Python and other tools inside WSL.
- Create virtual environments inside WSL.
- Install packages inside WSL.

You may follow the same instructions presented earlier to install **uv** and set up your Python environment. The overall process is the same, although the commands will be the Linux versions rather than the Windows versions.

Note that if you install `uv` in Windows, it is not automatically available inside WSL. Similarly, software installed inside WSL is generally not available in Windows.

In practice, it is usually easiest to keep an entire project inside WSL and perform all Python-related work there.

 Tip

Avoid storing virtual environments inside the Windows file system when working primarily in WSL. For best performance and compatibility, create your project folders inside your Linux home directory (for example, `~/Project`) rather than in mounted Windows directories (for example, `/mnt/c/...`).

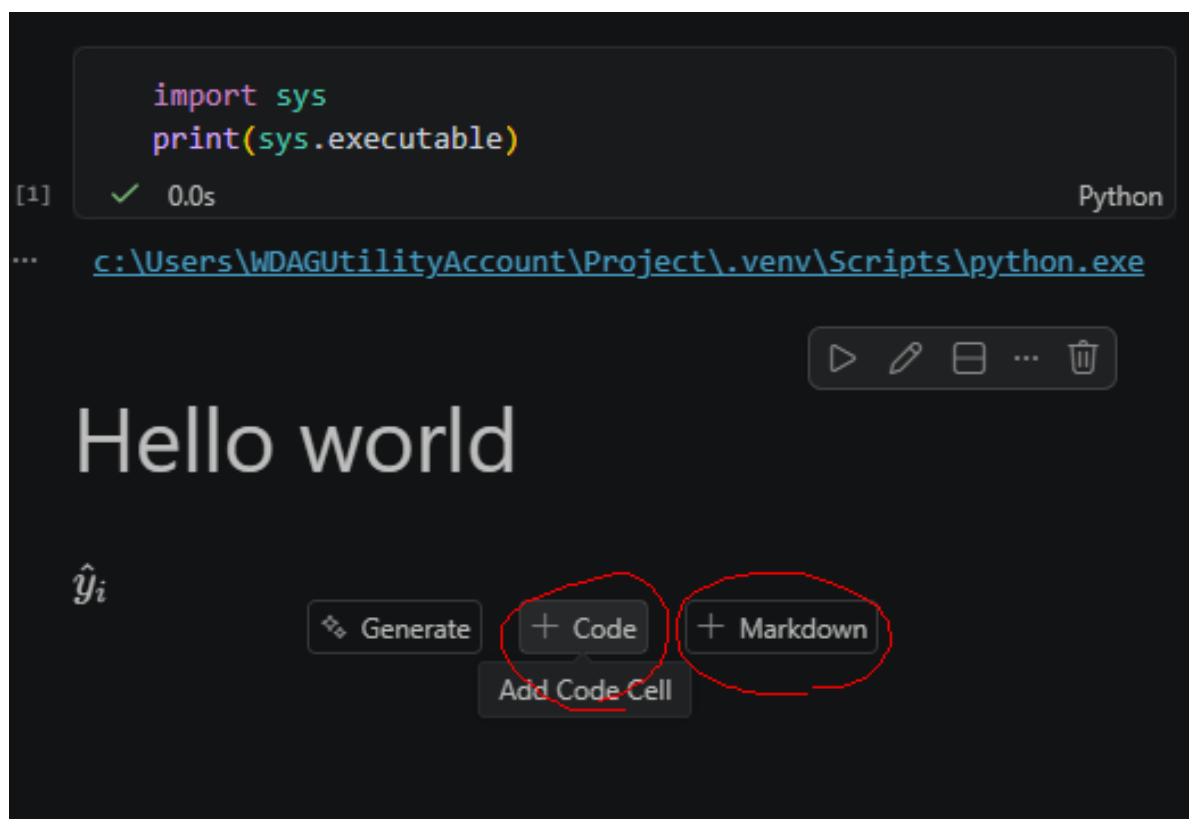
B Jupyter Notebook and Markdown

One of the best features of a Jupyter Notebook is that it can combine code, narrative text, and output in a single file.

There are two types of cells in a notebook: Code cells and Markdown cells.

- Code cells are used to run Python code.
- Markdown cells are used to write text, explanations, equations, and documentation.

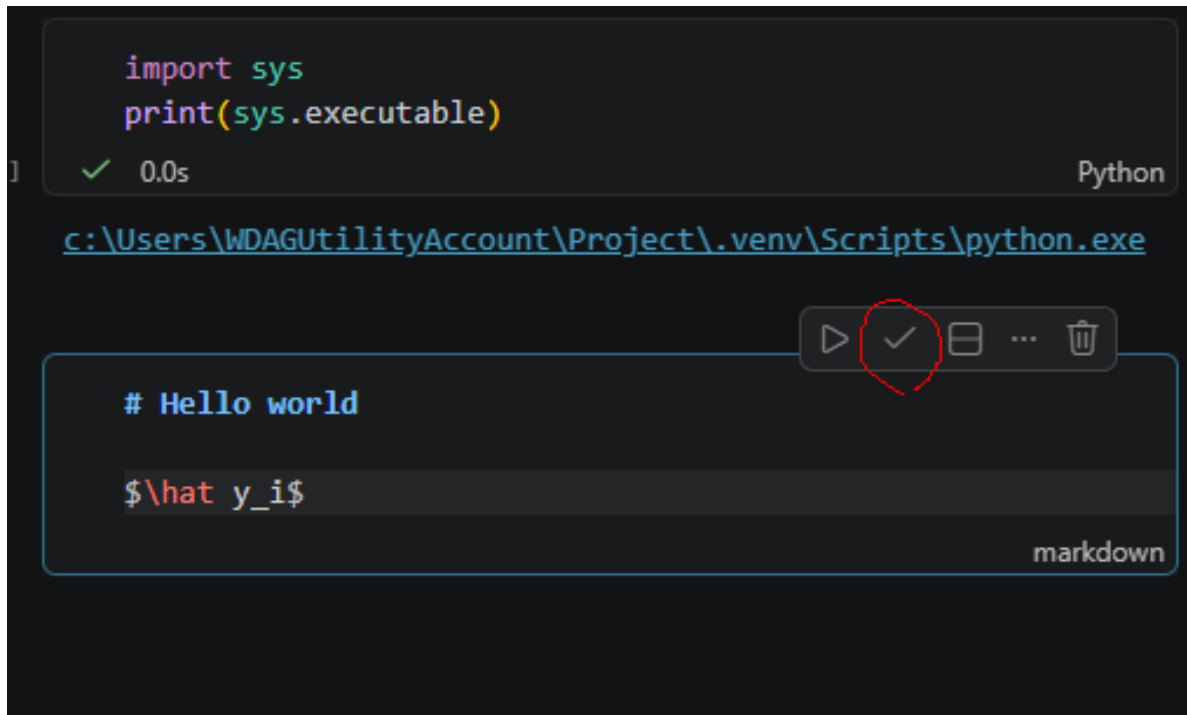
In statistical learning, code cells are typically used for data analysis, model fitting, and computations, while Markdown cells are used to explain the methods, interpret the results, and document the workflow.



A notebook combines code, output, and narrative in a single document, making it an ideal environment for data science and machine learning.

 Tip

You can always click the small **Edit / Stop Editing** button to preview how your Mark-down text will be rendered.




```
import sys
print(sys.executable)
```

[1] ✓ 0.0s Python

... <c:\Users\WDAGUtilityAccount\Project\.venv\Scripts\python.exe>

▶ ✎ 📄 ... 🗑

Hello world

$\hat{y}_i$

Markdown is a lightweight text format. In this course, we will use only the most basic Markdown syntax.

B.1 Text Formatting

Markdown Syntax	Output
<code>*italics*, **bold**, ***bold italics***</code>	<i>italics</i> , bold , <i>bold italics</i>
<code>superscript^2~ / subscript~2~</code>	superscript ² / subscript ₂
<code>~~strikethrough~~</code>	strikethrough
<code>`verbatim code`</code>	verbatim code

B.2 Headings

Markdown Syntax	Output
<code># Heading 1</code>	Heading 1

Markdown Syntax	Output
<code>## Heading 2</code>	Heading 2
<code>### Heading 3</code>	Heading 3

Tip

Note that for heading, there is a space after #.

B.3 Lists

Markdown Syntax	Output
<pre>* unordered list + sub-item 1 + sub-item 2 - sub-sub-item 1</pre>	• unordered list – sub-item 1 – sub-item 2 * sub-sub-item 1
<pre>1. ordered list 2. item 2 i) sub-item 1 A. sub-sub-item 1</pre>	1. ordered list 2. item 2 i) sub-item 1 A. sub-sub-item 1

Tip

Note that for lists, there is a space after ..

B.4 Essential LaTeX

Use single dollar signs for inline math:

The fitted value is $\hat{y}_i$.

The fitted value is $\hat{y}_i$.

Use double dollar signs for displayed equations:

```


$$Y = \beta_0 + \beta_1 X + \epsilon.$$


```

$$Y = \beta_0 + \beta_1 X + \epsilon.$$

Common symbols:

<code>$\hat{y}$</code>	predicted value
<code>$\bar{x}$</code>	sample mean
<code>β_0</code>	parameter
<code>X^T</code>	transpose
<code>$\sum_{i=1}^n$</code>	summation

$\hat{y}$: predicted value

$\bar{x}$: sample mean

β_0 : parameter

X^T : transpose

$\sum_{i=1}^n$: summation

LaTeX supports a wide range of mathematical notation, but the basic symbols introduced here will be enough for all of the work in this course.

C Python Basics for sklearn

This note introduces only the Python syntax needed for the rest of these notes. The goal is not to cover Python as a full programming language. The goal is to make code using `numpy`, `matplotlib`, and `sklearn` readable.

C.1 Running Python Code

A Python code chunk looks like this:

```
# This is a comment

x = 10      # assignment uses =

a, b = 2, 3
a + b      # last expression is automatically displayed
```

5

Notebook

1. When a code cell is executed, the value of the last expression is automatically displayed. Any explicit output commands, such as `print()`, will also produce output.
2. Use `#` comments only for short notes that help explain the code. Detailed explanations and discussion should be written in Markdown cells, where they are easier to read and maintain.

C.1.1 Indentation

Python uses indentation to mark code blocks. R uses braces `{}`; Python uses indentation.

For example, a `for` loop:

```
for k in [1, 3, 5]:  
    print(k)
```

```
1  
3  
5
```

The indented block belongs to the loop. These notes use loops only when they are helpful for plotting or comparing models.

C.1.2 if and for

The logic of **if** statements and **for** loops is straightforward. When using them, pay attention to indentation. A colon `:` indicates the beginning of an indented code block.

```
x = 1  
if x == 2:  
    print('good')  
else:  
    print('not good')
```

```
not good
```

```
for k in range(4):  
    print(k)
```

```
0  
1  
2  
3
```

C.1.3 Functions and Objects

This is not an object-oriented programming course, so don't worry too much about the details. The important idea is the distinction between a function, a class, and an object created from a class.

- Function: performs an action and returns a result. Called with parentheses: `round(3.14, 2)`

- Class: a blueprint for creating objects.
- Object: stores information and provides methods. Accessed with a dot: `object.method()`

Example in scikit-learn:

```
model = LinearRegression()
```

Here

- `LinearRegression` is the class;
- `LinearRegression()` creates a linear regression model object;
- the object is assigned to the variable `model`.

```
model.fit(X_train, y_train)
coef = model.coef_
```

Here

- `fit()` is method;
- `coef_` stores information about the fitted model;
- methods are called using `object.method(...)`;
- stored information is accessed using `object.attribute`.

In almost all `scikit-learn` code, you should work with objects rather than classes. A common mistake is to pass a class where an object is required.

```
pipe = make_pipeline(StandardScaler, LinearRegression) # wrong
pipe = make_pipeline(StandardScaler(), LinearRegression()) # correct
```

The same idea applies to most `scikit-learn` components, including transformers, models, and pipelines. In general, if something is used as a step in a pipeline, you should create an object by adding `()`.

. (Dot)

The meaning of `.` is different in Python and R.

- In Python, `.` is used to access methods and attributes of an object.

```
model.fit(X, y)
model.coef_
```

- In R, `.` is usually just part of a variable or function name.

```
my.variable <- 1
```

Do not interpret dots in R names as object access.

C.1.4 Importing Packages

Python packages are loaded with `import`.

The usual imports in these notes are:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

The names on the right are aliases:

- `numpy` is used as `np`;
- `pandas` is used as `pd`;
- `matplotlib.pyplot` is used as `plt`.

After importing a package this way, you access its functions using the package name, e.g.

- `np.array([1, 2, 3])`
- `pd.DataFrame({"x": [1, 2, 3]})`
- `plt.plot([1, 2, 3], [4, 5, 6])`

Note

When a package is imported with `import ... as ...`, you must use the alias to access its functions and classes instead of the original name.

Sometimes only specific functions or classes are imported.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

These objects can then be used directly.

```
model = LinearRegression()
```

Note

After `from sklearn.linear_model import LinearRegression`, you may use `LinearRegression()` directly, but other objects from `sklearn.linear_model` are not automatically imported.

C.2 Data structure

C.2.1 Basic Objects

Python has several basic data types.

```
x = 3          # integer
y = 2.5        # float
name = "tree"  # string
flag = True    # Boolean
```

Use `type()` to check the type of an object.

```
type(y)
```

```
float
```

Python is case-sensitive. `x` and `X` are different objects.

C.2.2 Lists

A list is an **ordered collection** of objects. Lists are commonly used to store variable names, column names, or collections of parameter values.

Purpose	Example	Explanation
Create a list	<code>features = ["age", "income", "education"]</code>	Creates a list containing three strings.
Access the first element	<code>features[0]</code>	Returns <code>"age"</code> . Python indexing starts at 0.
Access the last element	<code>features[-1]</code>	Returns <code>"education"</code> . Negative indices count from the end.

Purpose	Example	Explanation
Select several elements	<code>features[0:2]</code>	Returns <code>["age", "income"]</code> . The stop index is not included.
Add an element	<code>features.append("gender")</code>	Adds <code>"gender"</code> to the end of the list.
Count elements	<code>len(features)</code>	Returns the number of elements in the list.

i Note

Unlike R, Python uses zero-based indexing, so the first element has index 0.

C.2.3 Dictionaries

A dictionary stores **key-value pairs**. It is similar to a named list in R and is commonly used to store settings, options, and model parameters.

Purpose	Example	Explanation
Create a dictionary	<code>params = {"n_neighbors": 5, "weights": "uniform"}</code>	Creates a dictionary with two keys and their values.
Access a value	<code>params["n_neighbors"]</code>	Returns 5.
Add or update a value	<code>params["metric"] = "euclidean"</code>	Adds a new key-value pair or updates an existing one.
Count entries	<code>len(params)</code>	Returns the number of key-value pairs.
Store model settings	<code>{"max_depth": 3, "min_samples_leaf": 5}</code>	A common use in machine learning.
Store parameter grids	<code>{"max_depth": [2,3,4]}</code>	Often used when tuning hyperparameters.

A typical parameter grid in `scikit-learn` looks like:

```
param_grid = {
    "max_depth": [2, 3, 4],
    "min_samples_leaf": [1, 5, 10],
}
```

For pipelines, parameter names often use two underscores:

```
param_grid = {  
    "model__C": [0.1, 1, 10],  
}
```

i Note

In a pipeline, a parameter name of the form `step__parameter` refers to a parameter inside a specific pipeline step. For example, `model__C` means the parameter `C` of the pipeline step named `"model"`. More terminologies will be discussed in related sections.

C.3 `numpy.array`

The most important data structure for numerical computation in Python is `numpy.array`.

```
import numpy as np  
  
x = np.array([1, 2, 3, 4])  
x
```

```
array([1, 2, 3, 4])
```

Arrays have a shape and a dimension.

```
print(x.shape)  
print(x.ndim)
```

```
(4,)  
1
```

So `x` is a 1D array containing 4 elements.

C.3.1 2D Arrays

A 2D array is similar to a matrix.

```
X = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])

X
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

```
print(X.shape)
print(X.ndim)
```

```
(3, 2)
2
```

Here `X` is a 2D array with 3 rows and 2 columns.

C.3.2 Indexing

Use `[row, column]` indexing.

```
X[0, 1]      # row 0, column 1
X[:, 0]      # first column
X[1, :]      # second row
```

The symbol `:` means “all indices” along that axis.

Slicing can select ranges.

```
X[0:2, 1]    # first 2 rows, column 1
X[0, 0:2]    # row 0, first 2 columns
X[0:2, 0:2]  # upper-left 2x2 block
```

Note

`X[:, 0]` returns a 1D array with shape `(n,)`, not a column vector.
To obtain a column vector, use

```
X[:, [0]]  
# or  
X[:, 0].reshape(-1, 1)
```

which both have shape `(n, 1)`.

C.3.3 Common Operations

Arrays support vectorized arithmetic.

```
x + 1  
x * 2  
np.mean(x)
```

Operations are applied elementwise.

There are two special operations that is worth mentioning.

Matrix multiplication

There are several ways to perform matrix multiplication in NumPy. Here we introduce the `@` operator because it has the simplest syntax.

```
import numpy as np  
  
X = np.array([[1, 2], [3, 4], [5, 6]])  
y = np.array([1, 2])  
X @ y
```

```
array([ 5, 11, 17])
```

```
X = np.array([[1, 2], [3, 4], [5, 6]])  
y = np.array([1, 2]).reshape(-1, 1)  
X @ y
```

```
array([[ 5],
```

```
[11],  
[17]])
```

Note that the result depends on the dimensions of the inputs. NumPy automatically applies the appropriate matrix multiplication rule based on the shapes of the arrays.

- In the first example, `y` is a one-dimensional array with shape `(2,)`, so the result is also one-dimensional with shape `(3,)`.
- In the second example, `y` is a column vector with shape `(2,1)`, so the result is a two-dimensional array with shape `(3,1)`.

i Broadcasting

Broadcasting is a NumPy mechanism that allows arithmetic operations between arrays of different shapes.

Instead of explicitly copying data, NumPy automatically expands arrays along dimensions of size 1 when the shapes are compatible.

A common example is arithmetic between an array and a scalar:

```
import numpy as np  
  
X = np.array([[1, 2], [3, 4], [5, 6]])  
  
X + 10  
  
array([[11, 12],  
       [13, 14],  
       [15, 16]])
```

The scalar is automatically applied to every element of the array.

Broadcasting can also be used to compute pairwise differences efficiently.

```
import numpy as np  
  
X1 = np.array([[1, 2],  
               [3, 4]])  
  
X2 = np.array([[1, 2],  
               [3, 4]])  
  
d = X1[:, None] - X2[None, :]  
d
```

```
array([[[ 0,  0],
        [-2, -2]],

       [[ 2,  2],
        [ 0,  0]]])
```

`None` inserts a new dimension at the specified location. Therefore `X1[:, None]` has shape (2,1,2) while `X2[None, :]` has shape (1,2,2).

NumPy automatically broadcasts these arrays to shape (2, 2, 2) and computes all pairwise differences at once.

In this example, each row of `X1` is subtracted from each row of `X2`. The result satisfies `d[i, j] = X1[i] - X2[j]` for all valid indices `i` and `j`.

This technique is frequently used when computing distance matrices in machine learning algorithms.

C.3.4 Axis-wise

Many functions accept an `axis=` argument.

```
X          # show X
```

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

Computes the sum across rows:

```
X.sum(axis=0)
```

```
array([ 9, 12])
```

Computes the mean across columns:

```
X.mean(axis=1)
```

```
array([1.5, 3.5, 5.5])
```

A useful rule:

- `axis=0`: operate down rows (one result per column)
- `axis=1`: operate across columns (one result per row)

C.3.5 Reshaping

A 1D array can be converted into a matrix. Consider a 1D array

```
x = np.array([1, 2, 3, 4])
```

To create a column vector:

```
x.reshape(-1, 1)
```

```
array([[1],  
       [2],  
       [3],  
       [4]])
```

To create a row vector:

```
x.reshape(1, -1)
```

```
array([[1, 2, 3, 4]])
```

The value -1 means “infer this dimension automatically.”

A common pattern is

```
test_x = np.linspace(0, 1, 101).reshape(-1, 1)
```

On the other side, to flatten an array back to 1D:

```
X.reshape(-1)
```

```
array([1, 2, 3, 4, 5, 6])
```

NumPy reads entries row by row and produces a one-dimensional array.

Tip

Other flattening methods include `.ravel()` and `.flatten()`. The difference is about how copy-and-view is handled. We mainly use `.reshape(-1)` because it makes the resulting

shape explicit.

i Note

One common source of bugs in statistical learning is using arrays with incorrect shapes. In `scikit-learn`, predictors `X` are usually stored as a 2D array and responses `y` as a 1D array. Other libraries may use different conventions, and the required shapes often depend on the specific model or loss function.

When shape-related errors occur, always check the documentation of the library you are using.

C.3.6 Arrays versus Lists

Lists are general-purpose containers, and its addition concatenates. For example

```
[1, 2, 3] + [4, 5, 6]
```

```
[1, 2, 3, 4, 5, 6]
```

Arrays are designed for numerical computation. Its operations are align with the regular math operations. Therefore for numerical work and machine learning, use `numpy.array` whenever possible.

C.4 Random Numbers

These notes often use `numpy` random number generators for simulation.

```
rng = np.random.default_rng(1)

x = rng.normal(size=5)
x
```

```
array([ 0.34558419,  0.82161814,  0.33043708, -1.30315723,  0.90535587])
```


Note

The number 1 is a seed. It makes the random output reproducible.
On a single machine, if the seed is fixed the results are supposed to be the same.
You may choose any number as the seed. You may use the same seed when you want to see the same result.

Some common random functions are:

```
rng.normal(loc=0, scale=1, size=5)
```

```
array([ 0.44637457, -0.53695324,  0.5811181 ,  0.3645724 ,  0.2941325 ])
```

```
rng.uniform(0, 1, size=5)
```

```
array([0.75351311, 0.53814331, 0.32973172, 0.7884287 , 0.30319483])
```

For a two-dimensional predictor matrix:

```
X = rng.normal(size=(10, 3))  
X.shape
```

```
(10, 3)
```

C.5 pandas

pandas can be treated as a wrapper of `numpy.array` which make it more readable. The complete **pandas** is very complicated. Here we only mention a few very important concepts.

To import **pandas**, in most cases we set **pd** as its alias.

```
import pandas as pd
```

C.5.1 DataFrame and Series

In **pandas**, **DataFrame** is the 2D data table, and **Series** is the 1D column. There are many ways to create a **DataFrame**. Here we focus on one of them, that is to convert a **dict** into a **DataFrame**.

```

results = {
    'round': [1, 2, 3, 4, 5],
    'score': [0.22, 0.33, 0.44, 0.55, 0.66],
    'mse': [0.12, 0.11, 0.10, 0.09, 0.08]
}
df_res = pd.DataFrame(results)
df_res

```

	round	score	mse
0	1	0.22	0.12
1	2	0.33	0.11
2	3	0.44	0.10
3	4	0.55	0.09
4	5	0.66	0.08

A single column is a **Series**.

```
df_res['score']
```

```

0    0.22
1    0.33
2    0.44
3    0.55
4    0.66
Name: score, dtype: float64

```

Note that similar to the `numpy.array` case, if we select the column as a set (even if the set contains only 1 element), we will get a 2D object, which is a **DataFrame** in this case.

```
df_res[['score']]
```

	score
0	0.22
1	0.33
2	0.44
3	0.55
4	0.66

C.5.2 Read and write files

In `scikit-learn`, predictors and responses can be stored either as `numpy` arrays or as `pandas` objects. Throughout this course, many examples use `numpy`, but most code works equally well with `pandas` data structures.

When working with external datasets, a common workflow is:

1. Read the data into a `DataFrame`.
2. Perform data cleaning and preprocessing.
3. Extract predictors and responses.
4. Fit a statistical learning model.

Two of the most common tabular file formats are CSV files and Excel spreadsheets.

CSV

CSV stands for comma-separated values. A CSV file is a plain-text file that stores tabular data using a separator between columns.

Common separators include:

- , (comma)
- ; (semicolon)
- \t (tab)

Consider the following file [example.csv](#).

```
round    score    mse
1    0.220000    0.120000
2    0.330000    0.110000
3    0.440000    0.100000
4    0.550000    0.090000
5    0.660000    0.080000
```

In Python we could use `pd.read_csv()` to read it.

```
import pandas as pd
df = pd.read_csv('example.csv')
df
```

	round\t	score\t	mse
0	1\t	0.220000\t	0.120000
1	2\t	0.330000\t	0.110000

```
2  3\t0.440000\t0.100000
3  4\t0.550000\t0.090000
4  5\t0.660000\t0.080000
```

The result is incorrect because this file uses tab characters (`\t`) rather than commas as separators.

Specify the separator explicitly:

```
df = pd.read_csv('example.csv', sep='\t')
df
```

	round	score	mse
0	1	0.22	0.12
1	2	0.33	0.11
2	3	0.44	0.10
3	4	0.55	0.09
4	5	0.66	0.08

Excel

Excel spreadsheets can be read using `pd.read_excel()`. We provide the `example.xlsx` as an example.

```
import pandas as pd
df = pd.read_excel('example.xlsx')
df
```

	round	score	mse
0	1	0.22	0.12
1	2	0.33	0.11
2	3	0.44	0.10
3	4	0.55	0.09
4	5	0.66	0.08

Unlike CSV files, Excel files are stored in a binary format. Therefore, **pandas** requires an additional package called an engine to read and write them. The most commonly used engine is **openpyxl**.

```
uv add openpyxl
```

Although not provided in the toml file, **openpyxl** was added to our environment as an example in the introduction. If you didn't add it at that time you may add it now.

Writing files is similar to reading files. The main difference is that writing starts from an existing **DataFrame** instead of starting from **pd**.

```
df.to_csv('example2.csv')
df.to_excel('example2.xlsx')
```

C.5.3 Relative paths

In this course, always use relative paths when reading or writing files.

A relative path starts from the current working directory rather than from the root of the file system.

Suppose your working directory is

```
C:/Users/WDAGUtilityAccount/Project/
```

and you have the following structure:

```
Project/
  example.csv
  foldername/
```

Then you can read and write files using:

```
df = pd.read_csv("example.csv")
df.to_csv("foldername/result.csv")
```

Notice that neither path begins with a drive name such as **C:** or **D:**.

If your files are stored outside the working directory, **copy them into the working directory** before reading them.

i Note

Although absolute paths such as

```
pd.read_csv("D:/Files/example.csv")
```

often work on your own computer, they may fail on another computer because the file locations are different.

Using relative paths makes your code more portable and easier to share.

C.6 matplotlib Basics

These notes use only basic `matplotlib`. The standard import is:

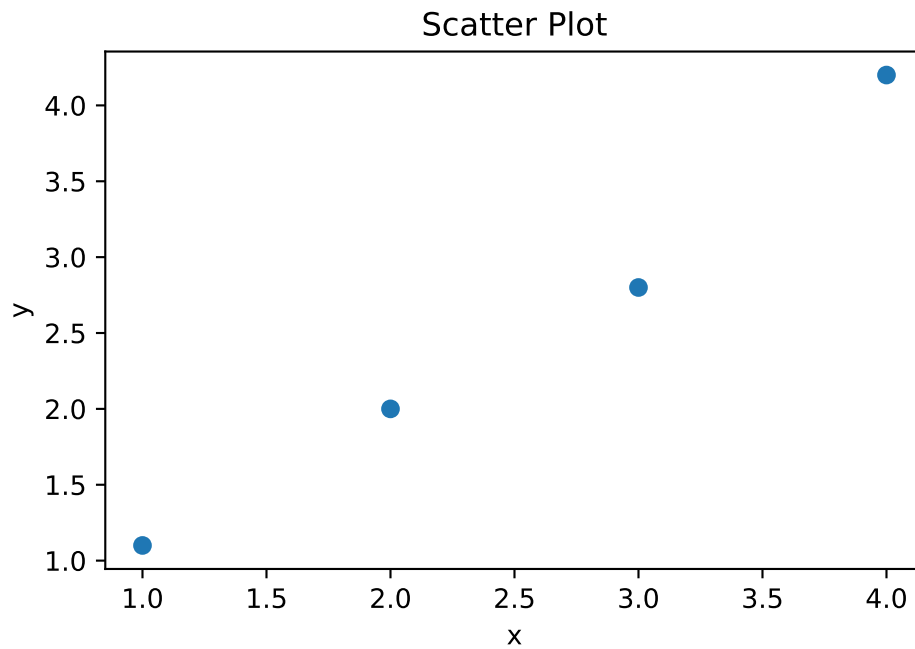
```
import matplotlib.pyplot as plt
```

We provide some code examples here. Most of the commands are straightforward. Please check the [official document](#) if you need more details.

C.6.1 Scatter Plot

```
x = np.array([1, 2, 3, 4])
y = np.array([1.1, 2.0, 2.8, 4.2])

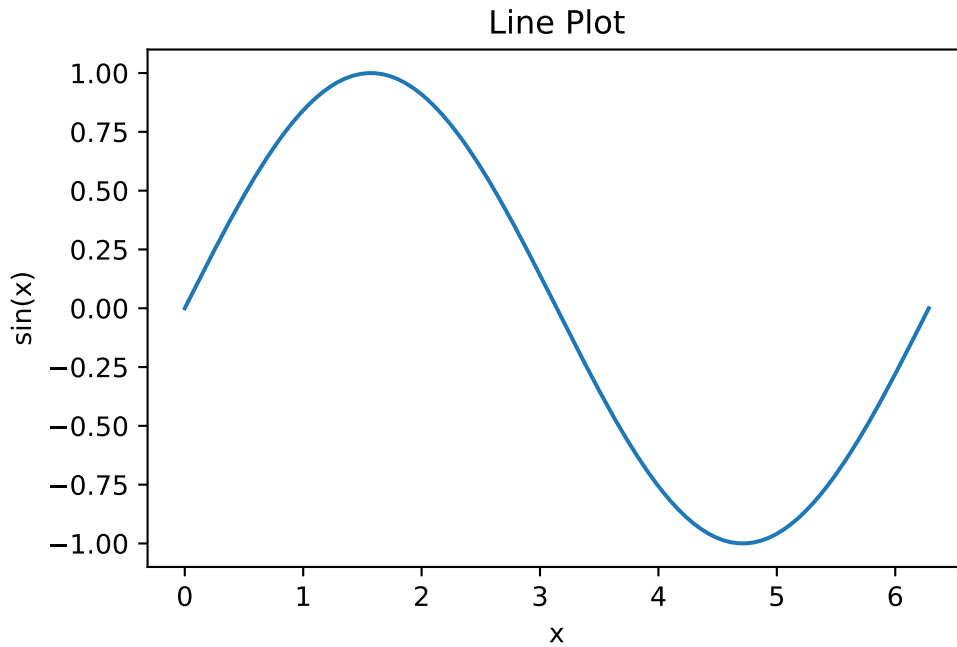
plt.scatter(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Scatter Plot");
```



C.6.2 Line Plot

```
x_grid = np.linspace(0, 2 * np.pi, 200)
y_grid = np.sin(x_grid)

plt.plot(x_grid, y_grid)
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("Line Plot");
```

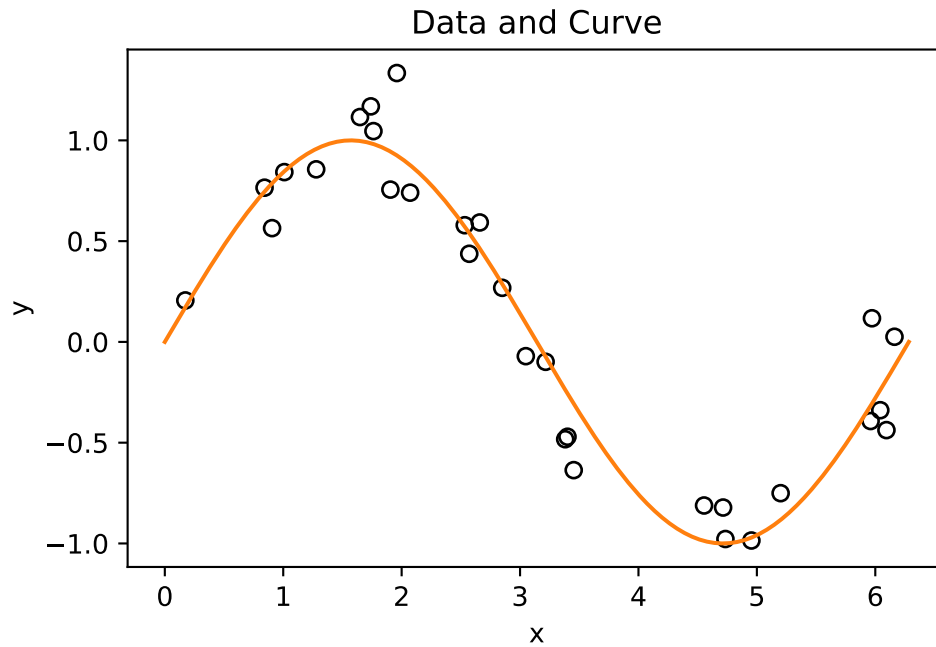


C.6.3 Scatter and Line Together

```
rng = np.random.default_rng(1)

x = rng.uniform(0, 2 * np.pi, size=30)
y = np.sin(x) + rng.normal(scale=0.2, size=30)
x_grid = np.linspace(0, 2 * np.pi, 200)

plt.scatter(x, y, facecolors="none", edgecolors="black")
plt.plot(x_grid, np.sin(x_grid), color="C1")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Data and Curve");
```

This pattern appears often when we plot data points and a fitted curve.

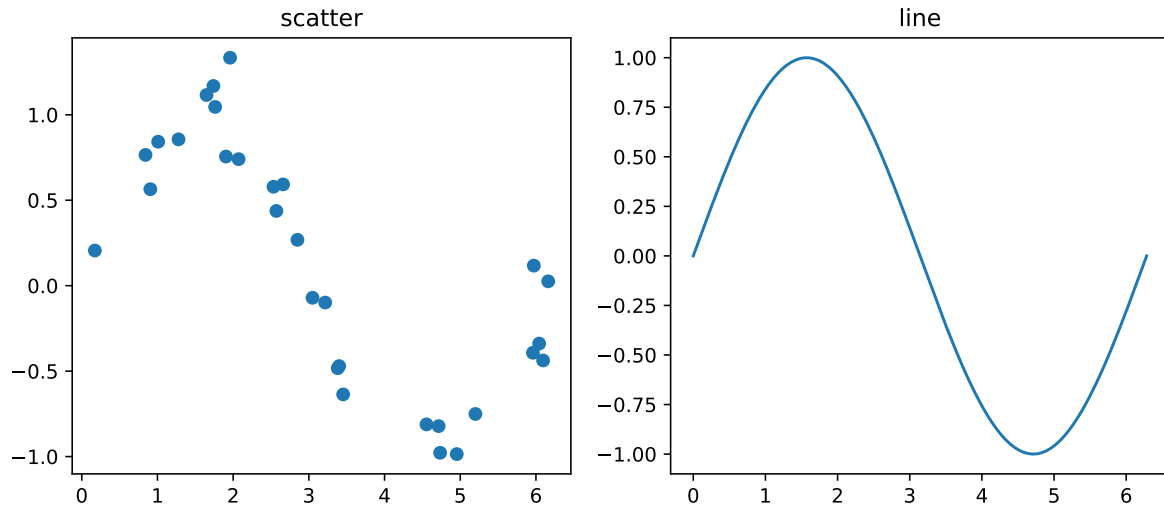
C.6.4 Multiple Plots

Use `plt.subplots` for multiple panels.

```
fig, axs = plt.subplots(1, 2, figsize=(10, 4))

axs[0].scatter(x, y)
axs[0].set_title("scatter")

axs[1].plot(x_grid, np.sin(x_grid))
axs[1].set_title("line");
```



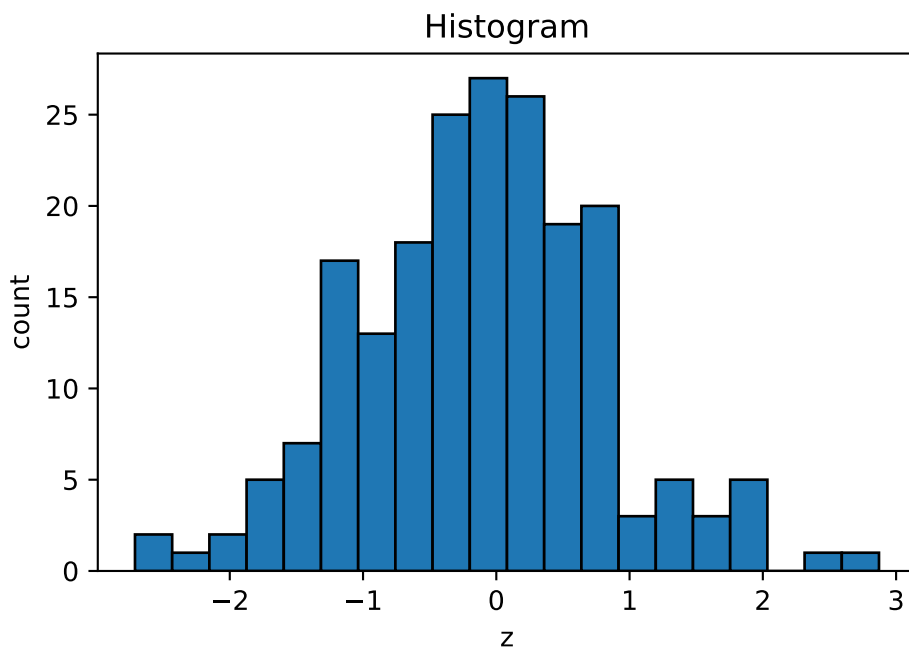
i Note

Notice that when using an axes object, many plotting commands have different names. For example, `plt.title()` becomes `ax.set_title()`. Therefore, you cannot always replace `plt` with `ax` directly. For details, consult the official documentation.

C.6.5 Histograms

```
z = rng.normal(size=200)

plt.hist(z, bins=20, edgecolor="black")
plt.xlabel("z")
plt.ylabel("count")
plt.title("Histogram");
```



C.7 Common Errors

C.7.1 One-Dimensional X

The most common error is passing a one-dimensional predictor array to `sklearn`.

Bad	Good
<pre>x = np.array([1, 2, 3, 4]) model.fit(x, y)</pre>	<pre>X = x.reshape(-1, 1) model.fit(X, y)</pre>

C.7.2 Class Name versus Model Object

Another common mistake is confusing an `sklearn` class with an object created from that class.

For example, `DecisionTreeClassifier` is the class itself.

Bad	Good
<pre>model = DecisionTreeClassifier().fit(X_train, y_train)</pre>	<pre>model = DecisionTreeClassifier().fit(X_train, y_train)</pre>

C.7.3 Mixing Lists and Arrays

Use arrays for elementwise numerical operations.

```
[1, 2, 3] * 2
```

```
[1, 2, 3, 1, 2, 3]
```

Plain lists do not always behave like numerical vectors.

```
import numpy as np
np.array([1, 2, 3]) * 2
```

```
array([2, 4, 6])
```

D Some Python concepts

D.1 Language

Python is a programming language. In other words, it is a collection of syntaxes.

D.2 Interpreters

Python needs to be interpreted into codes that computers can understand. Therefore there should be some programs that translate Python scripts. These programs are called *interpreters*.

CPython is the reference implementation of Python and the implementation most people use. It is written mainly in C and Python. New Python language features are normally developed and tested first in CPython.

Other implementations include [PyPy](#), [Jython](#) and [IronPython](#). They are useful in specific contexts, but they do not always support the same Python versions or the same package ecosystem as CPython. For example, PyPy focuses on speed through a JIT compiler, Jython integrates with the JVM but its current stable release is Python 2.7-based, and IronPython integrates with .NET but lags behind modern CPython versions.

Since **CPython** is the most widely used and best-supported implementation, it is the best choice, at least for beginners. And actually, if you have no idea about this topic, but you use Python, it is highly possible that you are using **CPython**.

Note

We mentioned “interpreter” here. There are mainly two types of implementations of programming languages: interpreters and compilers. There are also some additional types like just-in-time compilers which can be treated as combinations of the two.

Python is often described as an *interpreted* language, but **CPython** first compiles source code to bytecode and then executes that bytecode in the Python virtual machine. For beginners, the practical point is that Python supports an interactive workflow and does not require a separate manual compile step.

D.3 REPL

There are two ways to use Python interpreter. One way is to execute a Python script stored in a file. The second way is called *the interactive shell*, that Python interpreter read the input from user directly, and print the result immediately. The model is like code example: prompt the user for some code, and when they've entered it, execute it in the same process. This model is often called a **REPL**, or *Read-Eval-Print-Loop*.

Shell, *terminal*, *console* have different meanings in their original contexts. However, nowadays, especially when talking about Python interactive shell, these terminologies are used interchangeably. They are referred to the frontend of the system. In other words, the main task for the Python interactive shell is to handle the user inputs and communicate with the backend, which is also called a *kernel*. We won't distinguish the real differences between these terminologies. The kernel will be discussed in the next section.

The standard Python REPL can usually be started with `python` or `python3`. On Windows, `py` may also be available. To quit, use `quit()`, `exit()`, `Ctrl+D` on macOS/Linux, or `Ctrl+Z` then **Enter** on Windows.

Note

In the REPL model, the backend (evaluation) is basically handled by the Python interpreter. The frontend is dealing with the user interface. Some typically tasks include the *primary/secondary prompt* and multi-line commands. The original REPL is very limited.

D.4 IPython

IPython was initially designed as an Enhanced interactive Python shell. However after many year's development, the whole **IPython** project becomes too big to maintain as one single project. Therefore it is now split into many smaller projects. The two most popular projects are **IPython** and **Jupyter**. This is called [the Big Split](#).

The current **IPython** play two fundamental roles:

- Terminal **IPython** as the familiar REPL;
- The **IPython** kernel (which is defined below) that provides computation and communication with the frontend interfaces, like the notebook.

The core idea in the design of **IPython** is to abstract and extend the notion of a traditional REPL environment by decoupling the evaluation into its own process. We call this process a *kernel*: it receives execution instructions from clients and communicates the results back to them.

This decoupling allows us to have several clients connected to the same kernel, and even allows clients and kernels to live on different machines. This two-process model is now used by most of the **Jupyter** project.

You can launch the IPython shell on the command line with the `ipython` command (which similar to `python` case requires `PATH` configuration), and quit the shell with `exit/exit()/quit/quit()` commands.

The reference Python kernel provided by IPython is called `ipykernel`. With `ipykernel` you may create and maintain multiple kernels.

D.5 Jupyter

Jupyter is an ecosystem for interactive computing. It originated from the IPython Notebook project and later evolved into a language-independent platform supporting many programming languages. The name **Jupyter** is inspired by **Julia**, **Python**, and **R**, the three languages that were central to many early data science workflows.

Today, Jupyter includes several user interfaces, such as JupyterLab, Jupyter Notebook, Jupyter Console, and QtConsole. In this course, however, the important idea is not a particular interface, but the relationship among a notebook frontend, a kernel, and the notebook file stored on disk.

A Jupyter notebook is saved as a file with extension `.ipynb`. Internally, the file uses a structured JSON format that stores:

- code cells;
- Markdown cells;
- outputs;
- notebook metadata.

When you run code in a notebook, the frontend sends the code to a kernel, which executes the code and returns the results. In a Python notebook, a kernel can be viewed as a Python process running in the background. It executes code, stores variables, and communicates with the notebook interface.

Different programming languages can be supported by different kernels. For example:

- `ipykernel` for Python;
- `IRkernel` for R;
- `IJulia` for Julia.

Kernel

A kernel is associated with a particular Python environment, but it is not the environment itself. For example, multiple kernels may use the same Python environment while maintaining completely separate variables and memory. Conversely, multiple notebook windows may connect to the same kernel and therefore share the same variables and computational state.

For this course, you can think of the notebook as the user interface, the kernel as the Python process that runs your code, and the `.ipynb` file as the notebook document saved on disk.

D.5.1 Multi-kernels setup

This section is mainly following the [official document](#).

To install one IPython kernel, you may install `ipykernel` in the environment. If you want to have multiple IPython kernels for different venvs, you will need to specify unique names for the kernelspecs.

1. Activate the environment you want.
2. Install the kernel in the environment.

```
uv add ipykernel
uv run python -m ipykernel install --user --name myenv --display-name "Python (myenv)"
```

`--user` means that the kernel is installed in the user's folder instead of a system folder, and it can be removed. The `--name` value (in this case it is `myenv`) is used by Jupyter internally. These commands will overwrite any existing kernel with the same name. `--display-name` is what you see in the notebook menus.

3. You could use the command to find all kernels installed in your system.

```
uv run jupyter kernelspec list
```

Available kernels are shown, as well as the path to the kernel configuration file `kernel.json`. The most important configuration is the path to the Python interpreter executable file.

Notice that the path in `argv` points to the Python interpreter used by this kernel. Different kernels may point to different Python environments, or multiple kernels may point to the same environment.

Listing D.1 `kernel.json`

```
{
  "argv": [
    "C:\\Users\\WDAGUtilityAccount\\Project\\.venv\\Scripts\\python.exe",
    "-Xfrozen_modules=off",
    "-m",
    "ipykernel_launcher",
    "-f",
    "{connection_file}"
  ],
  "display_name": "(myenv)",
  "language": "python",
  "metadata": {
    "debugger": true
  },
  "kernel_protocol_version": "5.5"
}
```
